



曲芸飛行 - *Aerobatics*

JOI 2021 春合宿 Day1 解説
解説担当 — 米田 寛峻（よねだ ひろたか）

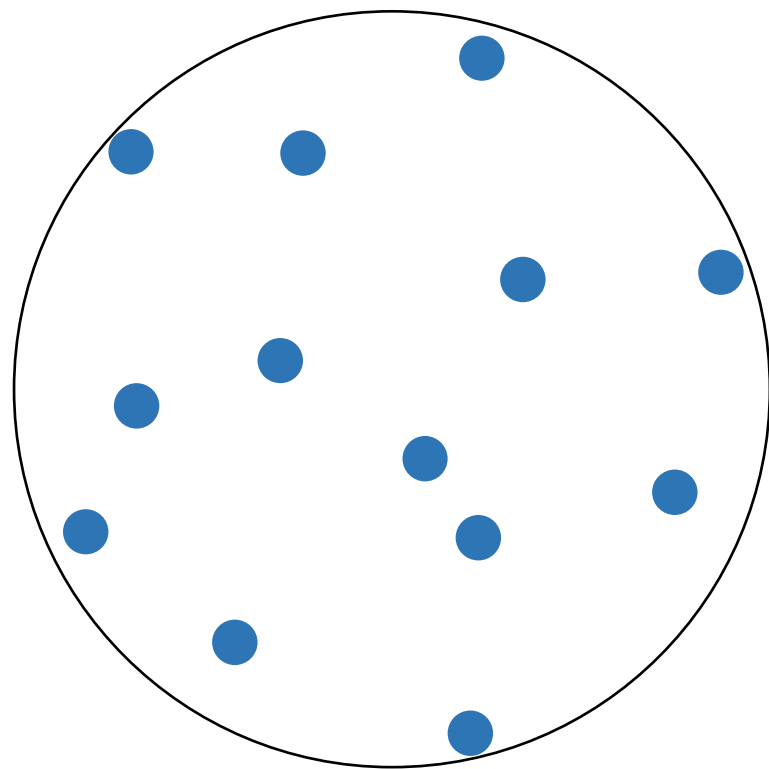
2021. 03. 20
square1001

問題概要 (1)



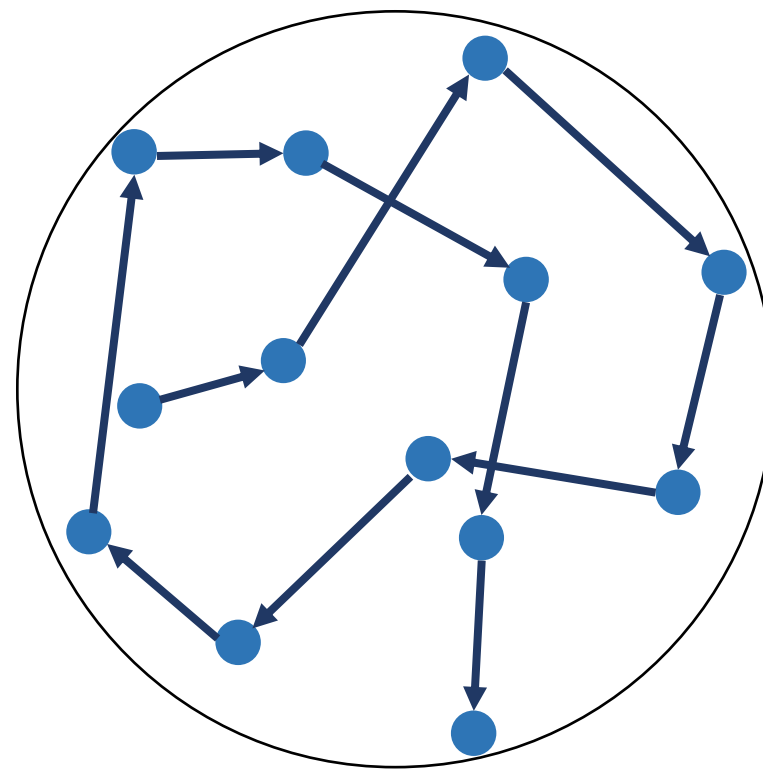
02 / 65

A



平面上に **N** 個の
チェックポイントがある

B



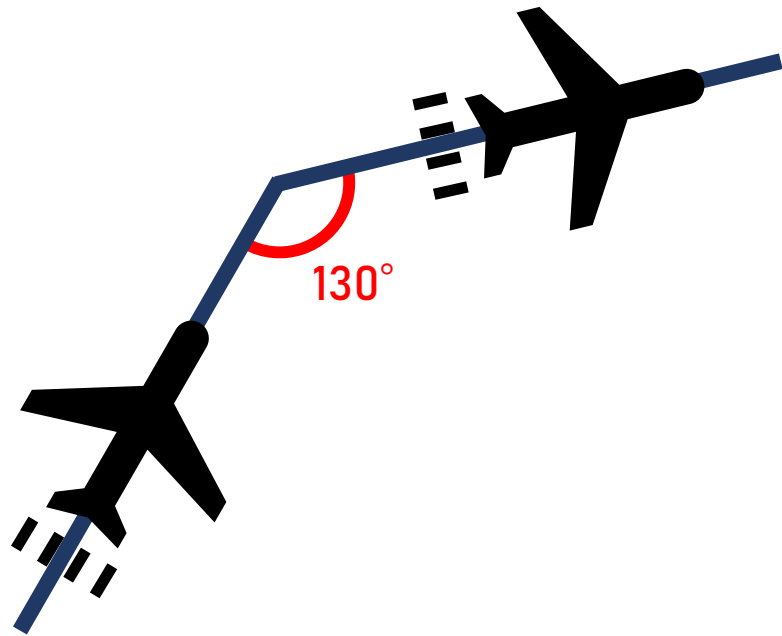
これらを **1** 回ずつ通る
経路を見つける

問題概要 (2)



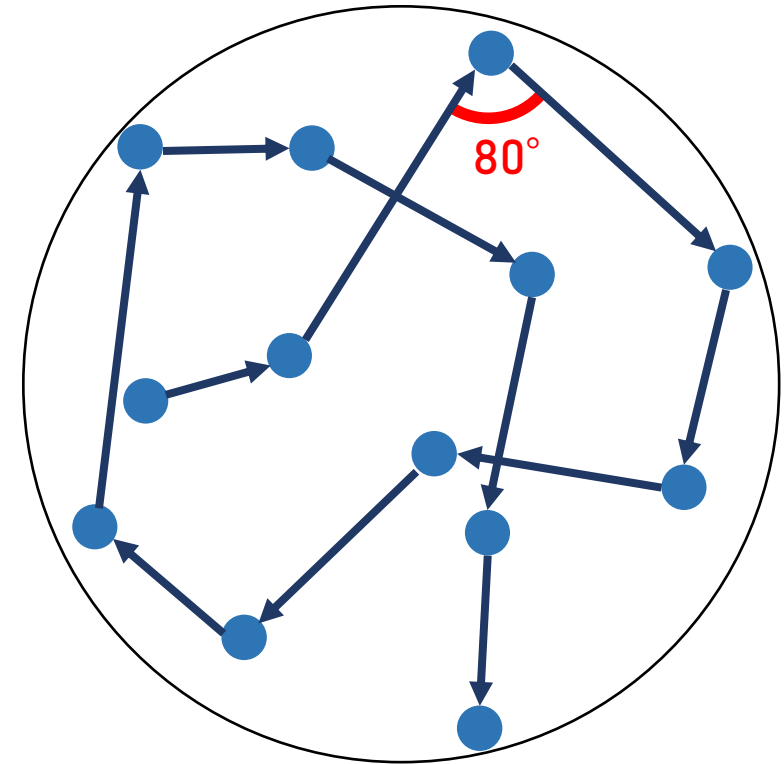
03 / 65

C



飛行機はあまり方向転換したくない
すなわち折れ線の角度を大きくしたい

D



経路の最小の角を可能な限り大きくし
曲芸飛行の成功率を上げよう！

問題概要 (3)



04 / 65

本課題は 6 個の入力データからなる
→ それぞれのデータに対して**手元で実行**し、出力結果を提出

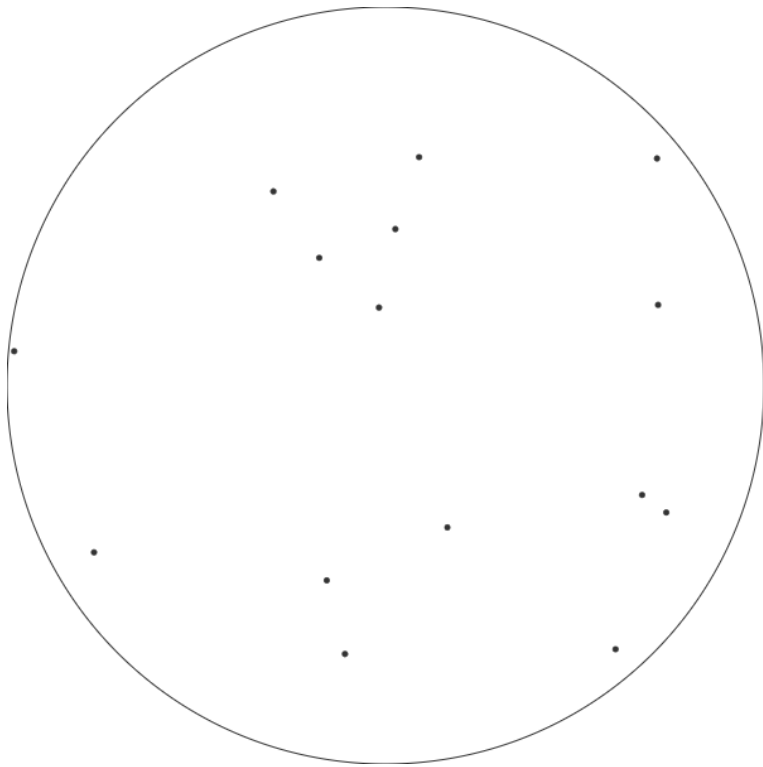
入力データ	N の値	満点のスコア	半分のスコア
01.txt	15	100.000 度	68.867 度
02.txt	200	143.000 度	110.954 度
03.txt	200	134.000 度	102.338 度
04.txt	1000	156.000 度	123.182 度
05.txt	1000	150.000 度	117.569 度
06.txt	1000	153.000 度	120.382 度

入力データ 1・2



05 / 65

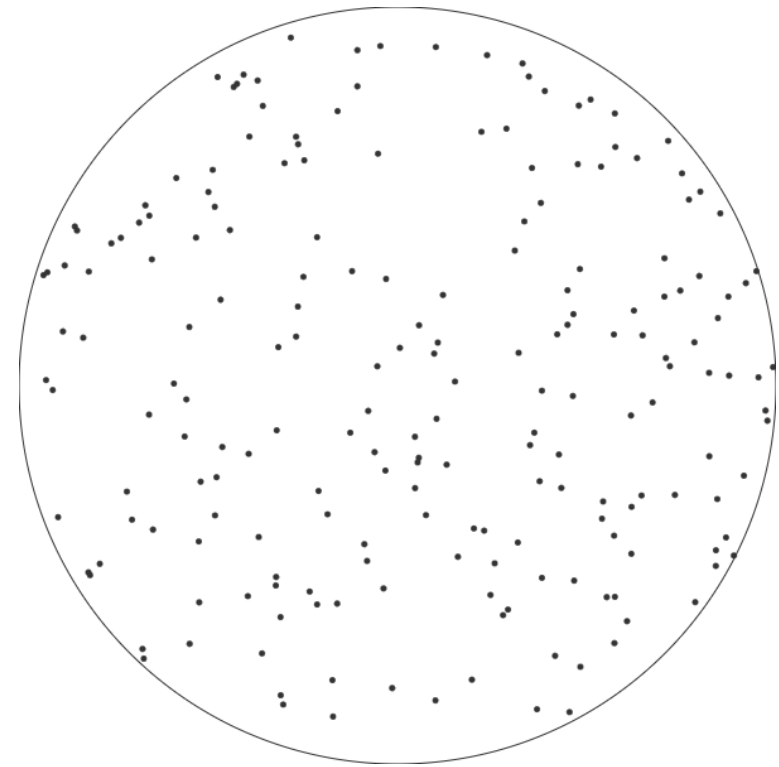
1



$$N = 15 / Z_0 = 100^\circ$$

(ランダムな感じの点の分布)

2



$$N = 200 / Z_0 = 143^\circ$$

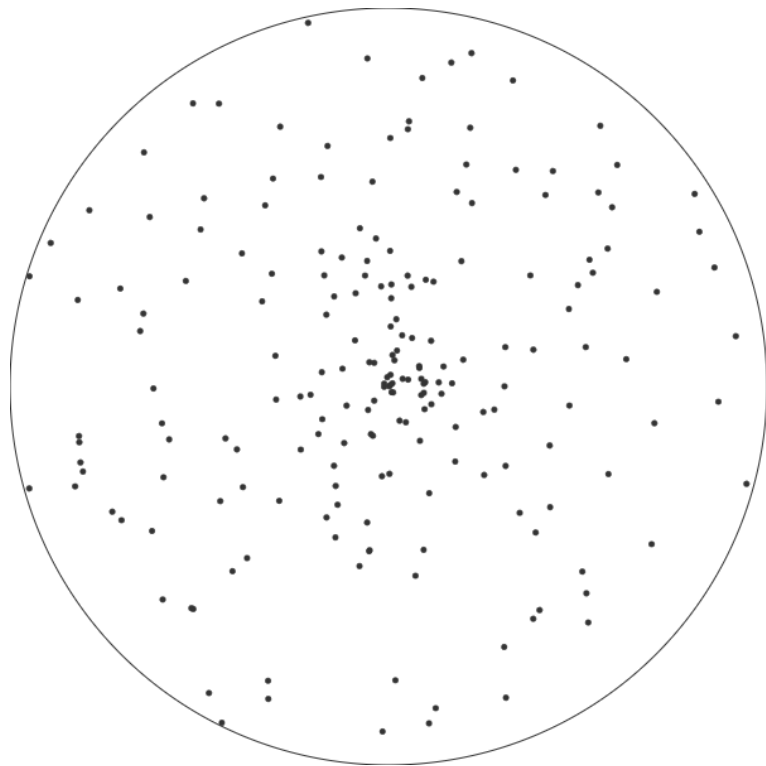
(ランダムな感じの点の分布)

入力データ 3・4



06 / 65

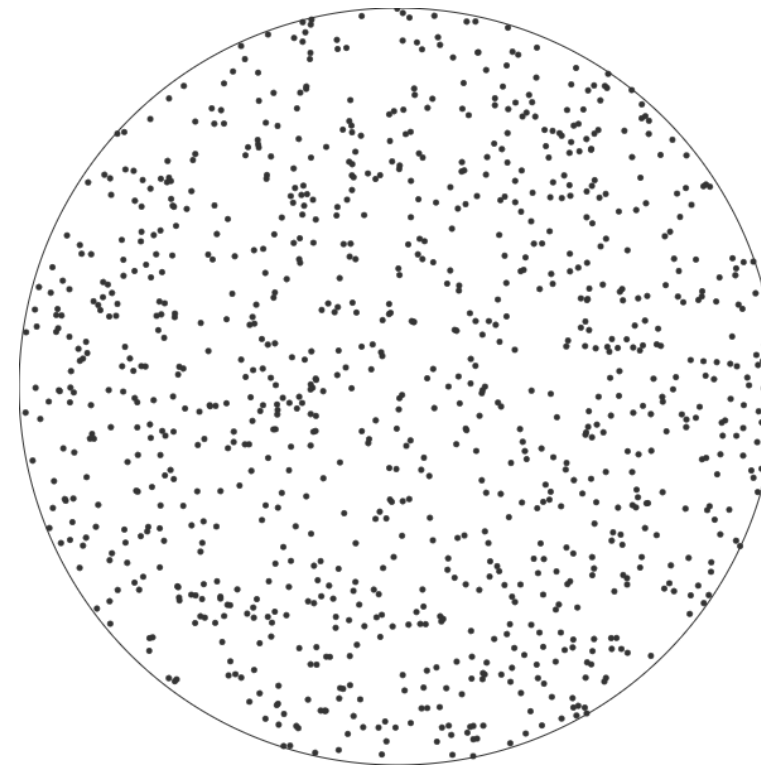
3



$$N = 200 \text{ } / \text{ } Z_0 = 134^\circ$$

(中心のほうが点の密度が高い分布)

4



$$N = 1000 \text{ } / \text{ } Z_0 = 156^\circ$$

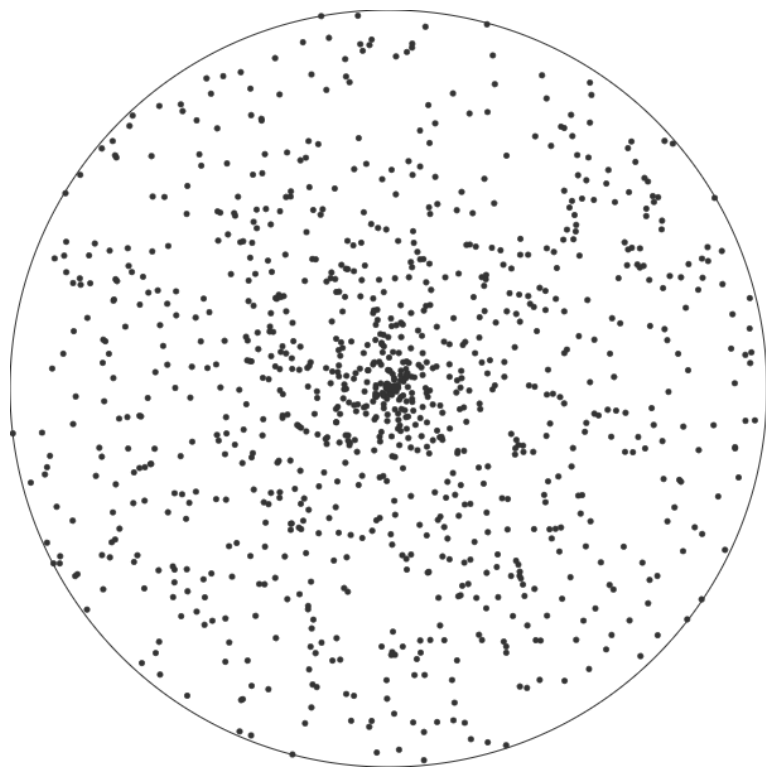
(ランダムな感じの点の分布)

入力データ 5・6



07 / 65

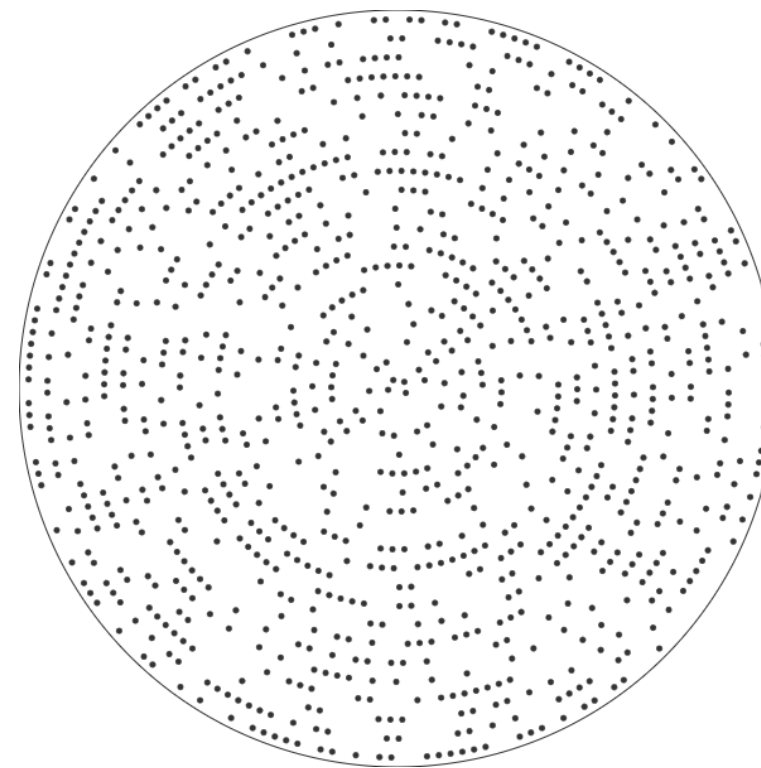
5



$N = 1000 / Z_0 = 150^\circ$

(中心のほうが点の密度が高い分布)

6



$N = 1000 / Z_0 = 153^\circ$

(特殊なタイプのデータ)

Output Only 形式について



08 / 65

課題の形式

- ✈ 入力データが公開されており
これに対する出力を提出しなければならない
- ✈ より良い答えを出せば高得点を取れる
- ✈ 方針によって大きく点数が変わることが多い
- ✈ 入力データの性質をつかむことが必要な場合も

過去の出題

JOI 春合宿における出題

- ☀ 2020 年「Legendary Dango Maker」
- ☀ 2018 年「Road Service」

IOI における出題

- ☀ 2019 年「Broken Line」
- ☀ 2017 年「Nowruz」

Output Only 形式について



09 / 65

課題の形式

- ✈ 入力データが公開されており
これに対する出力を提出しなければならない
- ✈ より良い答えを出せば高得点を取れる
- ✈ 方針によって大きく点数が変わることが多い
- ✈ 入力データの性質をつかむことが必要な場合も

過去の出題

JOI 春合宿における出題

☀ 2020 年「Legendary Dango Maker」

☀ 2018 年「Road Service」

IOI 日本代表合宿における出題

☀ 2019 年「Broken Line」

☀ 2017 年「Nowruz」

出題数はあまり多くないが
差が付きやすい問題になりうる

入力データ 1 に関して



10 / 65

入力データ 1 は「 $N = 15$ 」と小さいので...

入力データ 1 に関して



11 / 65

入力データ 1 は「 $N = 15$ 」と小さいので...

ビット DP

が使えます！

入力データ 1 に関して

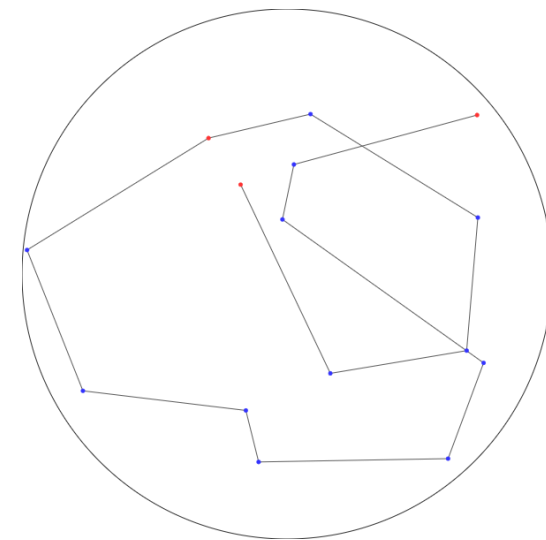


12 / 65

ビット DP で解いてみる

- ✈ dp[既に訪れた場所の集合][今いる場所][直前にいた場所] = (作った最小の角の最大値) を記録
- ✈ すると、**計算時間 $O(2^N \times N^3)$** ・ メモリ $O(2^N \times N^2)$ で解ける
- ✈ したがって、 $N = 15$ のケースだと数秒で最適解のうちひとつが求まる

Tip : 配布ライブラリも利用できる !



入力データ 1 の最適解 (約 100.008°)

10 点獲得

方針 1 - ランダムに経路を生成



13 / 65

最も簡単な方針：ランダムに経路を生成する

解法 A

チェックポイント $1 \rightarrow 2 \rightarrow 3 \rightarrow \dots \rightarrow N$ の順番に通る



解法 B

チェックポイントを通る順番をランダム生成
これを 1,000,000 回行い、スコアが最大のものに決める

方針 1 - ランダムに経路を生成



14 / 65

実験結果

入力データ	解法 A	解法 B
01.txt	1.540 度	80.517 度
02.txt	0.286 度	7.356 度
03.txt	0.139 度	5.354 度
04.txt	0.010 度	1.599 度
05.txt	0.048 度	1.069 度
06.txt	0.042 度	1.334 度
得点	0 点	7 点

※ 注意：解法 B については、ひとつの実験結果にすぎません。

7 点獲得

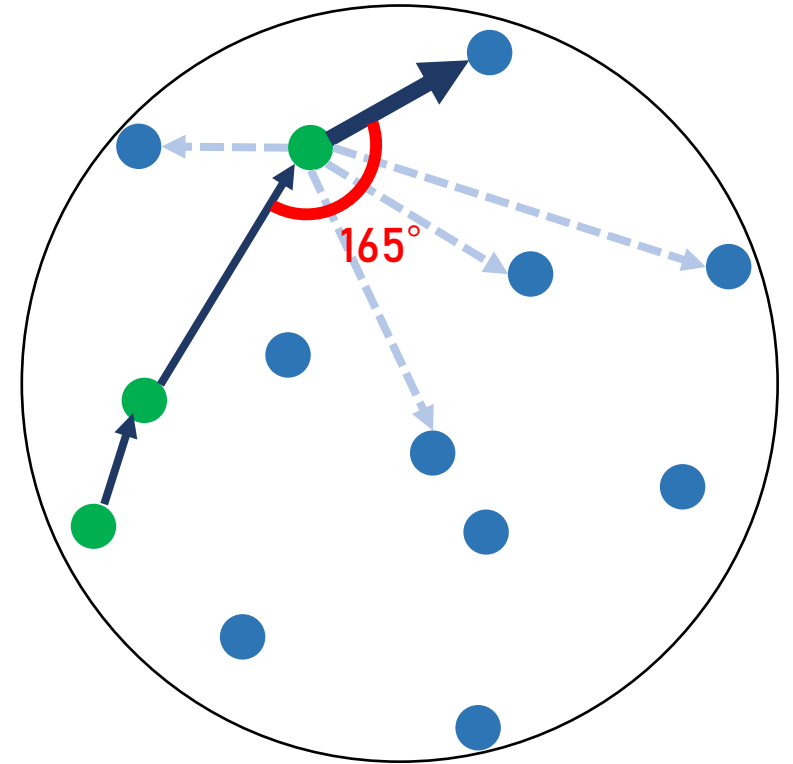
方針 2 – 貪欲法



15 / 65

貪欲法を使って解く

- ✈ 最初に通る 2 つのチェックポイントを決める
- ✈ 3 番目以降のチェックポイントを決めるときに
残った場所の中で「角が最大となるもの」を選ぶ

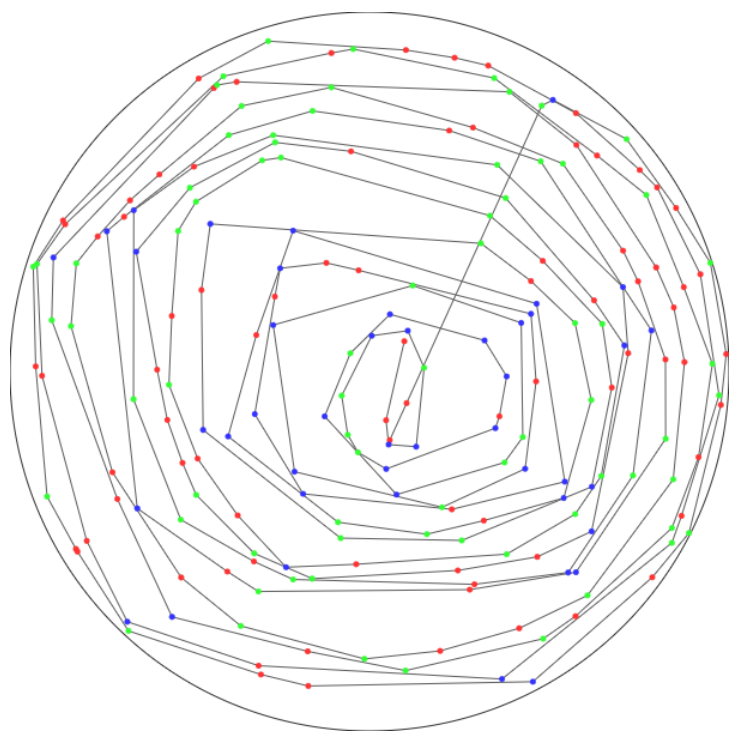


方針 2 – 貪欲法

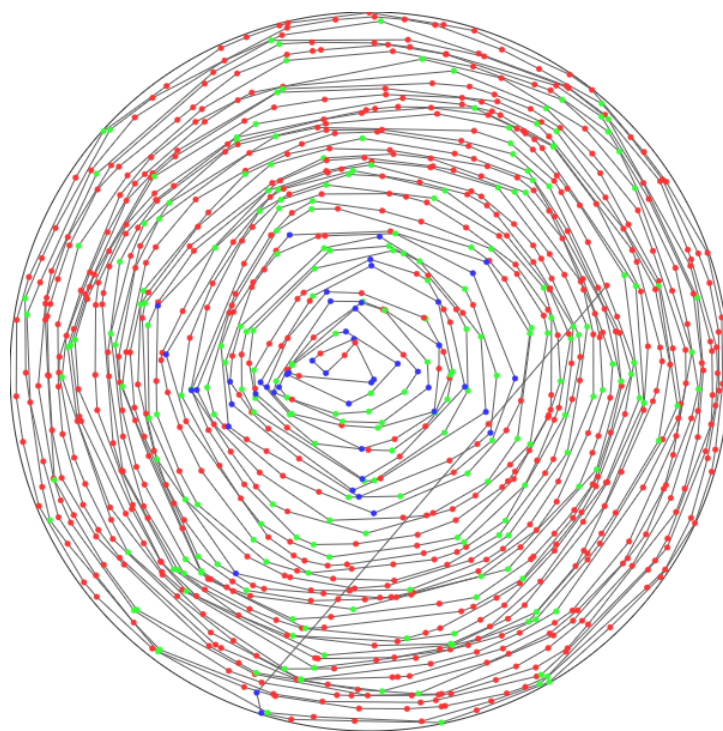


16 / 65

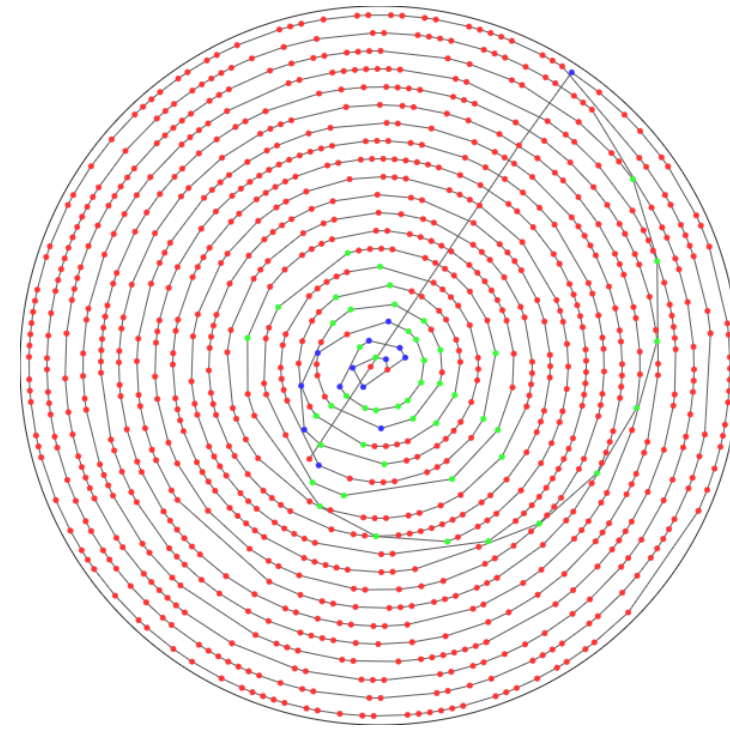
得られる答えは、らせん状になる



入力データ 2 の貪欲解



入力データ 4 の貪欲解



入力データ 6 の貪欲解

方針 2 – 貪欲法



17 / 65

貪欲法を使って解く

解法 A



解法 B

- X 座標が最大の頂点をスタート地点
- 2 番目の地点はここから最も上に近い方向に進むように選び
- 3 番目以降は貪欲法で答えを求める

- スタート地点・2 番目の地点を全探索し
- 3 番目以降は貪欲法で答えを求める

※ 解法 B の計算時間は $O(N^4)$ なので
入力データ 4, 5, 6 では実行に 20 分くらいかかる

方針 2 – 貪欲法



18 / 65

実験結果

入力データ	解法 A	解法 B
01.txt	67.638 度	91.822 度
02.txt	1.021 度	91.327 度
03.txt	47.791 度	58.807 度
04.txt	40.454 度	79.341 度
05.txt	33.773 度	78.051 度
06.txt	83.346 度	84.000 度
得点	14 点	28 点

28 点獲得

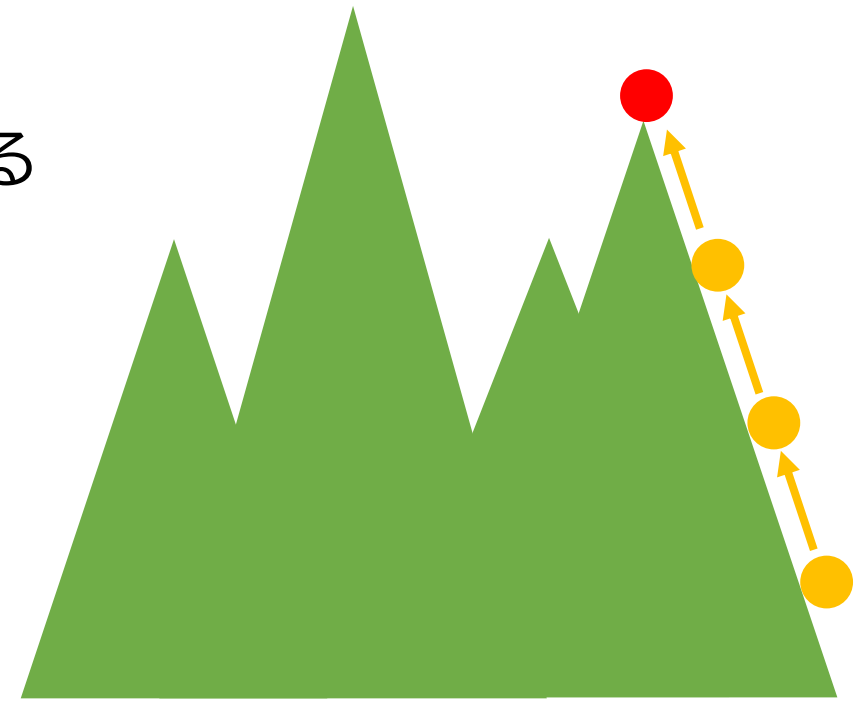
方針 3 – 山登り法 ①



19 / 65

山登り法とは？

- ✈ スコアが改善できるところを見つけて、これを改善する
- ✈ これを「改善できなくなるまで」繰り返す
上手くやれば、かなり良い解が得られることが多い！



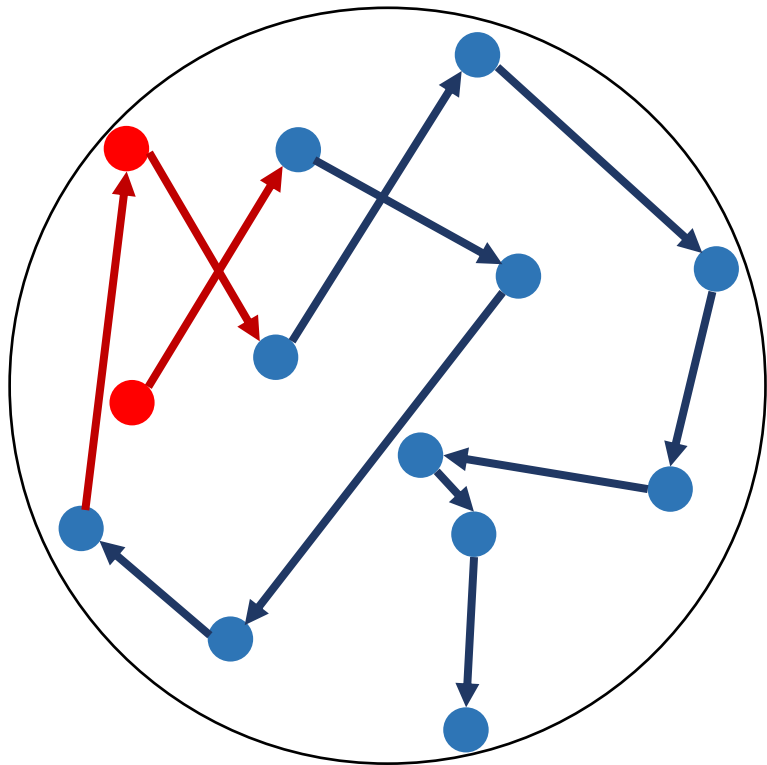
山登り法のイメージ
局所的改善を繰り返して、**頂上**に到達する

方針 3 – 山登り法 ①



20 / 65

山登り法のイメージ



Step 1 - 改善できる場所を見つける

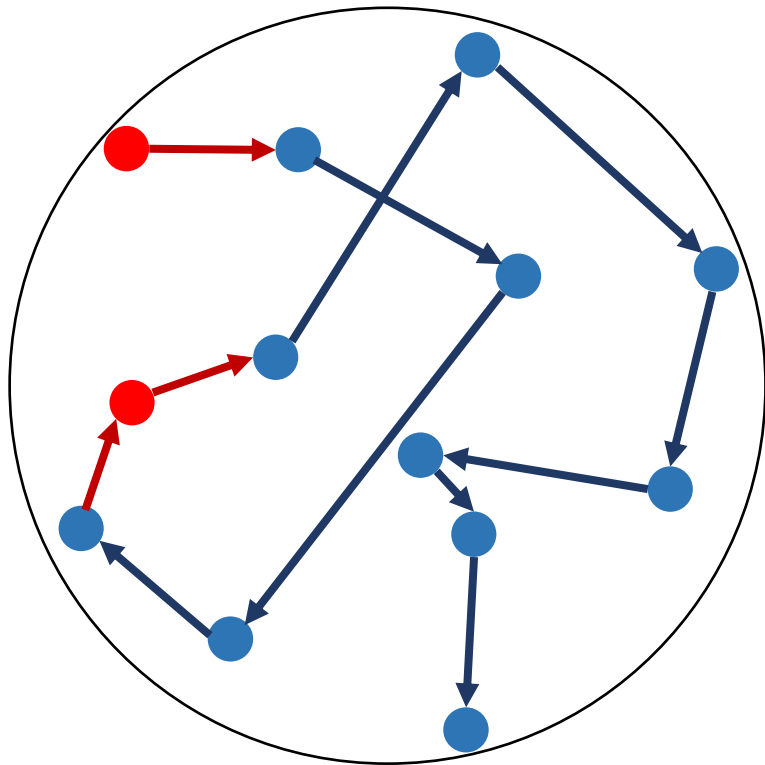
✈ 2つの赤い頂点の順番を交換できそう！

方針 3 – 山登り法 ①



21 / 65

山登り法のイメージ



Step 1 - 改善できる場所を見つける

✈ 2つの赤い頂点の順番を交換できそう！

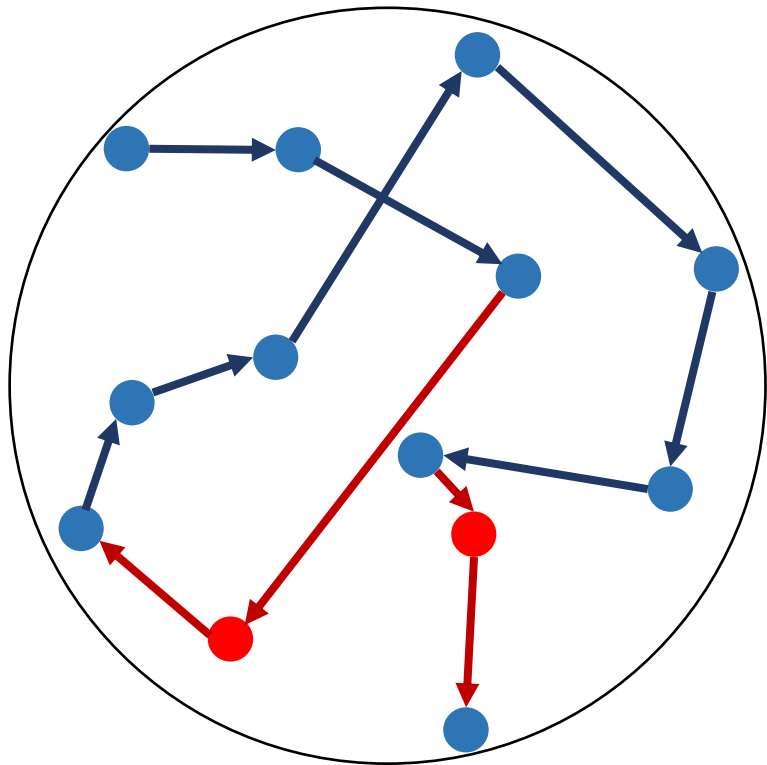
✈ スコアが改善するので、**順番を交換する**

方針 3 – 山登り法 ①



22 / 65

山登り法のイメージ



Step 1 - 改善できる場所を見つける

✈ 2つの赤い頂点の順番を交換できそう！

✈ スコアが改善するので、**順番を交換する**

Step 2 - 改善できる場所を見つける

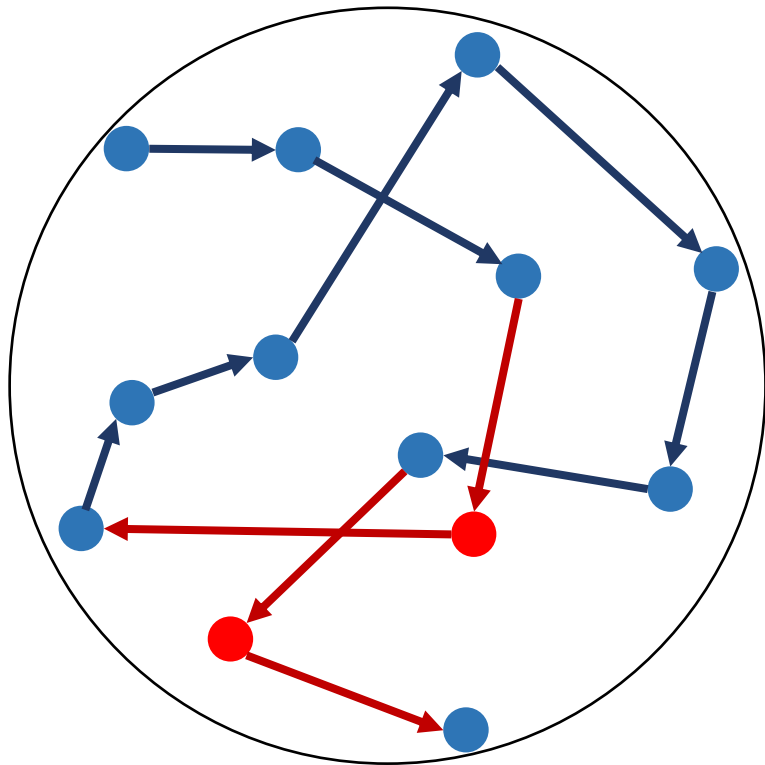
✈ 2つの赤い頂点の順番を交換できそう！

方針 3 – 山登り法 ①



23 / 65

山登り法のイメージ



Step 1 - 改善できる場所を見つける

✈ 2つの赤い頂点の順番を交換できそう！

✈ スコアが改善するので、**順番を交換する**

Step 2 - 改善できる場所を見つける

✈ 2つの赤い頂点の順番を交換できそう！

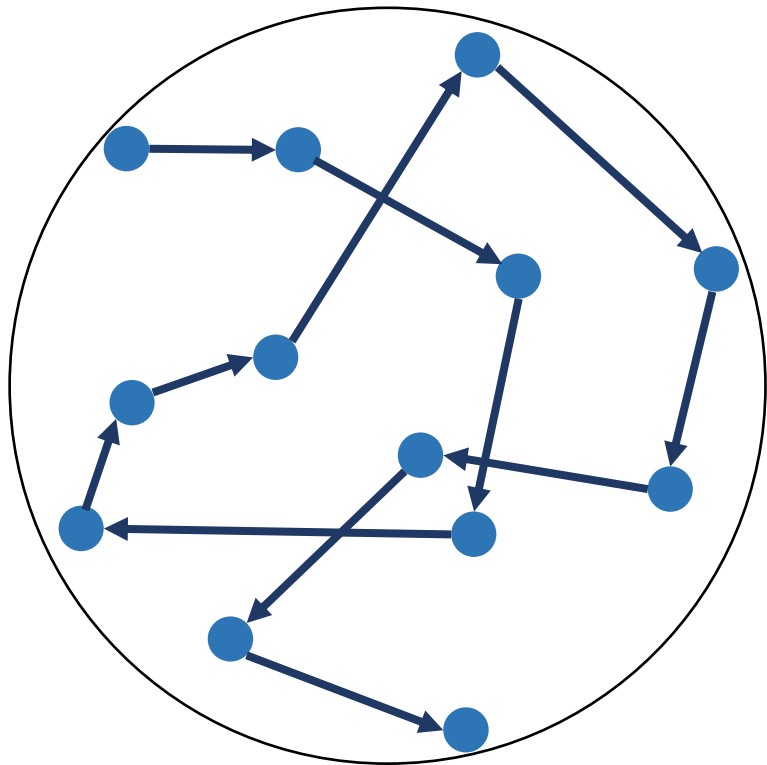
✈ スコアが改善するので、**順番を交換する**

方針 3 – 山登り法 ①



24 / 65

山登り法のイメージ



Step 1 - 改善できる場所を見つける

✈ 2つの赤い頂点の順番を交換できそう！

✈ スコアが改善するので、**順番を交換する**

Step 2 - 改善できる場所を見つける

✈ 2つの赤い頂点の順番を交換できそう！

✈ スコアが改善するので、**順番を交換する**

Step 3 - これ以上スコアを改善できないので
探索を終了する

方針 3 – 山登り法 ①



25 / 65

山登り法を使って解く

解法 A



解法 B

最もスコアが改善できる「2 つの順序の交換」を適用し続ける
改善する方法がひとつもなくなれば、探索を打ち切る

2 つの場所をランダムに選び、スコアが改善すれば交換する
ただし、解の自由度を上げるため、**スコアが同じでも交換を適用**
5N² 回以上スコアが更新されなければ、探索を打ち切る

※ ただし初期解は $1 \rightarrow 2 \rightarrow \dots \rightarrow N$ の順番とする

方針 3 – 山登り法 ①



26 / 65

実験結果

入力データ	解法 A	解法 B
01.txt	56.609 度	70.803 度
02.txt	57.037 度	102.910 度
03.txt	68.013 度	101.877 度
04.txt	64.315 度	118.428 度
05.txt	66.832 度	112.167 度
06.txt	71.712 度	111.941 度
得点	19 点	45 点

※ 注意：解法 B については、ひとつの実験結果にすぎません。

45 点獲得

方針 4 – 山登り法 ②



27 / 65

「改善」の方法を変えるということ

- ✈ 元々は 1 回の変化を「2 つの場所の訪れる順番を入れ替える」としていたが…
- ✈ これを上手く変えれば、“**局所的最適解**” にハマりにくくなるのではないか？

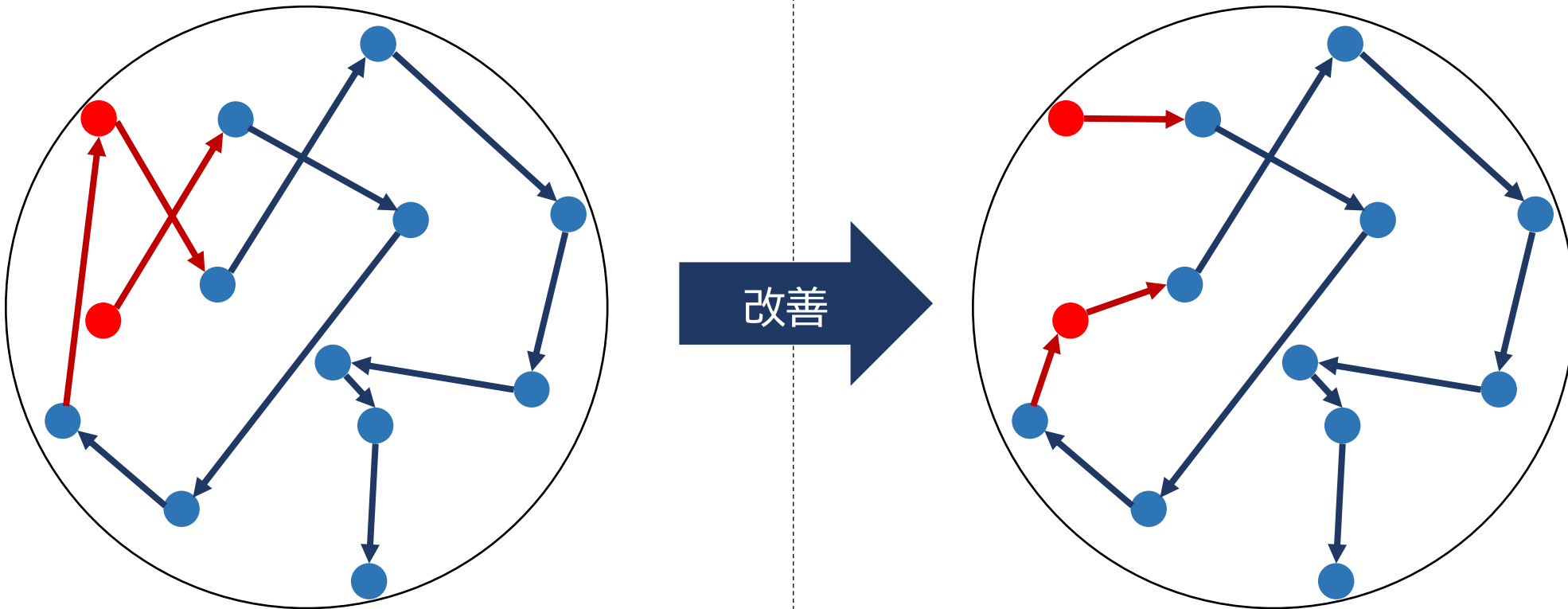
方針 4 – 山登り法 ②



28 / 65

元々の「1 回の変化」は

✳ 2 つの頂点を選び、これを訪れる順番を入れ替えること



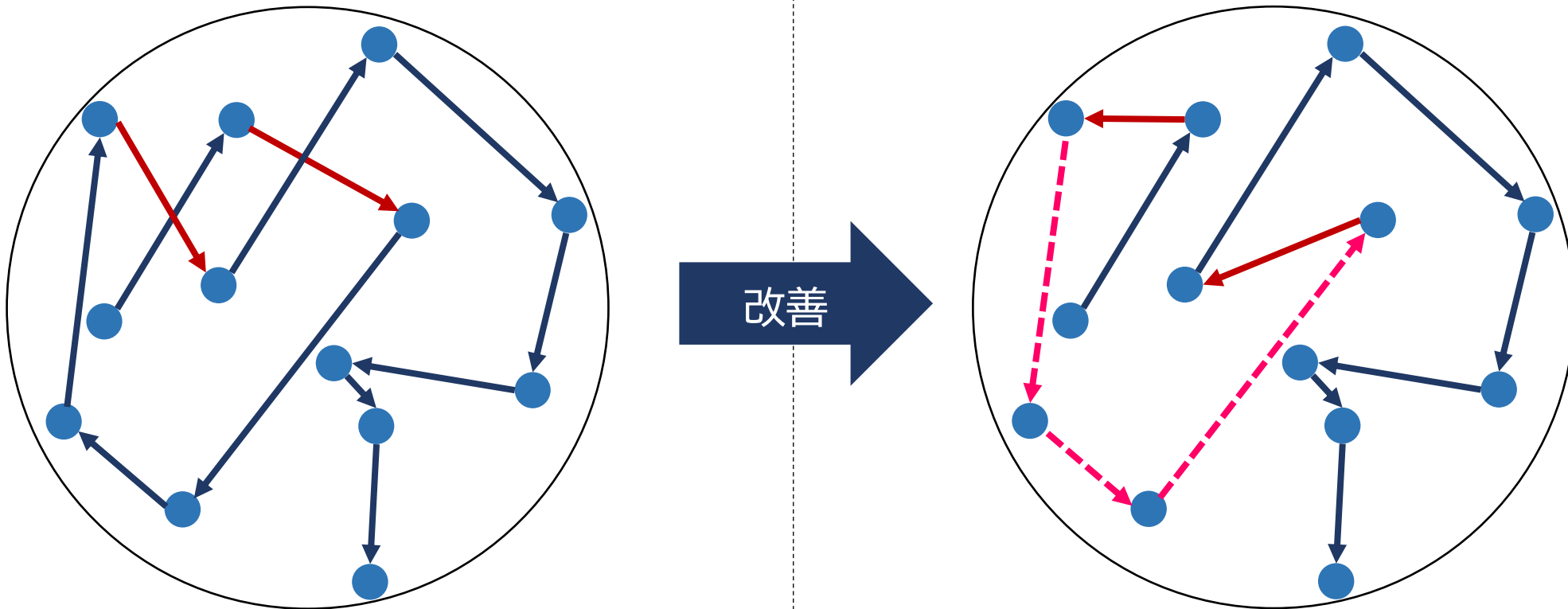
方針 4 – 山登り法 ②



29 / 65

これをもっと良いものにしよう！（Part 1）

☀ 2つの辺を選び、これを訪れる順番を入れ替えること



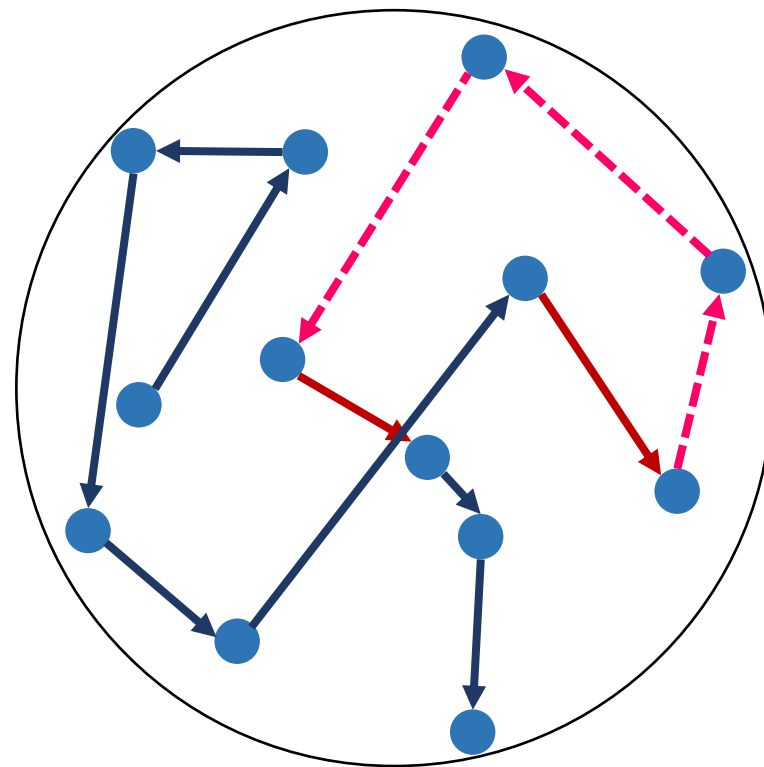
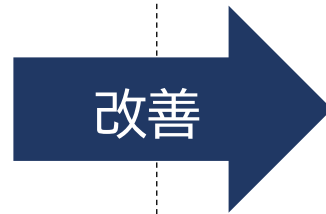
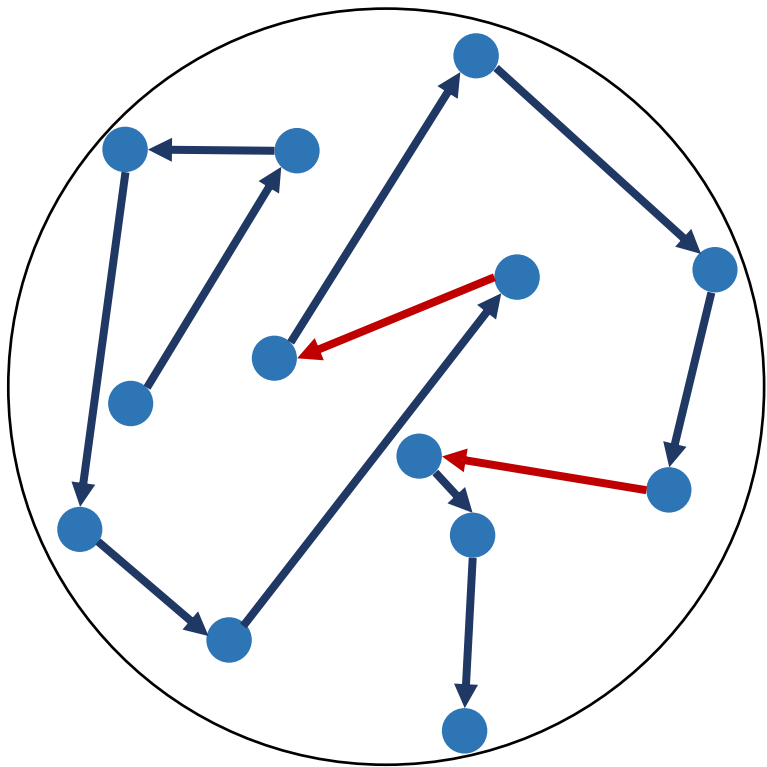
方針 4 – 山登り法 ②



30 / 65

これをもっと良いものにしよう！

☀ 2つの辺を選び、これを訪れる順番を入れ替えること



方針 4 – 山登り法 ②



31 / 65

1 回の変化で「2 つの頂点を swap する」

VS.

1 回の変化で「2 つの辺を swap する」

方針 4 – 山登り法 ②



32 / 65

1 回の変化で「2 つの頂点を swap する」

VS.

1 回の変化で「2 つの辺を swap する」

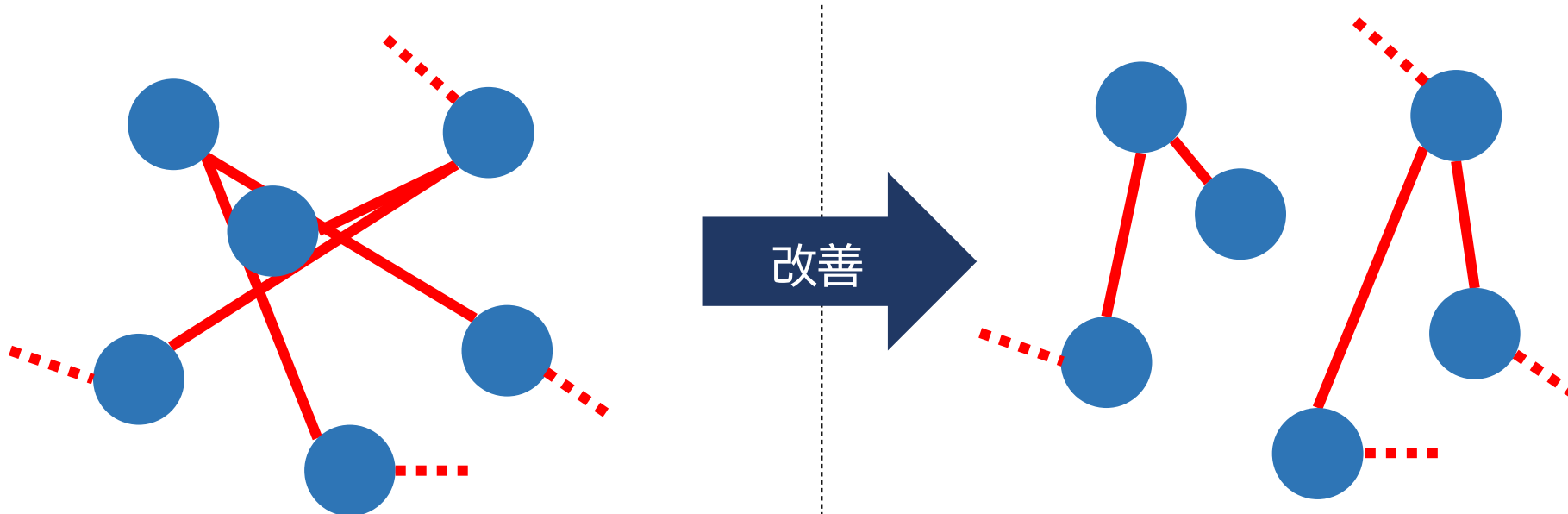
方針 4 – 山登り法 ②



33 / 65

2 つの頂点を swap する場合

- ✈ 変化する角は最大 6 個
- ✈ すなわち、1 回の変化における“制約”が大きく、解を動かしにくい…



方針 4 – 山登り法 ②

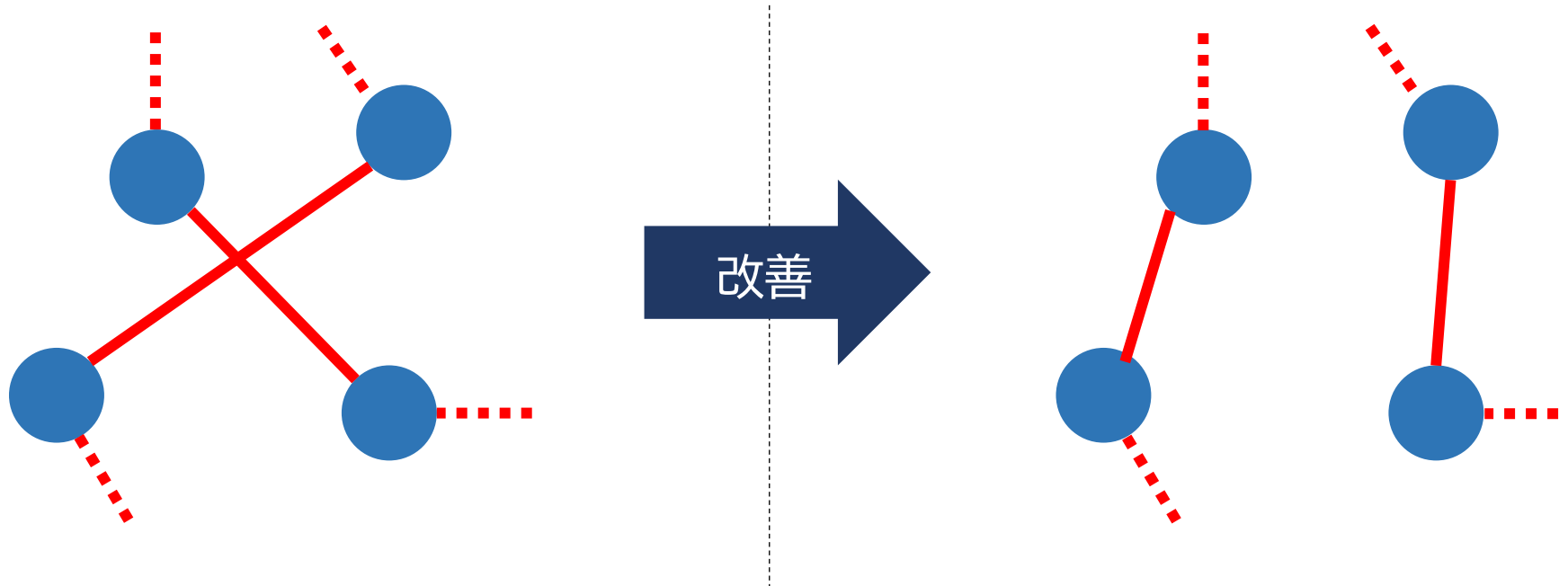


34 / 65

2 つの辺を swap する場合

✈ 変化する角は**最大 4 個**

✈ すなわち、1 回の変化における“制約”が大きく、**解を動かしやすい！**



方針 4 – 山登り法 ②



35 / 65

2 つの頂点を swap する場合

- ✈ 変化する角は最大 6 個
- ✈ すなわち、1 回の変化における“制約”が大きく、解を動かしにくい…

このような探索は「2-opt」とも呼ばれる

2 つの辺を swap する場合

巡回セールスマン問題にも使われる

- ✈ 変化する角は最大 4 個
- ✈ すなわち、1 回の変化における“制約”が小さく、解を動かしやすい！

方針 4 – 山登り法 ②



36 / 65

2 つの頂点を swap する場合の実装

- ✈ チェックポイントを通る順番を $p_1 \rightarrow p_2 \rightarrow \dots \rightarrow p_N$ とする
- ✈ 1 回の変化で、ランダムに x と y を選んで p_x と p_y を交換

2 つの辺を swap する場合の実装 (reverse 操作)

- ✈ チェックポイントを通る順番を $p_1 \rightarrow p_2 \rightarrow \dots \rightarrow p_N$ とする
- ✈ 1 回の変化で、ランダムに x と y を選んで $p_x, p_{x+1}, \dots, p_y$ を逆順にする

方針 4 – 山登り法 ②



37 / 65

実験結果

入力データ	swap 操作	reverse 操作
01.txt	70.803 度	100.008 度
02.txt	102.910 度	115.290 度
03.txt	101.877 度	106.645 度
04.txt	118.428 度	139.830 度
05.txt	112.167 度	130.771 度
06.txt	111.941 度	137.443 度
得点	45 点	69 点

※ 注意： これらはひとつの実験結果にすぎません。

69 点獲得

方針 5 – 評価関数の工夫



38 / 65

評価値 にも一工夫できないか？

これまでの評価値

角度の最小値を評価値として
これが大きくなる改善を適用していた

新しい評価値

角度を小さい順にソートした最初 $K = 5$ 個を評価値として
これが大きくなる改善を適用してみよう

※「角度の合計値」を評価値とする場合など、スコアが悪化する評価値もあります。

方針 5 – 評価関数の工夫



39 / 65

実験結果 (初期解を貪欲法により得られた解とする・ $K = 5$ とする)

入力データ	最小値を評価	小さい K 個を評価
01.txt	91.822 度	97.812 度
02.txt	119.025 度	131.056 度
03.txt	104.667 度	122.552 度
04.txt	140.513 度	140.897 度
05.txt	132.003 度	140.442 度
06.txt	137.875 度	142.881 度
得点	68 点	80 点

※ 注意： これらはひとつの実験結果にすぎません。

80 点獲得



より良い答えを出すための課題

1 回の reverse 操作に $O(N)$ かかってしまう

評価値を求めるのに $O(N)$ かかってしまう

初期解を貪欲法で作っても、この結果がほとんど保存されない



より良い答えを出すための課題

1 回の reverse 操作に $O(N)$ かかってしまう

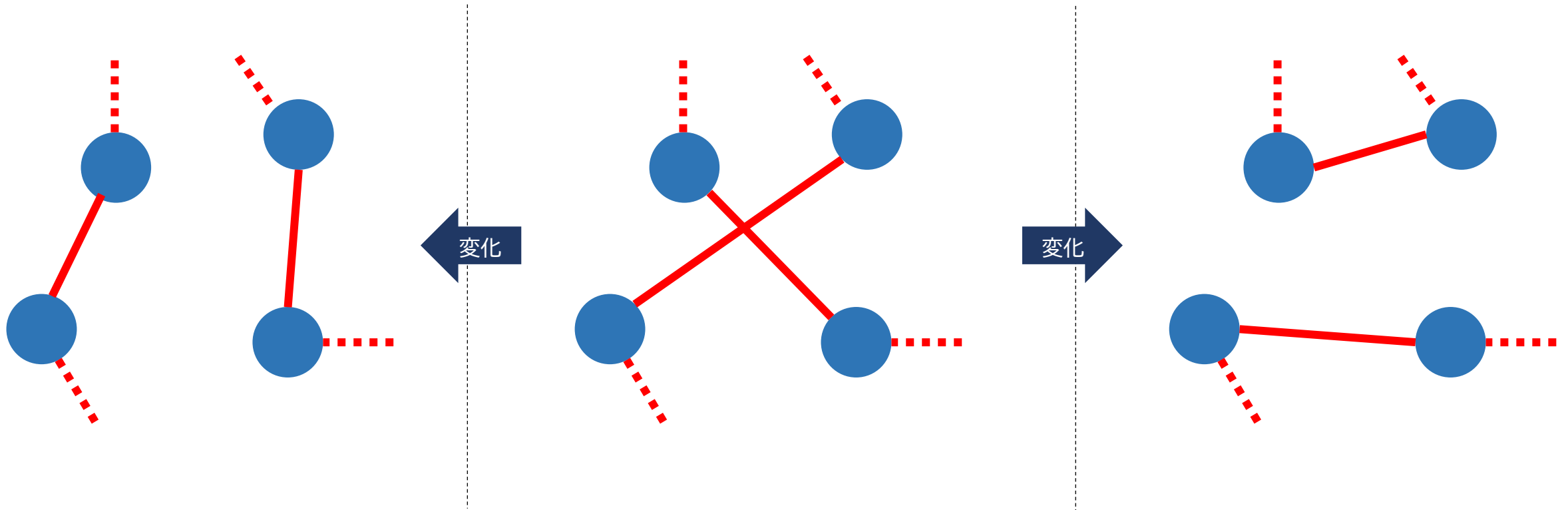
評価値を求めるのに $O(N)$ かかってしまう

初期解を貪欲法で作っても、この結果がほとんど保存されない



なぜ reverse 操作には $O(N)$ かかるのか？

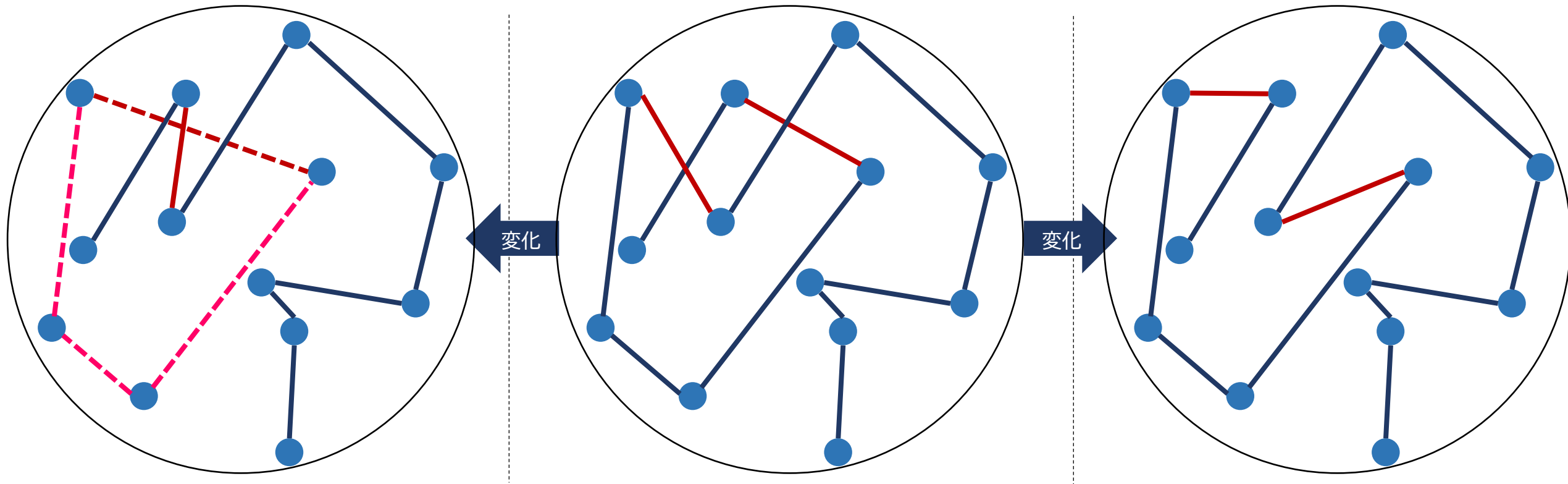
✈ 辺の交換の仕方には、以下の 2 通りがある





なぜ reverse 操作には $O(N)$ かかるのか？

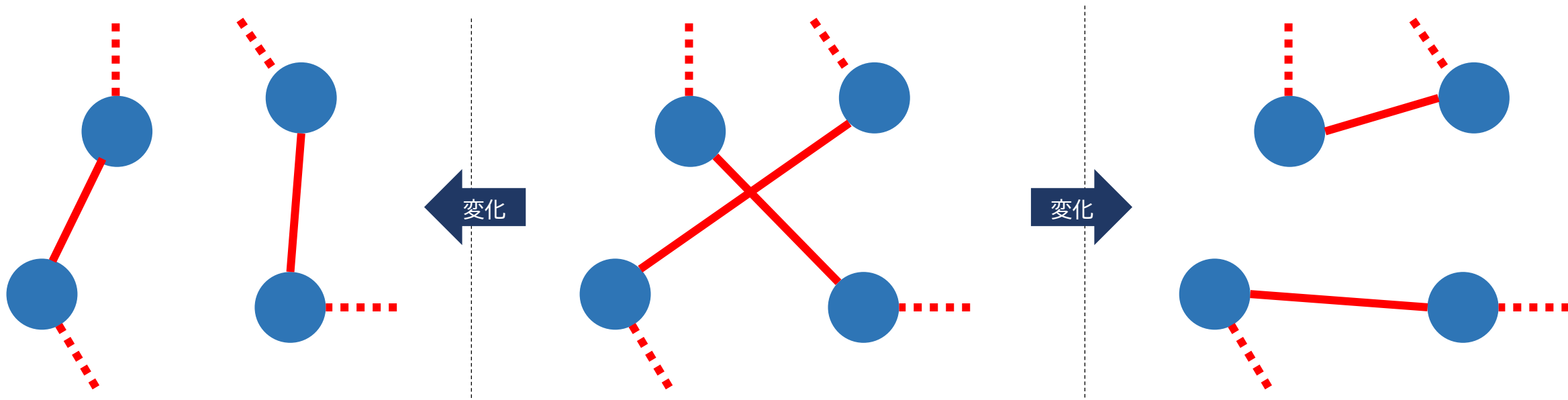
✈ このうち片方のやり方で **サイクル** ができてしまう





なぜ reverse 操作には $O(N)$ かかるのか？

- ✈ しかし、どちらの変化で サイクル が作られ、どちらの変化が正しいのか 判定するだけでも $O(N)$ かってしまう



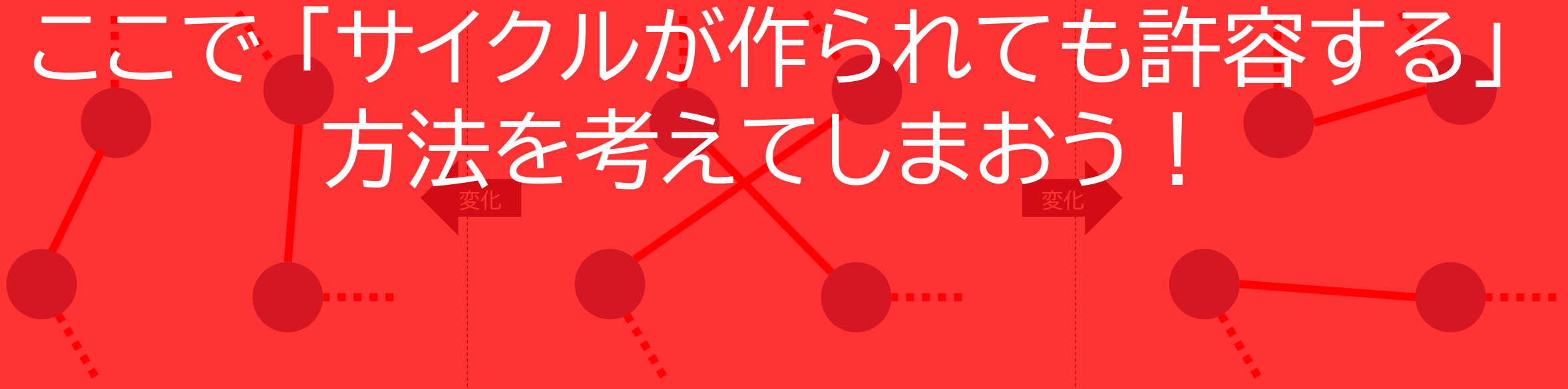
※ 平衡二分探索木などを用いると $O(\log N)$ でもできますが、 $N=200$ や 1000 では大きな高速化は見込めません。



なぜ reverse 操作には $O(N)$ かかるのか？

- ✈ しかし、どちらの変化で サイクル が作られ、どちらの変化が正しいのか 判定するだけでも $O(N)$ かってしまう

ここで「サイクルが作られても許容する」
方法を考えてしまおう！



※ 平衡二分探索木などを用いると $O(\log N)$ でもできますが、 $N=200$ や 1000 では大きな高速化は見込めません。



新しいアルゴリズム

✈ 2 つの辺を入れ替える変化で、途中でサイクルが作られるのも許容する

最終的に得られる解にサイクルが含まれている可能性もあるが…

☀ ほとんどの場合で、**サイクルの個数は 7 個くらい以下**になる
(完全ランダムな場合でも、個数の期待値は $\log_e N$ 個程度)

☀ 最小の角度を変えずに、2 辺の入れ替えで「1 個ずつサイクルを減らしていく」

→ かなりの確率で“復元”に成功する！



新しいアルゴリズム

✈ 2 つの辺を入れ替える変化で、途中でサイクルが作られるのも許容する

そのアイデアを使うと

最終的に得られる解にサイクルが含まれている可能性もあるが...

☀ 1 回の变化を定数時間で実現できる！
(完全ランダムな場合でも、個数の期待値は $\log_e N$ 個程度)

☀ 最小の角度を変えずに、2 辺の入れ替えで「1 個ずつサイクルを減らしていく」

→ かなりの確率で“復元”に成功する！

より良いスコアを求めて



48 / 65

より良い答えを出すための課題

1 回の reverse 操作に $O(N)$ かかってしまう

評価値を求めるのに $O(N)$ かかってしまう

初期解を貪欲法で作っても、この結果がほとんど保存されない

より良いスコアを求めて



49 / 65

現状の評価関数だと…

✈ 初期解を貪欲で生成すると、外周に近い頂点の角は 150° 以上のものも多い

しかし、最小値「だけ」を保存すると、 150° の角なんて評価に関係ないので…

元々良かった角が崩壊する

ということが起こってしまいます…

より良いスコアを求めて



50 / 65

評価関数に取り込みたい値

- ✈ 入力データごとに**基準値 θ** を決め打ちする
- ✈ 評価値には「基準値 θ 以上の角の個数」を入れてみたらどうか…？
- ✈ 全ての角が基準値 θ 以上になったらゲームクリア

今までの「最小の角度」みたいな評価値も残した方がよさそうだけど…

より良いスコアを求めて



51 / 65

新たな評価関数

基準値を θ / 各頂点の角度を $A_1, A_2, \dots, A_N$ とする

$$f(x) = \begin{cases} x - \theta & (x < \theta \text{ の場合}) \\ \text{complete_score} & (x \geq \theta \text{ の場合}) \end{cases} \quad \text{として}$$

評価値を $f(A_1) + f(A_2) + \dots + f(A_N)$ としてみよう

※ *complete_score* は 30~100 くらいに設定される定数

より良いスコアを求めて



52 / 65

新たな評価関数

基準値を θ / 各頂点の角度を $A_1, A_2, \dots, A_N$ とする

変わる角は最大で 4 個なので
評価値を定数時間で更新できる！

評価値を $f(A_1) + f(A_2) + \dots + f(A_N)$ としてみよう

※ `complete_score` は 30~100 くらいに設定される定数

より良いスコアを求めて



53 / 65

ここまでで手に入れた改善手法

1 回の reverse 操作は定数時間で実現できた

評価値も工夫すれば定数時間で求められる

初期解の良い部分は残した状態で解の改善がなされる

より良いスコアを求めて



54 / 65

新たな評価関数を使って解いてみよう！

解法 A

評価関数上がるか同じな場合、交換を適用する



解法 B

評価関数上がるか同じな場合、交換を適用する
評価関数がほんの少し落ちてても、確率的に交換を適用する
(焼きなまし法のようなアイデア)

※ ただし初期解は貪欲法で作るものとする

より良いスコアを求めて



55 / 65

解法 A の実験結果 (目安として 1 秒のループ回数は 400 万回・ループ回数制限は 4 億回)

入力データ	Z_0	θ	<i>complete_score</i>	スコア	ループ回数
01.txt	100°	100°	30	100.008 度	994
02.txt	143°	140°	50	140.135 度	24,921,224
03.txt	134°	132°	50	132.005 度	117,829,505
04.txt	156°	152°	50	152.004 度	218,769,334
05.txt	150°	147°	50	147.020 度	320,099,974
06.txt	153°	147°	100	147.007 度	369,905,056
得点				93 点	

※ 注意：これらはひとつの実験結果にすぎません。

93 点獲得

より良いスコアを求めて



56 / 65

解法 B の実験結果 (目安として 1 秒のループ回数は 400 万回・ループ回数制限は 4 億回)

入力データ	Z_0	θ	<i>complete_score</i>	スコア	ループ回数
01.txt	100°	100°	60	100.008 度	14,922
02.txt	143°	142°	80	142.014 度	370,739,181
03.txt	134°	134°	35	134.063 度	25,785,435
04.txt	156°	153°	60	153.003 度	325,223,062
05.txt	150°	147°	40	147.008 度	133,813,958
06.txt	153°	148°	100	148.000 度	110,789,454
得点				95 点	

※ 注意： これらはひとつの実験結果にすぎません。

95 点獲得



ギャラリー

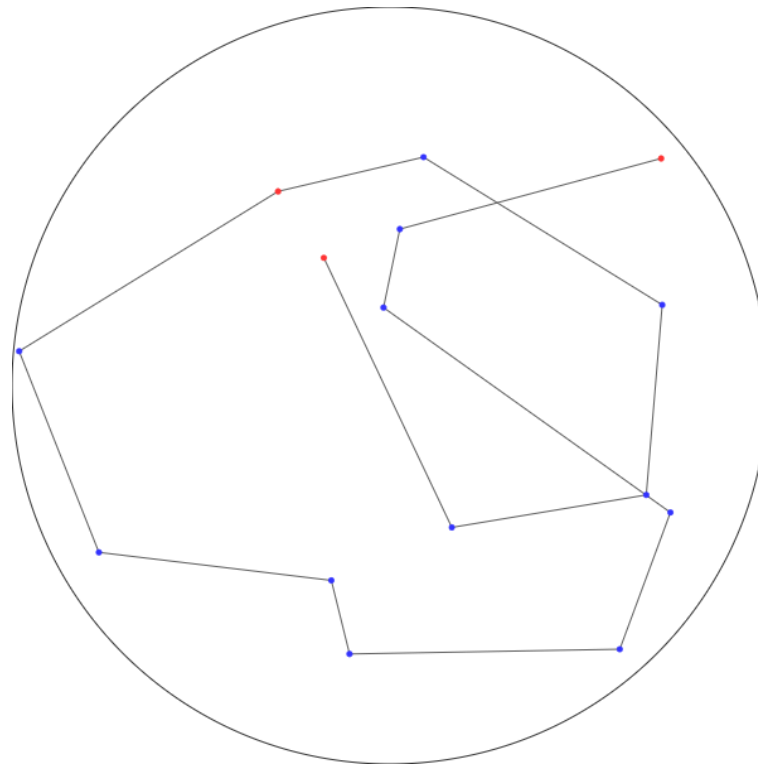
現在得られている最も良い解を
ビジュアライズしたものを紹介します

入力データ 1 (N = 15)



58 / 65

01.txt



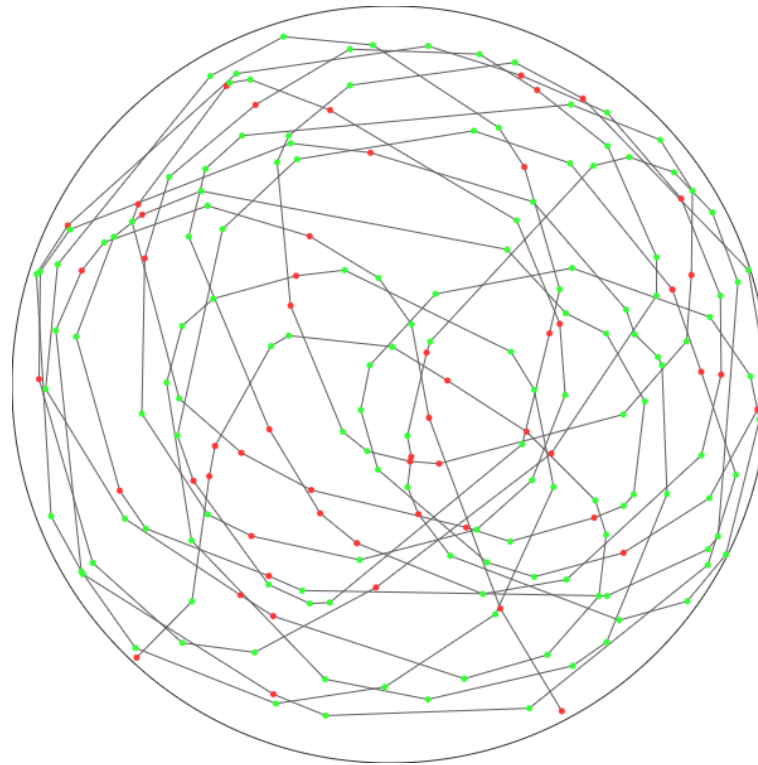
$Z = 100.008^\circ$ (最適解)

入力データ 2 (N = 200)



59 / 65

02.txt



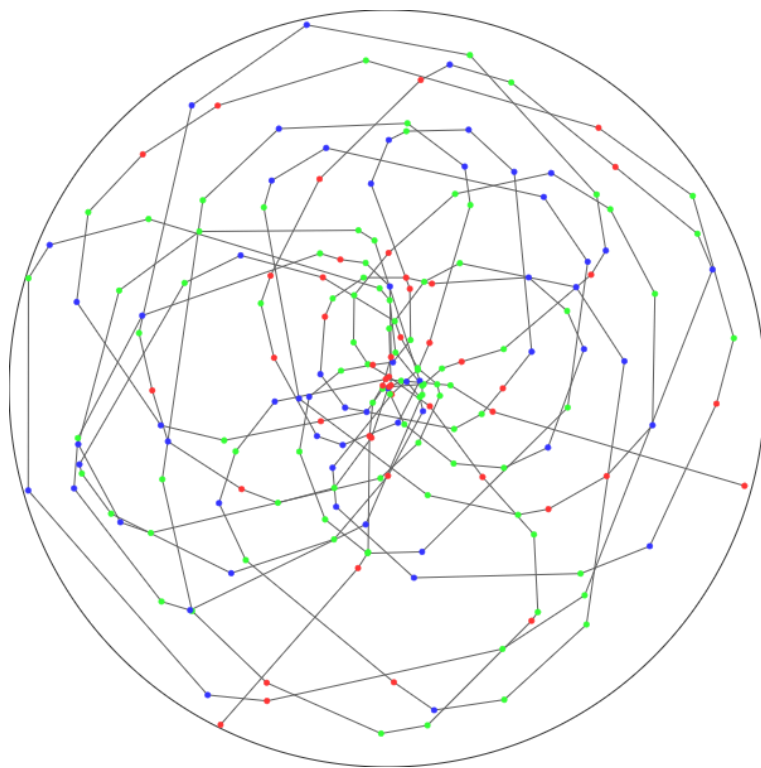
$$Z = 143.048^\circ$$

入力データ 3 (N = 200)



60 / 65

03.txt



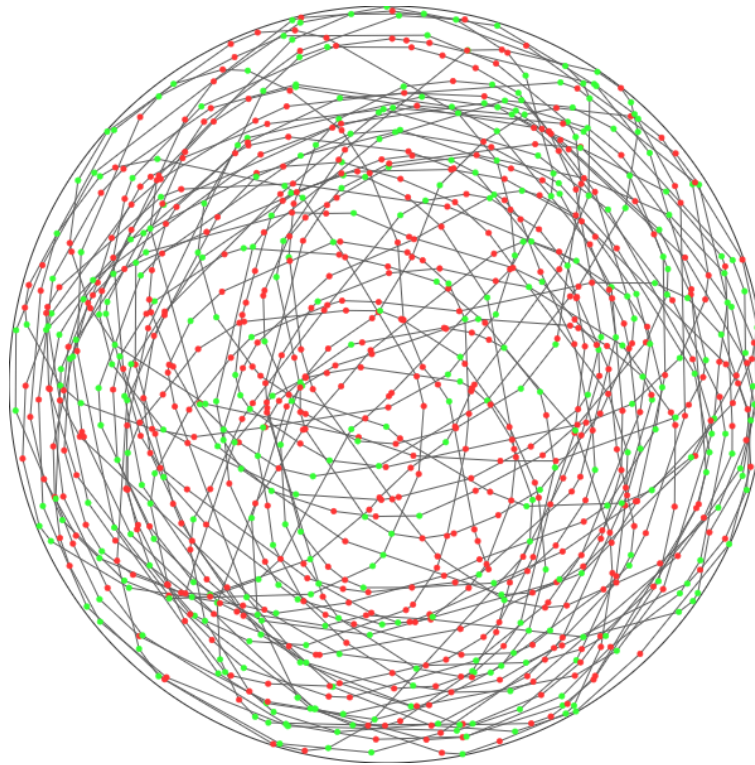
$Z = 134.554^\circ$ (最適解)

入力データ 4 (N = 1000)



61 / 65

04.txt



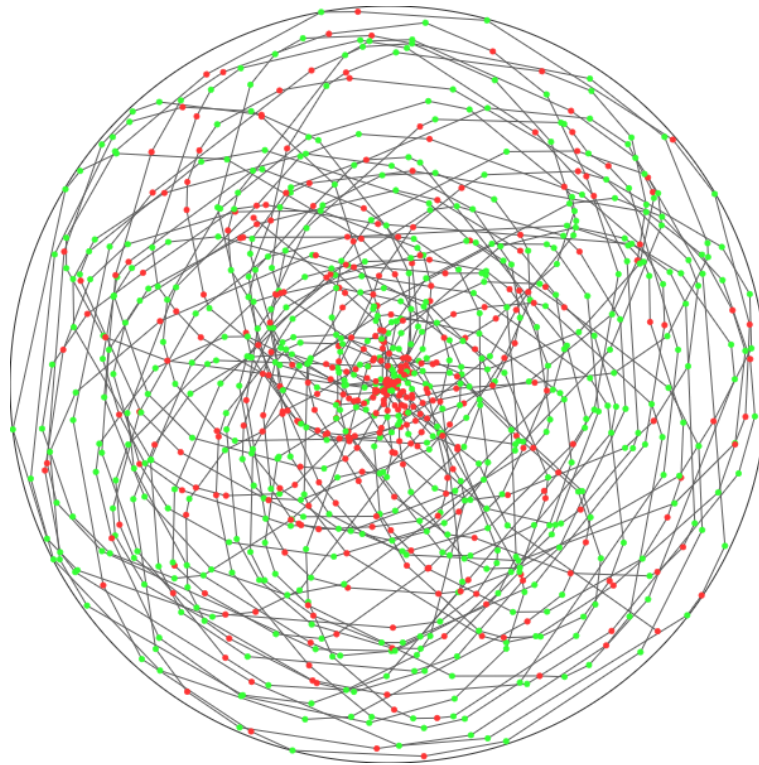
$$Z = 156.011^\circ$$

入力データ 5 (N = 1000)



62 / 65

05.txt



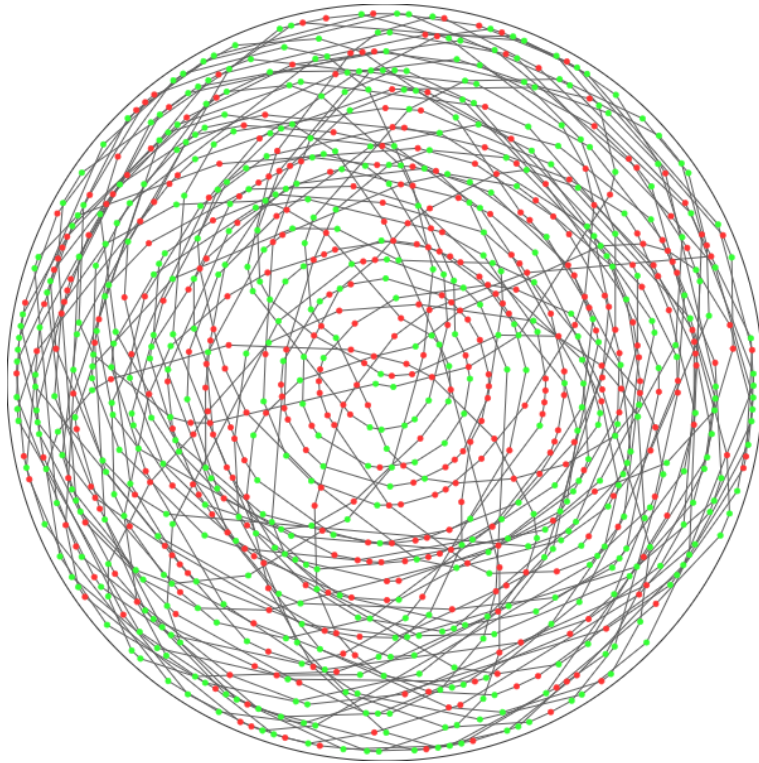
$$Z = 150.007^\circ$$

入力データ 6 (N = 1000)



63 / 65

06.txt



$$Z = 153.022^\circ$$

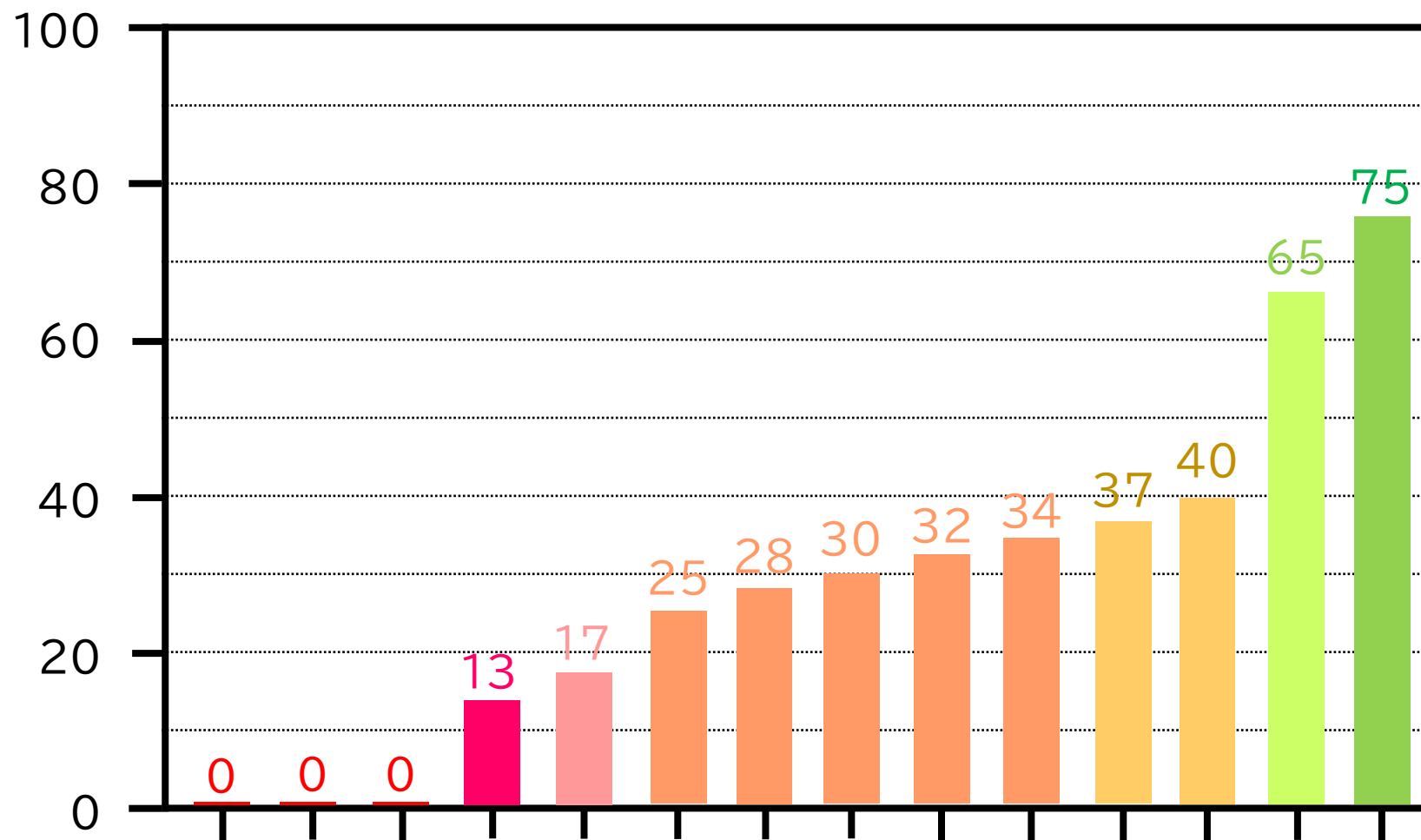


得点分布

得点分布



65 / 65



<統計データ>

最高点 75 点

最低点 0 点

平均点 28.3 点

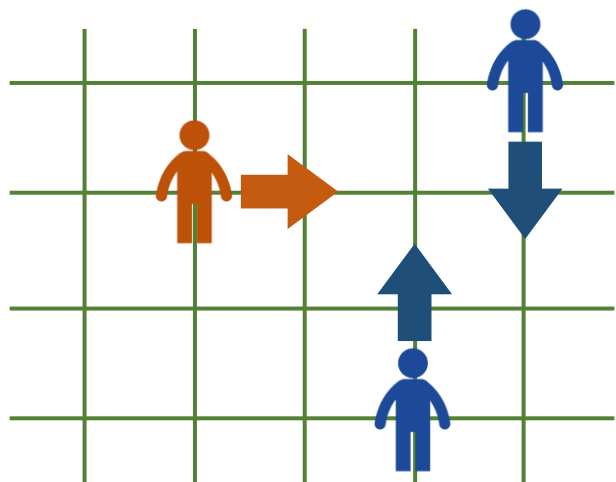
標準偏差 22.5 点

IOI 熱の感染拡大

解説：E869120

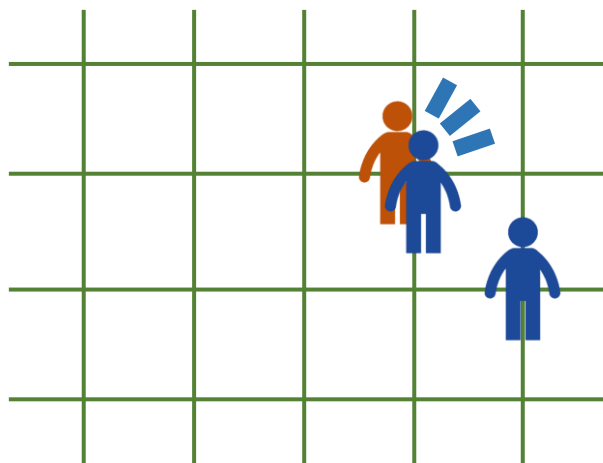
問題概要

1 最初は人 1 のみが感染



各人が 4 方向から 1 つ選び
選んだ方向に速度 1 で動く

2



感染者と同じ座標に来ると
新たに感染する

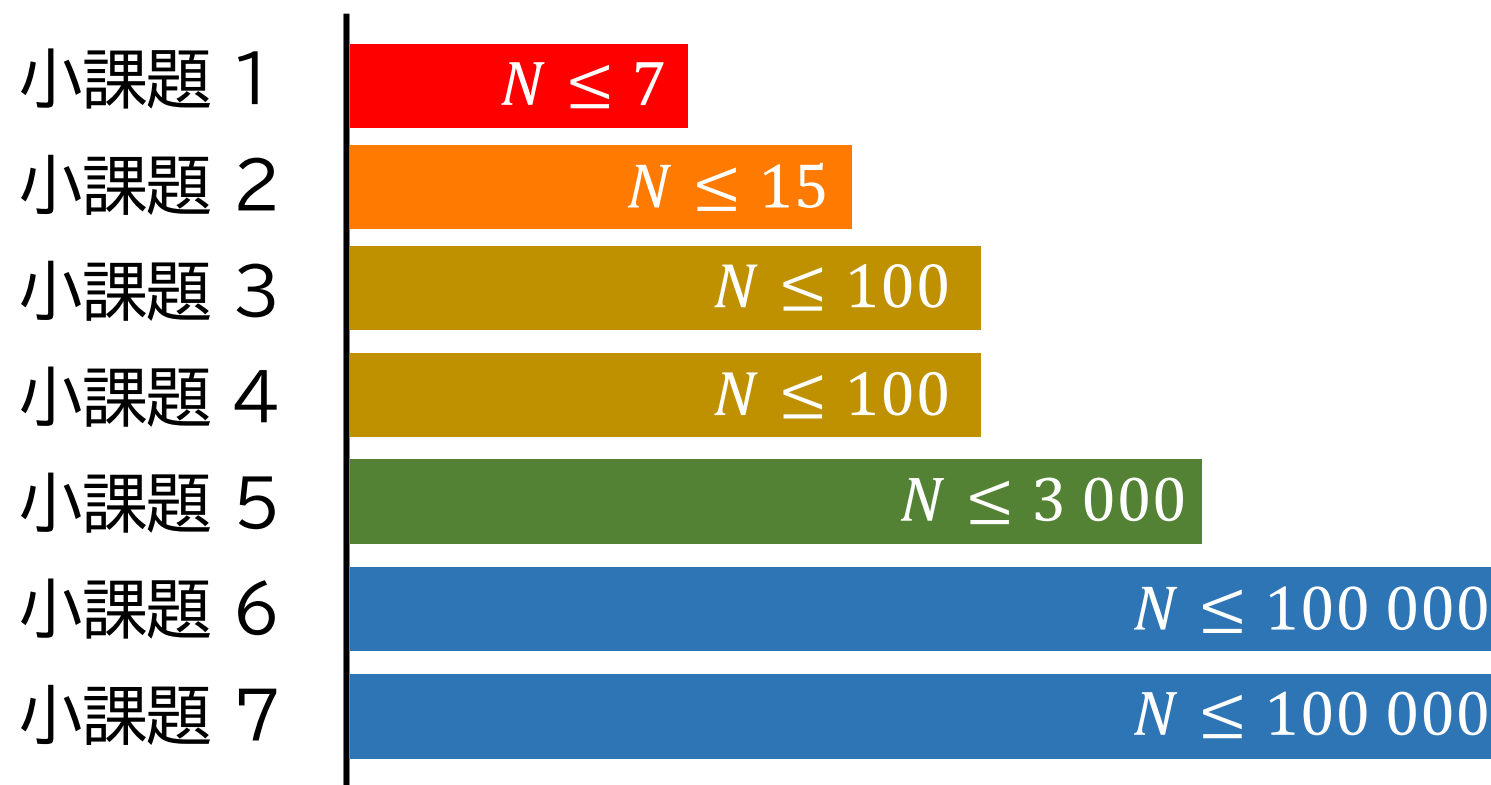
最終的な感染者数としてあり得る最大値は？

▶ 導入

- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (8点)
- ▶ 小課題 3 (6点)
- ▶ 小課題 4 (6点)
- ▶ 小課題 5 (12点)
- ▶ 小課題 6 (32点)
- ▶ 小課題 7 (31点)

問題概要

制約は以下の通り



※小課題 1, 2, 3, 4, 6 は $X_i \neq X_j, Y_i \neq Y_j$

※小課題 3 は $(X_0, Y_0) = (0, 0)$

▶ 導入

▶ 小課題 1 (5点)

▶ 小課題 2 (8点)

▶ 小課題 3 (6点)

▶ 小課題 4 (6点)

▶ 小課題 5 (12点)

▶ 小課題 6 (32点)

▶ 小課題 7 (31点)

解説の流れ

0

導入

4

小課題 4

1

小課題 1

5

小課題 5

2

小課題 2

6

小課題 6

3

小課題 3

7

満点

小課題 1

$N \leq 7$ のように制約が小さい場合...

全探索

をまず考えよう！

- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (8点)
- ▶ 小課題 3 (6点)
- ▶ 小課題 4 (6点)
- ▶ 小課題 5 (12点)
- ▶ 小課題 6 (32点)
- ▶ 小課題 7 (31点)

小課題 1 | 全探索

各国民が動く方法の数は何通り？

国民 1

「東」・「西」・「南」・「北」の 4 通り

国民 2

「東」・「西」・「南」・「北」の 4 通り

⋮

国民 N

「東」・「西」・「南」・「北」の 4 通り

全体の通り数は $4 \times 4 \times 4 \times \cdots \times 4 = 4^N$ 通り！

▶ 導入

▶ 小課題 1 (5点)

▶ 小課題 2 (8点)

▶ 小課題 3 (6点)

▶ 小課題 4 (6点)

▶ 小課題 5 (12点)

▶ 小課題 6 (32点)

▶ 小課題 7 (31点)

小課題 1 | 全探索

各国民が動く方法の数は何通り？

国民 1 「東」・「西」・「南」・「北」の 4 通り

国民 2 $4^7 = 16384$ 通りなので

全探索ができる

国民 N 「東」・「西」・「南」・「北」の 4 通り

全体の通り数は $4 \times 4 \times 4 \times \dots \times 4 = 4^N$ 通り！

- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (8点)
- ▶ 小課題 3 (6点)
- ▶ 小課題 4 (6点)
- ▶ 小課題 5 (12点)
- ▶ 小課題 6 (32点)
- ▶ 小課題 7 (31点)

小課題 1 | シミュレーションの方法

次の課題

- 各国民の移動方向が決まったとする
- どのように感染をシミュレーションするか？

問題点

- $X_i, Y_i \leq 500\,000\,000$
- 1 秒ごとにシミュレーションすることはできない

▶ 導入

▶ 小課題 1 (5点)

▶ 小課題 2 (8点)

▶ 小課題 3 (6点)

▶ 小課題 4 (6点)

▶ 小課題 5 (12点)

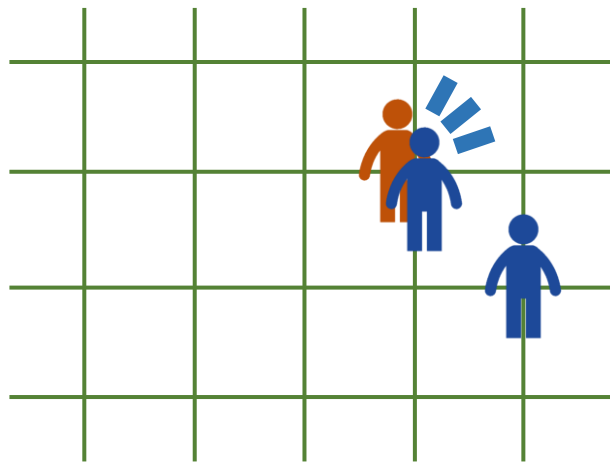
▶ 小課題 6 (32点)

▶ 小課題 7 (31点)

小課題 1 | シミュレーションの方法

重要な注意

以降のスライドでは、2 人の国民が同じ座標に来ることを「衝突」と書くことにします

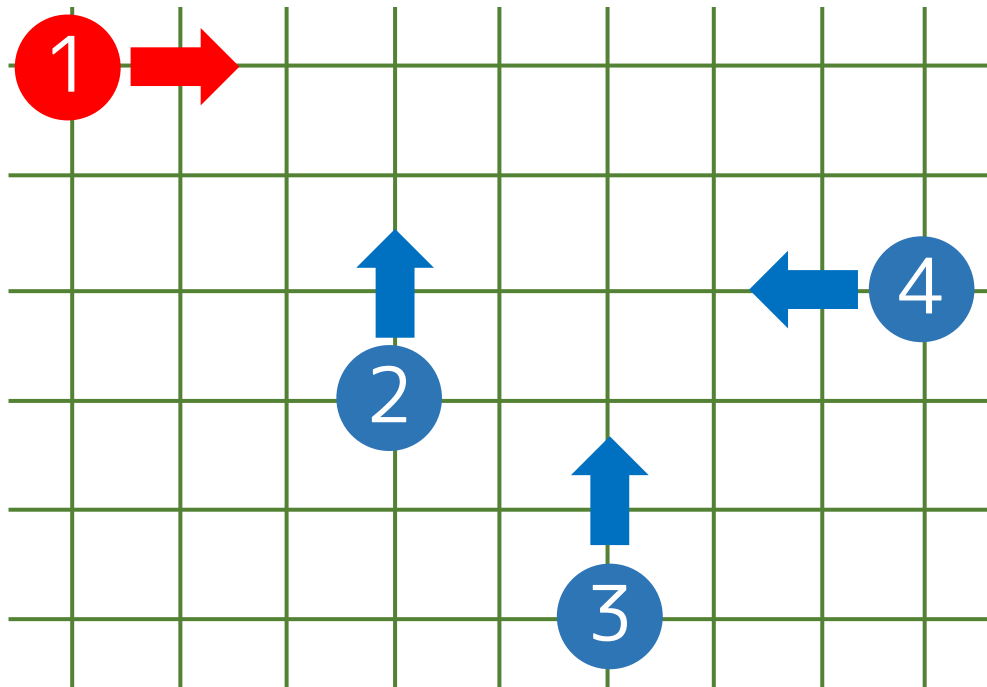


- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (8点)
- ▶ 小課題 3 (6点)
- ▶ 小課題 4 (6点)
- ▶ 小課題 5 (12点)
- ▶ 小課題 6 (32点)
- ▶ 小課題 7 (31点)

小課題 1 | シミュレーションの方法

「次に感染が発生する時刻」がいつか計算しよう！

例えば以下の場合



感染が起こり得る
国民の組

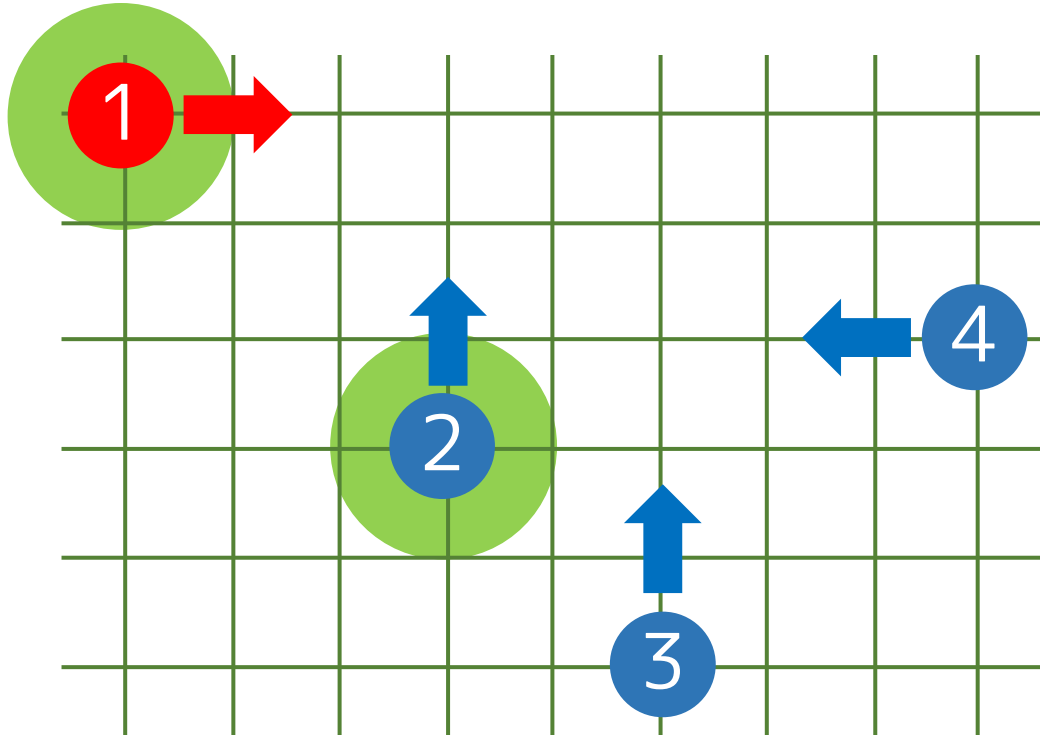
- (国民 1, 国民 2)
- (国民 1, 国民 3)
- (国民 2, 国民 3)

- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (8点)
- ▶ 小課題 3 (6点)
- ▶ 小課題 4 (6点)
- ▶ 小課題 5 (12点)
- ▶ 小課題 6 (32点)
- ▶ 小課題 7 (31点)

小課題 1 | シミュレーションの方法

「次に感染が発生する時刻」がいつか計算しよう！

例えば以下の場合



国民 1 と 2 は

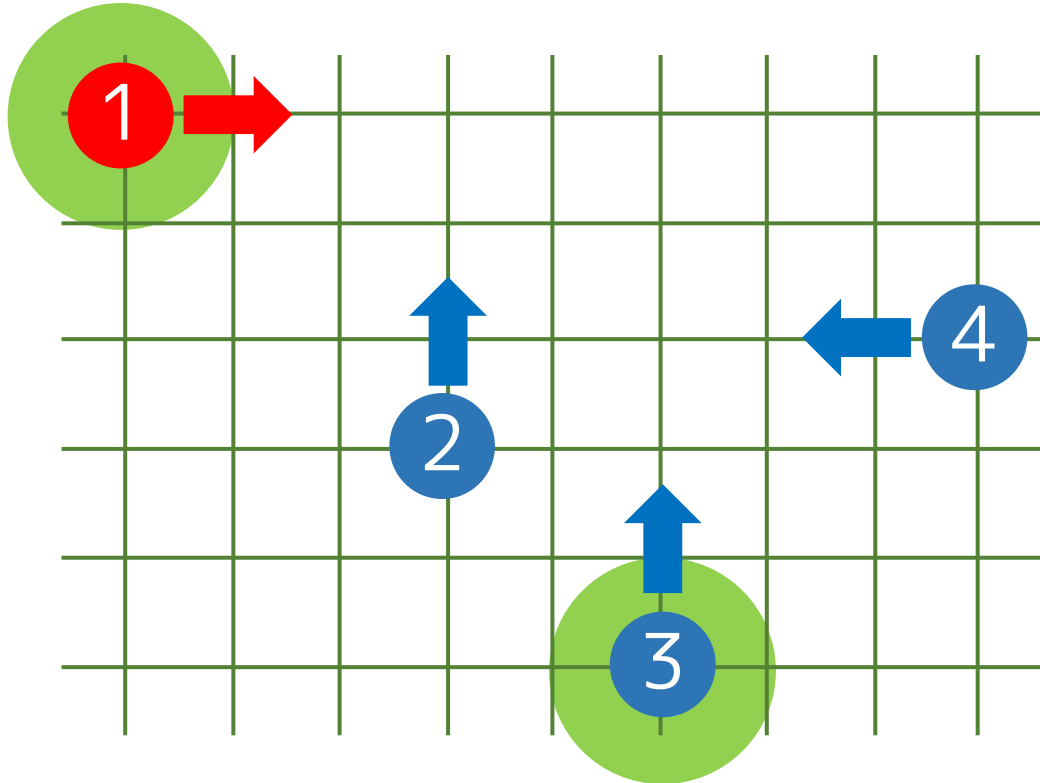
3秒後に衝突

- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (8点)
- ▶ 小課題 3 (6点)
- ▶ 小課題 4 (6点)
- ▶ 小課題 5 (12点)
- ▶ 小課題 6 (32点)
- ▶ 小課題 7 (31点)

小課題 1 | シミュレーションの方法

「次に感染が発生する時刻」がいつか計算しよう！

例えば以下の場合



国民 1 と 3 は

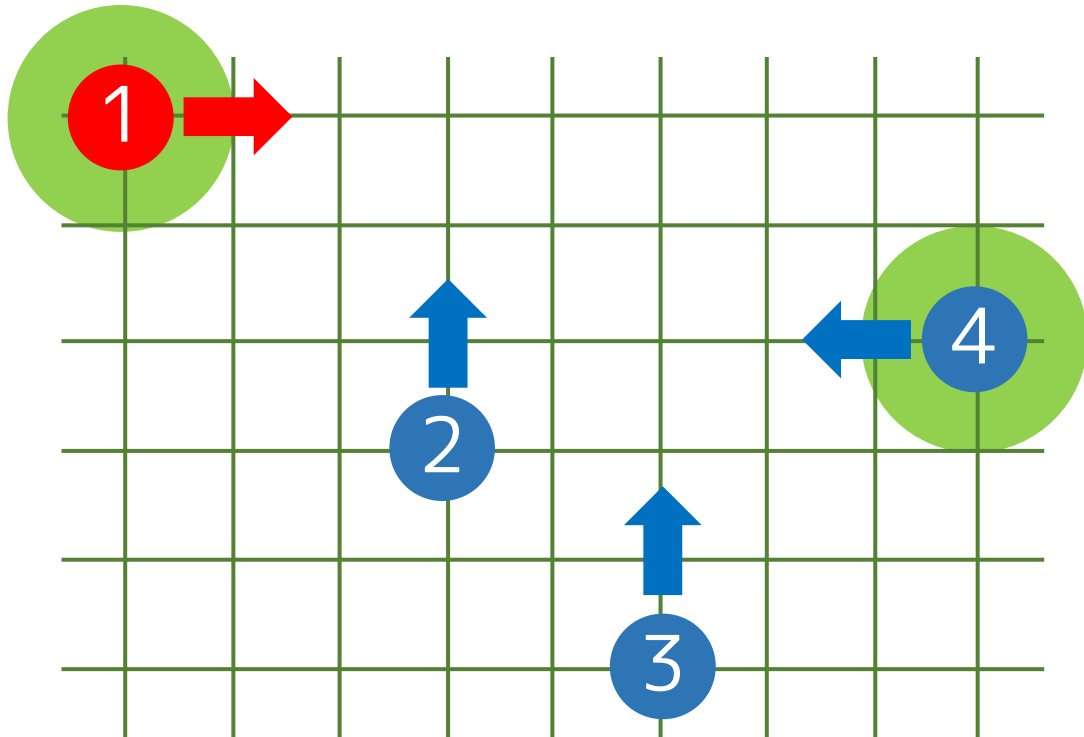
5秒後に衝突

- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (8点)
- ▶ 小課題 3 (6点)
- ▶ 小課題 4 (6点)
- ▶ 小課題 5 (12点)
- ▶ 小課題 6 (32点)
- ▶ 小課題 7 (31点)

小課題 1 | シミュレーションの方法

「次に感染が発生する時刻」がいつか計算しよう！

例えば以下の場合



国民 1 と 4 は
衝突しない

- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (8点)
- ▶ 小課題 3 (6点)
- ▶ 小課題 4 (6点)
- ▶ 小課題 5 (12点)
- ▶ 小課題 6 (32点)
- ▶ 小課題 7 (31点)

小課題 1 | シミュレーションの方法

「次に感染が発生する時刻」がいつか計算しよう！

例えば以下の場合



次の感染は

3 秒後に起こる

- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (8点)
- ▶ 小課題 3 (6点)
- ▶ 小課題 4 (6点)
- ▶ 小課題 5 (12点)
- ▶ 小課題 6 (32点)
- ▶ 小課題 7 (31点)

小課題 1 | シミュレーションの方法

そのように全ペアを調べれば、次に感染が発生する時刻を $O(N^2)$ で求めることができる

考察

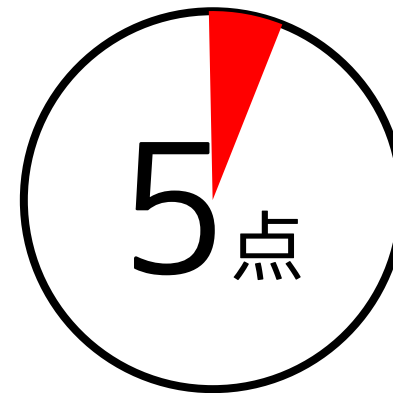
新たな感染が発生する回数は高々 $N - 1$ 回

計算量

$$O(N^2) \times (N - 1) = O(N^3)$$

全体

$$4^N \text{ 通り} \times O(N^3) = O(4^N \times N^3)$$



▶ 導入

▶ 小課題 1 (5点)

▶ 小課題 2 (8点)

▶ 小課題 3 (6点)

▶ 小課題 4 (6点)

▶ 小課題 5 (12点)

▶ 小課題 6 (32点)

▶ 小課題 7 (31点)

解説の流れ

0

導入

1

小課題 1

2

小課題 2

3

小課題 3

4

小課題 4

5

小課題 5

6

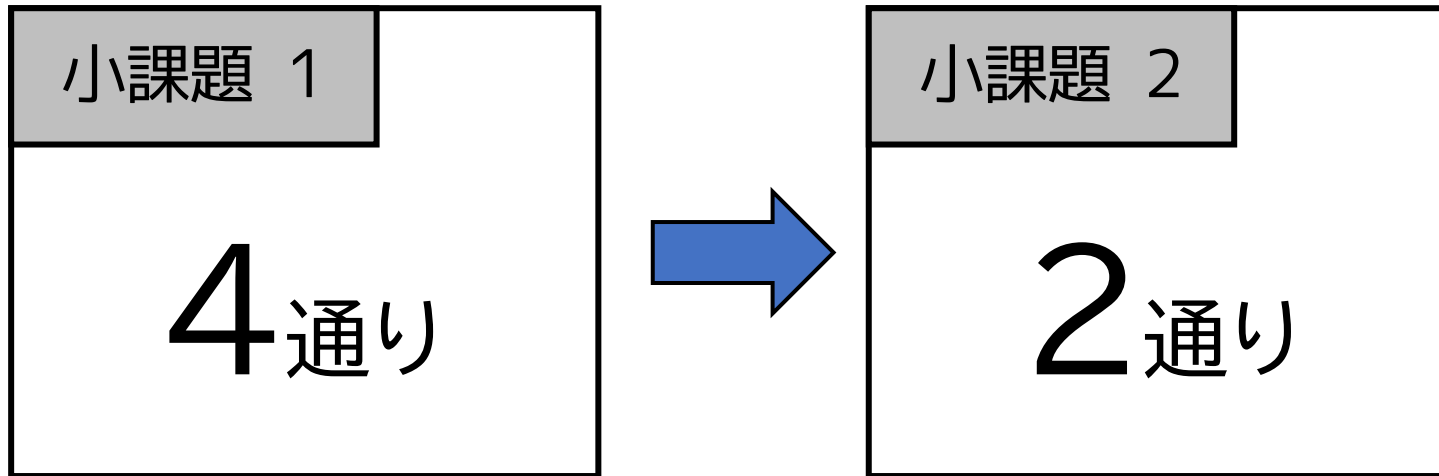
小課題 6

7

満点

小課題 2

- $N \leq 14$
- 各国民の進む方法を以下の通りに絞れないか？

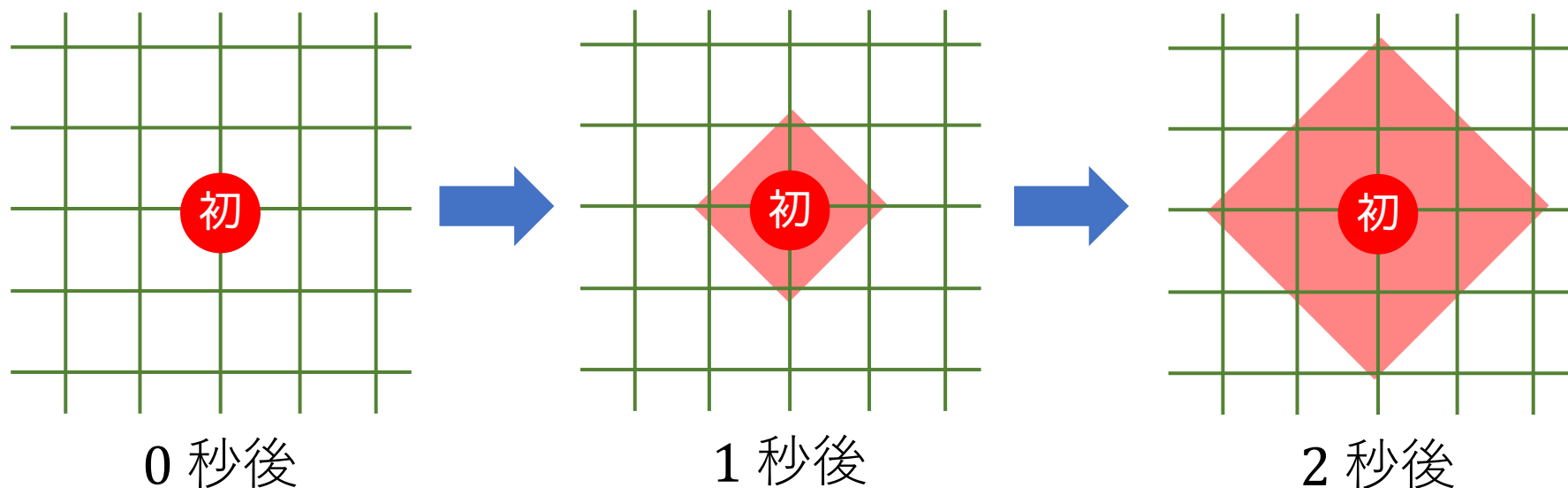


- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (8点)
- ▶ 小課題 3 (6点)
- ▶ 小課題 4 (6点)
- ▶ 小課題 5 (12点)
- ▶ 小課題 6 (32点)
- ▶ 小課題 7 (31点)

小課題 2 | 重要な考察

重要な性質

t 秒後には、国民 1 初期位置からのマンハッタン距離が t 以下の場所までしか感染しない



■ : 感染し得る範囲

- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (8点)
- ▶ 小課題 3 (6点)
- ▶ 小課題 4 (6点)
- ▶ 小課題 5 (12点)
- ▶ 小課題 6 (32点)
- ▶ 小課題 7 (31点)

小課題 2 | 重要な考察

問い

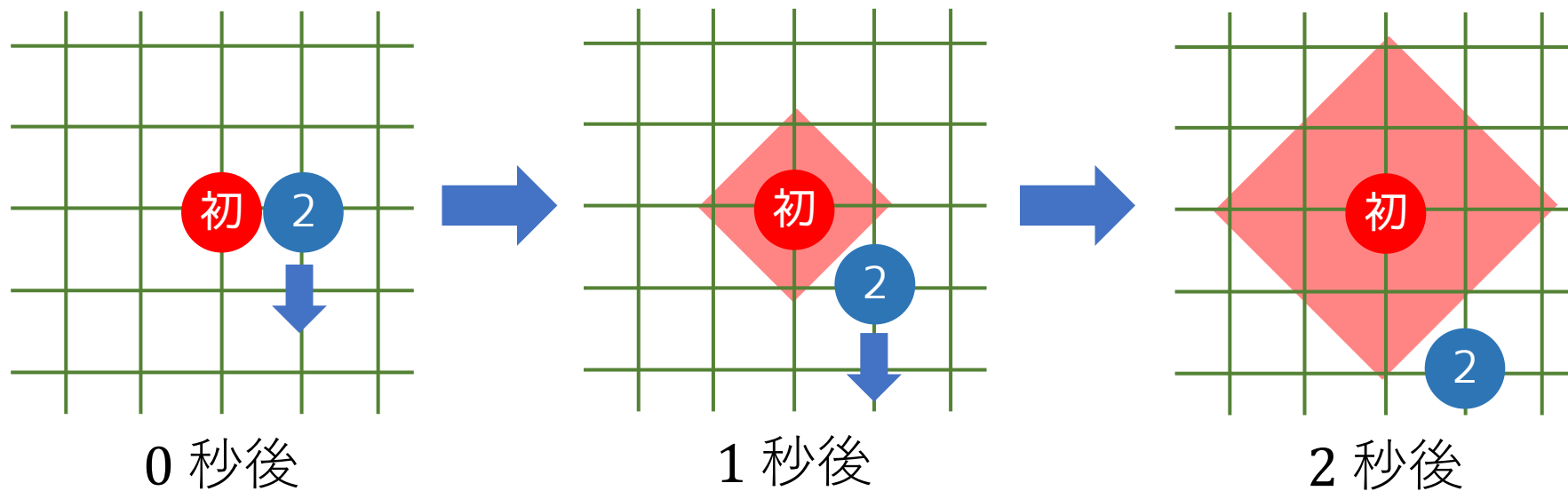
国民 1 の初期位置から離れる方向に
移動したらどうなる？

- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (8点)
- ▶ 小課題 3 (6点)
- ▶ 小課題 4 (6点)
- ▶ 小課題 5 (12点)
- ▶ 小課題 6 (32点)
- ▶ 小課題 7 (31点)

小課題 2 | 重要な考察

問い

国民 1 の初期位置から離れる方向に
移動したらどうなる？



- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (8点)
- ▶ 小課題 3 (6点)
- ▶ 小課題 4 (6点)
- ▶ 小課題 5 (12点)
- ▶ 小課題 6 (32点)
- ▶ 小課題 7 (31点)

小課題 2 | 重要な考察

問い

国民 1 の初期位置から離れる方向に

移動した 国民 1 初期位置からの

マンハッタン距離が 1 ずつ大きくなるため

永遠に感染範囲に入らない！

0 秒後

1 秒後

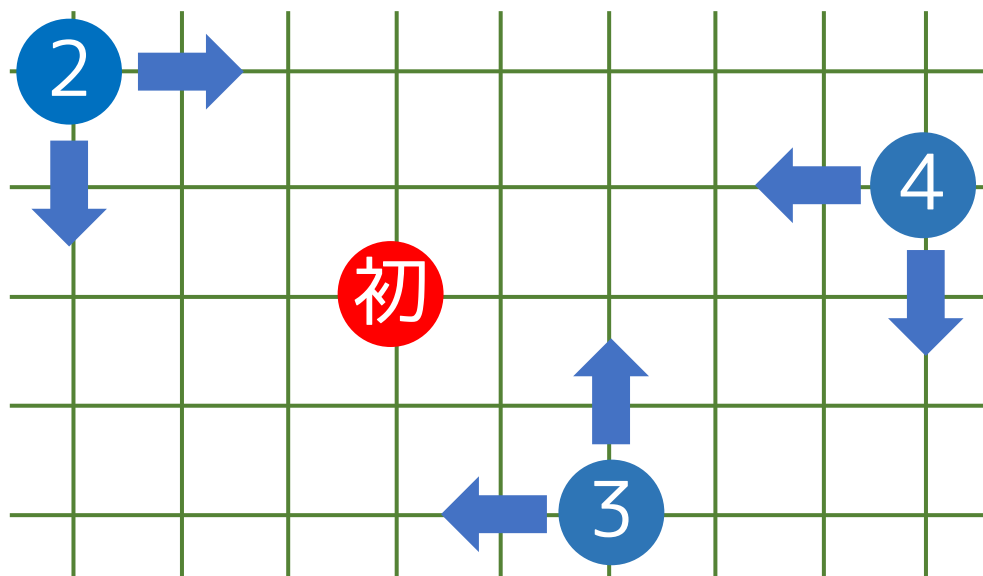
2 秒後

- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (8点)
- ▶ 小課題 3 (6点)
- ▶ 小課題 4 (6点)
- ▶ 小課題 5 (12点)
- ▶ 小課題 6 (32点)
- ▶ 小課題 7 (31点)

小課題 2 | 通り数を絞り込め！

結論

各国民は、**国民 1 に近づく方向**にしか移動しない
さもなければ永遠に感染しない



- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (8点)
- ▶ 小課題 3 (6点)
- ▶ 小課題 4 (6点)
- ▶ 小課題 5 (12点)
- ▶ 小課題 6 (32点)
- ▶ 小課題 7 (31点)

小課題 2 | 通り数を絞り込め！

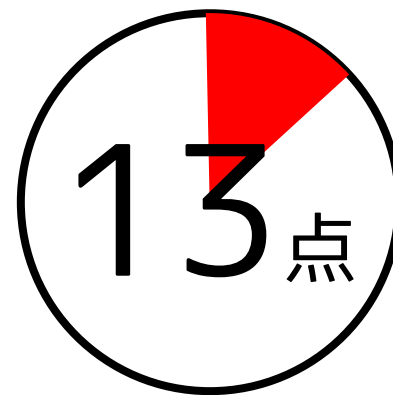
(X_1, Y_1) に近づくような移動方法は高々 2 通り

よって、考えるべき向きの組合せは 2^N 通り

シミュレーションは、小課題 1 で述べた通り $O(N^3)$

→ 全体計算量 $O(2^N \times N^3)$

→ $N \leq 15$ で間に合う！



- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (8点)
- ▶ 小課題 3 (6点)
- ▶ 小課題 4 (6点)
- ▶ 小課題 5 (12点)
- ▶ 小課題 6 (32点)
- ▶ 小課題 7 (31点)

解説の流れ

0

導入

1

小課題 1

2

小課題 2

3

小課題 3

4

小課題 4

5

小課題 5

6

小課題 6

7

満点

小課題 3

- $N \leq 100$
- そもそも指数時間アルゴリズムは通用しない…

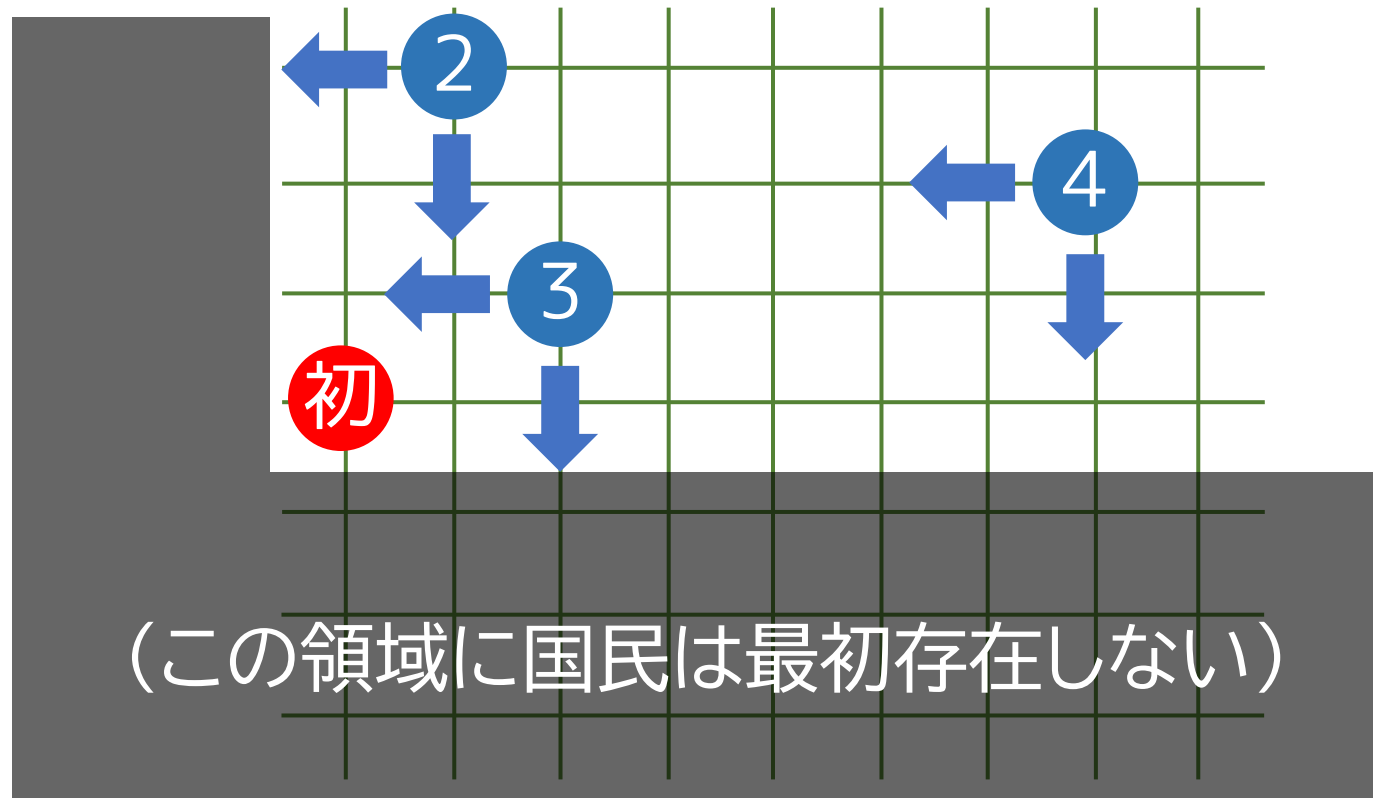
$(X_1, Y_1) = (0, 0)$ の場合を考えよう

※実際はこのような小課題が無くても、特殊ケースを考えることは多くの問題の考察において重要です

- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (8点)
- ▶ 小課題 3 (6点)
- ▶ 小課題 4 (6点)
- ▶ 小課題 5 (12点)
- ▶ 小課題 6 (32点)
- ▶ 小課題 7 (31点)

小課題 3 | $(X_1, Y_1) = (0, 0)$ のとき

このケースの場合、国民 2, 3, ..., N は
下か左に動くのが最適



- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (8点)
- ▶ 小課題 3 (6点)
- ▶ 小課題 4 (6点)
- ▶ 小課題 5 (12点)
- ▶ 小課題 6 (32点)
- ▶ 小課題 7 (31点)

小課題 3 | $(X_1, Y_1) = (0, 0)$ のとき

再掲

t 秒後には、国民 1 初期位置からのマンハッタン距離が t 以下の場所までしか感染しない

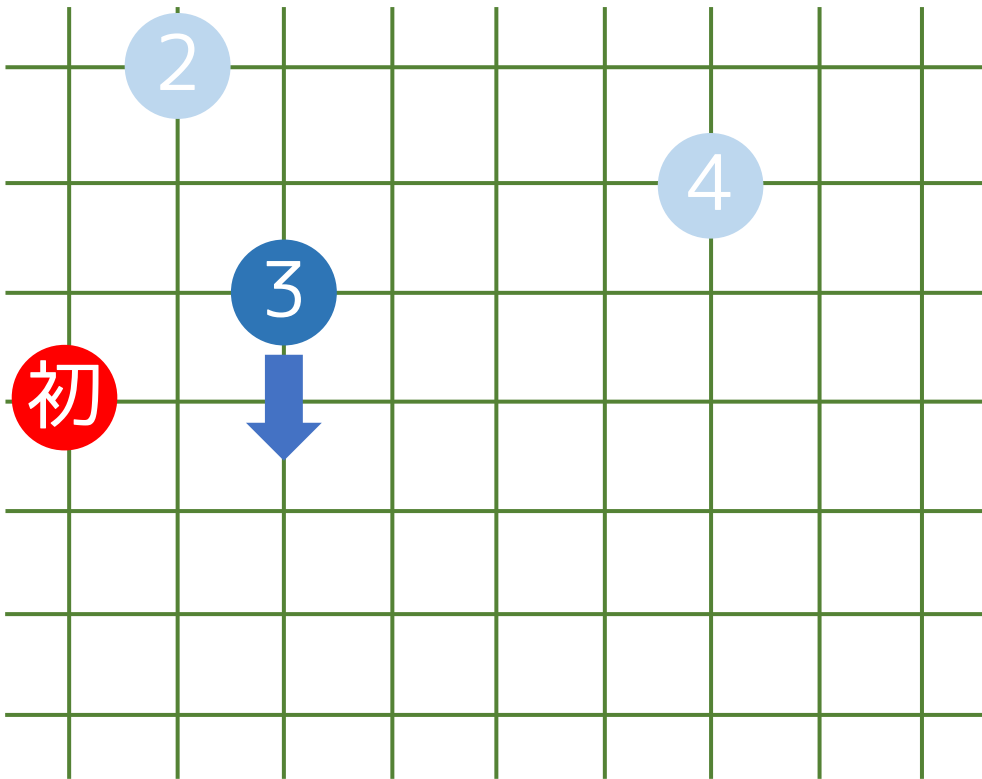
次のスライド以降ではこの性質を使います

- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (8点)
- ▶ 小課題 3 (6点)
- ▶ 小課題 4 (6点)
- ▶ 小課題 5 (12点)
- ▶ 小課題 6 (32点)
- ▶ 小課題 7 (31点)

小課題 3 | $(X_1, Y_1) = (0, 0)$ のとき

問い

国民 3 が下に動いた場合どうなるか？



時刻

0

(X_1, Y_1) からの
マンハッタン距離

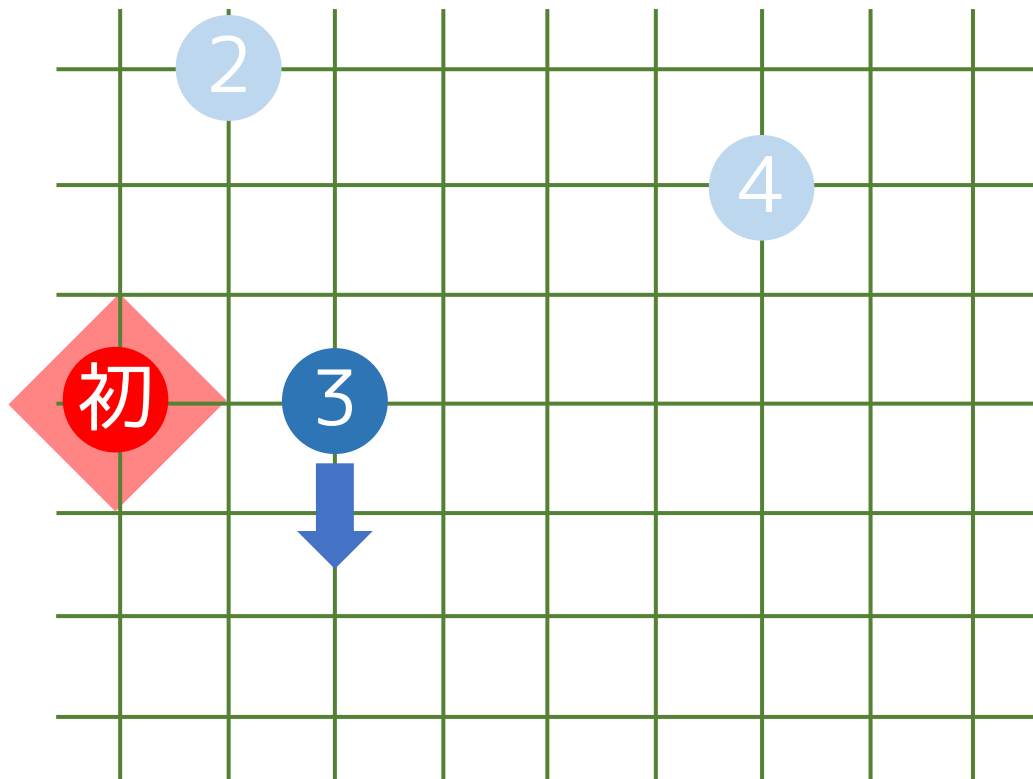
3

- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (8点)
- ▶ 小課題 3 (6点)
- ▶ 小課題 4 (6点)
- ▶ 小課題 5 (12点)
- ▶ 小課題 6 (32点)
- ▶ 小課題 7 (31点)

小課題 3 | $(X_1, Y_1) = (0, 0)$ のとき

問い

国民 3 が下に動いた場合どうなるか？



時刻

1

(X_1, Y_1) からの
マンハッタン距離

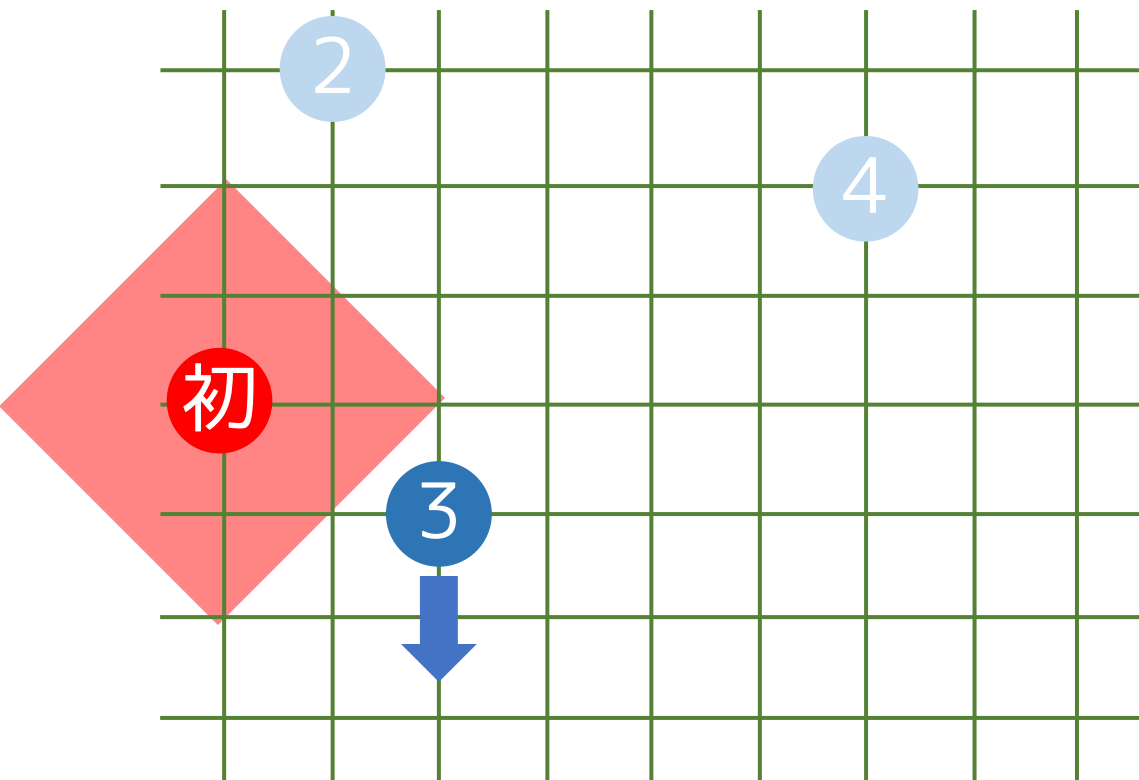
2

- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (8点)
- ▶ 小課題 3 (6点)
- ▶ 小課題 4 (6点)
- ▶ 小課題 5 (12点)
- ▶ 小課題 6 (32点)
- ▶ 小課題 7 (31点)

小課題 3 | $(X_1, Y_1) = (0, 0)$ のとき

問い

国民 3 が下に動いた場合どうなるか？



時刻

2

(X_1, Y_1) からの
マンハッタン距離

3

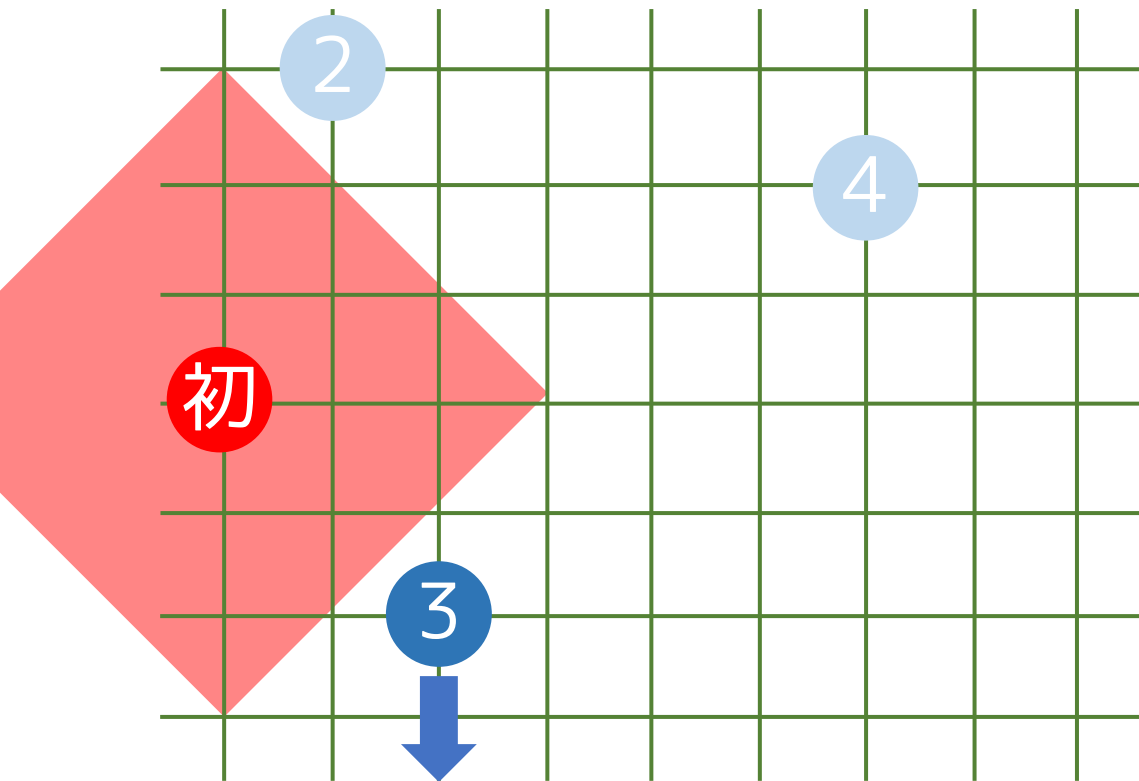
- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (8点)
- ▶ 小課題 3 (6点)
- ▶ 小課題 4 (6点)
- ▶ 小課題 5 (12点)
- ▶ 小課題 6 (32点)
- ▶ 小課題 7 (31点)

30

小課題 3 | $(X_1, Y_1) = (0, 0)$ のとき

問い

国民 3 が下に動いた場合どうなるか？



時刻

3

(X_1, Y_1) からの
マンハッタン距離

4

- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (8点)
- ▶ 小課題 3 (6点)
- ▶ 小課題 4 (6点)
- ▶ 小課題 5 (12点)
- ▶ 小課題 6 (32点)
- ▶ 小課題 7 (31点)

小課題 3 | $(X_1, Y_1) = (0, 0)$ のとき

問い

国民 3 が下に動いた場合どうなるか？

国民 1 初期位置からの
マンハッタン距離が 1 ずつ大きくなるため
結局永遠に感染範囲に入らない！

- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (8点)
- ▶ 小課題 3 (6点)
- ▶ 小課題 4 (6点)
- ▶ 小課題 5 (12点)
- ▶ 小課題 6 (32点)
- ▶ 小課題 7 (31点)

小課題 3 | 感染範囲に入る条件は？

条件の整理

同様に考えると、 $(X_1, Y_1) = (0, 0)$ のとき、

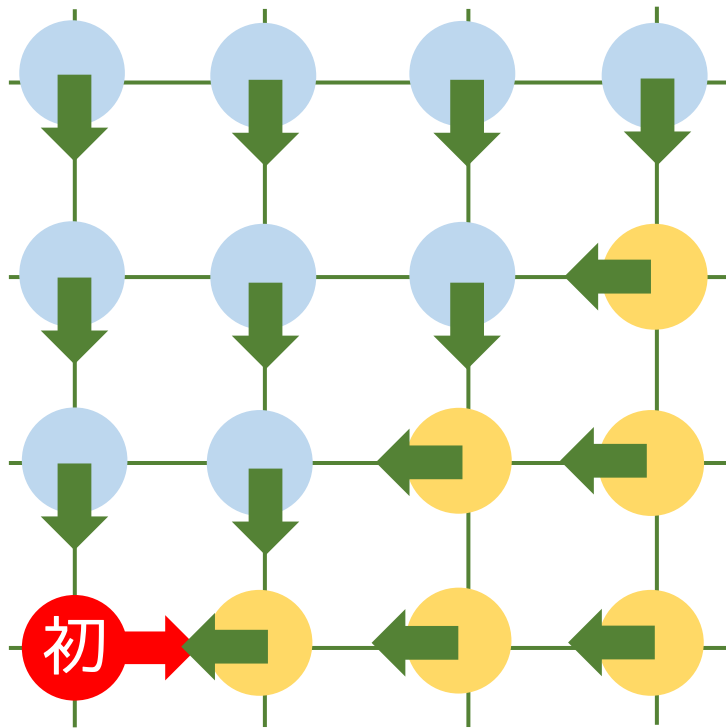
- $X_i > Y_i$: 左方向
- $X_i < Y_i$: 下方向
- $X_i = Y_i$: 左と下のうち、国民 1 の移動方向と垂直な方向

さもないければ国民 i は永遠に感染しない

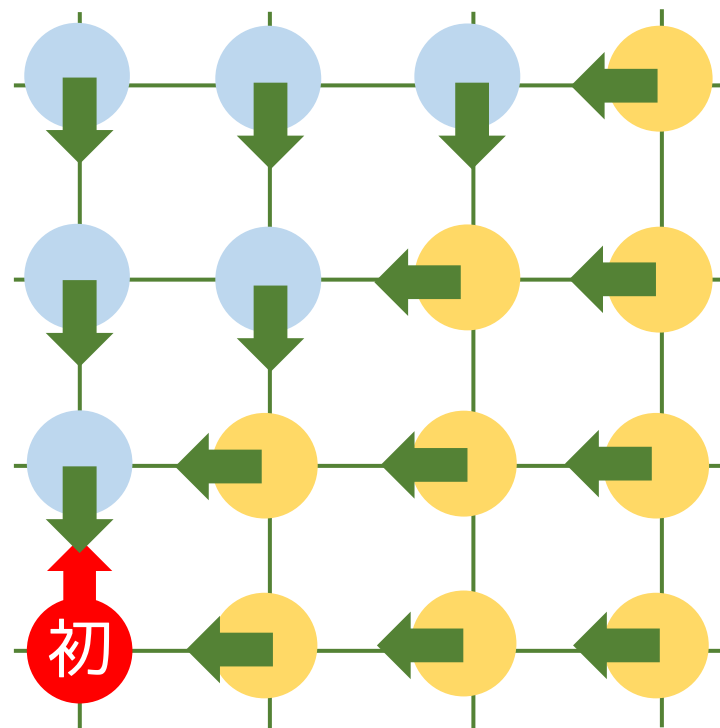
- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (8点)
- ▶ 小課題 3 (6点)
- ▶ 小課題 4 (6点)
- ▶ 小課題 5 (12点)
- ▶ 小課題 6 (32点)
- ▶ 小課題 7 (31点)

小課題 3 | 感染範囲に入る条件は？

つまりそういうこと



国民 1 が右方向の場合



国民 1 が上方向の場合

- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (8点)
- ▶ 小課題 3 (6点)
- ▶ 小課題 4 (6点)
- ▶ 小課題 5 (12点)
- ▶ 小課題 6 (32点)
- ▶ 小課題 7 (31点)

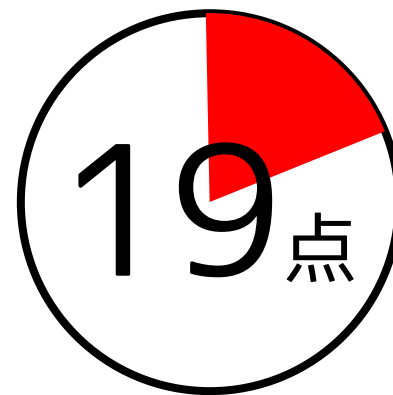
小課題 3 | あとはシミュレーション

そうすると、国民 1 の向きが決まった場合、**他のすべての国民の向きが決まる**

本小課題では、国民 1 の向きは**上か右**しか考えられないので、シミュレーションすべき通り数は高々 2 通り

小課題 1 の通りに実装すると

計算量は $2 \times O(N^3) = O(N^3)$



- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (8点)
- ▶ 小課題 3 (6点)
- ▶ 小課題 4 (6点)
- ▶ 小課題 5 (12点)
- ▶ 小課題 6 (32点)
- ▶ 小課題 7 (31点)

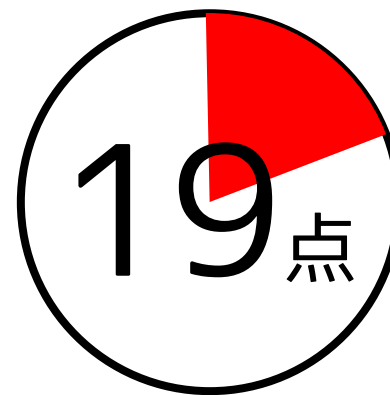
小課題 3 | 補足

重要な性質

この小課題のテストケースでは、

- $X_i \neq Y_i$: 絶対感染しない
- $X_i = Y_i$: 国民 1 と垂直な方向に動けば感染する

よって、 $X_i = Y_i$ ($2 \leq i \leq N$) となる
 i の個数が答え (実装 10 行)



- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (8点)
- ▶ 小課題 3 (6点)
- ▶ 小課題 4 (6点)
- ▶ 小課題 5 (12点)
- ▶ 小課題 6 (32点)
- ▶ 小課題 7 (31点)

解説の流れ

0

導入

1

小課題 1

2

小課題 2

3

小課題 3

4

小課題 4

5

小課題 5

6

小課題 6

7

満点

小課題 4

問い

- 小課題 3 で紹介した性質を拡張できないか？
- 国民 1 の移動方法が決まったら他の国民の移動方法も全部決まるのではないか？

- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (8点)
- ▶ 小課題 3 (6点)
- ▶ 小課題 4 (6点)
- ▶ 小課題 5 (12点)
- ▶ 小課題 6 (32点)
- ▶ 小課題 7 (31点)

小課題 4

問い 3 で紹介した性質を拡張できないか？

- ・国民 1 の移動方法が決まったら他の国民の移動方法も全部決まるのではないかな？

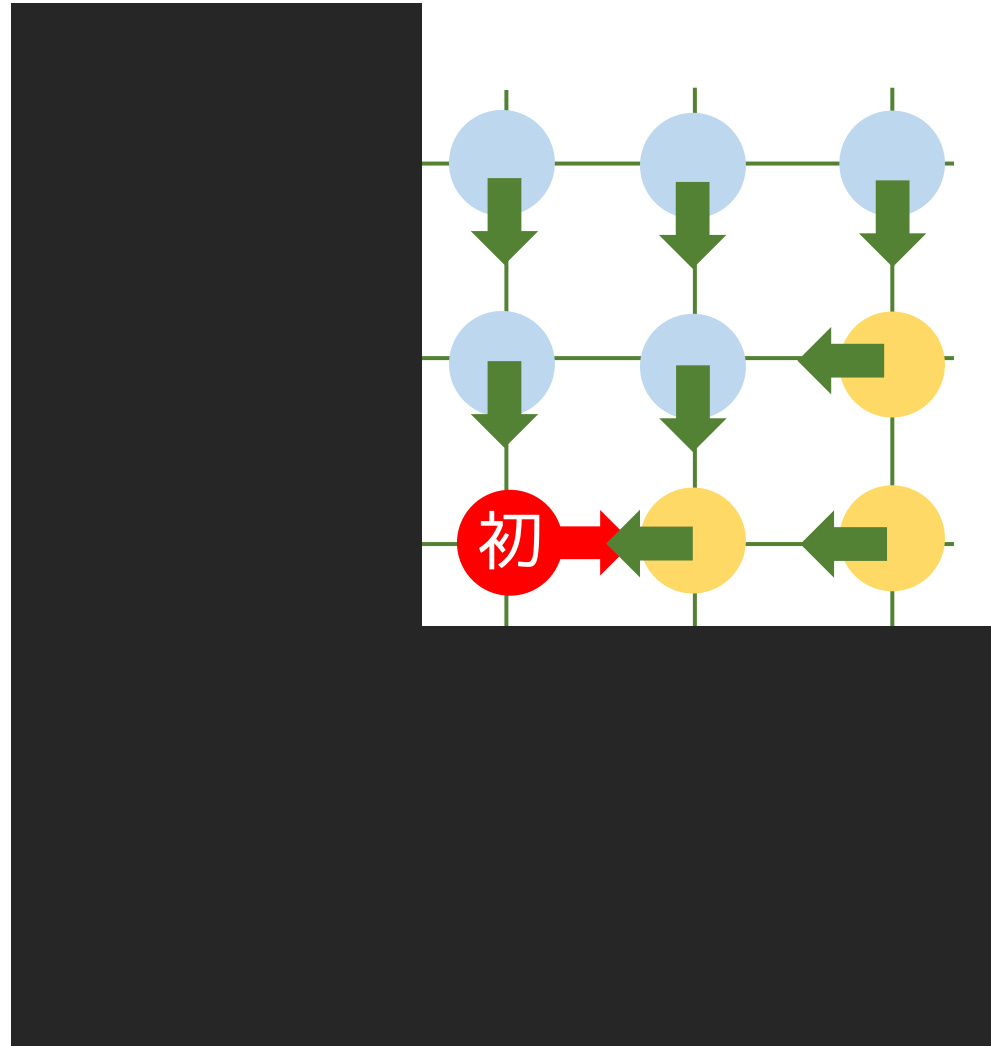
実は決まります

- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (8点)
- ▶ 小課題 3 (6点)
- ▶ 小課題 4 (6点)
- ▶ 小課題 5 (12点)
- ▶ 小課題 6 (32点)
- ▶ 小課題 7 (31点)

小課題 4 | 国民 1 が右に移動する場合

小課題 3 までで考察した領域は、右図の通りです

そこから拡張していきます

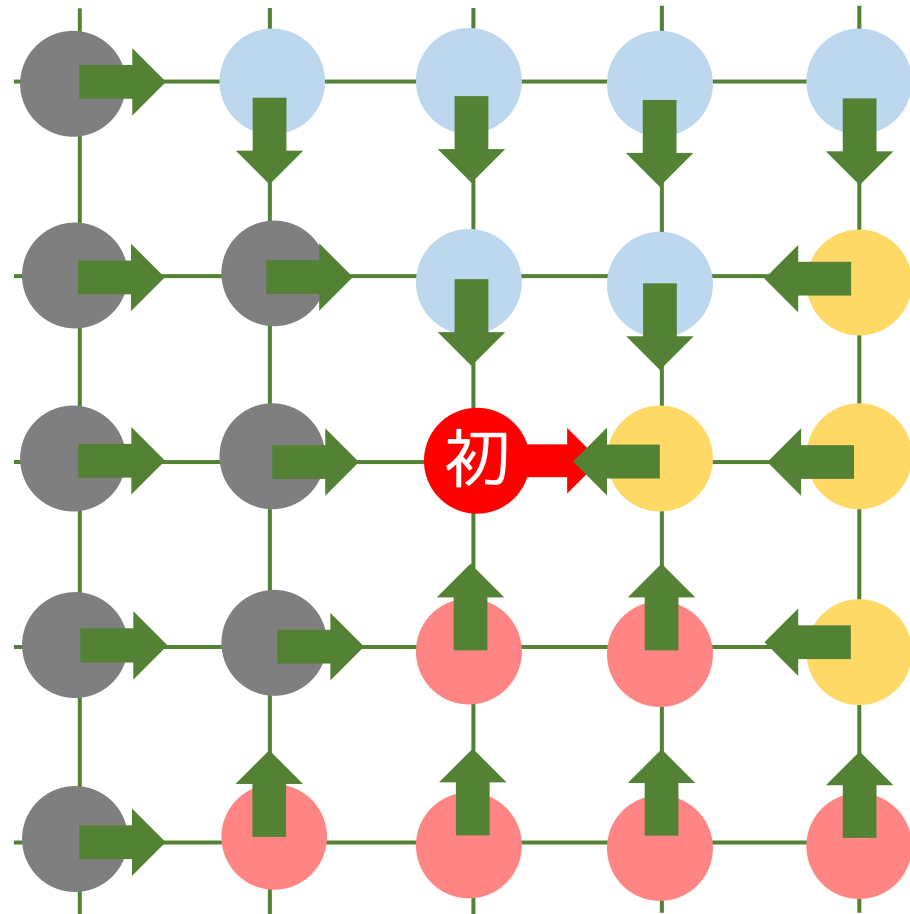


- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (8点)
- ▶ 小課題 3 (6点)
- ▶ 小課題 4 (6点)
- ▶ 小課題 5 (12点)
- ▶ 小課題 6 (32点)
- ▶ 小課題 7 (31点)

小課題 4 | 国民 1 が右に移動する場合

右図の通りです

実は国民 1 が右に
移動する場合、他の
国民の移動方法が
1 通りに定まります



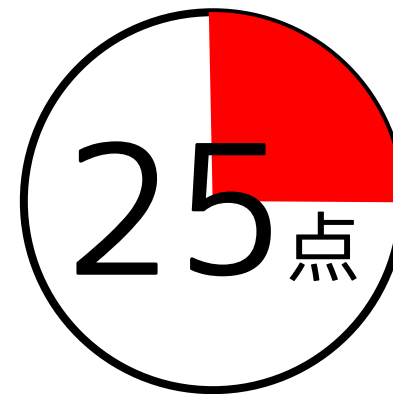
- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (8点)
- ▶ 小課題 3 (6点)
- ▶ 小課題 4 (6点)
- ▶ 小課題 5 (12点)
- ▶ 小課題 6 (32点)
- ▶ 小課題 7 (31点)

小課題 4 | 国民 1 が右に移動する場合

国民 1 が右以外の方向に移動する場合も、同様に 1 通りに定まることが証明できます

国民 1 の方向を全探索する上で、90 度回転をすると実装が楽になります

• 類題: M-SOLUTIONS 2020 F「Air Safety」



- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (8点)
- ▶ 小課題 3 (6点)
- ▶ 小課題 4 (6点)
- ▶ 小課題 5 (12点)
- ▶ 小課題 6 (32点)
- ▶ 小課題 7 (31点)

解説の流れ

0

導入

1

小課題 1

2

小課題 2

3

小課題 3

4

小課題 4

5

小課題 5

6

小課題 6

7

満点

小課題 5

課題

- 取り敢えず向きは $O(1)$ 通りに定まったが...
- シミュレーションに $O(N^3)$ かけると間に合わない

問い

- シミュレーションに $O(N^2 \log N)$ にできないか？

- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (8点)
- ▶ 小課題 3 (6点)
- ▶ 小課題 4 (6点)
- ▶ 小課題 5 (12点)
- ▶ 小課題 6 (32点)
- ▶ 小課題 7 (31点)

小課題 5

重要な性質

国民 i と国民 j は、高々 1 回しか衝突しない

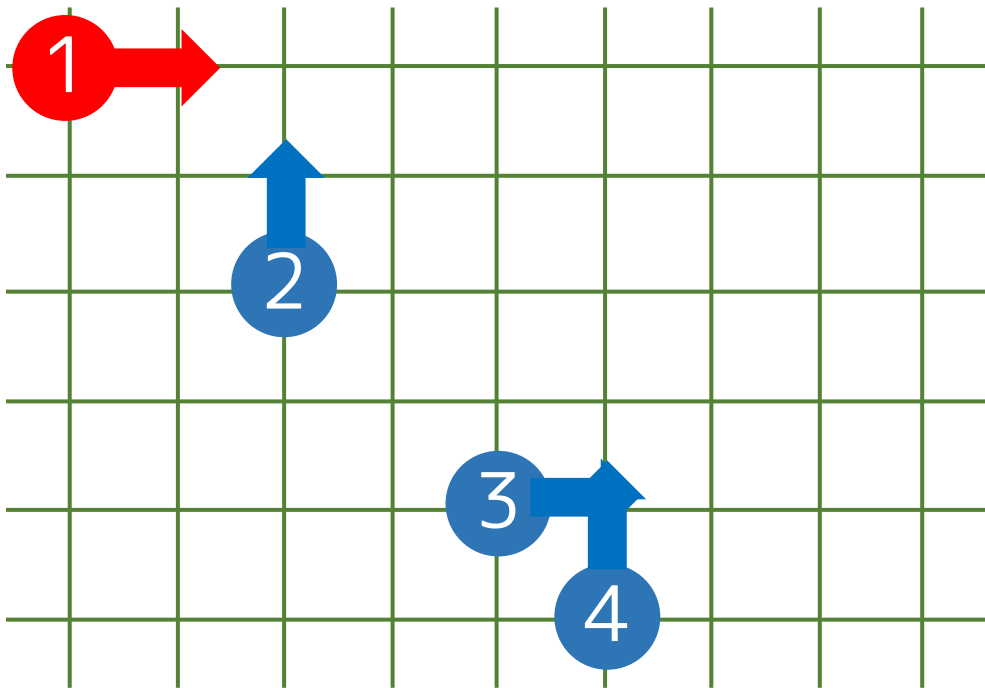


衝突時刻の早い順に予めソートしておくことで
シミュレーションを高速化できないか？

- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (8点)
- ▶ 小課題 3 (6点)
- ▶ 小課題 4 (6点)
- ▶ 小課題 5 (12点)
- ▶ 小課題 6 (32点)
- ▶ 小課題 7 (31点)

小課題 5 | シミュレーションの高速化

具体例



国民の組 衝突時刻

3 と 4 時刻 1

1 と 2 時刻 2

1 と 4 時刻 5

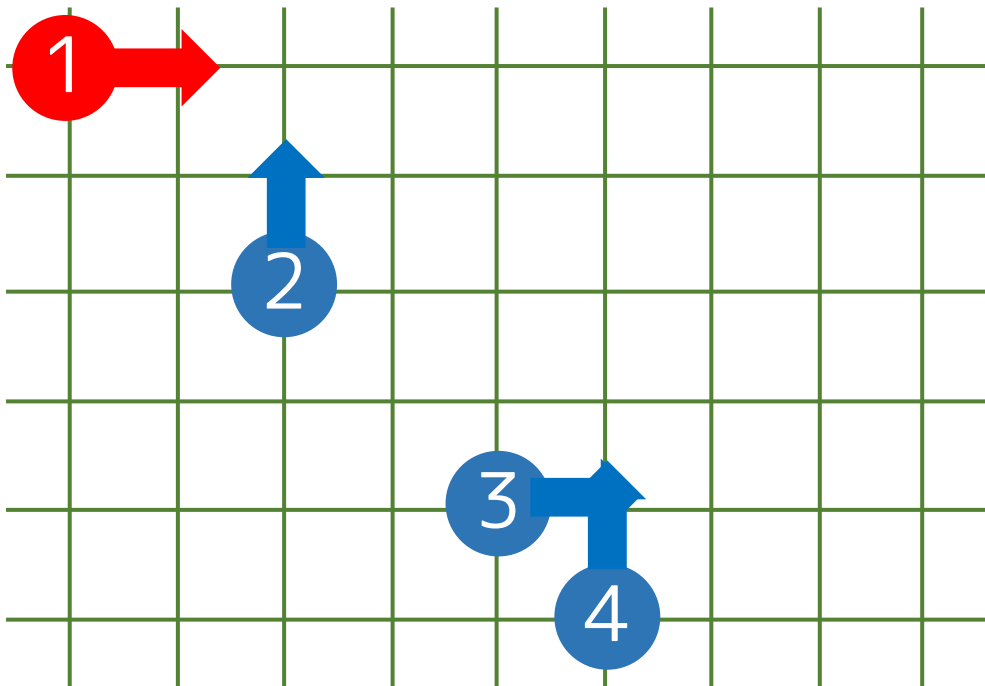
時刻 0 のとき

- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (8点)
- ▶ 小課題 3 (6点)
- ▶ 小課題 4 (6点)
- ▶ 小課題 5 (12点)
- ▶ 小課題 6 (32点)
- ▶ 小課題 7 (31点)

小課題 5 | シミュレーションの高速化

具体例

3, 4 は両方非感染者なので
新たに感染が発生しない



国民の組 衝突時刻

3 と 4 時刻 1

1 と 2 時刻 2

1 と 4 時刻 5

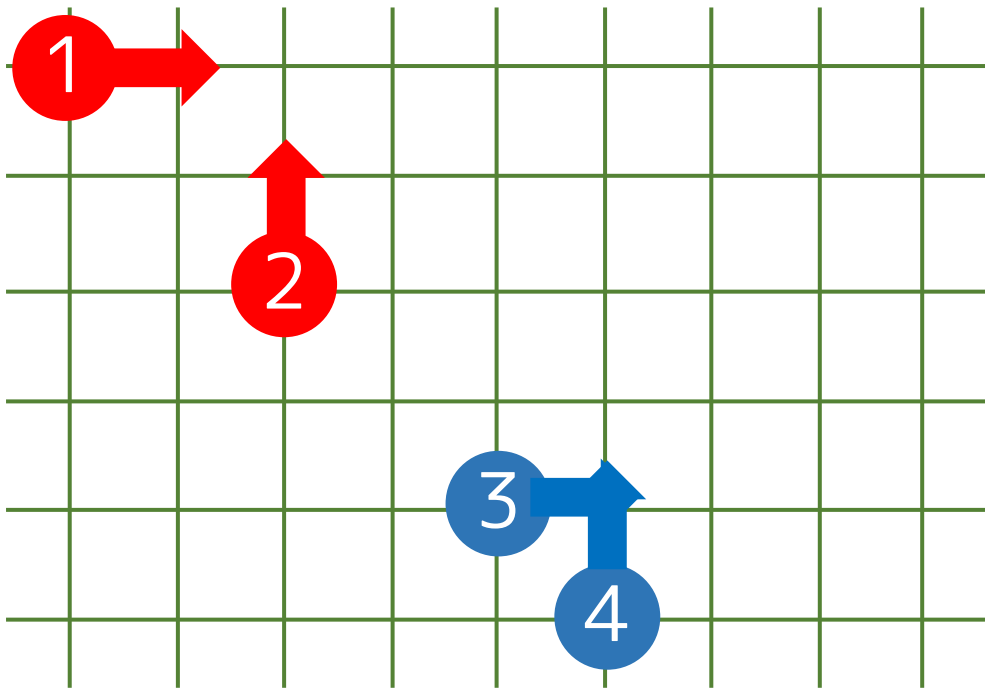
時刻 1 のとき

- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (8点)
- ▶ 小課題 3 (6点)
- ▶ 小課題 4 (6点)
- ▶ 小課題 5 (12点)
- ▶ 小課題 6 (32点)
- ▶ 小課題 7 (31点)

小課題 5 | シミュレーションの高速化

具体例

1 は感染者なので新たに 2 が感染する



国民の組 衝突時刻

3 と 4 時刻 1

1 と 2 時刻 2

1 と 4 時刻 5

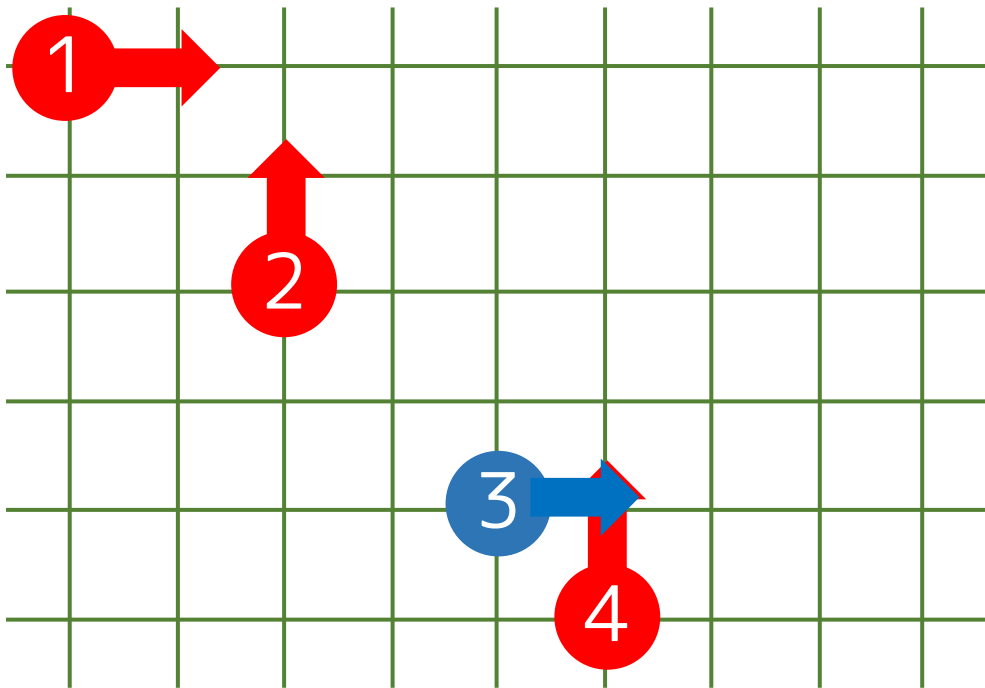
時刻 2 のとき

- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (8点)
- ▶ 小課題 3 (6点)
- ▶ 小課題 4 (6点)
- ▶ 小課題 5 (12点)
- ▶ 小課題 6 (32点)
- ▶ 小課題 7 (31点)

小課題 5 | シミュレーションの高速化

具体例

1 は感染者なので新たに 4
が感染する



国民の組 衝突時刻

3 と 4 時刻 1

1 と 2 時刻 2

1 と 4 時刻 5

時刻 5 のとき

- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (8点)
- ▶ 小課題 3 (6点)
- ▶ 小課題 4 (6点)
- ▶ 小課題 5 (12点)
- ▶ 小課題 6 (32点)
- ▶ 小課題 7 (31点)

小課題 5

具体例

1 は感染者なので新たに 4
が感染する

感染者数: 3 人

国民の組 衝突時刻

3 と 4 時刻 1

1 と 2 時刻 2

1 と 4 時刻 5

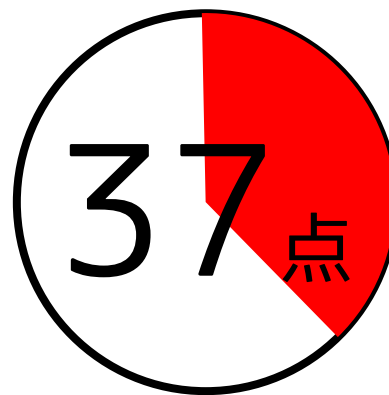
- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (8点)
- ▶ 小課題 3 (6点)
- ▶ 小課題 4 (6点)
- ▶ 小課題 5 (12点)
- ▶ 小課題 6 (32点)
- ▶ 小課題 7 (31点)

小課題 5

このように、最悪計算量 $O(N^2 \log N)$ でシミュレーションを行うことができます

$N \leq 3000$ であれば、余裕を持って間に合います

時間の都合上、小課題 6 は飛ばします



- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (8点)
- ▶ 小課題 3 (6点)
- ▶ 小課題 4 (6点)
- ▶ 小課題 5 (12点)
- ▶ 小課題 6 (32点)
- ▶ 小課題 7 (31点)

解説の流れ

0

導入

1

小課題 1

2

小課題 2

3

小課題 3

4

小課題 4

5

小課題 5

6

小課題 6

7

満点

満点解法

課題

- 取り敢えず向きは $O(1)$ 通りに定まったが...
- シミュレーションに $O(N^2 \log N)$ かけると間に合わない

問い

- シミュレーションに $O(N \log N)$ にできないか？

- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (9点)
- ▶ 小課題 3 (7点)
- ▶ 小課題 4 (22点)
- ▶ 小課題 5 (11点)
- ▶ 小課題 6 (28点)
- ▶ 小課題 7 (18点)

満点解法

全体の方針

最短経路問題に帰着させる

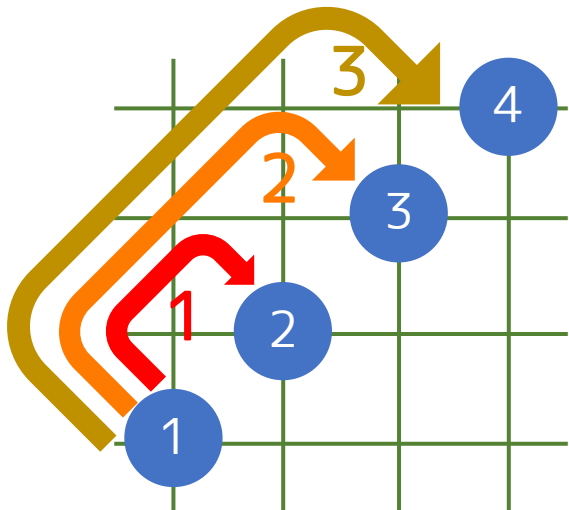
$dist[i]$ = (国民 i が感染する最初の時刻)
として、ダイクストラ法に上手く帰着させる

- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (9点)
- ▶ 小課題 3 (7点)
- ▶ 小課題 4 (22点)
- ▶ 小課題 5 (11点)
- ▶ 小課題 6 (28点)
- ▶ 小課題 7 (18点)

満点解法

愚直ダイクストラ法

- $pos = Q.top()$ とする
- 時刻 $dist[pos]$ 以降に pos と j が衝突する場合、その時刻を t として、 $pos \rightarrow j$ に長さ $t - dist[pos]$ の辺を張る



$pos = 1, dist[pos] = 0$ の場合
右図の通りに辺が張られる
計算量 $O(N^2 \log N)$

- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (9点)
- ▶ 小課題 3 (7点)
- ▶ 小課題 4 (22点)
- ▶ 小課題 5 (11点)
- ▶ 小課題 6 (28点)
- ▶ 小課題 7 (18点)

満点解法

愚直ダイクストラ法

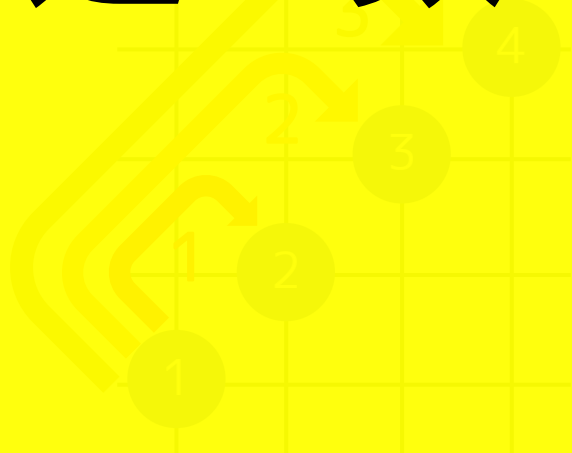
- $pos = Q.top()$ とする
- 時刻 $dist[pos]$ 以降に pos と j が衝突する場合、その時刻を t として、 $pos \rightarrow j$ に長さ $t - dist[pos]$ の辺を張る

辺の数を減らせないか？

$pos = 1, dist[pos] = 0$ の場合

右図の通りに辺が張られる

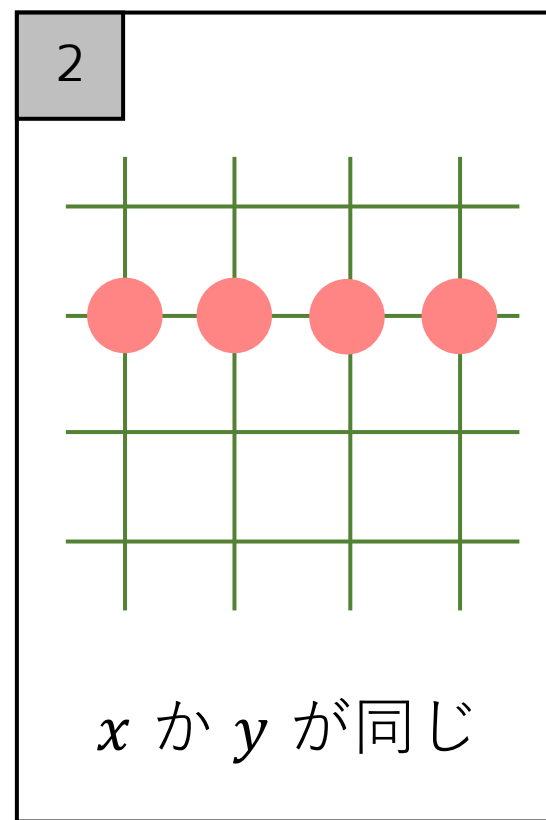
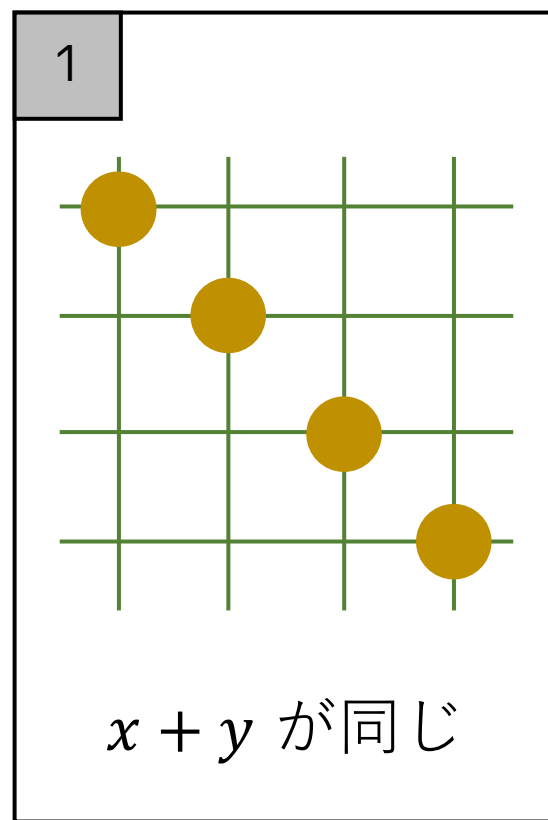
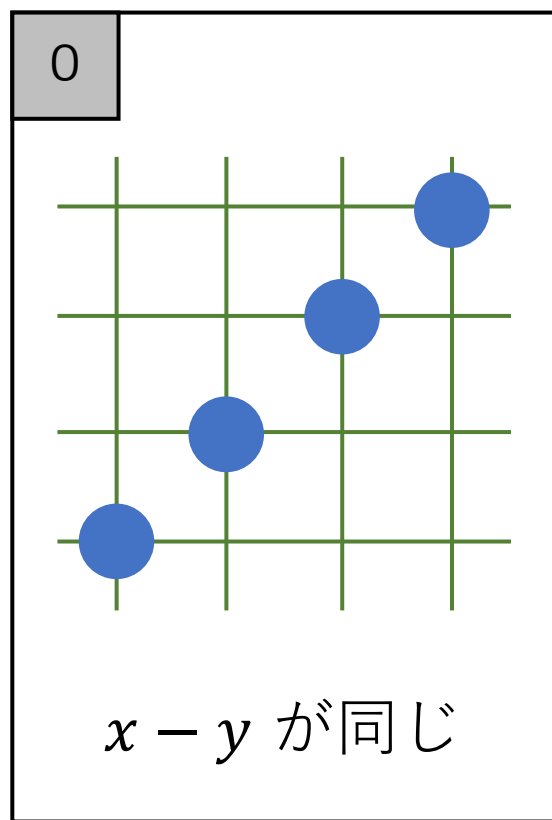
計算量 $O(N^2 \log N)$



- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (9点)
- ▶ 小課題 3 (7点)
- ▶ 小課題 4 (22点)
- ▶ 小課題 5 (11点)
- ▶ 小課題 6 (28点)
- ▶ 小課題 7 (18点)

満点解法

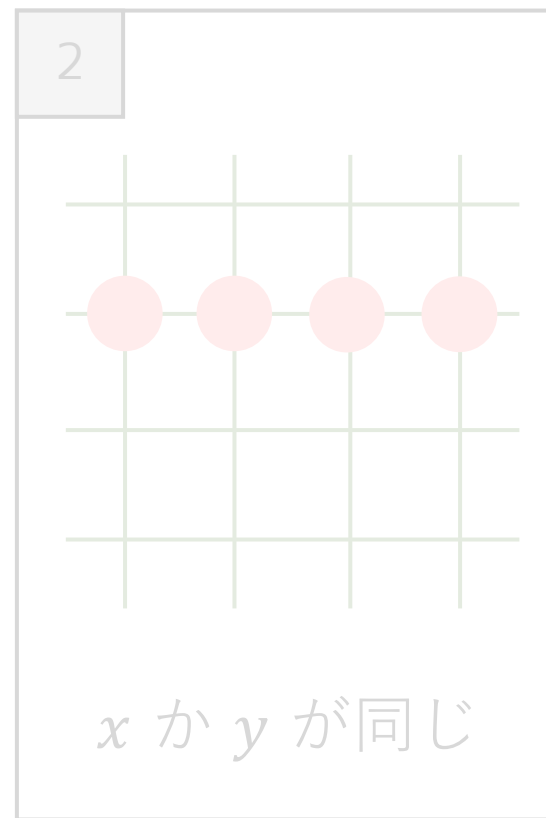
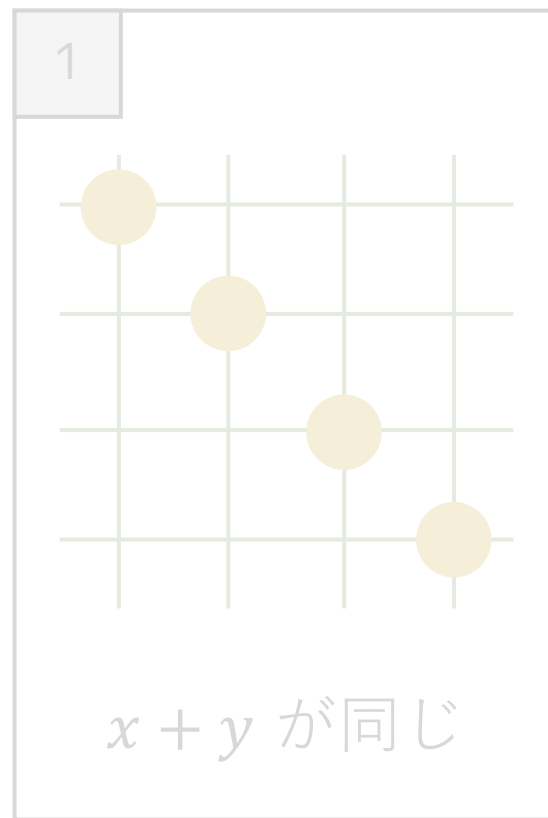
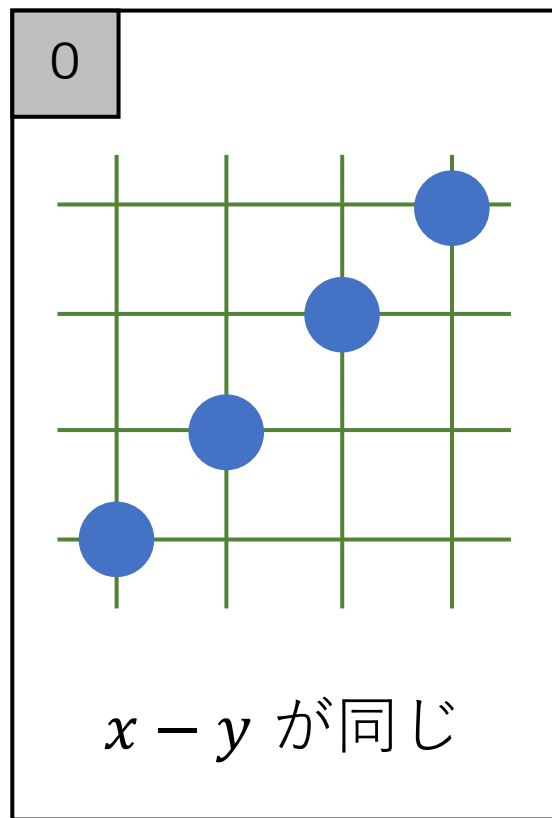
実は「衝突が起こるパターン」は以下の 3 通り



- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (9点)
- ▶ 小課題 3 (7点)
- ▶ 小課題 4 (22点)
- ▶ 小課題 5 (11点)
- ▶ 小課題 6 (28点)
- ▶ 小課題 7 (18点)

満点解法

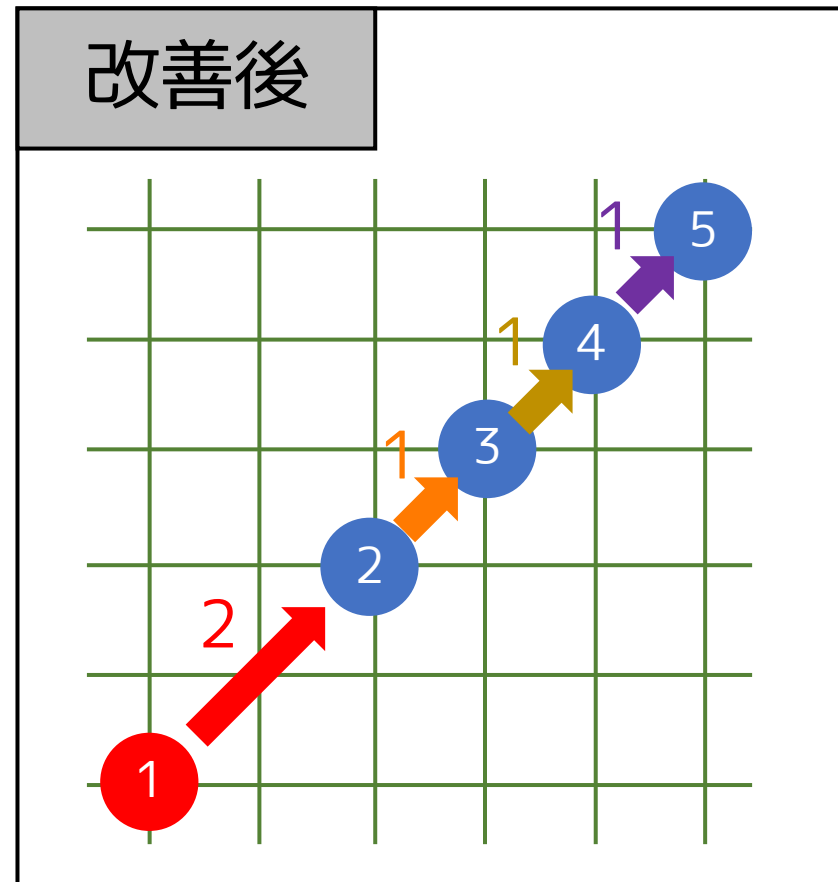
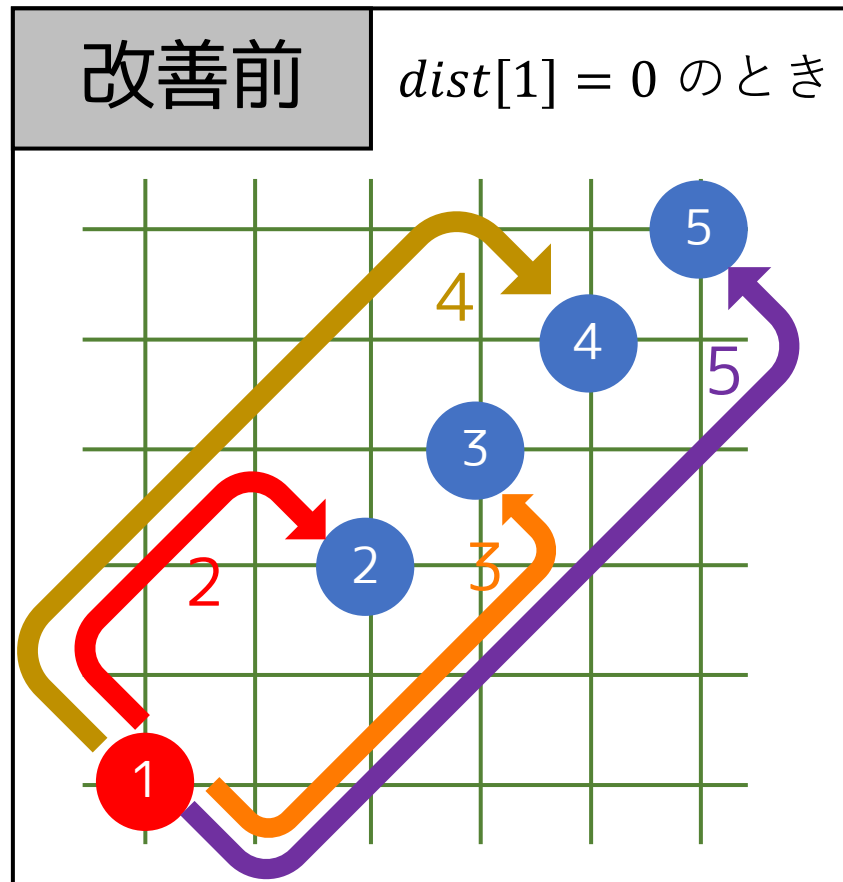
そのうち [0] のみの場合について考える



- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (9点)
- ▶ 小課題 3 (7点)
- ▶ 小課題 4 (22点)
- ▶ 小課題 5 (11点)
- ▶ 小課題 6 (28点)
- ▶ 小課題 7 (18点)

満点解法 | $x - y$ の値が同じ場合はどう解くか？

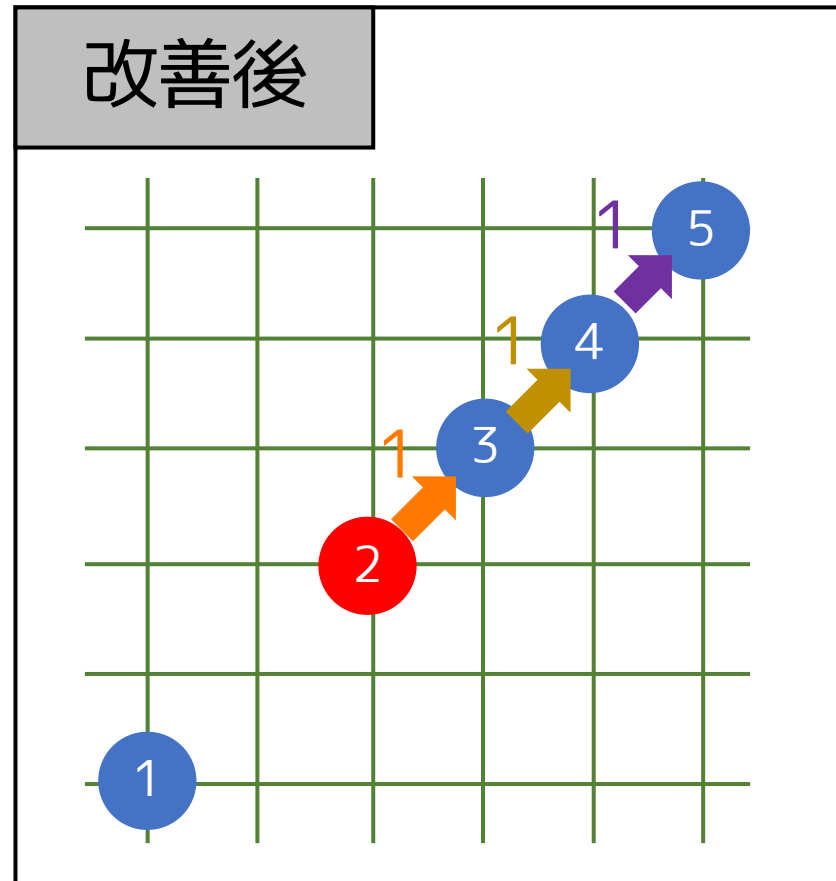
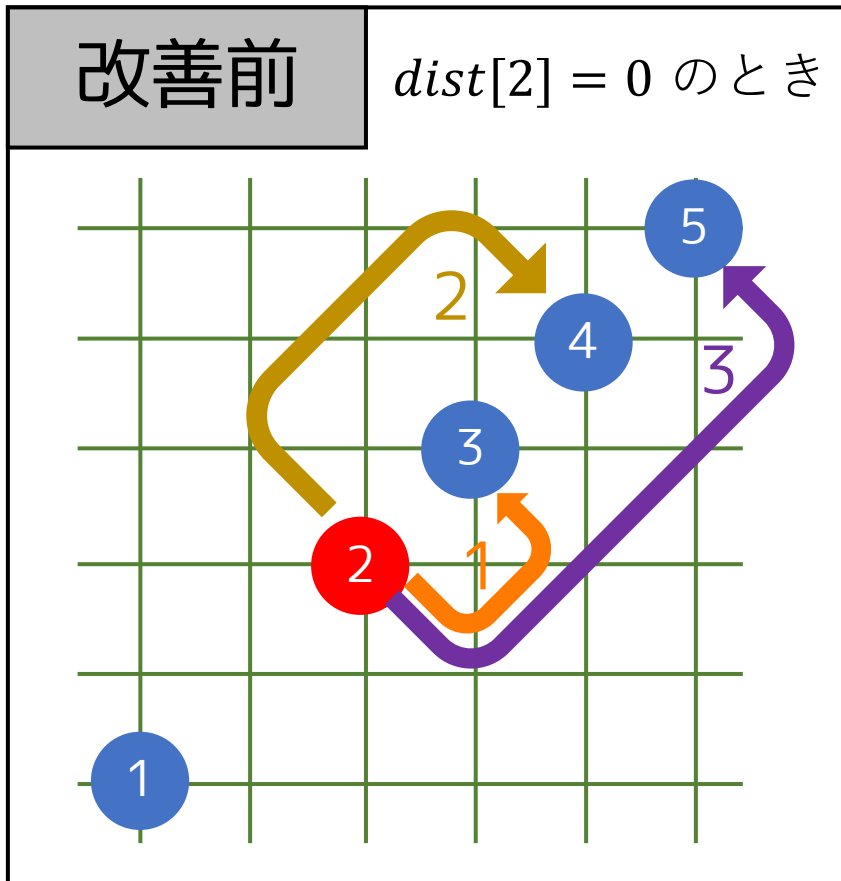
以下のように辺の貼り方を改善できると良さそう



- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (9点)
- ▶ 小課題 3 (7点)
- ▶ 小課題 4 (22点)
- ▶ 小課題 5 (11点)
- ▶ 小課題 6 (28点)
- ▶ 小課題 7 (18点)

満点解法 | $x - y$ の値が同じ場合はどう解くか？

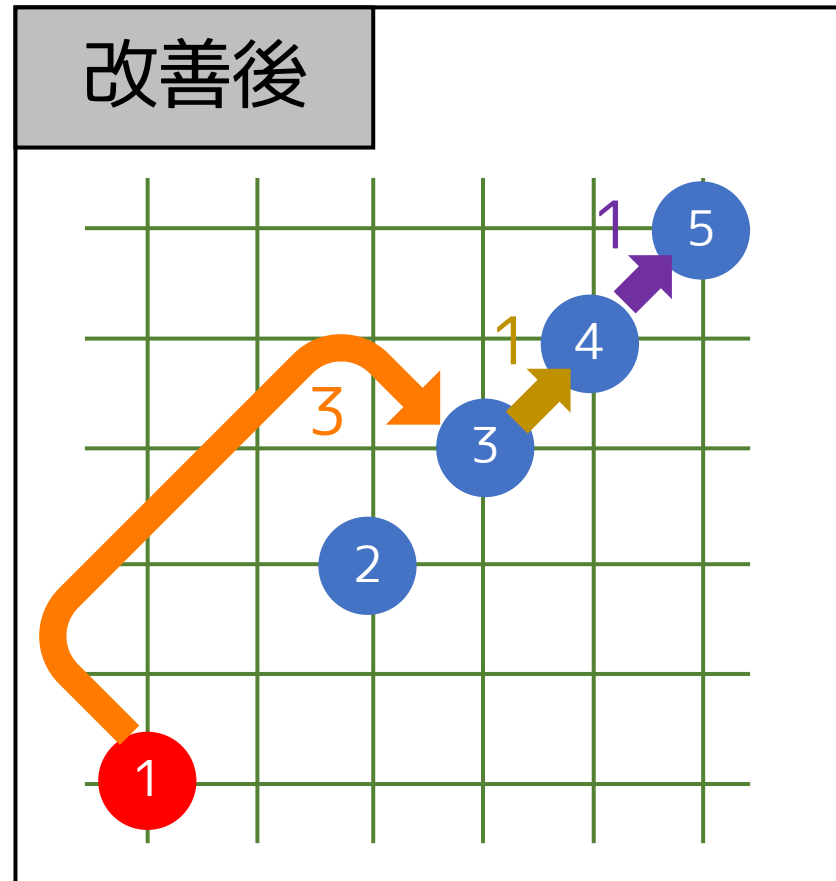
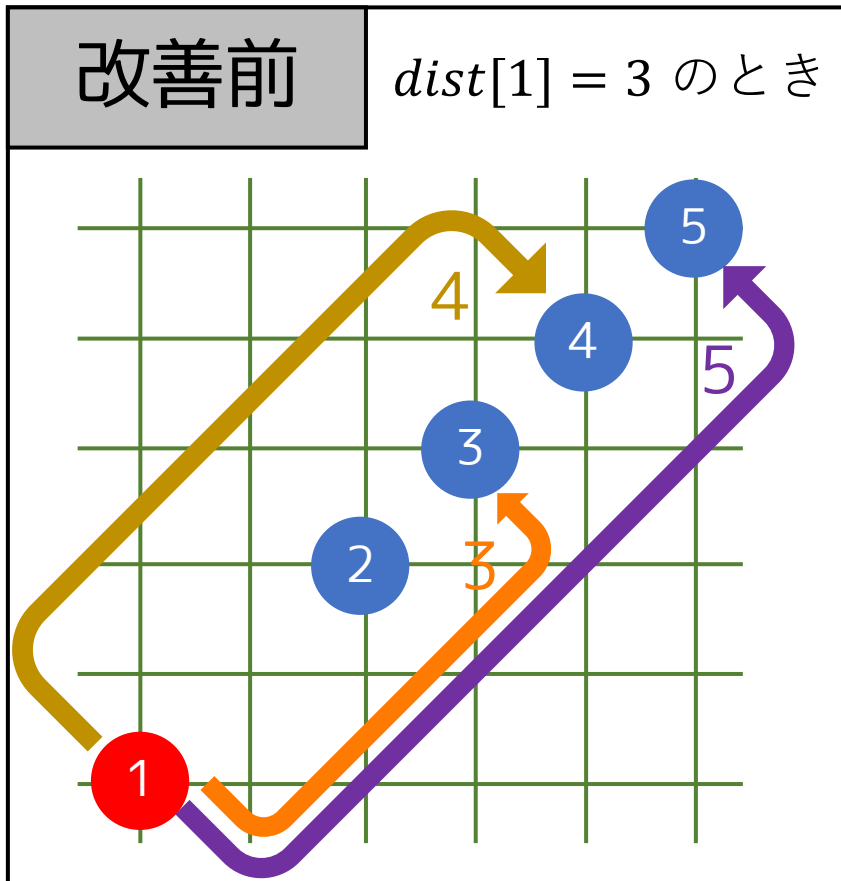
以下のように辺の貼り方を改善できると良さそう



- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (9点)
- ▶ 小課題 3 (7点)
- ▶ 小課題 4 (22点)
- ▶ 小課題 5 (11点)
- ▶ 小課題 6 (28点)
- ▶ 小課題 7 (18点)

満点解法 | $x - y$ の値が同じ場合はどう解くか？

以下のように辺の貼り方を改善できると良さそう



- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (9点)
- ▶ 小課題 3 (7点)
- ▶ 小課題 4 (22点)
- ▶ 小課題 5 (11点)
- ▶ 小課題 6 (28点)
- ▶ 小課題 7 (18点)

満点解法 | $x - y$ の値が同じ場合はどう解くか？

3 → 4、4 → 5 などは何回も使われている



使われている辺の数が減っているのではないか？

- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (9点)
- ▶ 小課題 3 (7点)
- ▶ 小課題 4 (22点)
- ▶ 小課題 5 (11点)
- ▶ 小課題 6 (28点)
- ▶ 小課題 7 (18点)

満点解法 | 実は辺の数が減っている

- 全部の頂点について $X_i - Y_i = T$ (T は定数) を満たすとき、改善後の手法では以下の 2 種類の辺しか張られない

- A. 点を x_i 昇順に並べたとき隣り合っている 2 つの頂点間を結ぶ辺
- B. 時刻 $dist[pos]$ 以降で、点 pos と初めて衝突する頂点の番号 (ただし、 $pos = Q.top()$ / この辺はダイクストラ法の結果に応じて動的に決められる)

- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (9点)
- ▶ 小課題 3 (7点)
- ▶ 小課題 4 (22点)
- ▶ 小課題 5 (11点)
- ▶ 小課題 6 (28点)
- ▶ 小課題 7 (18点)

満点解法 | 実は辺の数が減っている

- 全部の頂点について $x_i - y_i = T$ (T は定数) を満たすとき、改善後の手法では以下の 2 種類の辺しか張られない

A. と B. は両方

$O(N)$ 辺しかない

1. 点を x_i 昇順に並べたとき隣り合っている 2 つの頂点間を結ぶ辺
2. 時刻 $dist[pos]$ 以降で、点 pos と初めて衝突する頂点の番号 (ただし、 $pos = Q.top()$ / この辺はダイクストラ法の結果に応じて動的に決められる)

- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (9点)
- ▶ 小課題 3 (7点)
- ▶ 小課題 4 (22点)
- ▶ 小課題 5 (11点)
- ▶ 小課題 6 (28点)
- ▶ 小課題 7 (18点)

満点解法 | パターンの乗り換え

これまでの方法で、[0], [1], [2] のうち 1 パターンしかない場合は解けた

そこで、 $dist[pos][0 \text{ or } 1 \text{ or } 2]$ のように状態を持っておくと、上手く実装すれば「パターンの乗り換え」のようなことができる

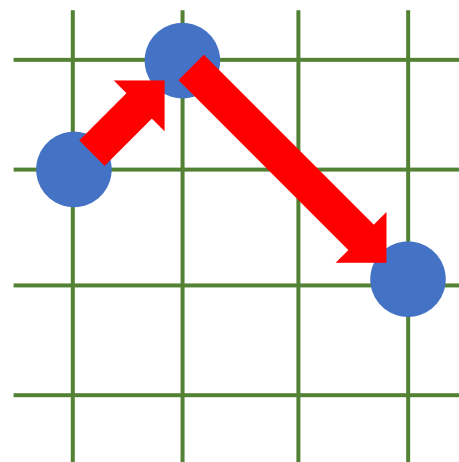
- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (9点)
- ▶ 小課題 3 (7点)
- ▶ 小課題 4 (22点)
- ▶ 小課題 5 (11点)
- ▶ 小課題 6 (28点)
- ▶ 小課題 7 (18点)

満点解法 | パターンの乗り換え

$dist[pos][0 \text{ or } 1 \text{ or } 2]$ は以下の値を記録することを考える

- $dist[pos][0]$: 直前に通った辺がパターン [0] の方向である
- $dist[pos][1]$: 直前に通った辺がパターン [1] の方向である
- $dist[pos][2]$: 直前に通った辺がパターン [2] の方向である

上手く辺を張れば、右図のようにパターンや向きが入れ替わる場合にも対応できる（詳細は紙面の都合上割愛させていただきます）



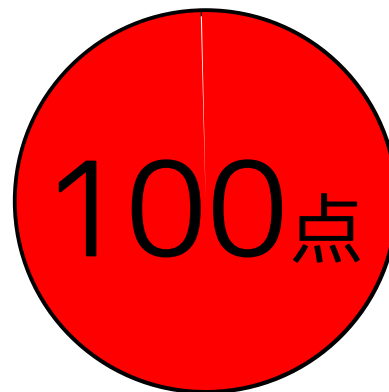
- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (9点)
- ▶ 小課題 3 (7点)
- ▶ 小課題 4 (22点)
- ▶ 小課題 5 (11点)
- ▶ 小課題 6 (28点)
- ▶ 小課題 7 (18点)

満点解法 | 実装

したがって、頂点数 $O(N)$ 辺の数 $O(N)$ のダイクストラ法になったので、計算量は $O(N \log N)$

実装がかなり大変です

- チューター解: 266 行、10261 バイト



- ▶ 導入
- ▶ 小課題 1 (5点)
- ▶ 小課題 2 (9点)
- ▶ 小課題 3 (7点)
- ▶ 小課題 4 (22点)
- ▶ 小課題 5 (11点)
- ▶ 小課題 6 (28点)
- ▶ 小課題 7 (18点)

得点分布

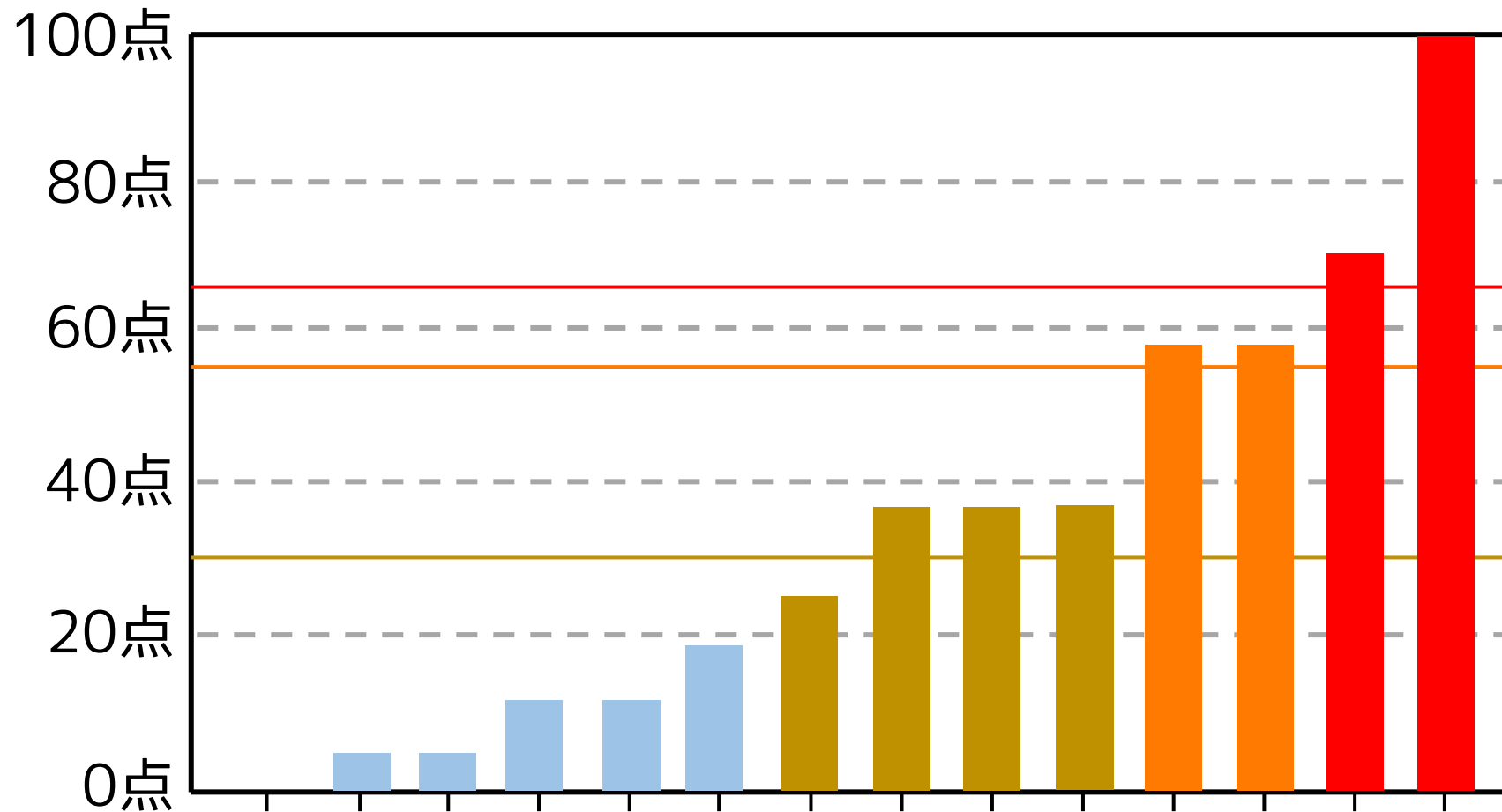
得点分布



【参考】
標準偏差
11.0

- ▶ 導入
- ▶ この問題の解法
- ▶ 得点分布

得点分布



▶ 導入

▶ この問題の解法

▶ 得点分布



解説：平木 康傑

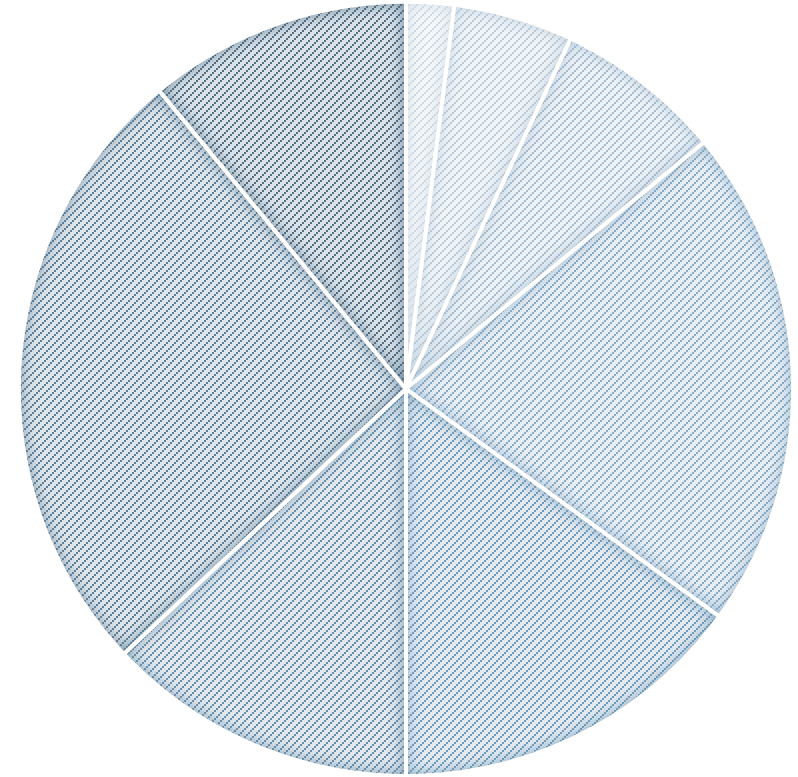
JOI 2021 春合宿 Day1 – Food Court

問題概要

- N 個の待機列 (queue) に整数を出し入れするクエリが時系列順に Q 個ある
- 以下の 3 種類
 - 「加入」：待機列 $L \sim R$ に整数 C を K 個ずつ push
 - 「脱退」：待機列 $L \sim R$ から K 個ずつ pop (K 個に満たなければ空になる)
 - 「サービス」：待機列 A の要素数が B 個以上か判定し, そうなら B 番目を出力

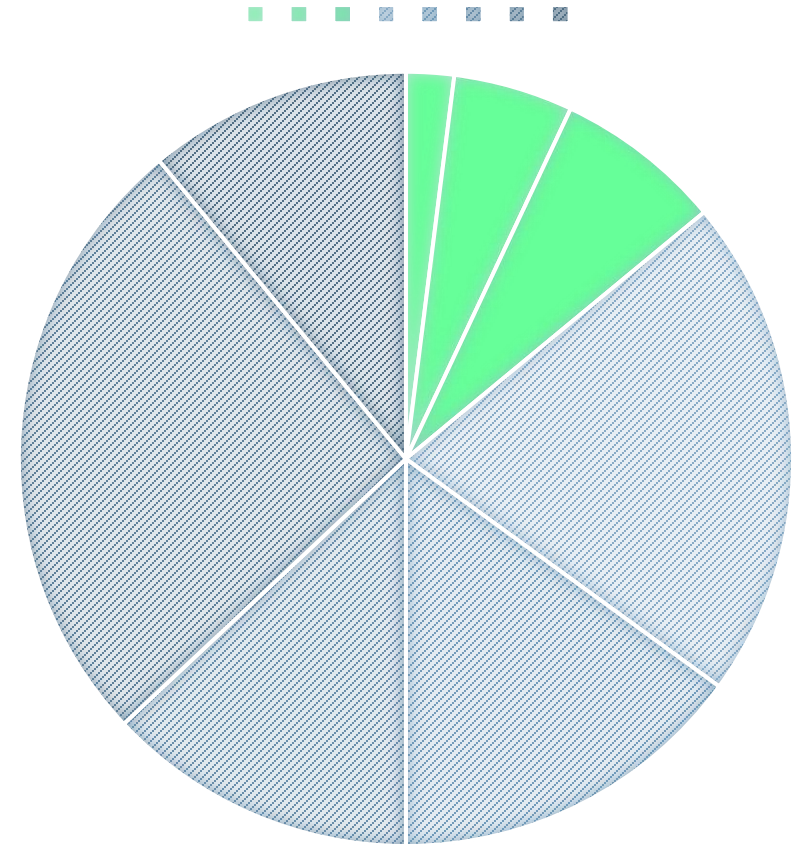
小課題

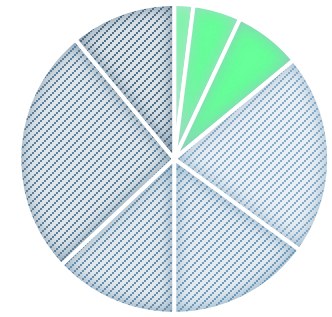
- (2 点) $O(NQ), K_i = 1$
- (5 点) $O(NQ)$
- (7 点) push クエリは狭い範囲に 1 個ずつ
- (21 点) $M = 1$
- (15 点) $K_i = 1$
- (13 点) pop なし
- (26 点) $\log^2$
- (11 点) $\log^1$



小課題

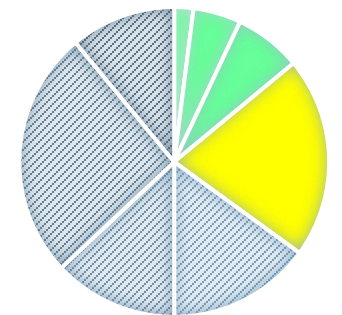
- ~~• (2 点) $O(NQ)$, $K_t = 1$ ✓~~
- ~~• (5 点) $O(NQ)$ ✓~~
- ~~• (7 点) push クエリは狭い範囲に 1 個ずつ ✓~~
- (21 点) $M = 1$
- (15 点) $K_i = 1$
- (13 点) pop なし
- (26 点) $\log^2$
- (11 点) $\log^1$





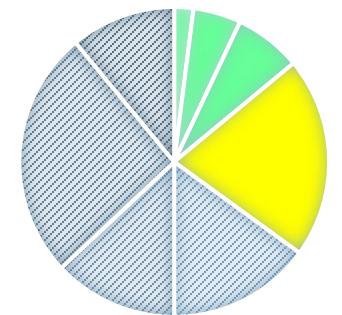
解法の方針

- どの小課題も糸口として使えるし，最終的な解法も多様
- 予告：2通りの解法を示す



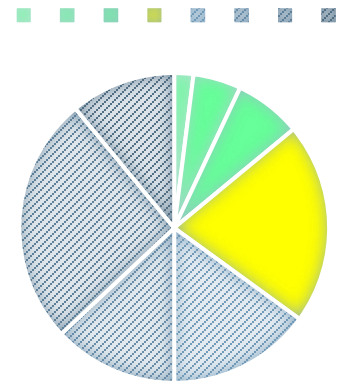
小課題 4 の考察: $M = 1$

- 出し入れする整数は全て 1
- 「サービス」クエリにおいて,
待機列 A の要素数が B 以上なら 1, そうでなければ 0 が答え
- → クエリ時点での待機列 A の要素数がわかればよい



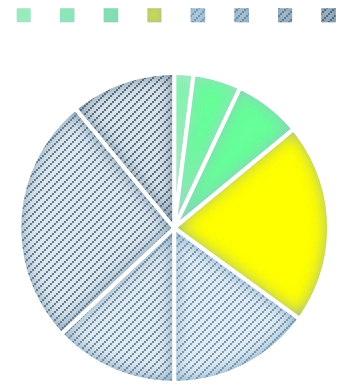
小課題 4 の考察：要素数を管理する

- 各クエリを要素数の観点から捉え直すようになる
 - 「加入」：区間 $[L, R]$ に K を加算
 - 「脱退」：区間 $[L, R]$ から K を減算し， 0 との $\max$ をとる
 - 「サービス」：位置 $[A]$ の値を求める



小課題 4 の考察：要素数を管理する

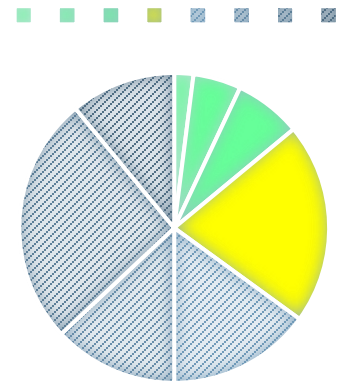
- 各クエリを要素数の観点から捉え直すようになる
 - 「加入」：区間[L, R] に K を加算
 - ~~「脱退」：区間[L, R] から K を減算し, 0 との max をとる~~
 - 「サービス」：位置[A] の値を求める
- こうだったなら有名問題：区間加算・一点取得
- いわゆる「双対セグメント木」



小課題 4 の考察：要素数を管理する

- 各クエリを要素数の観点から捉え直すようになる
 - 「加入」：区間[L, R] に K を加算
 - ~~「脱退」：区間[L, R] から K を減算し, 0 との max をとる~~
 - 「サービス」：位置[A] の値を求める
- こうだったなら有名問題：区間加算・一点取得
- いわゆる「双対セグメント木」

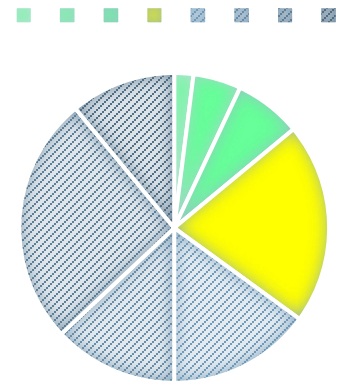
区間[1, 8] に 4 を加算



小課題 4 の考察：要素数を管理する

- 各クエリを要素数の観点から捉え直すようになる
 - 「加入」：区間[L, R] に K を加算
 - ~~「脱退」：区間[L, R] から K を減算し, 0 との max をとる~~
 - 「サービス」：位置[A] の値を求める
- こうだったなら有名問題：区間加算・一点取得
- いわゆる「双対セグメント木」

区間[1, 8] に 4 を加算

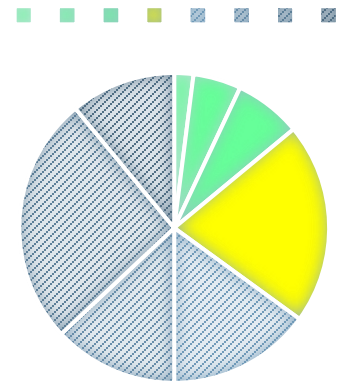


小課題 4 の考察：要素数を管理する

- 各クエリを要素数の観点から捉え直すようになる
 - 「加入」：区間 $[L, R]$ に K を加算
 - ~~「脱退」：区間 $[L, R]$ から K を減算し、 0 との $\max$ をとる~~
 - 「サービス」：位置 A の値を求める
- こうだったなら有名問題：区間加算・一点取得
- いわゆる「双対セグメント木」

+4															

✓

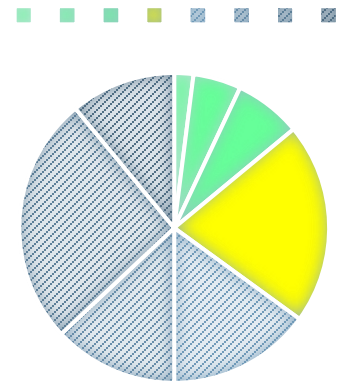


小課題 4 の考察：要素数を管理する

- 各クエリを要素数の観点から捉え直すようになる
 - 「加入」：区間[L, R] に K を加算
 - ~~「脱退」：区間[L, R] から K を減算し、0 との max をとる~~
 - 「サービス」：位置[A] の値を求める
- こうだったなら有名問題：区間加算・一点取得
- いわゆる「双対セグメント木」

+4															
												+3			
+6						+1		+2						+5	

区間[4, 11] に 3 を加算

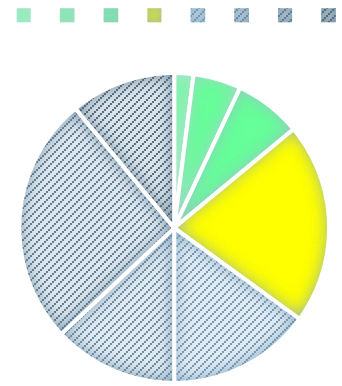


小課題 4 の考察：要素数を管理する

- 各クエリを要素数の観点から捉え直すようになる
 - 「加入」：区間[L, R] に K を加算
 - 「脱退」：区間[L, R] から K を減算し、0 との max をとる
 - 「サービス」：位置[A] の値を求める
- こうだったなら有名問題：区間加算・一点取得
- いわゆる「双対セグメント木」

+4															
												+3			
+6						+1		+2						+5	

区間[4, 11] に 3 を加算

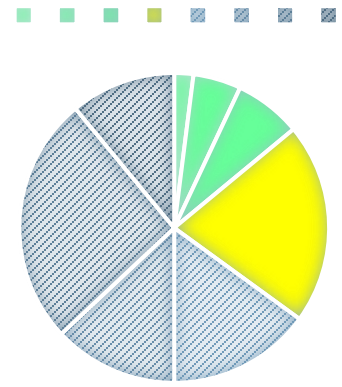


小課題 4 の考察：要素数を管理する

- 各クエリを要素数の観点から捉え直すようになる
 - 「加入」：区間[L, R] に K を加算
 - 「脱退」：区間[L, R] から K を減算し、0 との max をとる
 - 「サービス」：位置[A] の値を求める
- こうだったなら有名問題：区間加算・一点取得
- いわゆる「双対セグメント木」

+4															
												+3			
+6						+1		+2						+5	

区間[4, 11] に 3 を加算

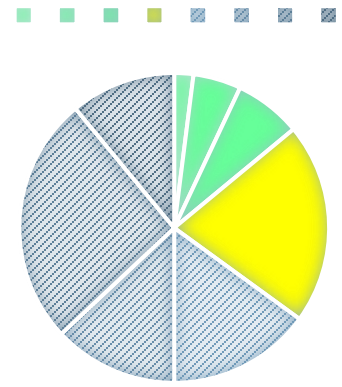


小課題 4 の考察：要素数を管理する

- 各クエリを要素数の観点から捉え直すようになる
 - 「加入」：区間[L, R] に K を加算
 - 「脱退」：区間[L, R] から K を減算し、0 との max をとる
 - 「サービス」：位置[A] の値を求める
- こうだったなら有名問題：区間加算・一点取得
- いわゆる「双対セグメント木」

+4															
												+3			
+6						+1		+2						+5	

区間[4, 11] に 3 を加算

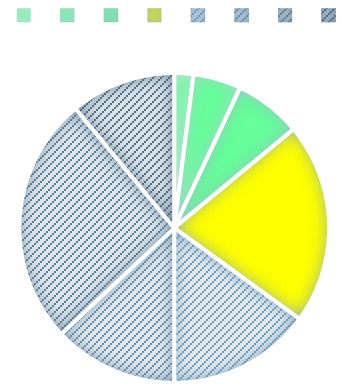


小課題 4 の考察：要素数を管理する

- 各クエリを要素数の観点から捉え直すようになる
 - 「加入」：区間[L, R] に K を加算
 - 「脱退」：区間[L, R] から K を減算し、0 との max をとる
 - 「サービス」：位置[A] の値を求める
- こうだったなら有名問題：区間加算・一点取得
- いわゆる「双対セグメント木」

+4				+4								+3			
+6						+1		+2						+5	

区間[4, 11] に 3 を加算

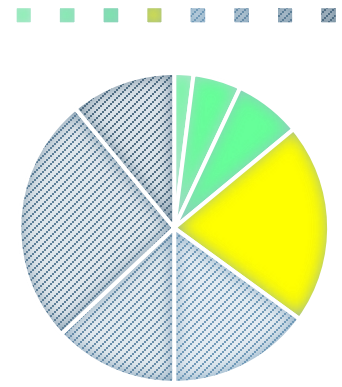


小課題 4 の考察：要素数を管理する

- 各クエリを要素数の観点から捉え直すようになる
 - 「加入」：区間[L, R] に K を加算
 - 「脱退」：区間[L, R] から K を減算し、0 との max をとる
 - 「サービス」：位置[A] の値を求める
- こうだったなら有名問題：区間加算・一点取得
- いわゆる「双対セグメント木」

				+4								+3			
+10	+4				+1			+2					+5		

区間[4, 11] に 3 を加算

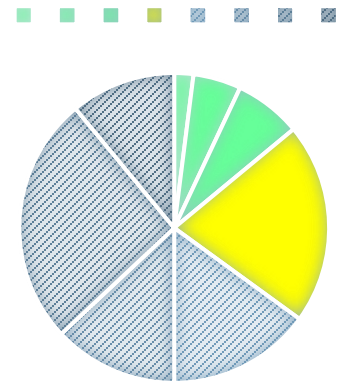


小課題 4 の考察：要素数を管理する

- 各クエリを要素数の観点から捉え直すようになる
 - 「加入」：区間[L, R] に K を加算
 - ~~「脱退」：区間[L, R] から K を減算し, 0 との max をとる~~
 - 「サービス」：位置[A] の値を求める
- こうだったなら有名問題：区間加算・一点取得
- いわゆる「双対セグメント木」

				+4								+3			
+10						+1		+2						+5	
		4	4												

区間[4, 11] に 3 を加算

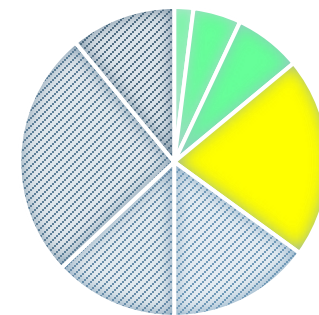


小課題 4 の考察：要素数を管理する

- 各クエリを要素数の観点から捉え直すようになる
 - 「加入」：区間[L, R] に K を加算
 - ~~「脱退」：区間[L, R] から K を減算し, 0 との max をとる~~
 - 「サービス」：位置[A] の値を求める
- こうだったなら有名問題：区間加算・一点取得
- いわゆる「双対セグメント木」

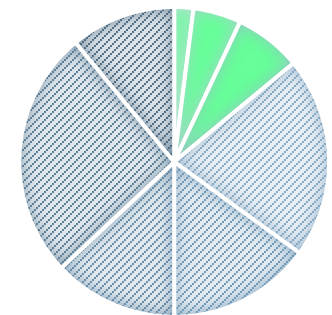
				+7								+3			
+10						+1		+5						+5	
		4	7							3					

✓



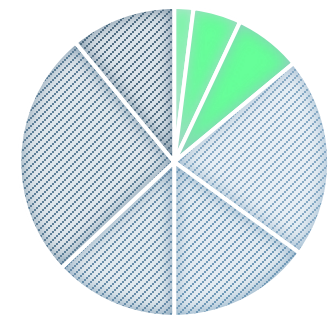
小課題 4 の考察：要素数を管理する

- 各クエリを要素数の観点から捉え直すようになる
 - 「加入」：区間 $[L, R]$ に K を加算
 - 「脱退」：区間 $[L, R]$ から K を減算し、 0 との $\max$ をとる
 - 「サービス」：位置 $[A]$ の値を求める
- これが元の問題の「前半」にあたり，小課題 4 が対応する



前半が解けた後の考察

- クエリ時点での待機列 A の先頭から B 番目の要素が知りたいとする
- クエリ時点での待機列 A の要素数 s ($\leq B$) がわかったものとする
 - (つまり, 前半が解けた)



前半が解けた後の考察

- クエリ時点での待機列 A の先頭から B 番目の要素が知りたいとする
- クエリ時点での待機列 A の要素数 s ($\leq B$) がわかったものとする
 - (つまり, 前半が解けた)
- クエリ時点までに待機列 A に加わった要素数 t は双対セグメント木でわかる
 - 2ページ前に挙げた通り, 区間加算・一点取得



前半が解けたら，pop なしに帰着

- クエリ時点での待機列 A の先頭から B 番目の要素が知りたいとする
- クエリ時点での待機列 A の要素数 s ($\leq B$) がわかったものとする
 - (つまり，前半が解けた)
- クエリ時点までに待機列 A に加わった要素数 t は双対セグメント木でわかる
 - 2ページ前に挙げた通り，区間加算・一点取得
- すると目的の要素は，初めから数えて $t - s + B$ 番目に待機列 A に加わったということがわかる
- したがって pop の無い場合 (小課題 6) に帰着された(これが「後半」)



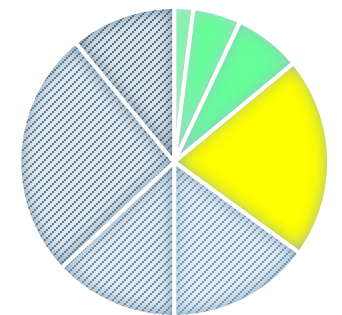
方針の整理

- クエリ時点の要素数を求めよう (前半; 小課題 4)
- すると, pop のない場合に帰着できる
- それを解こう (後半; 小課題 6)



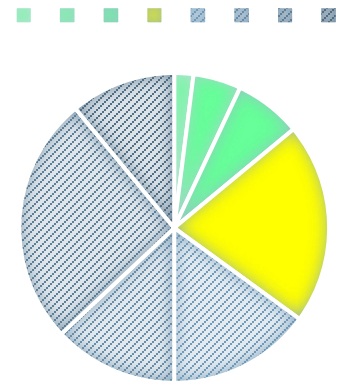
方針の整理

- クエリ時点の要素数を求めよう（前半；小課題 4）
- すると，pop のない場合に帰着できる
- それを解こう（後半；小課題 6）
- ここからは多様な解法がある



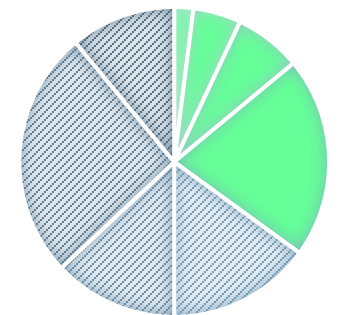
前半(小課題 4)を解こう

- 「加入」：区間 $[L, R]$ に K を加算
- 「脱退」：区間 $[L, R]$ から K を減算し， 0 との $\max$ をとる
- 「サービス」：位置 $[A]$ の値を求める



前半(小課題 4)を解こう

- 「加入'」：区間[L, R] に K を加算
 - 「脱退'」：区間[L, R] から K を減算し，0 との max をとる
 - 「サービス'」：位置[A] の値を求める
-
- 双対セグメント木で解くことを考える
 - 単純な区間加算のみの場合，作用素は $x \leftarrow x + A$



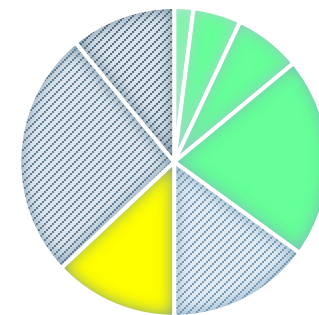
前半(小課題 4)を解こう

- 「加入'」：区間[L, R] に $x \leftarrow \max(x + K, -\infty)$ を適用
- 「脱退'」：区間[L, R] に $x \leftarrow \max(x - K, 0)$ を適用
- 「サービス'」：位置[A] の値を求める
- 双対セグメント木で解くことを考える
- 作用素を $x \leftarrow \max(x + A, B)$ とすることで，上のように適用できる
 - 作用素 2 つの合成が出来る
 - もちろん結合法則も成り立つ

よって解けた



後半(小課題 6)を解こう

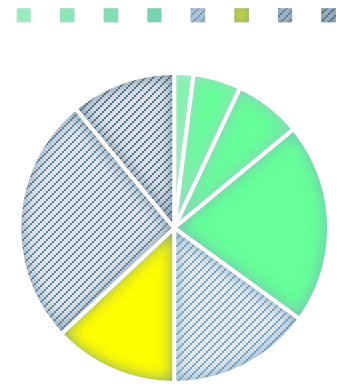


- 簡単のため，クエリで聞かれる位置はすべて異なるものとする



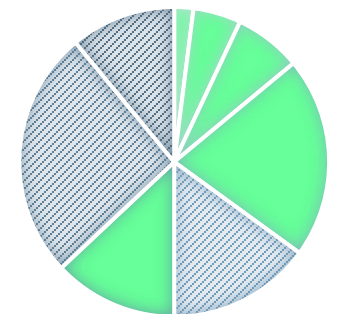
後半(小課題 6)を解こう

- 簡単のため，クエリで聞かれる位置はすべて異なるものとする
- 次のように初期化した配列を用意する
 - 位置 A を聞くクエリで B 番目が問われる場合，配列の位置 A の値は $-B$
 - クエリで聞かれない位置は $-\infty$
- そして，時系列順に「加入」クエリを見て，次のように処理する
 - 「加入」される位置に K を加える
 - 値が 0 以上の位置があるあいだ，その位置にクエリ番号を記録してからその位置の値を $-\infty$ にすることを繰り返す



後半(小課題 6)を解こう

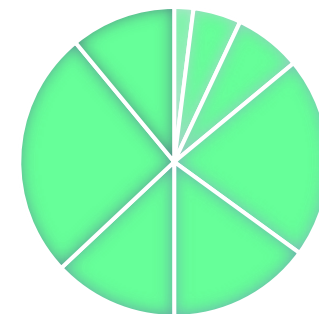
- 簡単のため，クエリで聞かれる位置はすべて異なるものとする
- 次のように初期化した配列を用意する
 - 「 B 番目」を聞かれる位置の値は $-B$
 - クエリで聞かれない位置の値は $-\infty$
- そして，時系列順に「加入」クエリを見て，次のように処理する
 - 「加入」される位置に K を加える
 - 値が 0 以上の位置があるあいだ，
その位置にクエリ番号を記録してからその位置の値を $-\infty$ にする
ことを繰り返す
- B 番目の要素を含む push をした「加入」クエリではじめてその位置が 0 以上になるので，
ある位置に記録した番号の「加入」クエリは，
その位置の「サービス」クエリにおける目的の要素を含むものである



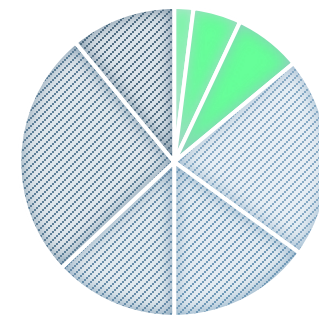
後半(小課題 6)を解けた

- したがって、「サービス」クエリの答えとなる要素がどの「加入」クエリ由来かを求めることが出来たので、後半も解けた

前半 + 後半 = 満点

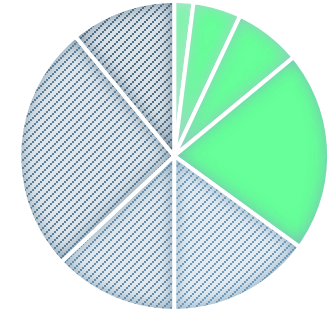


- したがって、「サービス」クエリの答えとなる要素がどの「加入」クエリ由来かを求めることが出来たので、後半も解けた
- 前半と後半を組み合わせて $O((N + Q) \log(N + Q))$ 時間の解法が得られ、満点をもぎとれる



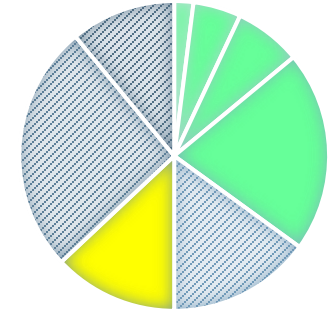
別解(平面走査)

- あらかじめ全てのクエリが与えられている
- そこで、待機列の番号を時間軸とした順番で処理することを考える
- 前半と後半に分割するところまでは同じ



前半(小課題 4)

- 位置 $[Q]$ に作用素 $x \leftarrow x + K$ を置く/取り除く
- 位置 $[Q]$ に作用素 $x \leftarrow \min(x - K, 0)$ を置く/取り除く
- 区間 $[0, Q]$ の作用素を順に合成した結果を求める
- 双対セグメント木(区間更新・一点取得)ではなく,
本来のセグメント木(一点更新・区間取得)の形になった
- 同様に解ける



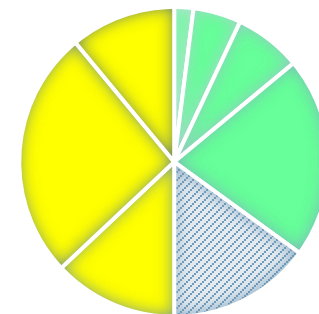
後半(小課題 6)

- 位置 $[Q]$ に整数 K を置く/取り除く
- 区間 $[0, x]$ の総和が B 以上になる最小の x を求める



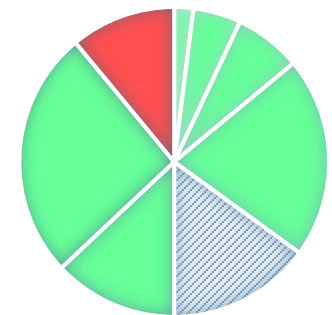
後半(小課題 6)

- 位置 $[Q]$ に整数 K を置く/取り除く
- 区間 $[0, x]$ の総和が B 以上になる最小の x を求める
- セグメント木上で二分探索をすればよい



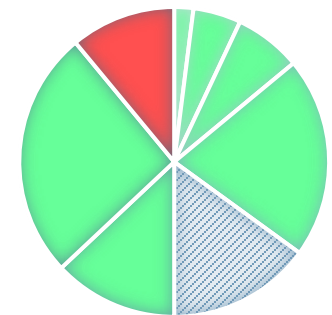
後半(小課題 6)

- 位置 $[Q]$ に整数 K を置く/取り除く
- 区間 $[0, x]$ の総和が B 以上になる最小の x を求める
- セグメント木上で二分探索をすればよい
- 前半と一緒にそのまま実装して...



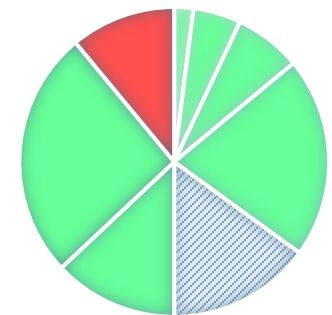
後半(小課題 6)

- 位置 $[Q]$ に整数 K を置く/取り除く
- 区間 $[0, x]$ の総和が B 以上になる最小の x を求める
- セグメント木上で二分探索をすればよい
- 前半と一緒にそのまま実装して…**TLE**



後半(小課題 6)

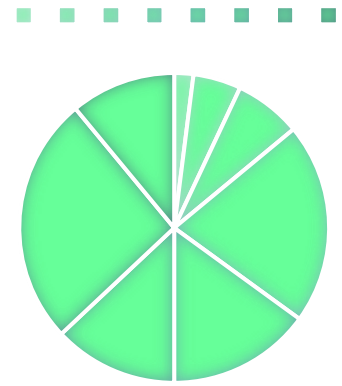
- 位置 $[Q]$ に整数 K を置く/取り除く
- 区間 $[0, x]$ の総和が B 以上になる最小の x を求める
- セグメント木上で二分探索を $O(\log N)$ 時間で行いたい



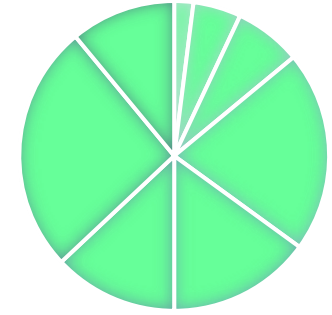
後半(小課題 6)

- 位置 $[Q]$ に整数 K を置く/取り除く
- 区間 $[0, x]$ の総和が B 以上になる最小の x を求める
- セグメント木上で二分探索を $O(\log N)$ 時間で行いたい
- ノードを上から見て,
 - 左の子が x 以上なら左へ
 - 左の子が x 未満なら x から左の子の値を減じてから右へ
- 移動することを, 葉に行きあたるまで繰り返す
- これで $O(\log N)$ 時間になり,

後半(小課題 6)



- 位置 $[Q]$ に整数 K を置く/取り除く
- 区間 $[0, x]$ の総和が B 以上になる最小の x を求める
- セグメント木上で二分探索を $O(\log N)$ 時間で行いたい
- ノードを上から見て,
 - 左の子が x 以上なら左へ
 - 左の子が x 未満なら x から左の子の値を減じてから右へ
- 移動することを, 葉に行きあたるまで繰り返す
- これで $O(\log N)$ 時間になり, **AC**



後半(小課題 6)

- 位置 $[Q]$ に整数 K を置く/取り除く
- 区間 $[0, x]$ の総和が B 以上になる最小の x を求める
- 平面走査をしなくともパラレル二分探索によって解けるが、結局 $\log$ が 2 つつくため満点は得られない(と思う)

得点分布

得点分布

- 2 点: 1 人
- 7 点: 4 人
- 14 点: 3 人
- 21 点: 1 人
- 27 点: 1 人
- 35 点: 1 人
- 89 点: 2 人
- 100 点: 1 人

逃走経路 (Escape Route)

解説: gotoloop

問題概要

- N 頂点 M 辺の無向な連結グラフが与えられる
- 各辺には通過にかかる時間があり, 通れる時間も周期的である(周期はすべての辺で共通)
- Q 個のクエリが与えられ, それぞれのクエリは「あるタイミングで頂点 U_j から V_j まで移動するとき, どれだけ時間がかかるか」というもの
- 制約は $N \leq 90$, $M \leq N(N-1)/2$, $Q \leq 3000000$

小課題1

- $N \leq 40, Q \leq 1000$
- クエリごとに普通のDijkstra法をおこなう
- $O(MQ \log M)$
- 5点

小課題2

- $N \leq 40, U_j = 0$
- まず、その日のうちに目的の都市にたどり着けるかどうかを考える
- これは、目的の都市に時刻**S-1**に到着することを想定して、すべての都市から逆方向にDijkstra法をおこなうことによって $O(NM \log M)$ で事前計算できる

小課題2

- 次に、目的の都市にその日のうちにたどり着けない場合を考える
- この場合、必ずある都市からは時刻0に出発することになる
- そのため、その日のうちに都市0からたどり着ける都市を列挙し、それらの都市を時刻0に出発したときに目的の都市にたどり着くのにかかる時間を比較すればよい
- $O(NM \log M + NQ)$
- 以降は、その日のうちにたどり着ける場合のみを考える

小課題2

- 最後に、その日のうちに目的の都市にたどり着ける場合を考える
- 都市0から変形Dijkstra法をおこなう
- 出発時刻が遅くなると、都市0からある都市への最短経路は最大でM回切り替わる
 - 切り替わりうるのは、最短経路に使っていた道が通れなくなったときのみ
- 切り替わる時刻を持ってDijkstra法をおこなう

小課題2

- 都市0を時刻S-1に出発することをまず想定する
- そして道を通るときに、その道を通れるようなギリギリの出発時刻での計算も行うようにする
(各頂点が出発時刻ごとの所要時間を持っているイメージ)
- $O((M^2 + Q) \log M)$
- すべて合わせて $O(M^2 \log M + NQ)$ なので20点

小課題3

- $N \leq 40$
- 小課題2の変形Dijkstra法をすべての都市からおこなう
- $O((NM^2 + Q) \log M)$
- ブロックサイズを B として $O(N^2Q/B + N^3MB)$ となる解法もあるらしい
- どちらにせよ35点

小課題4

- $N \leq 60$
- あるルートについて、出発時刻をだんだんと遅くしてゆくと、ある時刻からある道の制約によってそのルートが使えなくなる
- 逆に、そのルートはこの「ある道」の制約に引っかかりさえしなければ使うことができる

小課題4

- この「あるルート」が都市Uから都市Vに行く最短経路である必要条件是、「ある道」の端点のうち「あるルート」上で都市Uに近いものを都市X、都市Vに近いものを都市Yとすると
 - 「あるルート」内での都市Uから都市Xまでのルートが、都市Xに時刻 $C[\text{ある道}] - L[\text{ある道}]$ にたどりつくという条件における最短経路となっていること
 - 「あるルート」内での都市Yから都市Vまでのルートが、都市Yを時刻 $C[\text{ある道}]$ に出発するという条件における最短経路となっていること
- となる

小課題4

- この条件を満たすようなルートは都市Uから都市VまででM個以下しかない
- 前処理として、すべての辺の端点について、都市Xとなるときと都市Yとなるときを両方を考え、Dijkstra法を行う
- ここは全体で $O(M^2 \log M)$

小課題4

- 都市Uを出発して都市Vに到着するクエリに答えるには、都市Uから都市Vへの「あるルート」で使えるもののうち、最も短いものがわかればよい
- それぞれの「あるルート」は使える時刻の上限が決まっているので、その順番でソートして処理をすればクエリに高速に答えられる
- $O(N^2M \log M + Q \log M)$

小課題4

- 以上より全体で $O(N^2M \log M + NQ)$ となる
- $O(N^5 + NQ)$ となる解法もあるらしい
- どちらにせよ70点

小課題5(満点)

- 追加の制約なし ($N \leq 90$)
- 小課題4を高速化する
- まず、Dijkstra法の対象が密なグラフなので、 $O(M \log M)$ だったものが $O(N^2)$ になる
- これによりDijkstra法の部分が全体で $O(N^2M)$ になる

小課題5(満点)

- 次に「あるルート」をソートする側であるが、使える時刻の上限は都市Uから都市Xへのルートで決定されるので、都市Uからすべての都市Xまでのルートをあらかじめソートしておけば、 $\log$ を落とすことができる
- 両方の高速化を使って $O(N^2M + NQ)$
- 100点

JOI 2020/2021 春合宿 Day2

道路の建設案 (Road Construction)

解説: 木ノ下 恭範

問題概要

- N 個の町があり、町 i の座標は (X_i, Y_i) 。
- 町 i と町 j をつなぐ道路は $|X_i - X_j| + |Y_i - Y_j|$ 円かかる。
- $N(N - 1)/2$ 通りのうち、安い方から K 個の道路の費用を順に出力してください。
- $N \leq 250000$
- $K \leq 250000$

小課題 1 : $N \leq 1000$

- $N(N - 1)/2$ 通りを全て計算して、ソートして出力すればよい。
- 時間計算量 $\Theta(N^2 \log(N))$
- 5 点

小課題 2 : $Y_i = 0$

- 全ての点が X 軸上に並んでいる。
- 道路の費用は $|X_i - X_j|$ となり、数直線上の距離と一致する。
- 町は X の昇順にソートされているとする。
- 費用の安い順に、町の組を列挙していくアルゴリズムが作れる。
- 全ての町について、その左隣の町との距離を考える。最も安い組は、このうちのいずれかである。その組を $(i-1, i)$ とすれば、2 番目に安い組は $(i-1, i)$ を除外して代わりに $(i-2, i)$ を入れたときの最小である。これを繰り返せばよい。

小課題 2 : $Y_i = 0$

- 費用の安い順に、町の組を列挙していくアルゴリズムが作れる。
- 全ての町について、その左隣の町との距離を考える。最も安い組は、このうちのいずれかである。その組を $(i-1, i)$ とすれば、2 番目に安い組は $(i-1, i)$ を除外して代わりに $(i-2, i)$ を入れたときの最小である。これを繰り返せばよい。
- 優先度付きキュー (std::priority_queue) を用いると効率的に実装できる。
- 時間計算量 $O((N + K) \log(N))$
- 6 点

小課題 3 : $K = 1$

- 町は X の昇順にソートされているとする。
- 町 i と町 j ($i < j$) をつなぐ費用を整理する。
- X 昇順にソートしたので、 $X_i \leq X_j$ 。
- $Y_i \leq Y_j$ ならば、費用は $(X_j - X_i) + (Y_j - Y_i)$ 。
- $Y_i \geq Y_j$ ならば、費用は $(X_j - X_i) + (Y_i - Y_j)$ 。
- 絶対値がなくなったので、 i に関係する項と j に関係する項に分解することが出来る。
- $Y_i \leq Y_j$ ならば、費用は $(-X_i - Y_i) + (X_j + Y_j)$ 。
- $Y_i \geq Y_j$ ならば、費用は $(-X_i + Y_i) + (X_j - Y_j)$ 。

小課題 3 : $K = 1$

- 町 i と町 j ($i < j$) をつなぐ費用を整理する。
- $Y_i \leq Y_j$ ならば、費用は $(-X_i - Y_i) + (X_j + Y_j)$ 。
- $Y_i \geq Y_j$ ならば、費用は $(-X_i + Y_i) + (X_j - Y_j)$ 。
- 各 j について、最も安い相手 i ($i < j$) を求める。
- 町を X 昇順に読み、 Y 座標を添え字とする Segment Tree で $-X_i - Y_i$ と $-X_i + Y_i$ それぞれの最小値を管理する。 j の手前まで読んだ段階で、 $Y_i \leq Y_j$ の範囲での $-X_i - Y_i$ の最小値を取得し、 $X_j + Y_j$ を加算すればよい。 $Y_i \geq Y_j$ についても同様。
- Y 座標は大きいので、添え字に使うために座標圧縮しておく。

小課題 3 : $K = 1$

- 各 j について、最も安い相手 i ($i < j$) を求める。
- 町を X 昇順に読み、 Y 座標を添え字とする Segment Tree で $-X_i - Y_i$ と $-X_i + Y_i$ それぞれの最小値を管理する。 j の手前まで読んだ段階で、 $Y_i \leq Y_j$ の範囲での $-X_i - Y_i$ の最小値を取得し、 $X_j + Y_j$ を加算すればよい。 $Y_i \geq Y_j$ についても同様。
- Y 座標は大きいので、添え字に使うために座標圧縮しておく。
- 時間計算量 $\Theta(N \log(N))$
- 7 点

小課題 4 : $K \leq 10$

- $K = 2$ で考えることにする。
- 小課題 3 のアルゴリズムを用いて、最も安い町の組 (a, b) を求める。
- $N(N - 1)/2$ 通りある町の組を、以下の 2 種類に分類する。
 - a または b を含むもの
 - a も b も含まないもの
- 2 番目に安い組は、前者に含まれるか、さもなくば後者の中で最も安い。
- 前者は $2N - 3$ 組しかないため列挙できる。
- 後者は a, b を除いた町群に小課題 3 を適用すればよい。

小課題 4 : $K \leq 10$

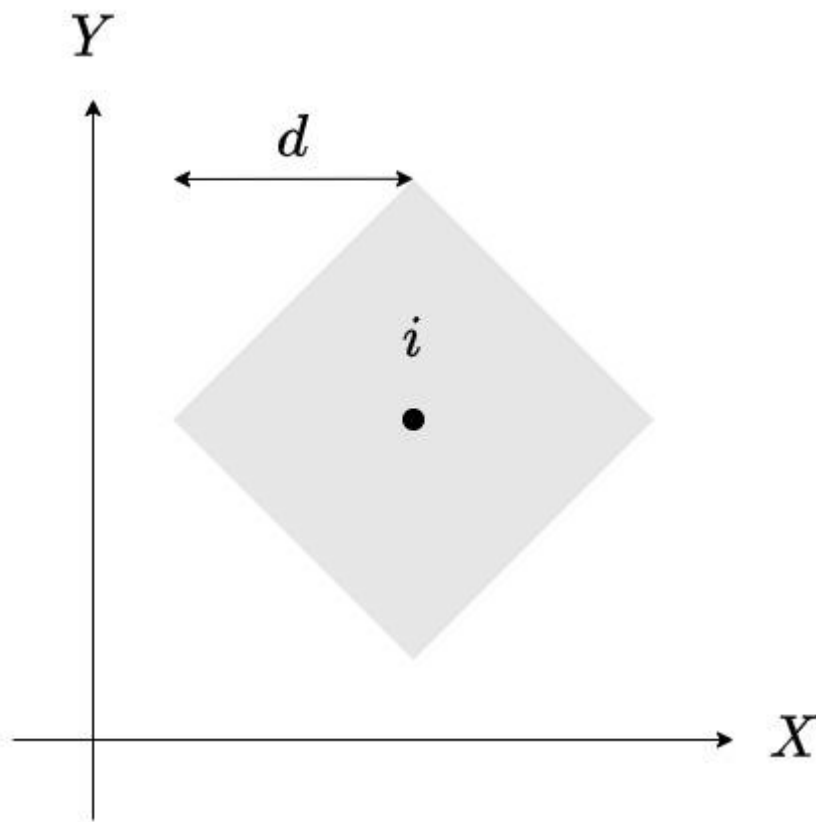
- $K \leq 10$ も同様に解くことができる。
- 以下の操作を K 回繰り返す
 - 最も安い組 (a, b) を計算する。
 - a, b を含む組を全て列挙する。
 - 町の集合から a, b を取り除く。
- 列挙された組のうち、安い方から K 組を出力する。
- 時間計算量 $\Theta(KN \log(N))$
- 20 点

小課題 5 : $N \leq 10^5$

- 費用 d 以下の町の組の数を数えることができれば、二分探索を用いることで、 K 番目に安い道路の費用を計算することが出来る。その値を D とすれば、費用 $D - 1$ 以下の町の組を列挙した後、 D を何回か出力すればよい。
- 各町 i について、町 i とつなぐ費用が d 以下の町の個数を数えたい。また、そのような街を列挙したい。

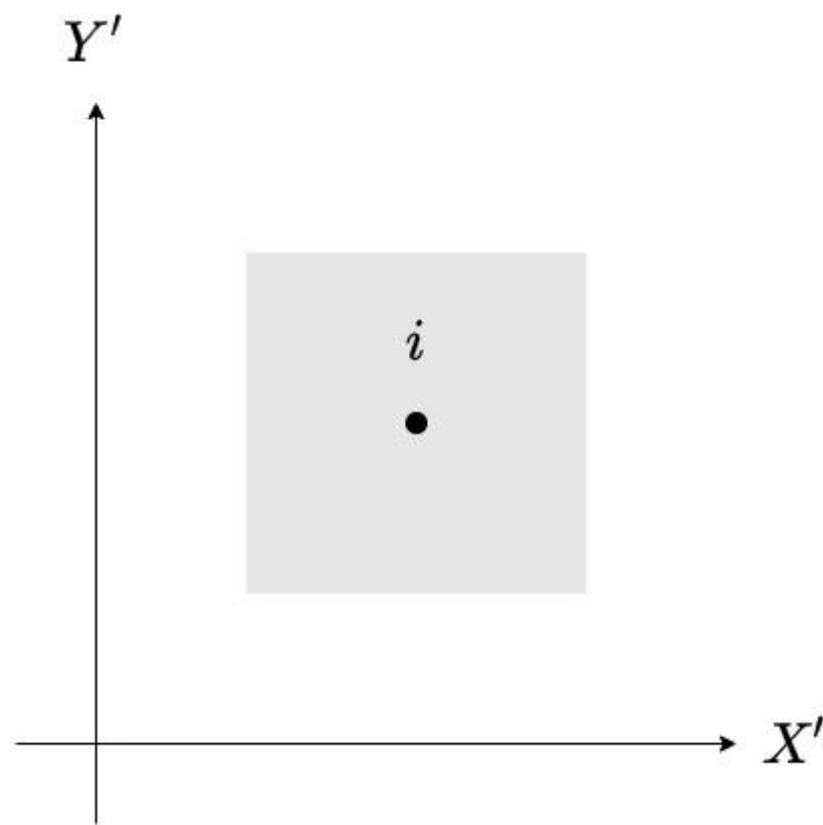
小課題 5 : $N \leq 10^5$

- 町 i とつなぐ費用が d 以下の町の座標範囲を図示すると、
下図のようになる。



小課題 5 : $N \leq 10^5$

- 45° 回転させることで正方形の領域となり、扱いやすくなる。
- $X' := X + Y, \quad Y' := X - Y$ とすれば、回転できる。
- 費用は $\max\{|X'_i - X'_j|, |Y'_i - Y'_j|\}$ となる。



小課題 5 : $N \leq 10^5$

- $X' := X + Y, \quad Y' := X - Y$
- 費用は $\max\{|X'_i - X'_j|, |Y'_i - Y'_j|\}$ となる。
- 町を X' の昇順にソートし、各 j について、費用 d 以下の範囲に入っている町 i ($i < j$) の個数を数え、足し合わせる。
- そのような i の条件は、 $X'_j - d \leq X'_i$ かつ $Y'_j - d \leq Y'_i \leq Y'_j + d$ となる。
- X' について尺取り法を用い、 Y' を添え字とする Segment Tree で Y'_i が特定の範囲に入る i の個数を高速に取得できるようにすると、全体で $\Theta(N \log(N))$ で数えられる。

小課題 5 : $N \leq 10^5$

- X' について尺取り法を用い、 Y' を添え字とする Segment Tree で Y'_i が特定の範囲に入る i の個数を高速に取得できるようにすると、全体で $\Theta(N \log(N))$ で数えられる。
- Segment Tree ではなく `std::set` を用いれば、費用 d 以下の組を全て列挙することができる。
- 時間計算量 $\Theta(N \log(N) \log(A) + K \log(K))$
- 27 点
- 速度次第では、満点が取れる。

小課題 6：満点

- 小課題 3 ($K = 1$) で用いたアルゴリズムを発展させる。
- 町を X 昇順に読み、 Y 座標を添え字とする Segment Tree で $-X_i - Y_i$ と $-X_i + Y_i$ それぞれの最小値を管理する。 j の手前まで読んだ段階で、 $Y_i \leq Y_j$ の範囲での $-X_i - Y_i$ の最小値を取得し、 $X_j + Y_j$ を加算すればよい。 $Y_i \geq Y_j$ についても同様。(小課題 3)
- $-X_i - Y_i$ の最小値を $+\infty$ で更新して再び最小値を取得すれば、($Y_i \leq Y_j$ の範囲で) 2 番目に安い町を取得できる。

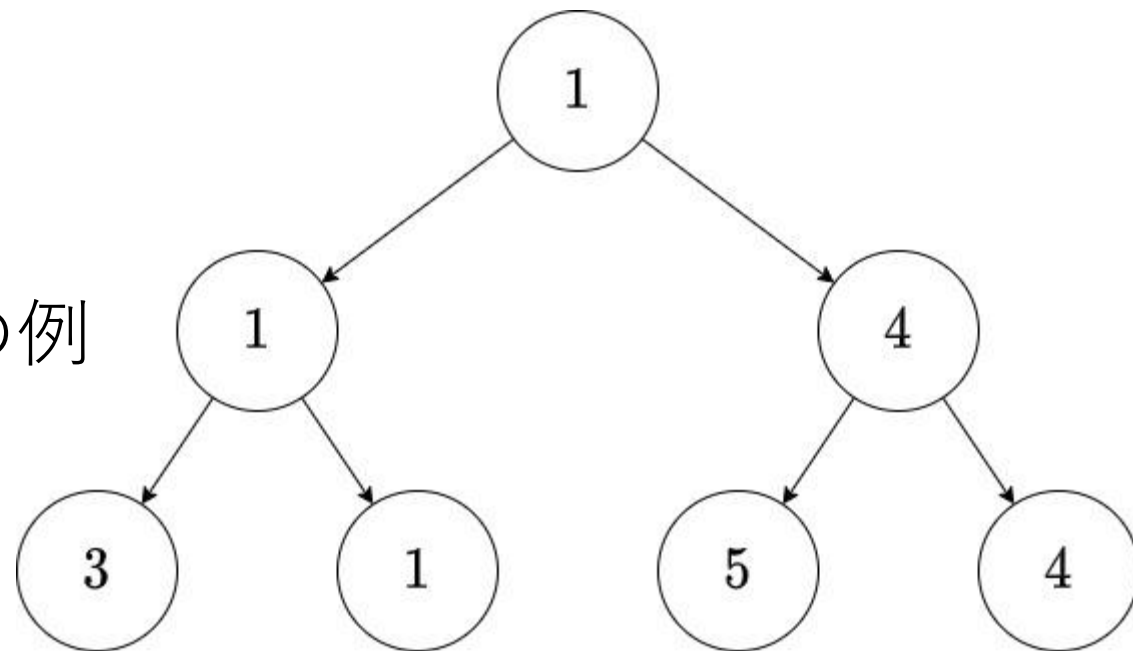
小課題 6：満点

- $-X_i - Y_i$ の最小値を $+\infty$ で更新して再び最小値を取得すれば、($Y_i \leq Y_j$ の範囲で) 2 番目に安い町を取得できる。
- 小課題 2 ($Y_i = 0$) で用いたアルゴリズムのように、最も安い組 (i, j) を出力し、 (i, j) を同じ j で 2 番目に安い組 (i', j) に置き換えることを K 回繰り返す。
- これを行うためには、町を X 昇順に読む際に刻々と変化する Segment Tree の全ての時刻に後からアクセスする必要がある。 $(+\infty$ への更新も行う)
- 過去のデータにアクセスできるデータ構造を、永続データ構造と呼ぶ。Segment Tree を永続的にできるだろうか？

小課題 6：満点

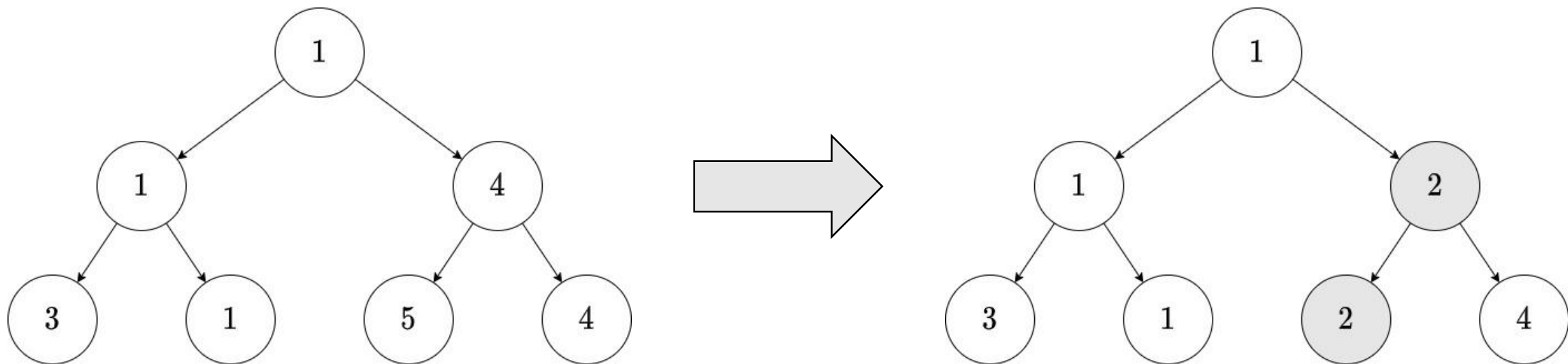
- Segment Tree を永続的にできるだろうか？
- Segment Tree の木構造を、それぞれのノードが左右の子を指すことで表現する。例えば、ノードの配列を用意して、左右の子をその添え字で表せばよい。

最小値を管理する Segment Tree の例



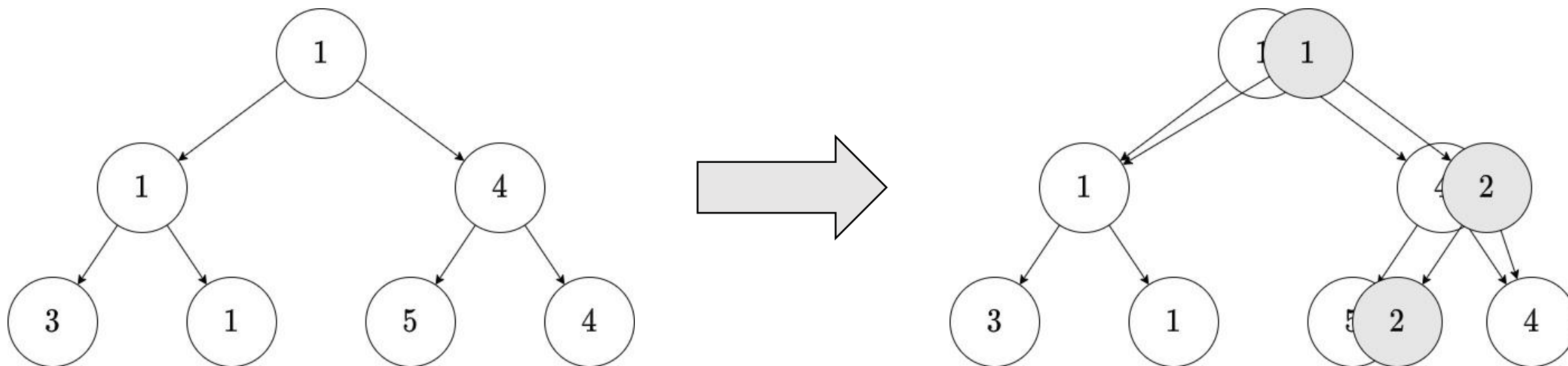
小課題 6：満点

- 5 と書かれた位置を 2 に更新することを考える。
- 通常の Segment Tree は、単にいくつかのノードの値を書き換える。(書き換わったノードを灰色で示した)



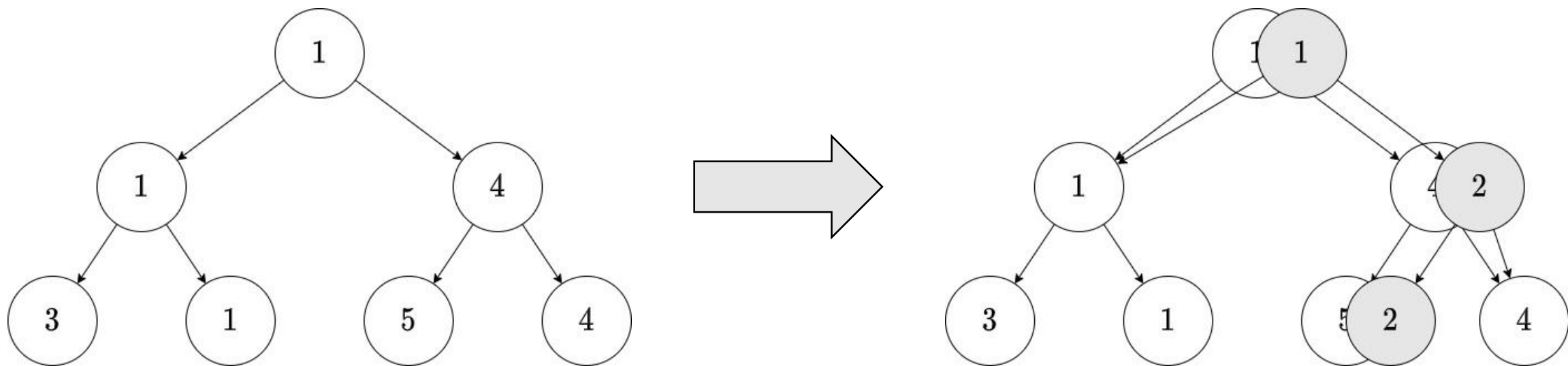
小課題 6：満点

- 5 と書かれた位置を 2 に更新することを考える。
- 永続的な Segment Tree では、書き換えは行わずに新たにノードを作成する。
- 更新前と後のそれぞれの根に着目すると、その根から辺を辿って到達できるノードの集合は、通常の Segment Tree として見ることができる。

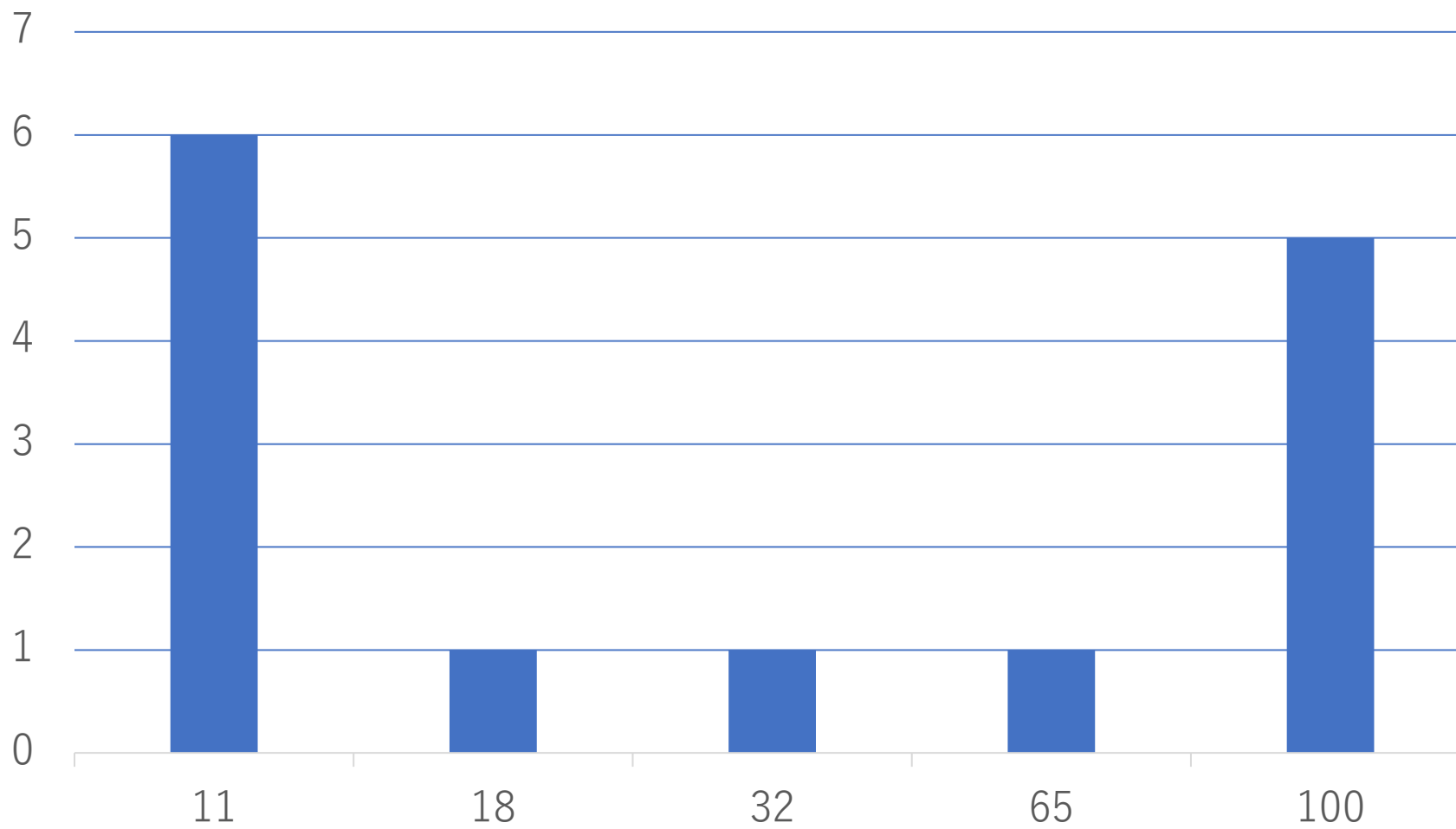


小課題 6：満点

- 1 回の更新の時間計算量は $\Theta(\log(N))$ となる。
- 解法全体の時間計算量 $\Theta((N + K) \log(N))$
- 35 点



得点分布



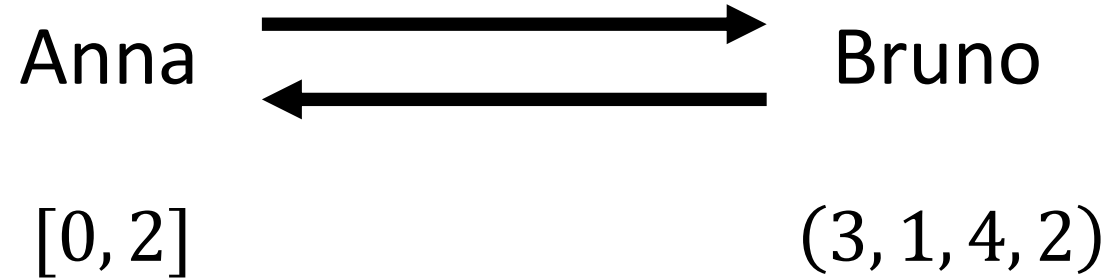
Shopping 解説

担当：高谷 悠太

問題概要

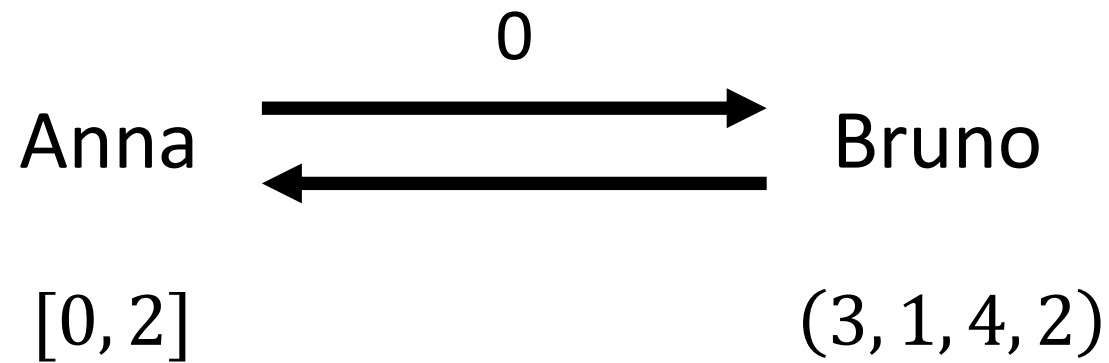
- N 個の商品がある.
- Anna は L 番目から R 番目の商品のうち、最も安い商品を購入したい.
- 各商品の値段を直接知ることとはできないため、値段を把握している Bruno と通信を行う.

やり取りの例



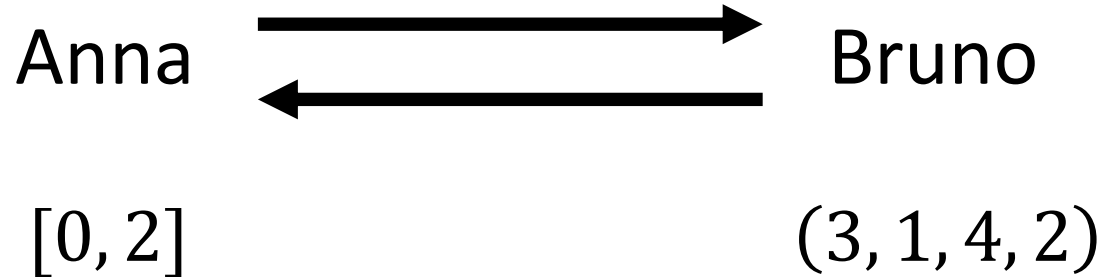
$L = 0$ と伝えたい

やり取りの例



$L = 0$ と伝えたい

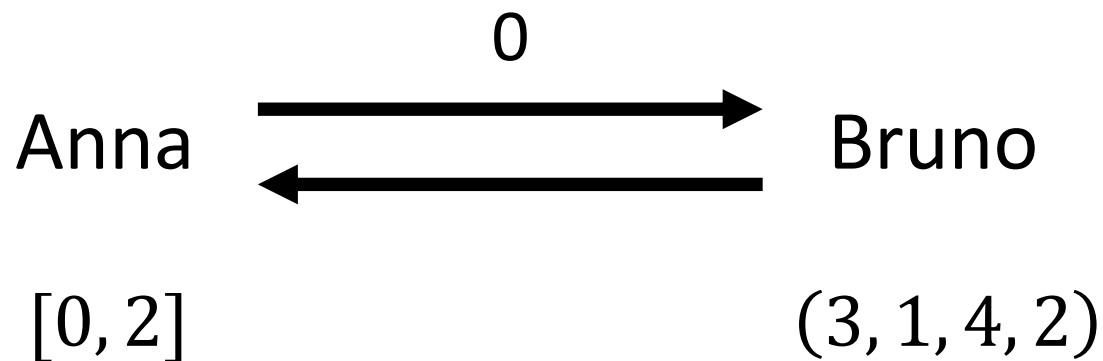
やり取りの例



$L = 0$ と伝えたい

1st bit : 0

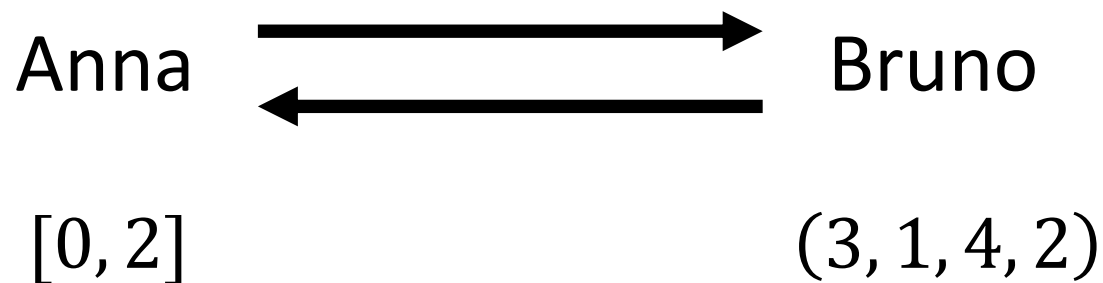
やり取りの例



$L = 0$ と伝えたい

1st bit : 0

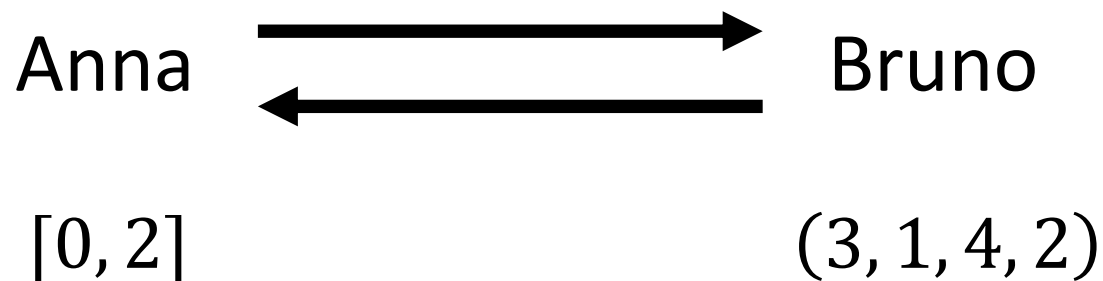
やり取りの例



$L = 0$ と伝えたい

1st bit : 0
2nd bit : 0

やり取りの例



$L = 0$ と伝えたい

1st bit : 0

2nd bit : 0

$L = 0$ と気づく

注意

- 厳密な通信方法を把握するのはとても大切.
- 例えば, 双方向の通信といっても
 - 0 と 1 からなる**文字列**を送りあう
 - 0 と 1 の**文字**を送りあうでは情報の送り方が大きく異なる.

注意

- 今回の通信法は後者.
- つまり，相手が1ステップの通信を終えるまで待機し，その後自分が次のステップの通信を行う，などは不可能.

注意

- 双方向通信はあまり見ない形式だが、
クエリの扱い方だけでなく、クエリ内容まで
参加者側に自由度がある通信課題
といえる.
- クエリの効率的な扱い方を考える一般的な通信
課題と違い、どのようなクエリを相手に質問す
るかが重要となる.

過去の例

- ふたつの交通機関 (JOI 2018/2019 春合宿)

準備

- 以下のように定式化しておく.
- Anna は区間 $[L, R]$ を, Bruno は数列 P を持っている.
- Anna は通信を行うことで, $[L, R]$ のうち数列 P において最小値をとる index を求めたい. どの程度の通信量で可能か.

小課題 1 : $N \leq 1000$

- Bruno が P のすべての値を伝える.
 - 一つの値を伝えるには $\log_2 N$ bit 必要.
- 以上より, 以下の通信量で可能.
 - Anna : 0 bit
 - Bruno : $N \log_2 N$ bits

小課題 2 : $N \leq 10,000$

- Anna は L の値を伝える.
 - これは $\log_2 N$ bits 必要.
- Bruno は L 以上の各整数 X について,
数列 P が $[L, X]$ において X で最小値をとるか
かどうか伝える.
 - これは N bits 必要.

小課題 2 : $N \leq 10,000$

- $[L, X]$ において X で最小値をとるような X のうち, R 以下最大のものが求める index.
- 以上より, 以下の通信量で可能.
 - Anna : $\log_2 N$ bits
 - Bruno : N bits

小課題 3 : $N \leq 1,000,000$

- ここでは三つの解法を紹介する.
 1. バケット法 (32 点)
 2. Cartesian Tree 上での重心分解 (80 点)
 3. 二分探索 (100 点)

小課題 3 : バケツ法

- バケツサイズ B を固定する.
 - Anna と Bruno の間でこの値を共有しておく.
- 数列 P を N/B 個の長さ B の数列に分割する.
- Anna は L, R が属するバケツの番号を送る.
 - 合計で $2 \log_2 \left(\frac{N}{B} \right) - 1$ bits 必要.

小課題 3 : バケツト法

- L, R が属するバケツトの番号を l, r とする.
- このとき, 求める index は
 - バケツト l に属す
 - バケツト $l + 1, \dots, r - 1$ で最小値をとる index
 - バケツト r に属すのいずれかを満たす.

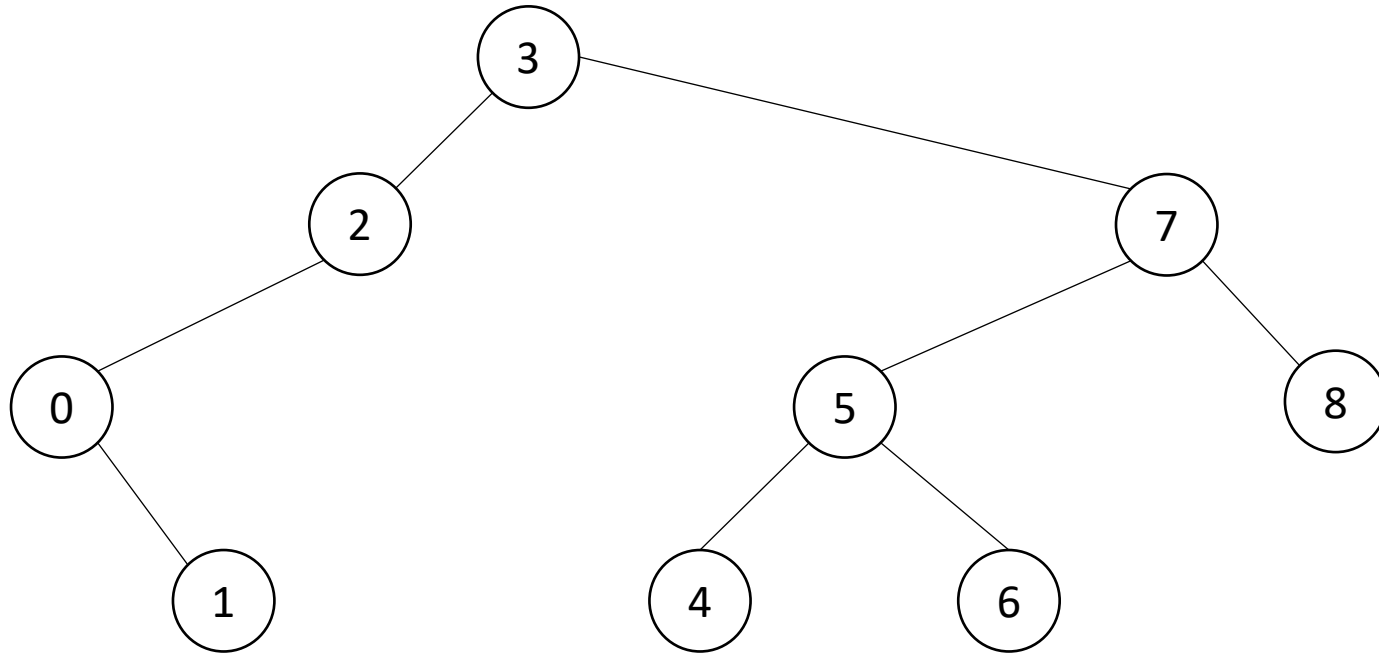
小課題 3 : バケツ 法

- Bruno はまずバケツ $l + 1, \dots, r - 1$ で最小値をとる **index** を送る.
 - これは $\log_2 N$ bits 必要.
- そして, $2B + 1$ 個の候補についての **Cartesian Tree** の情報を送る.

Cartesian Tree

- 数列 Q に対する Cartesian Tree の定義：
 1. 数列 Q の各 index を頂点とする根付き二分木.
 2. 最小値をとる index を根とする.
 3. 根の左にある部分木は、最小値をとる index より左の部分列に対する Cartesian Tree である.
 4. 根の右にある部分木は、最小値をとる index より右の部分列に対する Cartesian Tree である.

Cartesian Tree



5	9	3	1	7	4	6	2	8
---	---	---	---	---	---	---	---	---

Cartesian Tree の性質

- Cartesian Tree は最小値の情報は覚えていないが、最小値をとる index は覚えている.
- $[L, R]$ で最小値をとる index に対応する頂点は、頂点 L と頂点 R の LCA である.
- よって、数列全体の情報は必要なく、対応する Cartesian Tree の情報を送れば十分である.

Cartesian Tree の送り方

- 根付き二分木の個数は Catalan 数で表せ、頂点数を M とすると高々 4^{M-1} 個である.
- 実際, Cartesian Tree の構成を考えると, $2M - 2$ bits で送ることが可能.

Cartesian Tree の構成

1. 空の stack S を用意し, Q の要素を先頭から見ていく.
2. 今見ている値より S の先頭の値が大きい場合, S の先頭を **pop** する.
3. 今見ている値より S の先頭の値が小さい場合, S の先頭に今見ている値を **push** し, 次の要素に進む.

Cartesian Tree の構成送り方

1. 空の stack S を用意し, Q の要素を先頭から見っていく.
2. 今見ている値より S の先頭の値が大きい場合, S の先頭を **pop** する. **このとき 0 を送る.**
3. 今見ている値より S の先頭の値が小さい場合, S の先頭に今見ている値を **push** し, 次の要素に進む. **このとき 1 を送る.**

小課題 3 : バケツ法

- Anna は L, R が属するバケツの番号を送る.
 - 合計で $2 \log_2 \left(\frac{N}{B} \right) - 1$ bits 必要.
- Bruno はバケツ $l + 1, \dots, r - 1$ で最小値をとる index を送る.
 - これは $\log_2 N$ bits 必要.
- そして, $2B + 1$ 個の候補についての Cartesian Tree の情報を送る.
 - これは $4B$ bits 必要.

小課題 3 : バケツト法

- まとめると, 以下で可能.
 - Anna : $2 \log_2 \left(\frac{N}{B} \right) - 1$ bits
 - Bruno : $4B + \log_2 N$ bits
- $B = 1400$ 程度にすると 32 点が得られる.

小課題 3 : 重心分解

- (再掲)

$[L, R]$ で最小値をとる index に対応する頂点は、頂点 L と頂点 R の LCA である.

- つまり、問題は以下のように言い換えられる.

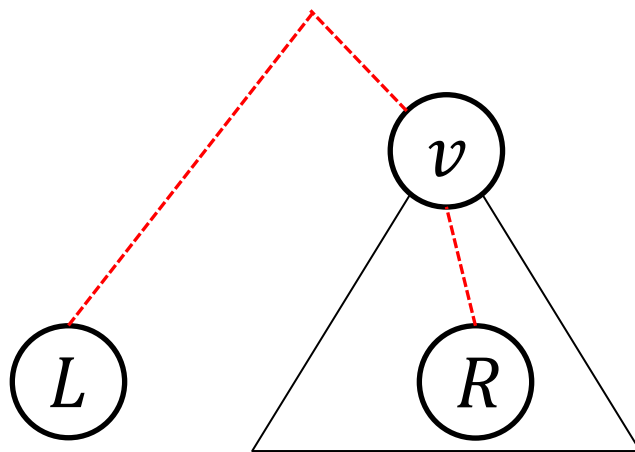
- Anna は二頂点 L, R を, Bruno は二分木を持っている.
- 通信を行うことで, Anna は Bruno の木における LCA を求めたい.

小課題 3 : 重心分解

- ある頂点 v を根とする部分木に対応する区間を $[l, r]$ とする. Bruno はまず l, r を送る.
- Anna は以下のようにして 01 を返す.
 - $[L, R]$ が $[l, r]$ に含まれていれば 1 を返す.
 - そうでない場合 0 を返す.
- Bruno は Anna の返答から何が分かるか？

小課題 3 : 重心分解

- $[L, R]$ が $[l, r]$ に含まれていれば,
当然 L と R の LCA は $[l, r]$ に含まれる.
- そうでない場合, $[l, r]$ が頂点 v を根とする部分木に対応する区間であることから, LCA が $[l, r]$ に含まれることはないことが分かる.



小課題 3 : 重心分解

- Bruno は Anna からの返答により, 求める答えが $[l, r]$ に含まれるかどうか判定できる.
- できるだけ $[l, r]$ の長さを $N/2$ に近づければ, 毎回頂点数が半減するのではないか?
 - 18 回繰り返せば最後 4 頂点しか残らず AC する?

小課題 3 : 重心分解

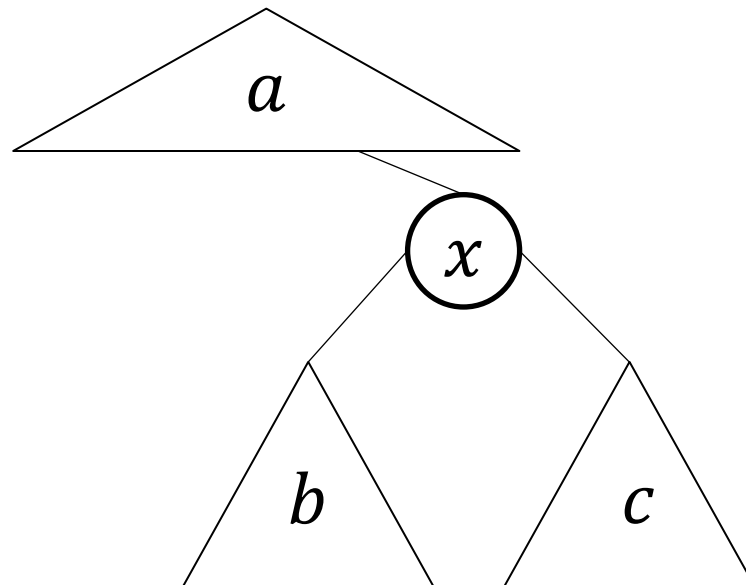
- Bruno は Anna からの返答により, 求める答えが $[l, r]$ に含まれるかどうか判定できる.
- できるだけ $[l, r]$ の長さを $N/2$ に近づければ, 毎回頂点数が半減するのではないか?
 - 18 回繰り返せば最後 4 頂点しか残らず AC する?
 - 実はそんなに甘くない.

小課題 3 : 重心分解 (algorithm)

- まず以下を 18 回繰り返す.
 - Bruno は $[l, r]$ を長さが $V/2$ に最も近くなるようにとり, それを送る. V : 残り頂点数
 - これは $2 \log_2 V - 1$ bits で可能.
 - Anna は $[L, R]$ が $[l, r]$ に含まれるかどうか判定する. そして, 不必要な頂点を削除する.
- 最後に残った頂点について, Bruno は Cartesian Tree を送る.
 - これは $2V - 2$ bits で可能. V : 残り頂点数

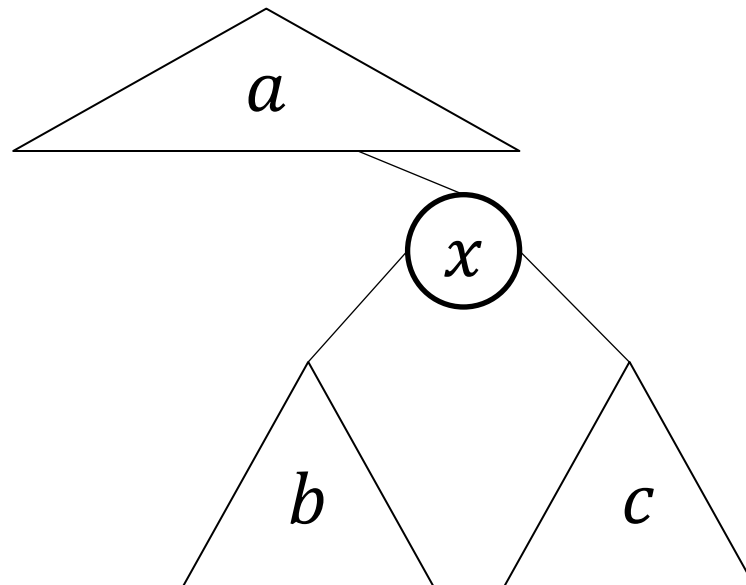
小課題 3 : 重心分解 (estimation)

- 1回でどの程度頂点数が減るか見積もる.
- x を重心とし, その周りの部分木の大きさを a, b, c とする.



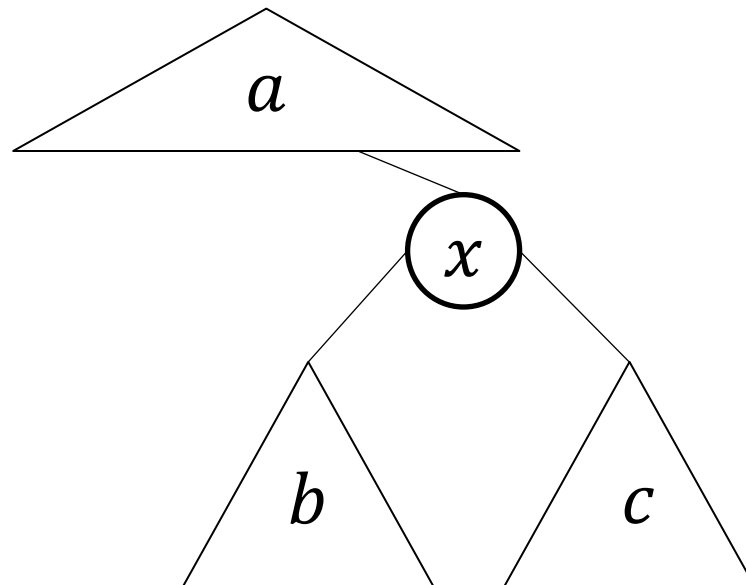
小課題 3 : 重心分解 (estimation)

- x を重心なので, $a, b, c \leq V/2$ である.
 - 特に, 次数が 2 以下であれば 1 回で半減する.
以下, 次数は 3 とする.



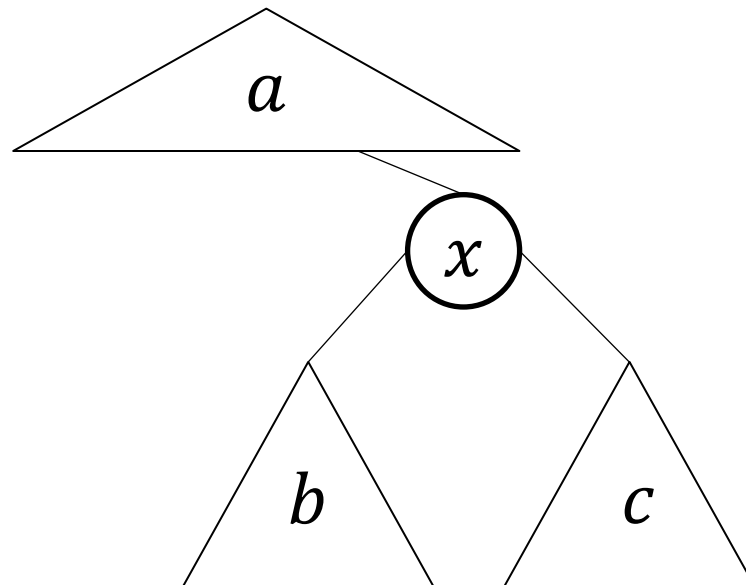
小課題 3 : 重心分解 (estimation)

- 対称性より, $a \geq b \geq c$ とする.
- 1 回行うと, サイズは高々 $b + c$ になり,
2 回行うと, サイズは高々 b になる.



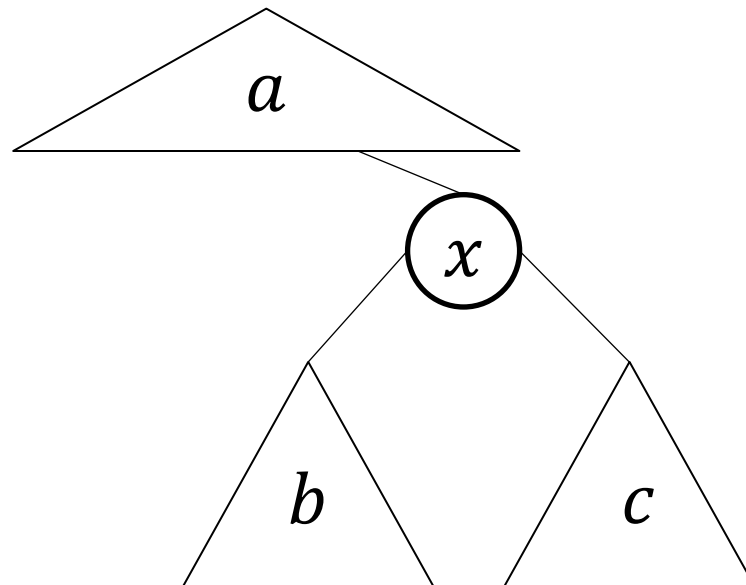
小課題 3 : 重心分解 (estimation)

- α を以下を満たす実数とすれば, (最後の 1 回外) 1 回あたり頂点数は α 倍となる.
 - $b + c \leq \alpha V$ もしくは $b \leq \alpha^2 V$



小課題 3 : 重心分解 (estimation)

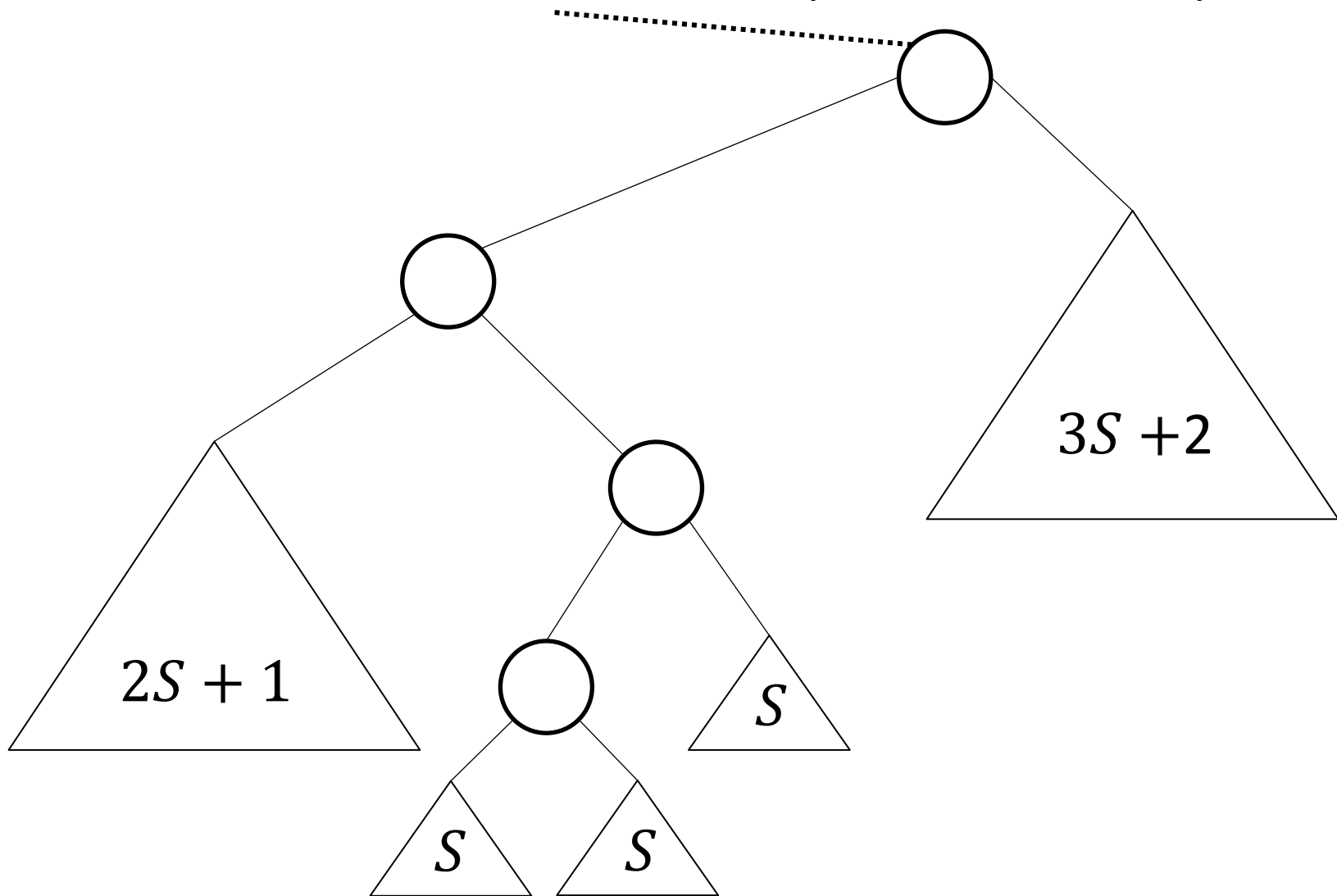
- $b + c > \alpha V$ かつ $b > \alpha^2 V$ となると
 $V > a + \alpha V \geq b + \alpha V > (\alpha^2 + \alpha)V$
なので. $\alpha^2 + \alpha = 1$ なる α は条件を満たす.
 - 実は α は黄金比の逆数.



小課題 3 : 重心分解 (estimation)

- 最後の 1 回はサイズは高々 $b + c$ にしかならないので、効率は $2/3$ である.
- よって、最後に残る頂点数は高々
$$1,000,000 \times \alpha^{17} \times \frac{2}{3} \approx 186$$
となる.

小課題 3 : 重心分解 (worst case)



小課題 3 : 重心分解 (estimation)

- 最初の 18 回の通信量 :
 - $2 \log_2 V - 1$ の値は 1 回目は 39, 18 回目は 16 程度なので, 合計 $(39 + 16) \times 18 \div 2 \approx 500$ bits 程度
- 最後の Cartesian Tree の通信量 :
 - $2 \times 186 - 2 = 370$ bits 程度
- 合計 870 bits で可能. 80 点程度.

小課題 3 : 二分探索

- 次のような場合を考える.

Bruno が $[L, R]$ に含まれる index X を一つ知っている.

- この場合, 1 回で要素数を半減させることができる.
 - Cartesian Tree における X の親について二分探索を行えばよいが, 応用上別の方法を示す.

小課題 3 : 二分探索

- $[X, X]$ から始め, 以下のように区間を伸ばし, 長さ 1 から N までの区間を得る.
 1. 長さ $n + 1$ の区間は, 長さ n の区間を左右いずれかに長さ 1 伸ばした区間である.
 2. ただし, 追加する値が**大きい**方向に区間を伸ばす.
 - つまり, 区間内の最小値ができるだけ大きくなる方向に区間を伸ばしていく.
- 長さ n の区間を I_n とする.

小課題 3 ： 二分探索

- $[L, R]$ が I_n に含まれていたとき，当然求める index は I_n に含まれる.
- 一方， I_n に含まれない場合は，求める index は以下のいずれかを満たす.
 1. I_n に含まれない.
 2. I_n で最小値をとる index と等しい.

小課題 3 : 二分探索

- $[L, R]$ が I_l に含まれず, I_r に含まれる, という l, r が求まると, 後は以下を送ればよい.
 1. I_l で最小値をとる index
 2. $I_r \setminus I_l$ および 先の値を含めた数列の Cartesian Tree
- これは $\log_2 N + 2(r - l)$ bits 必要.

小課題 3 : 二分探索

- 二分探索を k 回行えば $r - l = N/2^k$ となる.
- 二分探索 1 回には, Anna は 1 bit, Bruno は $\log_2 V - 1$ bits (V : 残り要素数) かかる.
- よって, 以下の通信量で可能.
 - Anna : k bits
 - Bruno : $\frac{k(2\log_2 N - k - 1)}{2} + N/2^{k-1}$ bits

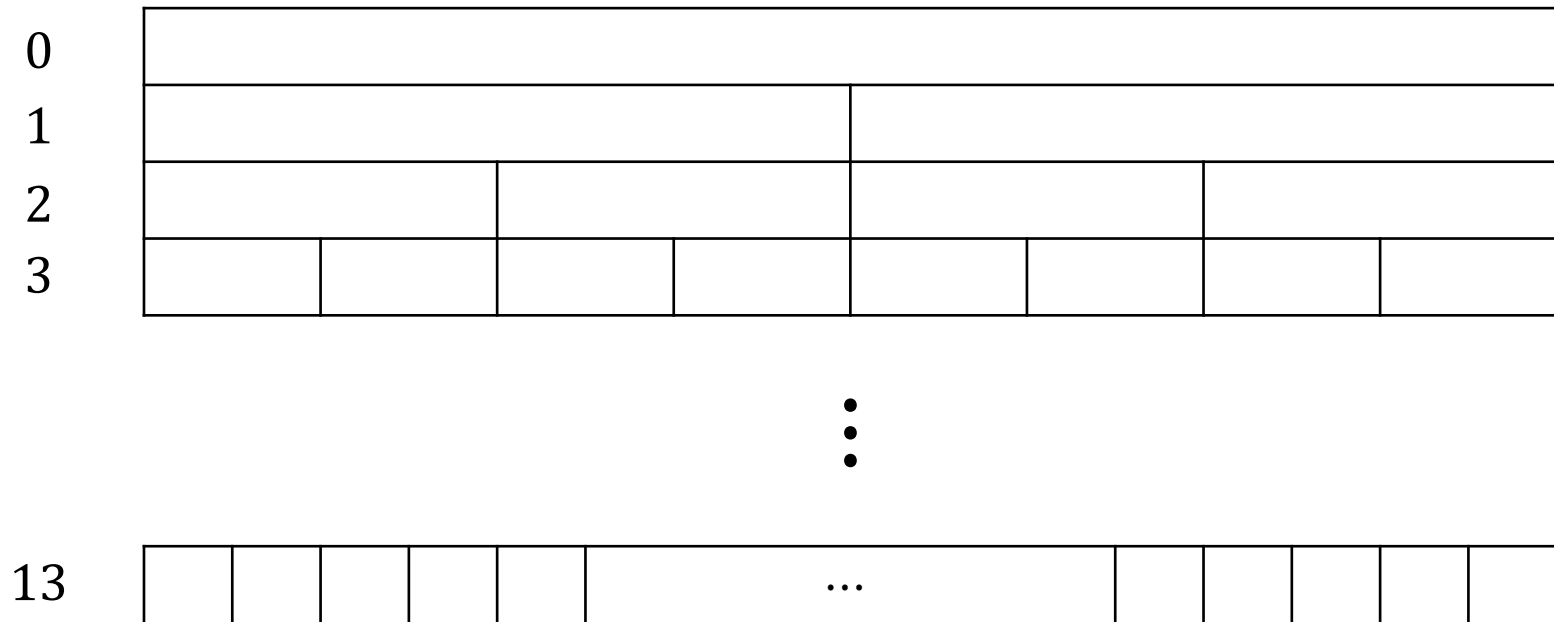
小課題 3 : 二分探索

- 最初の仮定 :

Bruno が $[L, R]$ に含まれる index X を一つ知っている.

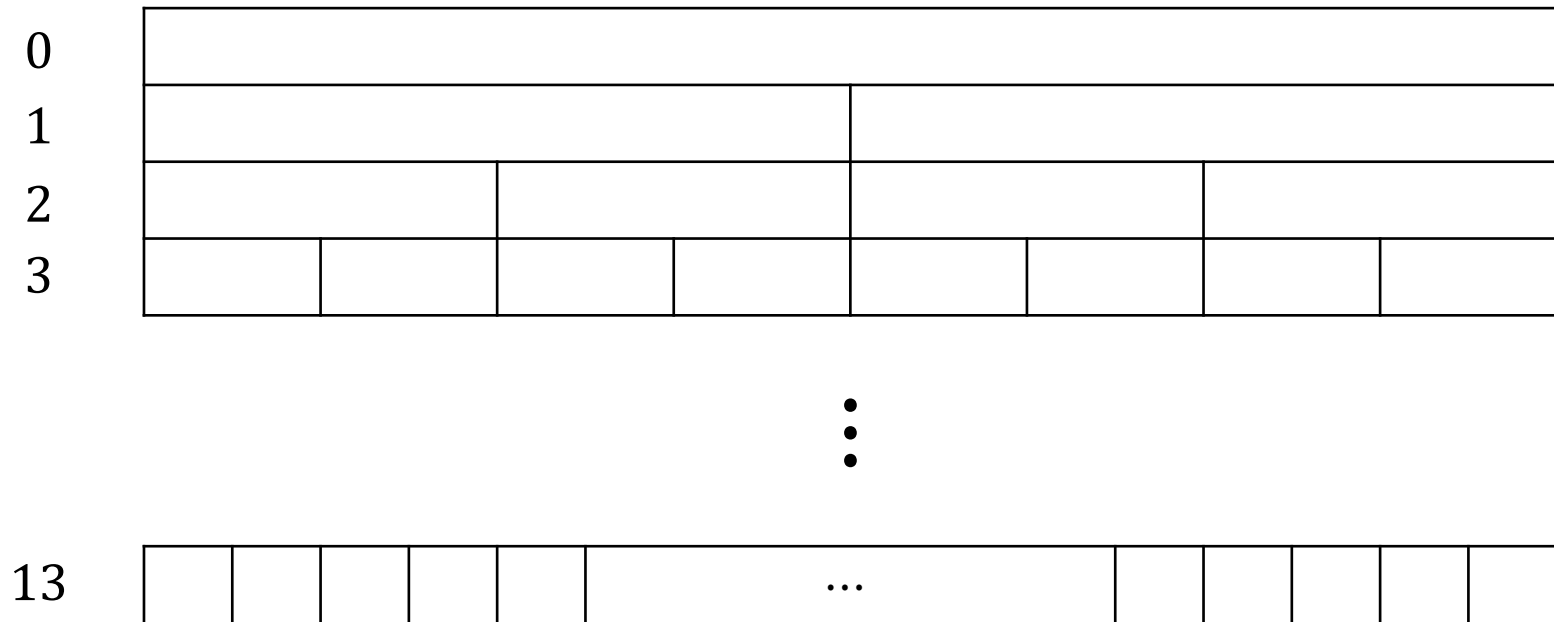
- 実際には, このような仮定は成立しない.
- 今度は, この情報を送ることを考える.

小課題 3 : 二分探索



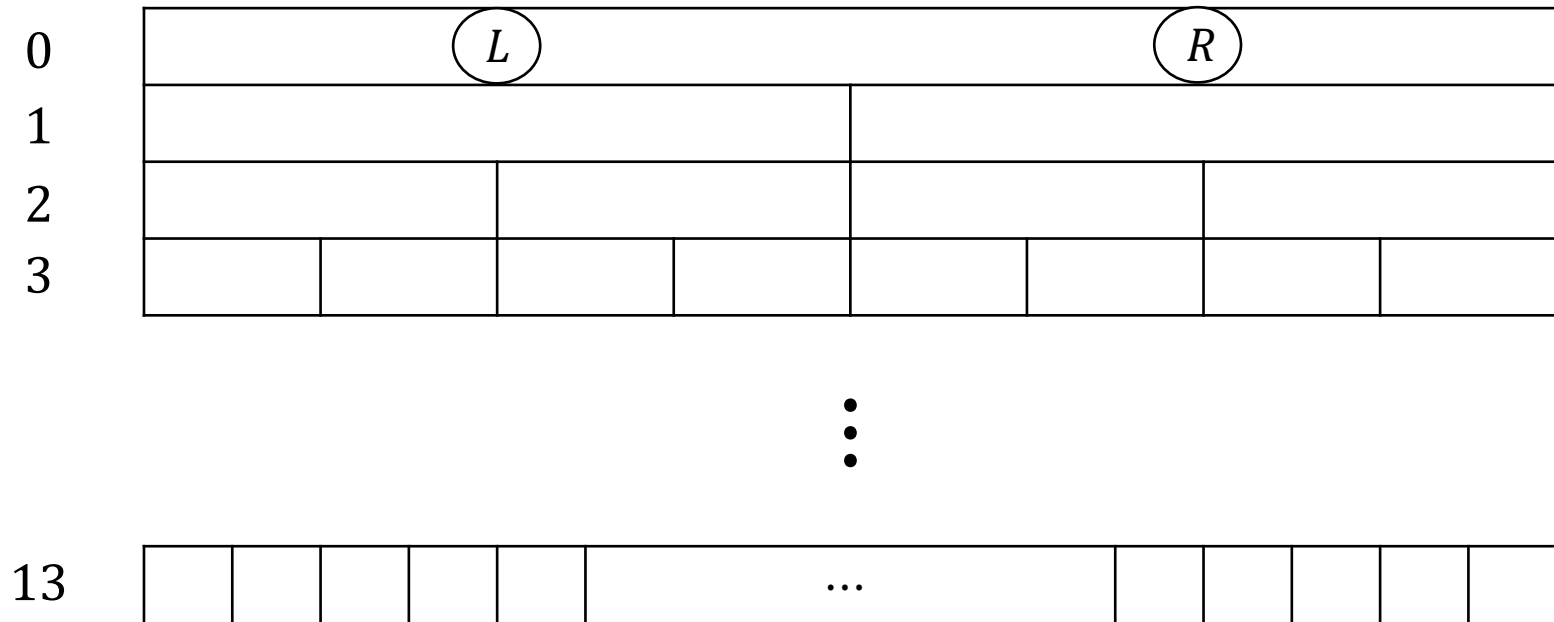
- segment tree (の葉を広くしたもの) を考える.

小課題 3 : 二分探索



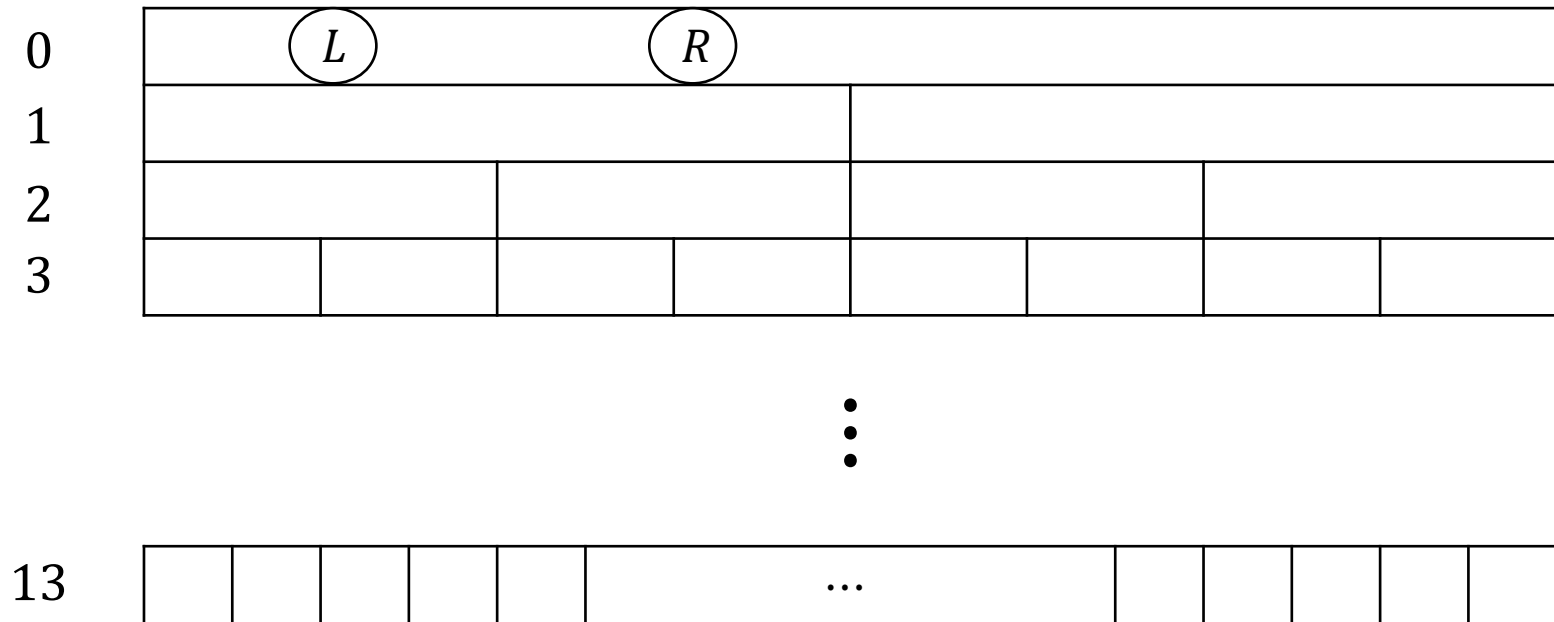
- $[L, R]$ が含まれる頂点のうち, 最も深い頂点を送る.

小課題 3 : 二分探索



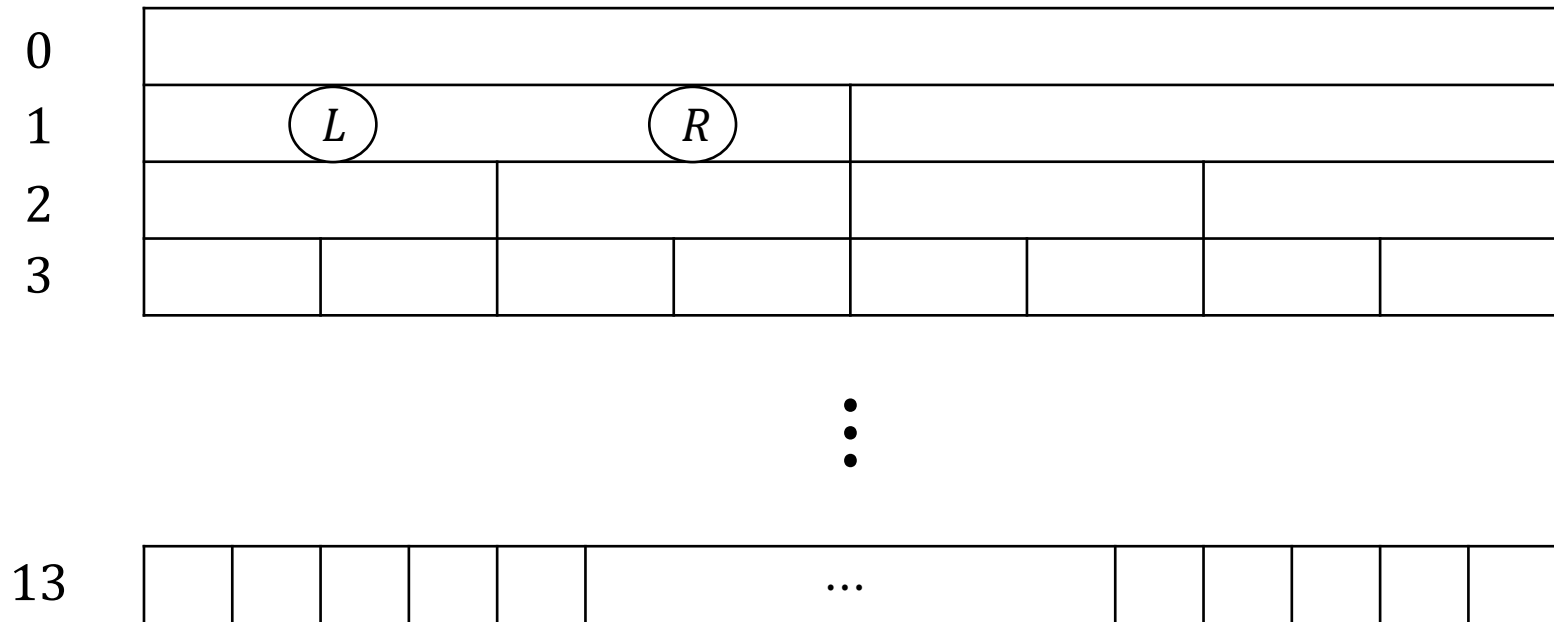
- $[L, R]$ が含まれる頂点のうち, 最も深い頂点を送る.

小課題 3 : 二分探索



- $[L, R]$ が含まれる頂点のうち, 最も深い頂点を送る.

小課題 3 : 二分探索



- $[L, R]$ を含む頂点のうち, 最も深い頂点を送る.

小課題 3 ： 二分探索

- 深さが 1 増えるごとに，ありうる範囲が半減している．
- つまり，二分探索を行う回数を減らしても，その分深さが深ければ効率はほとんど同じである．

小課題 3 : 二分探索 (algorithm)

- Anna は $[L, R]$ を含む頂点のうち、最も深い頂点の**深さ**を以下のようにして送る.
 - 深さが 0, 1 のとき 3 bits 用いて,
深さが 2 以上 13 以下のとき 4 bits 用いる.
 - $1/8 \times 2 + 1/16 \times 12 = 1$ より可能.
- それから、対応する頂点を送る.
 - その深さを d としたとき、 d bits で可能.

小課題 3 : 二分探索 (algorithm)

- Anna に残された通信量を k bits として, Bruno は k 回二分探索をし, Cartesian Tree を送る.
 - Anna : k bits
 - Bruno : $\frac{k(2\log_2 N - k - 2d - 1)}{2} + N/2^{k+d-1}$ bits
- $d = 2$ のときに最大となり,
$$12 \times 23 \div 2 + 1,000,000/2^{13} \approx 260$$
- 合計 300 bits に収まる. これで 100 点.

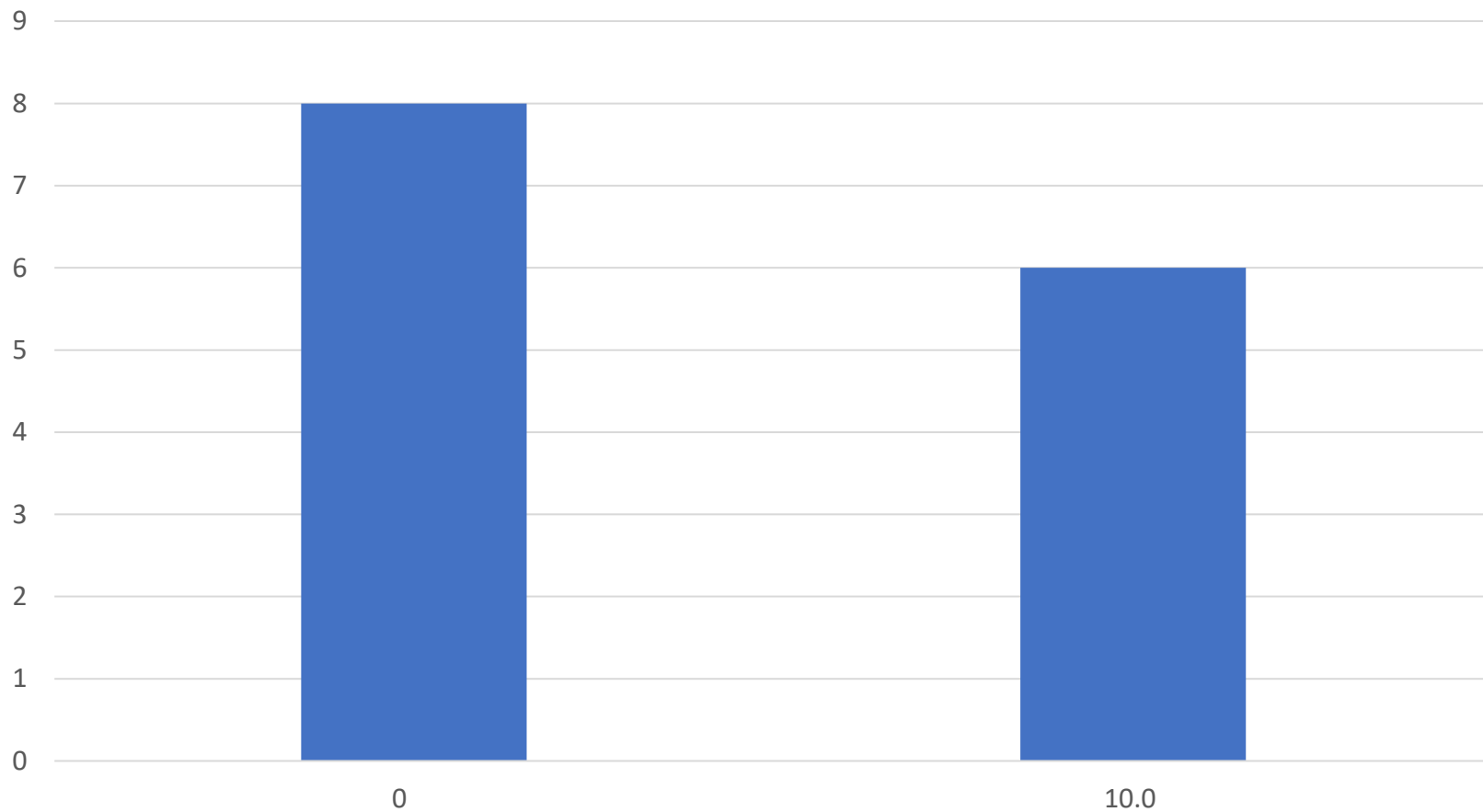
小課題 3 : 二分探索 (in general)

- Anna は $[L, R]$ が含まれる頂点のうち、最も深い頂点の**深さ**を送る.
 - これは $\log \log N$ bits で可能.
- それから、対応する頂点を送る.
 - その深さを d としたとき、 d bits で可能.

小課題 3 : 二分探索 (in general)

- Bruno は $\log N - \log \log N - d$ 回二分探索を行い, 最後に Cartesian Tree を送る.
 - Anna : $\log N - \log \log N - d$ bits
 - Bruno : $\frac{(\log N - \log \log N - d)(\log N + \log \log N - d - 1)}{2} + 2 \log N$ bits
- よって, 一般に以下の通信量で可能.
 - Anna : $\log N$ bits
 - Bruno : $\frac{1}{2}(\log N)^2 + O(\log N)$ bits
($\log$ の底は 2)

得点分布



古代の機械 (Ancient Machine)

解説 戸高空

問題概要

- XYZ からなる長さNの文字列
- 文字列がなくなるまで、文字を1つずつ消していく
- X --- Y -- Z のような並びでYを消す回数を最大化したい
- Anna ... 文字列を知っている
 - Brunoに0,1をLビット送る
- Bruno ... 文字列を知らない
 - 文字を消す役割

制約

- $N \leq 18$ (5点)
- $N \leq 100000$
 - $L \leq 200000$ (累積 30点)
 - $L \leq 160000$ (累積 40点)
 - $L \leq 100000$ (累積 70点)
 - $L \leq 70000$ (満点)

$N \leq 18$ (5点)

- AnnaはBrunoに文字列の情報を全部送ることができる
- Brunoがよい消し方の最大値を $O(2^N)$ くらいで達成できれば良い
- $dp[S]$... 残っている集合が S のときのよい消し方の最大値でbitDP
- dpテーブルをもとに最大値を達成する消し方を復元できる

$L \leq 200000$ (累積 30点)

- AnnaはBrunoに文字列の情報を全部送ることができる
- $X \rightarrow 00, Y \rightarrow 01, Z \rightarrow 10$ に変換して1文字につき2ビット使って送る
- Brunoがよい消し方の最大値を $O(N)$ くらいで達成できれば良い

よい消し方の最大値

- 一番左のXより左の文字はよい消し方はできない
- 一番右のZより右の文字はよい消し方はできない
- 連続するYはそのうち1つしかよい消し方はできない
- 最大値は一番左のXと一番右のZの間にある連続するYの区間の個数以下になる
- 実際にこれは達成できる

最大値の達成の仕方

- 一番左のX（場所をaとする）より左の文字は消す
- 残った文字で一番左のZを見る（場所をbとする）
- aとbの間の文字はXかYのみである
- bの一つ左を順番に消していく
- 連続したYの中で一番左のYは必ずよい消し方になる
- 最後にbを消す
- 同じように残った文字で一番左のZを見ていく

例

YZXYYZZYXXYZZZ

--XYYZZYXXYZZZ

--XY--ZZYXXYZZZ

--X--ZZYXXYZZZ good

--X---Z YXXYZZZ

--X----YXXYZZZ

--X-----YXX-ZZZ good

例

--X-----YXX-ZZZ

--X-----YX--ZZZ

--X-----Y---ZZZ

--X----- - - - -ZZZ good

--X----- - - - -ZZ

--X----- - - - -Z

--X----- - - - - -

$L \leq 160000$ (累積 40点)

- AnnaはBrunoに文字列の情報を全部送る
- 文字は3種類しかないので1文字に2ビット使うのは無駄
- 文字列の種類は 3^N 通りしかない
- 理論的には $\log_2 3^N \doteq 1.58N$ ビットで十分
- まとめて送るほど理論値近づく
- $3^5 \leq 2^8$ より5文字ずつを8ビットで送ると
160000ビットを達成できる

$L \leq 100000$ (累積 70点)

- Brunoが最大値を達成するために必要な情報のみ送る

$L \leq 100000$ (累積 70点)

- 最大値の達成の仕方
- 一番左のX以降で、左のZから順にZの左隣を消していく
- 必要な情報は一番左のXとそれ以降のZの位置
- 各文字1ビット使って送ることができる
- 一番左のXまたはそれ以降のZなら1、それ以外なら0を送る
- 例：YZXYYZZYXXYZZZ

00100110000111

$L \leq 100000$ (累積 70点)

- 最大値の達成の仕方
- 一番左のX以降で、左のZから順にZの左隣を消していく
- 必要な情報は一番左のXとそれ以降のZの位置
- 各文字1ビット使って送ることができる
- 一番左のXまたはそれ以降のZなら1、それ以外なら0を送る
- 例：YZXYYZZYXXYZZZ

00100110000111

$L \leq 70000$ (満点)

- 先ほどの解法ではNビットのあらゆるビット列が登場する
- アルゴリズムを変えないとこれ以上減らせない

最大値の達成の仕方（改）

- 一番左のX（場所をaとする）より左の文字は消す
- 残った文字で一番左のZを見る（場所をbとする）
- ただしそのZが連続している場合はその中で一番右のZを見る
- bの一つ左を順番に消していく
- 連続したYの中で一番左のYは必ずよい消し方になる
- 最後にbを消す
- 同じように残った文字で一番左のZを見ていく

$L \leq 70000$ (満点)

- このようにアルゴリズムを変えても最大値を達成できる
- またこの改善によって送るべきビット列に次の性質が生まれた
- 「1は連続して現れない」
- 登場するビット列が制限された

$L \leq 70000$ (満点)

- 「1が連続して現れない」 N ビット列は 2^N 種類より少ない
- 「1が連続して現れない」 ビット列に辞書順で番号をつけてその番号を2進数で送ればよい
- Bruno側では辞書順で(送られてきた番号)番目の「1が連続して現れない」ビット列を求めて復元する
- 各 i について「1が連続して現れない i ビット列の数」を計算しておくことができる
- 実装の都合上、番号がオーバーフローしないように、何ビットかずつに分けて送る

L ≤ 70000 (満点)

- 「1が連続して現れない」 Nビット列は何通りあるだろうか
- $dp[i][0]$... i 文字目が0のとき i ビット列は何通りあるか

$dp[i][1]$... i 文字目が1のとき i ビット列は何通りあるか

$$dp[i][0] = dp[i - 1][0] + dp[i - 1][1]$$

$$dp[i][1] = dp[i - 1][0] \text{ だから}$$

$$dp[i][0] = dp[i - 1][0] + dp[i - 2][0]$$

これはフィボナッチ数列

$L \leq 70000$ (満点)

- 「1が連続して現れない」 N ビット列は $fib(N + 2)$ 種類ある
- ただし $fib(i)$ はフィボナッチ数列の i 項目を表す
- 理論的には $\log_2 fib(N + 2) \doteq 0.69N$ ビットで送ることができる
- 例えば $fib(63 + 2) \leq 2^{44}$ なので63ビットずつを44ビットで送ると69872ビットが達成できる

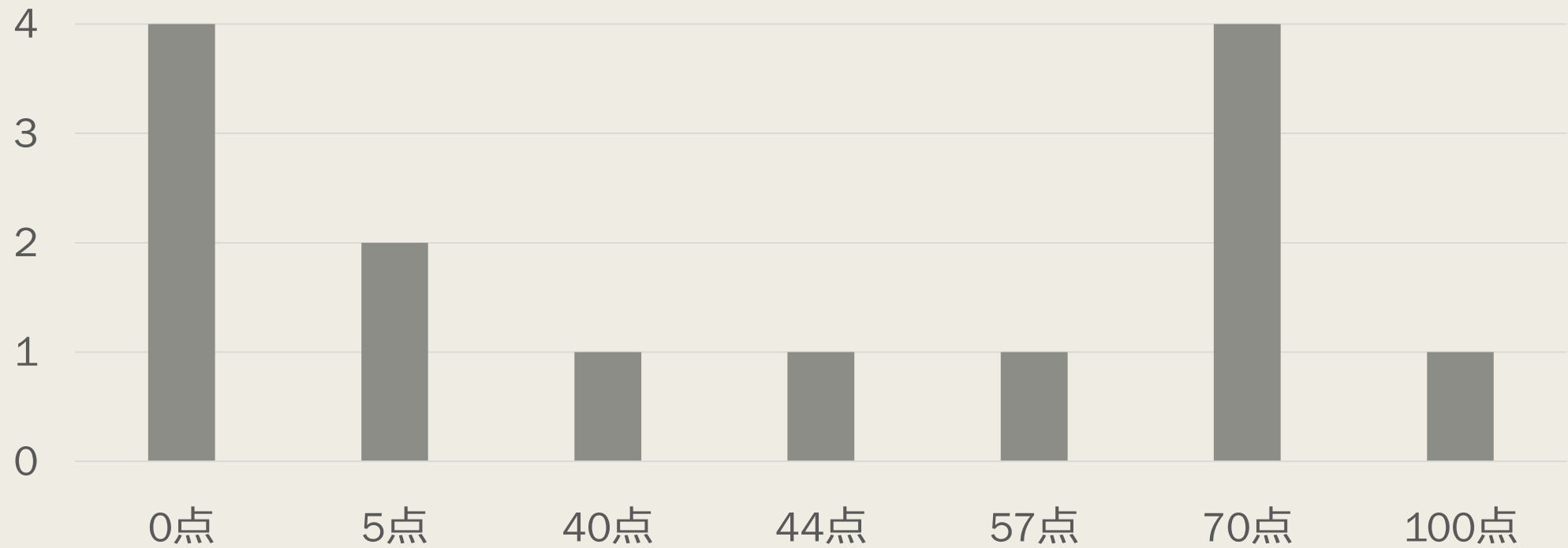
最大値の達成の仕方(別解)

- 一番左のX（場所をaとする）より左の文字は消す
- aの右隣りの文字を見ていく
- Xのとき...消してよい
- Yのとき...その次がXなら、このYXを残して次の文字をみる
そうでなければ消してよい
- Zのとき...消してよい（最後のZの場合は残しておいた文字を後ろから消していく）

最大値の達成の仕方(別解)

- これで最大値が達成できる
- 必要な情報は最初のX、それ以降のYXの並び、最後のZ
- これを各文字ごとに0or1で送ると100000ビット
- 例：YZXYZZYXXYXZZZX
0010000100100010
- またYXで送る1は連続しないのでさきほどと同様に満点もとれる

得点分布



ボディーガード(Bodyguard)

解説:星井智仁

問題概要

- 要人 i は時刻 T_i に座標 A_i を出発して、速度1で移動し、時刻 $T_i + |A_i - B_i|$ に座標 B_i に到着する
- 要人 i と一緒にいる時間 $\times C_i$ だけ報酬が貰える
- 時刻 P_i に座標 X_i から動き始めた時、報酬の最大値を求めよ

<制約>

- $N \leq 2800$
- $Q \leq 3 \times 10^6$
- $T_i, A_i, B_i, C_i, P_i, X_i \leq 10^9$
- C_i は偶数

問題概要

- 要人 i は時刻 T_i に座標 A_i を出発して、速度1で移動し、時刻 $T_i + |A_i - B_i|$ に座標 B_i に到着する
- 要人 i と一緒にいる時間 $\times C_i$ だけ報酬が貰える
- 時刻 P_i に座標 X_i から動き始めた時、報酬の最大値を求めよ

<制約>

- $N \leq 2800$
- $Q \leq 3 \times 10^6$
- $T_i, A_i, B_i, C_i, P_i, X_i \leq 10^9$
- C_i は偶数

問題概要

- 要人 i は時刻 T_i に座標 A_i を出発して、速度1で移動し、時刻 $T_i + |A_i - B_i|$ に座標 B_i に到着する
- 要人 i と一緒にいる時間 $\times C_i$ だけ報酬が貰える
- 時刻 P_i に座標 X_i から動き始めた時、報酬の最大値を求めよ

<制約>

- $N \leq 2800$
- $Q \leq 3 \times 10^6$
- $T_i, A_i, B_i, C_i, P_i, X_i \leq 10^9$
- C_i は偶数←なぜ？

問題概要

- 要人 i は時刻 T_i に座標 A_i を出発して、速度1で移動し、時刻 $T_i + |A_i - B_i|$ に座標 B_i に到着する
- 要人 i と一緒にいる時間 $\times C_i$ だけ報酬が貰える
- 時刻 P_i に座標 X_i から動き始めた時、報酬の最大値を求めよ

<制約>

- $N \leq 2800$
- $Q \leq 3 \times 10^6$
- $T_i, A_i, B_i, C_i, P_i, X_i \leq 10^9$
- C_i は偶数←なぜ？

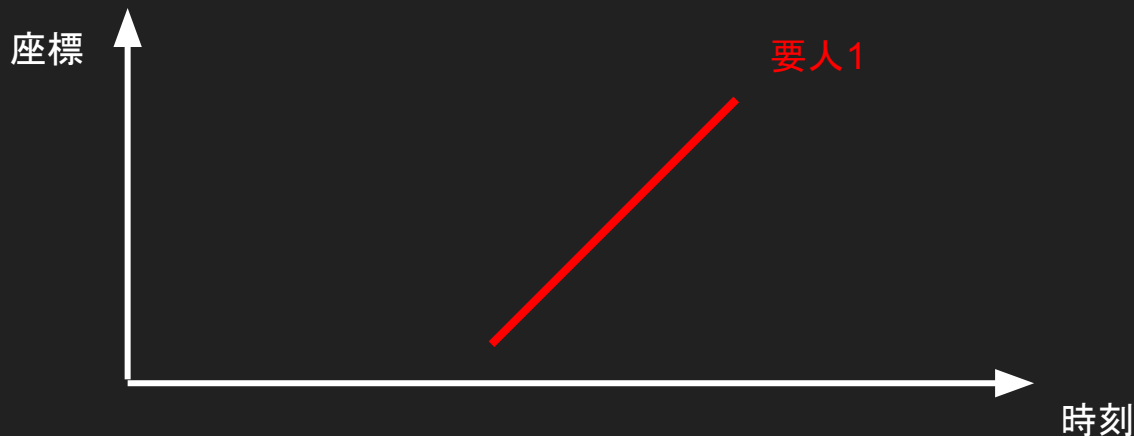


問題概要

- 要人 i は時刻 T_i に座標 A_i を出発して、速度1で移動し、時刻 $T_i + |A_i - B_i|$ に座標 B_i に到着する
- 要人 i と一緒にいる時間 $\times C_i$ だけ報酬が貰える
- 時刻 P_i に座標 X_i から動き始めた時、報酬の最大値を求めよ

<制約>

- $N \leq 2800$
- $Q \leq 3 \times 10^6$
- $T_i, A_i, B_i, C_i, P_i, X_i \leq 10^9$
- C_i は偶数←なぜ？

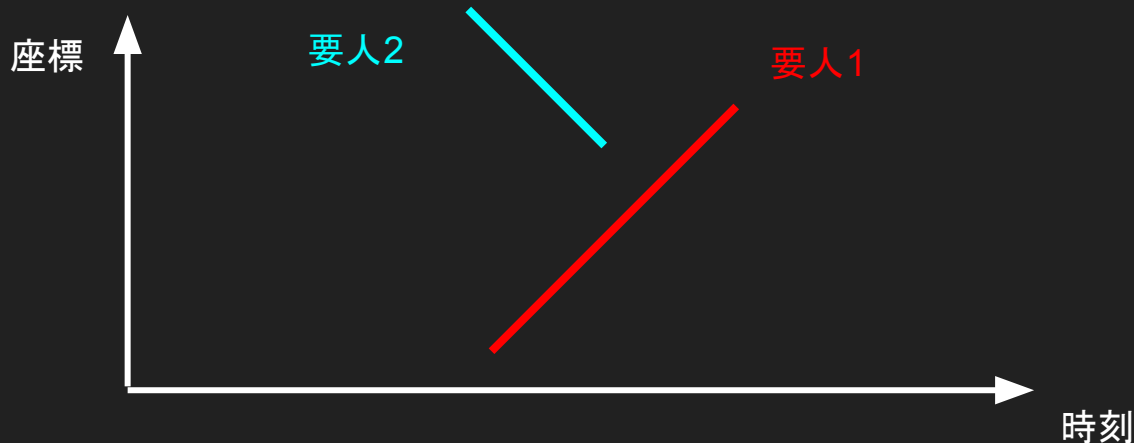


問題概要

- 要人 i は時刻 T_i に座標 A_i を出発して、速度1で移動し、時刻 $T_i + |A_i - B_i|$ に座標 B_i に到着する
- 要人 i と一緒にいる時間 $\times C_i$ だけ報酬が貰える
- 時刻 P_i に座標 X_i から動き始めた時、報酬の最大値を求めよ

<制約>

- $N \leq 2800$
- $Q \leq 3 \times 10^6$
- $T_i, A_i, B_i, C_i, P_i, X_i \leq 10^9$
- C_i は偶数←なぜ？

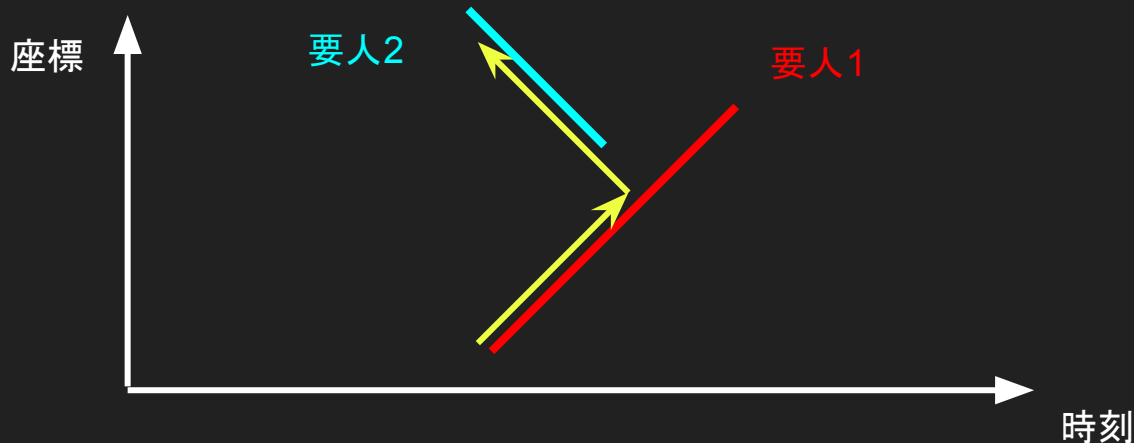


問題概要

- 要人 i は時刻 T_i に座標 A_i を出発して、速度1で移動し、時刻 $T_i + |A_i - B_i|$ に座標 B_i に到着する
- 要人 i と一緒にいる時間 $\times C_i$ だけ報酬が貰える
- 時刻 P_i に座標 X_i から動き始めた時、報酬の最大値を求めよ

<制約>

- $N \leq 2800$
- $Q \leq 3 \times 10^6$
- $T_i, A_i, B_i, C_i, P_i, X_i \leq 10^9$
- C_i は偶数←なぜ？

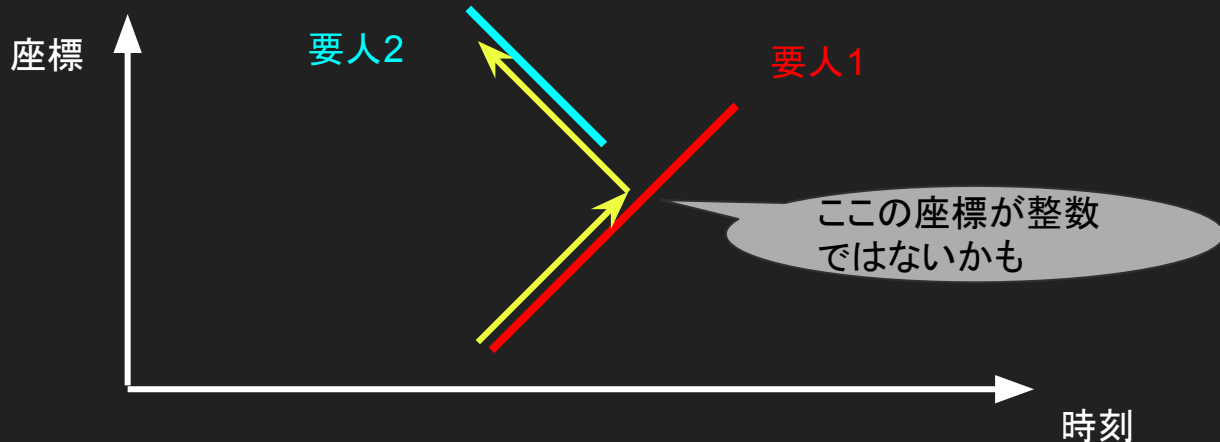


問題概要

- 要人 i は時刻 T_i に座標 A_i を出発して、速度1で移動し、時刻 $T_i + |A_i - B_i|$ に座標 B_i に到着する
- 要人 i と一緒にいる時間 $\times C_i$ だけ報酬が貰える
- 時刻 P_i に座標 X_i から動き始めた時、報酬の最大値を求めよ

<制約>

- $N \leq 2800$
- $Q \leq 3 \times 10^6$
- $T_i, A_i, B_i, C_i, P_i, X_i \leq 10^9$
- C_i は偶数←なぜ？

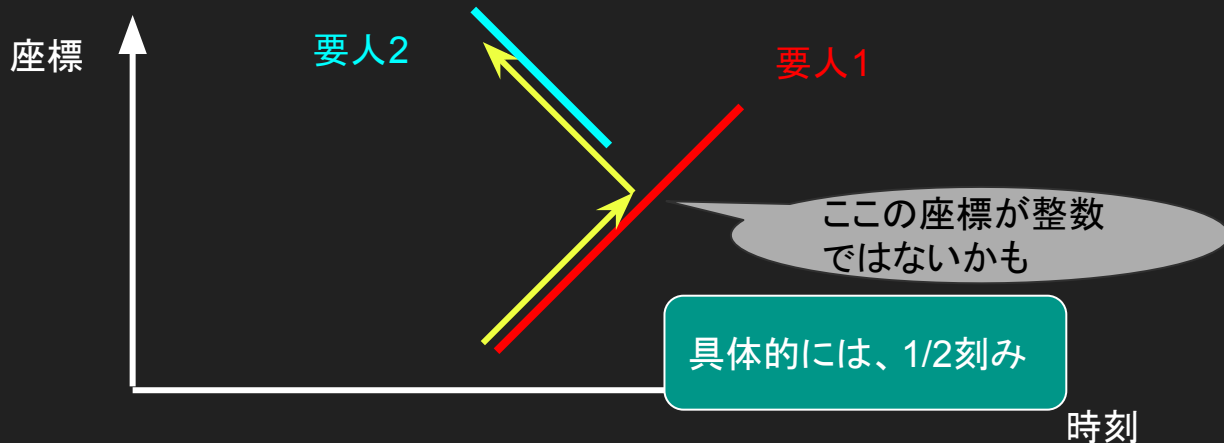


問題概要

- 要人 i は時刻 T_i に座標 A_i を出発して、速度1で移動し、時刻 $T_i + |A_i - B_i|$ に座標 B_i に到着する
- 要人 i と一緒にいる時間 $\times C_i$ だけ報酬が貰える
- 時刻 P_i に座標 X_i から動き始めた時、報酬の最大値を求めよ

<制約>

- $N \leq 2800$
- $Q \leq 3 \times 10^6$
- $T_i, A_i, B_i, C_i, P_i, X_i \leq 10^9$
- C_i は偶数←なぜ？



問題概要

- 要人 i は時刻 T_i に座標 A_i を出発して、速度1で移動し、時刻 $T_i + |A_i - B_i|$ に座標 B_i に到着する
- 要人 i と一緒にいる時間 $\times C_i$ だけ報酬が貰える
- 時刻 P

<制約>

- $N \leq 28$
- $Q \leq 3 \times 10^5$
- $T_i, A_i, B_i, C_i, P, X_i \leq 10^9$
- C_i は偶数←なぜ？

座標・時刻を2倍して考える

ここの座標が整数
ではないかも

具体的には、1/2刻み

時刻

小課題1(6点)

<制約>

$$T_i, A_i, B_i, P_i, X_i \leq 3000$$

小課題1(6点)

<制約>

$$T_i, A_i, B_i, P_i, X_i \leq 3000$$

小課題1(6点)

<制約>

$$T_i, A_i, B_i, P_i, X_i \leq 3000$$

DP[今の時刻][今の座標]=今後得られる報酬の最大値

時間を遡るように動的計画法をします 遷移に毎回 $O(N)$ かけるとTLEします
工夫しましょう

DPは $O(\text{座標}_{\max} \times \text{時刻}_{\max})$ 遷移を調べるのに $O(\sum |A_i - B_i|)$ とか

小課題2(7点)

Q=1

小課題2(7点)

Q=1

小課題2(7点)

Q=1←よくわからない

小課題2(7点)

Q=1←よくわからない

終

制作・著作

JOI

解けないときは

1. もっと考察

問題の性質を追求しよう、がんばれ

解けないときは

1. もっと考察

問題の性質を追求しよう、がんばれ

2. 制約からエスパー

与えられる数が小さかったり大きかったりするときすべきことは割と限られる 10^5 くらいが一番困る

解けないときは

1. もっと考察

問題の性質を追求しよう、がんばれ

2. 制約からエスパー

与えられる数が小さかったり大きかったりするときすべきことは割と限られる 10^5 くらいが一番困る

3. 実験

手を動かして実際にやってみよう 絵を描いてみるのもよかったり

解けないときは

1. もっと考察

問題の性質を追求しよう、がんばれ

2. 制約からエスパー

与えられる数が小さかったり大きかったりするときすべきことは割と限られる 10^5 くらいが一番困る

3. 実験

手を動かして実際にやってみよう 絵を描いてみるのもよかったり

4. 解法全列挙

自分の知っている解法を総ざらいして、この問題に当てはめられないか考える

解けないときは

1. もっと考察

問題の性質を追求しよう、がんばれ

2. 制約からエスパー

与えられる数が小さかったり大きかったりするときすべきことは割と限られる 10^5 くらいが一番困る

3. 実験

手を動かして実際にやってみよう 絵を描いてみるのもよかったり

4. 解法全列挙

自分の知っている解法を総ざらいして、この問題に当てはめられないか考える

5. 他の問題を考える

時には逃げることも戦略のうち

解けないときは

1. もっと考察

問題の性質を追求しよう、がんばれ

2. 制約からエスパー

与えられる数が小さかったり大きかったりするときすべきことは割と限られる 10^5 くらいが一番困る

3. 実験

手を動かして実際にやってみよう 絵を描いてみるのもよかったり

4. 解法全列挙

自分の知っている解法を総ざらいして、この問題に当てはめられないか考える

5. 他の問題を考える

時には逃げることも戦略のうち

小課題2(7点)

Q=1

小課題2(7点)

$Q=1$

$N \leq 2800 \leftarrow O(N^2)$ とか $O(N^2 \log N)$ くらい？

小課題2(7点)

$Q=1$

$N \leq 2800 \leftarrow O(N^2)$ とか $O(N^2 \log N)$ くらい？

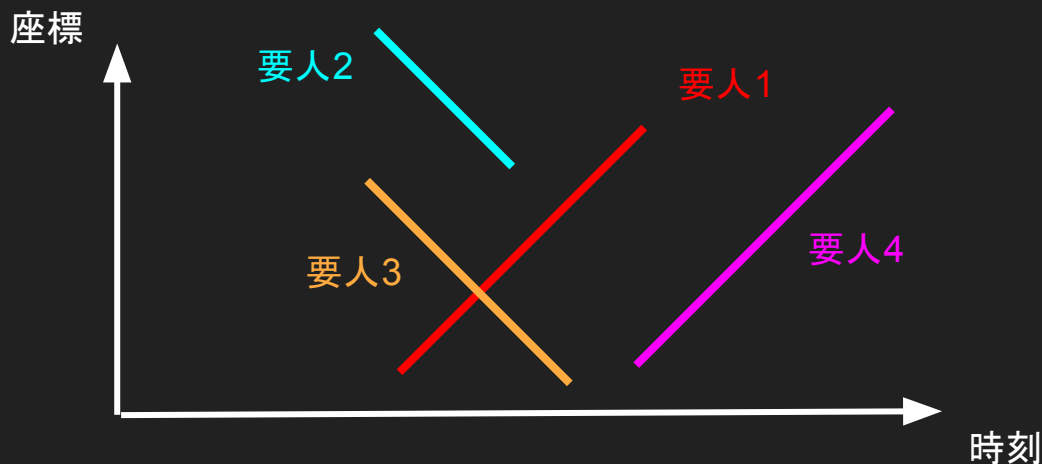
$T_i, A_i, B_i, C_i, P_i, X_i \leq 10^9 \leftarrow$ 座標を圧縮したい

小課題2(7点)

$Q=1$

$N \leq 2800 \leftarrow O(N^2)$ とか $O(N^2 \log N)$ くらい？

$T_i, A_i, B_i, C_i, P_i, X_i \leq 10^9 \leftarrow$ 座標を圧縮したい

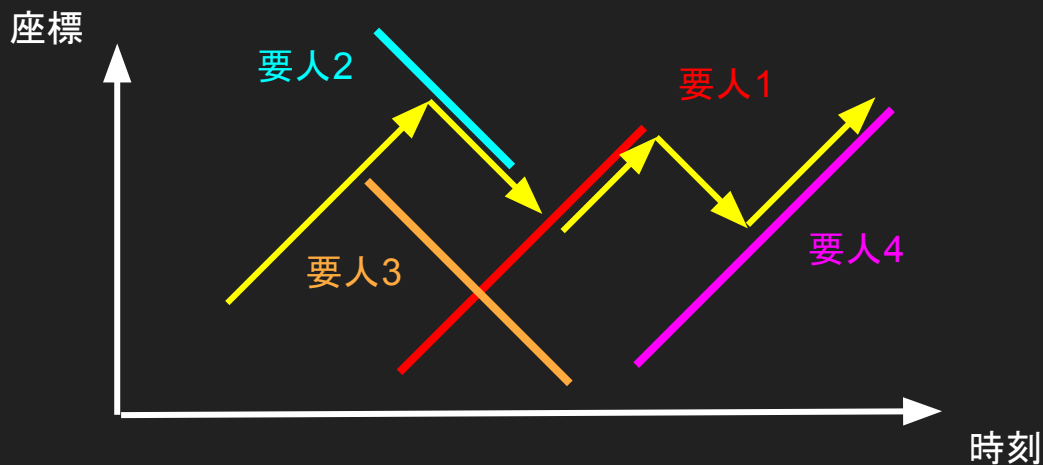


小課題2(7点)

$Q=1$

$N \leq 2800 \leftarrow O(N^2)$ とか $O(N^2 \log N)$ くらい？

$T_i, A_i, B_i, C_i, P_i, X_i \leq 10^9 \leftarrow$ 座標を圧縮したい



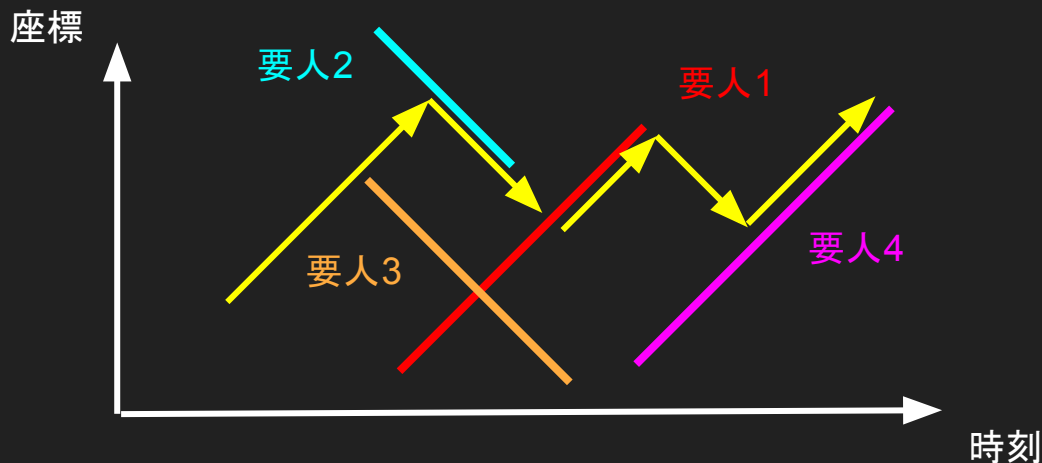
小課題2(7点)

$Q=1$

$N \leq 2800 \leftarrow O(N^2)$ とか $O(N^2 \log N)$ くらい？

$T_i, A_i, B_i, C_i, P_i, X_i \leq 10^9 \leftarrow$ 座標を圧縮したい

座標を圧縮したいが動きは斜め



小課題2(7点)

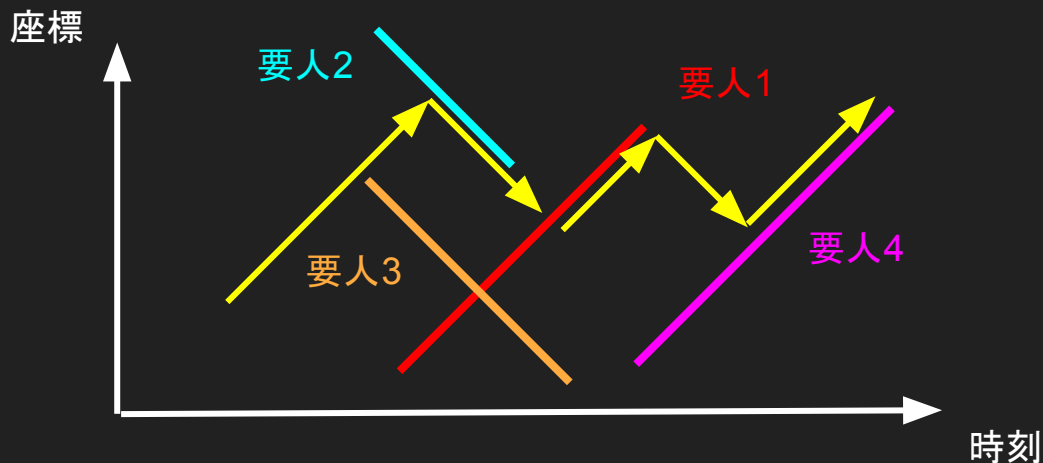
$Q=1$

$N \leq 2800 \leftarrow O(N^2)$ とか $O(N^2 \log N)$ くらい？

$T_i, A_i, B_i, C_i, P_i, X_i \leq 10^9 \leftarrow$ 座標を圧縮したい

座標を圧縮したいが動きは斜め

どうしよう



小課題2(7点)

$Q=1$

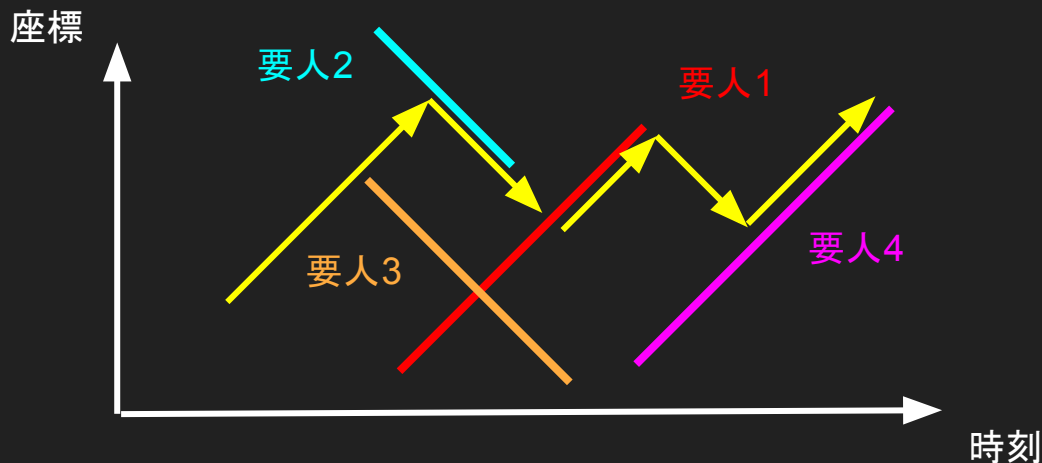
$N \leq 2800 \leftarrow O(N^2)$ とか $O(N^2 \log N)$ くらい？

$T_i, A_i, B_i, C_i, P_i, X_i \leq 10^9 \leftarrow$ 座標を圧縮したい

座標を圧縮したいが動きは斜め

どうしよう

→45度回転させればいい感じ



小課題2(7点)

$Q=1$

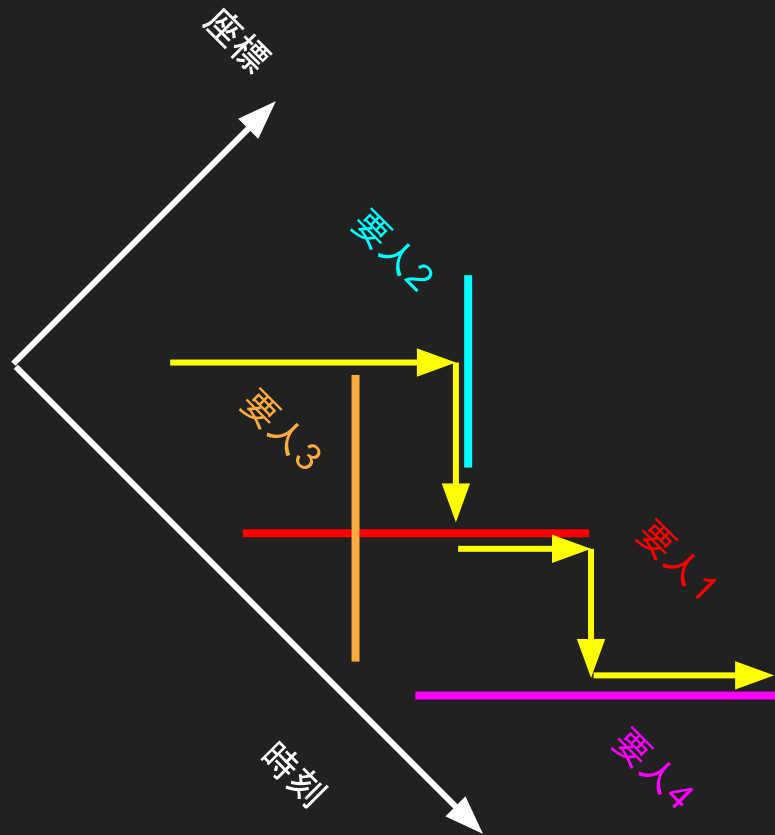
$N \leq 2800 \leftarrow O(N^2)$ とか $O(N^2 \log N)$ くらい？

$T_i, A_i, B_i, C_i, P_i, X_i \leq 10^9 \leftarrow$ 座標を圧縮したい

座標を圧縮したいが動きは斜め

どうしよう

→45度回転させればいい感じ



小課題2(7点)

$Q=1$

$N \leq 2800 \leftarrow O(N^2)$ とか $O(N^2 \log N)$ くらい？

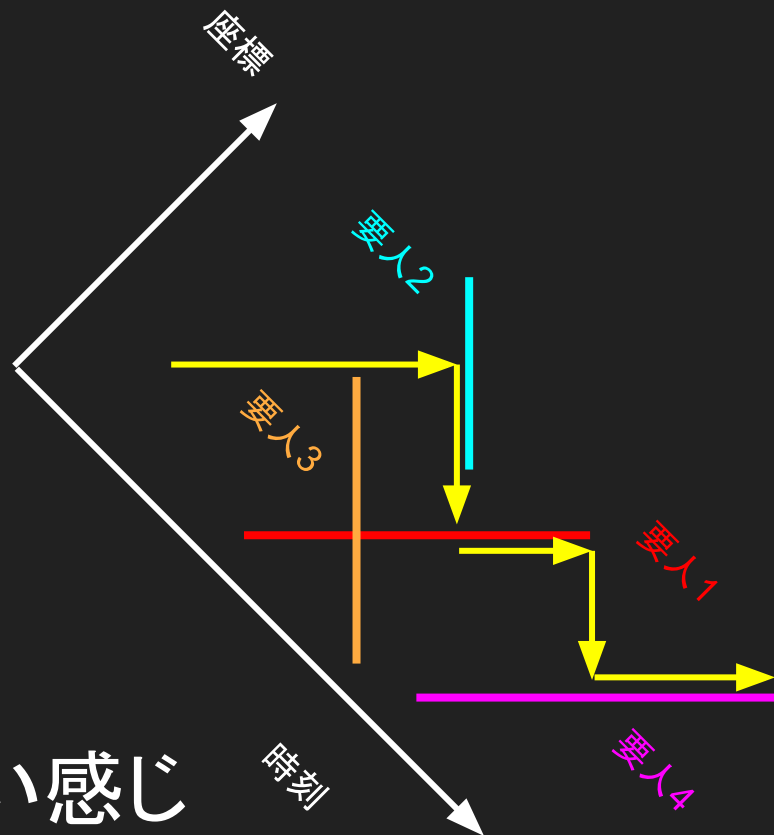
$T_i, A_i, B_i, C_i, P_i, X_i \leq 10^9 \leftarrow$ 座標を圧縮したい

座標を圧縮したいが動きは斜め

どうしよう

→45度回転させればいい感じ

→45度回転させればいい感じ



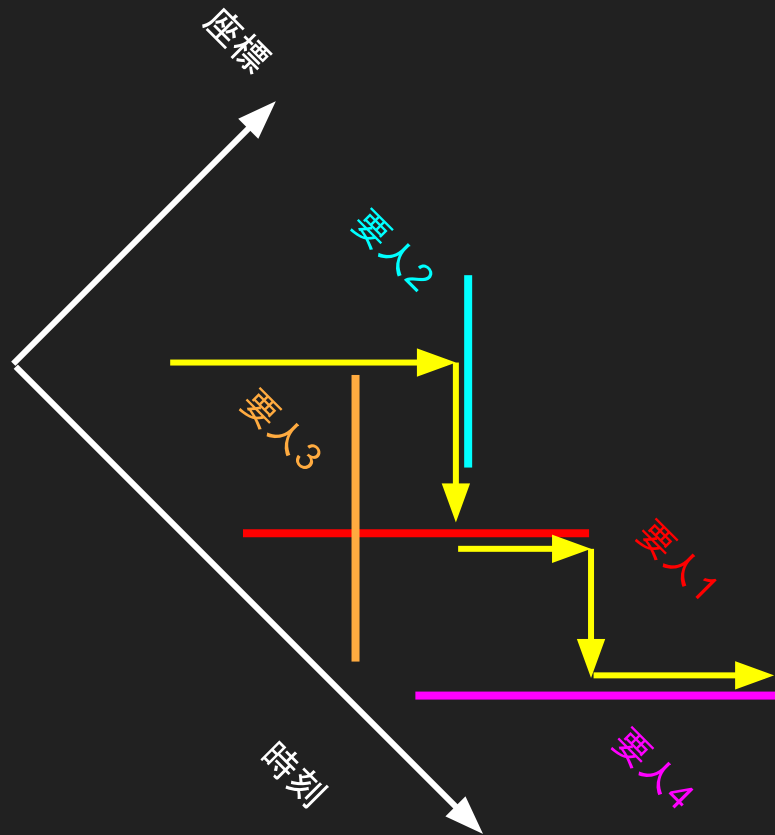
小課題2(7点)

45度回転したそれぞれの線分の座標と
クエリの始点を座標圧縮して
時系列の早い方からDP

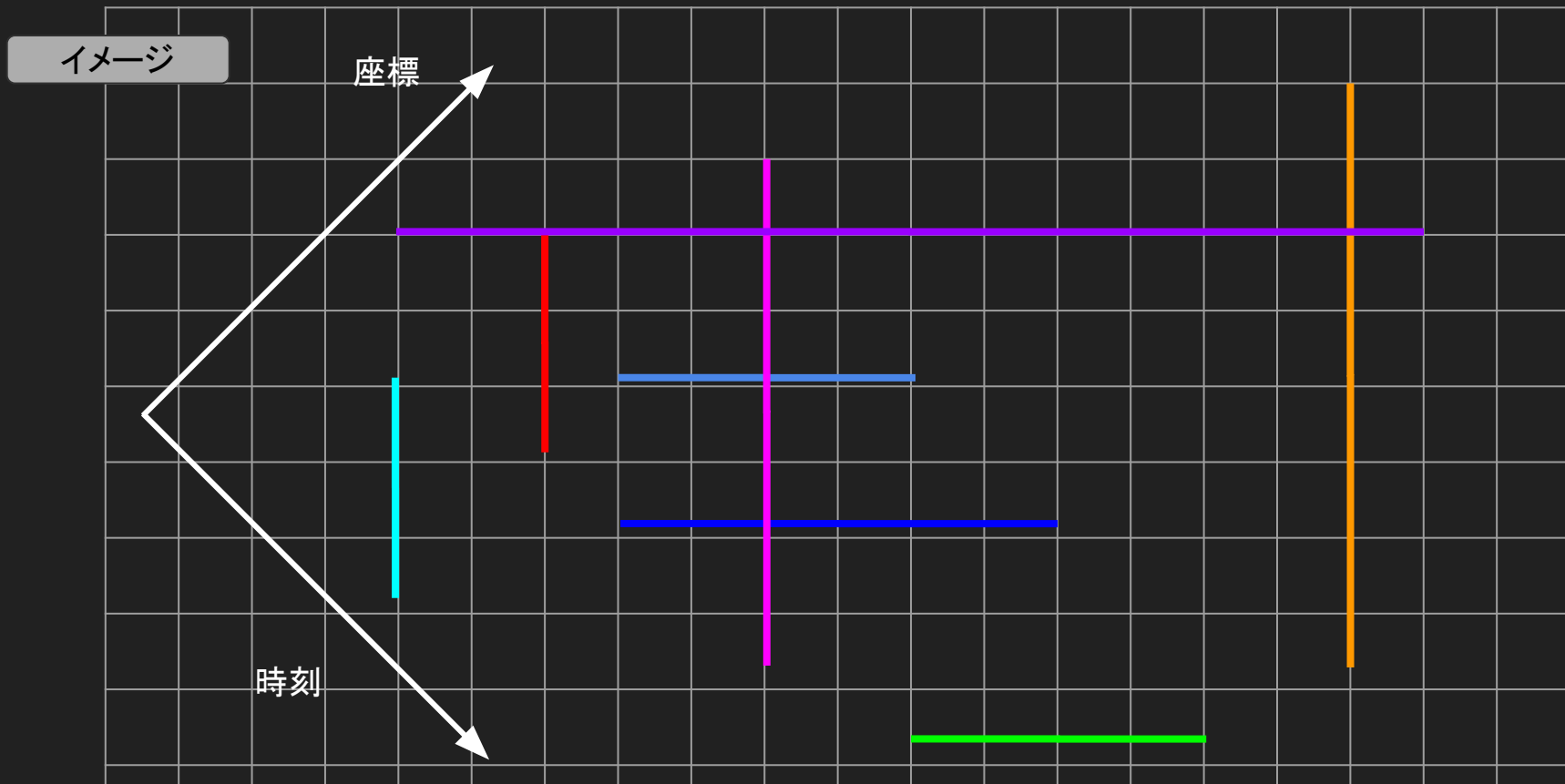
座標圧縮に $O(N \log N)$

DPに $O(N^2)$

$O(N^2)$ で解けました



小課題2(7点)



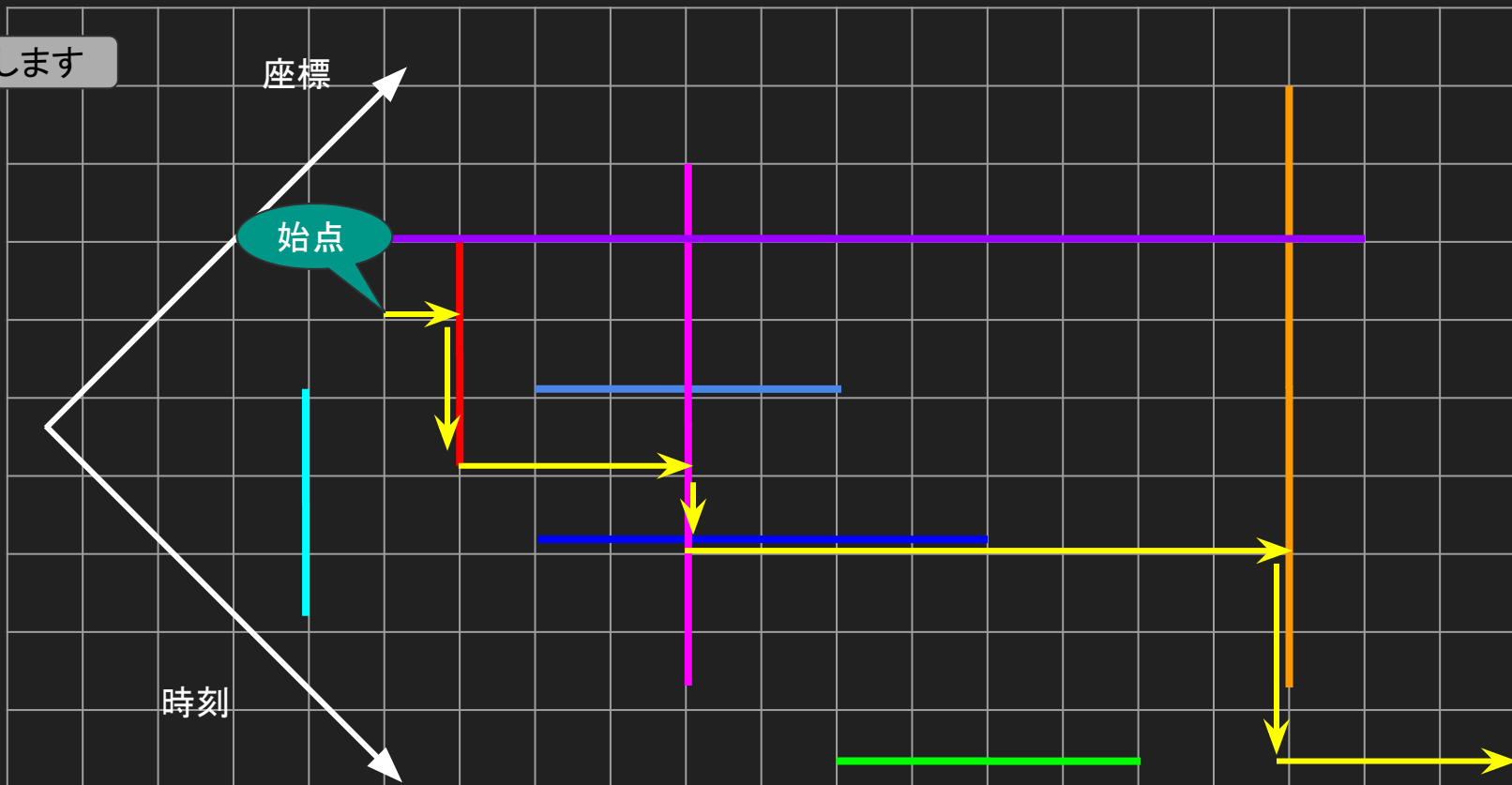
小課題2(7点)

DPをします

座標

始点

時刻



小課題3(15点)

45度回転したそれぞれの線分の座標と

クエリの始点を座標圧縮して

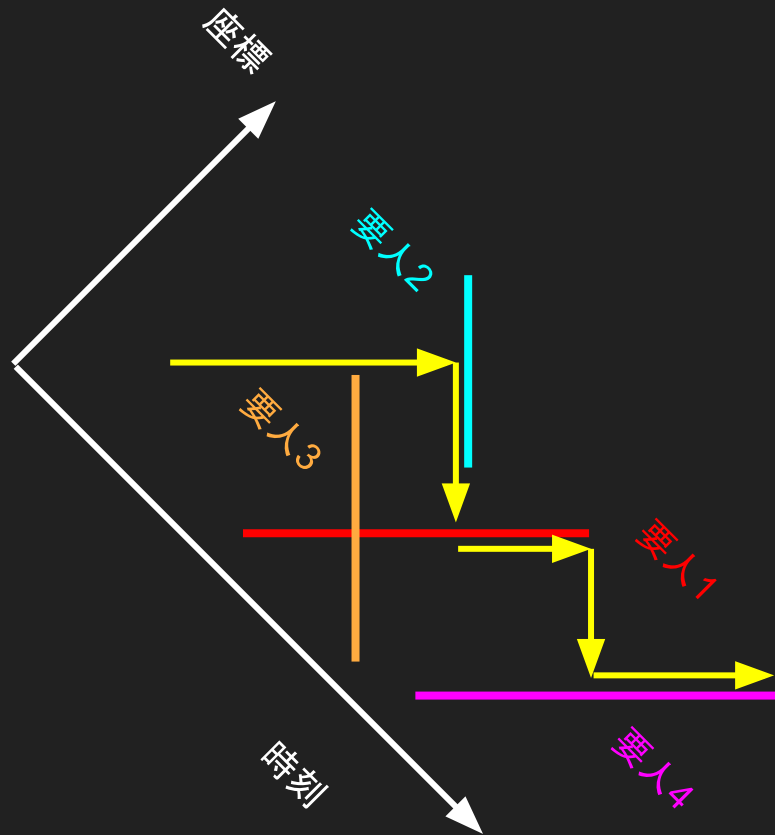
時系列の遅い方からDP

制約: $Q \leq 3000$

座標圧縮に $O((N+Q)\log(N+Q))$

DPに $O((N+Q)^2)$

$O((N+Q)^2)$ で解けました



小課題4(20点)

小課題2や3と同じように45度回転して座圧してDPLしたい

小課題4(20点)

小課題2や3と同じように45度回転して座圧してDPLしたい

制約: $Q \leq 40000$

小課題4(20点)

小課題2や3と同じように45度回転して座圧してDPLしたい

制約: $Q \leq 40000$

小課題3の解法だとDPIに $O((N+Q)^2)$

小課題4(20点)

小課題2や3と同じように45度回転して座圧してDPLしたい

制約: $Q \leq 40000$

小課題3の解法だとDPIに $O((N+Q)^2)$

終

制作・著作

JOI

小課題4(20点)

小課題2や3と同じように45度回転して座圧してDPLしたい

制約: $Q \leq 40000$

小課題3の解法だとDPIに $O((N+Q)^2)$

本当に各始点の座標も一緒に圧縮する必要がありますか？

小課題4(20点)

小課題2や3と同じように45度回転して座圧してDPLしたい

制約: $Q \leq 40000$

小課題3の解法だとDPIに $O((N+Q)^2)$

本当に各始点の座標も一緒に圧縮する必要がありますか？

各始点の座標は遷移に関わらない

小課題4(20点)

小課題2や3と同じように45度回転して座圧してDPLしたい

制約: $Q \leq 40000$

小課題3の解法だとDPIに $O((N+Q)^2)$

本当に各始点の座標も一緒に圧縮する必要がありますか？

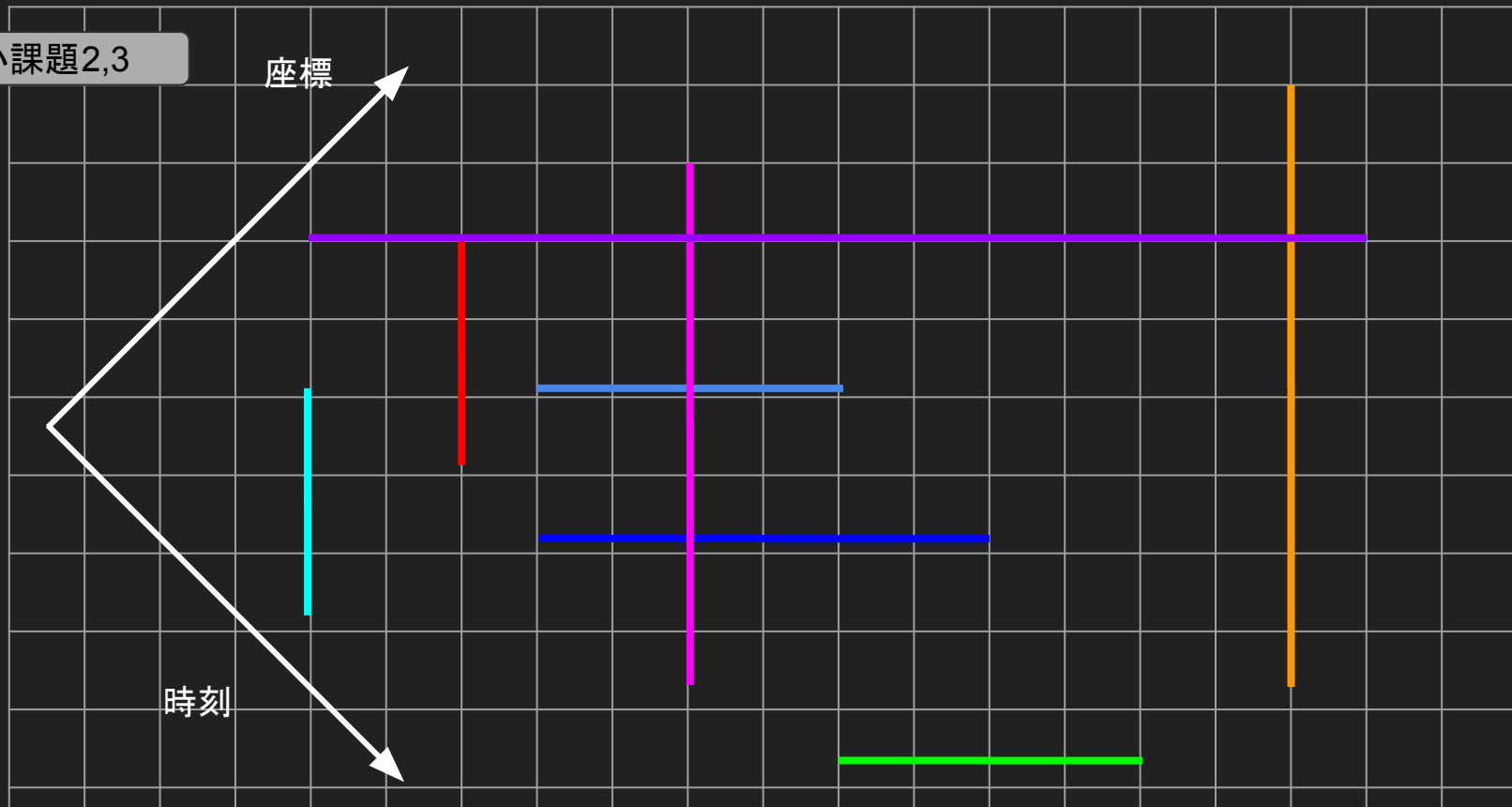
各始点の座標は遷移に関わらない→いらぬ

小課題4(20点)

小課題2,3

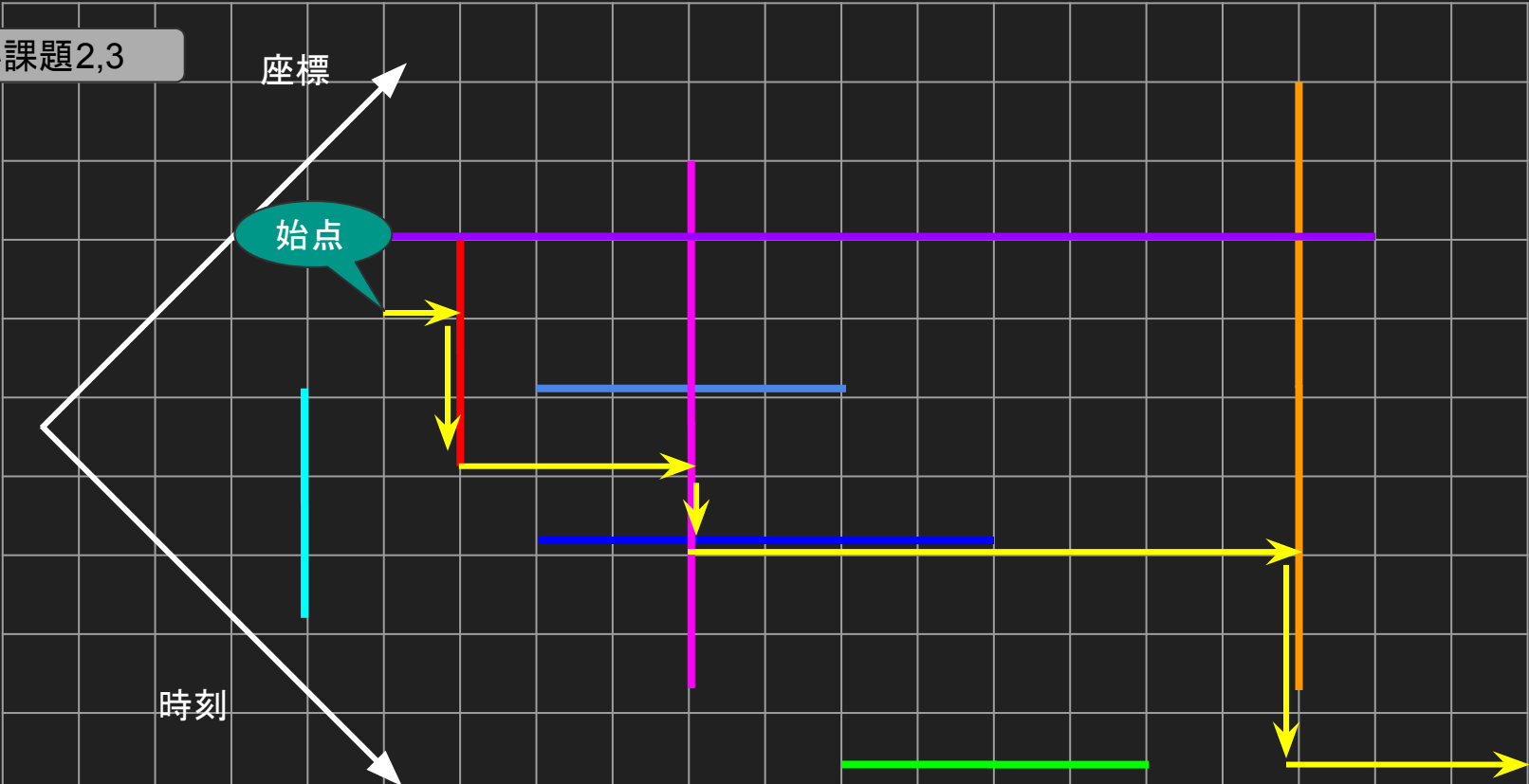
座標

時刻



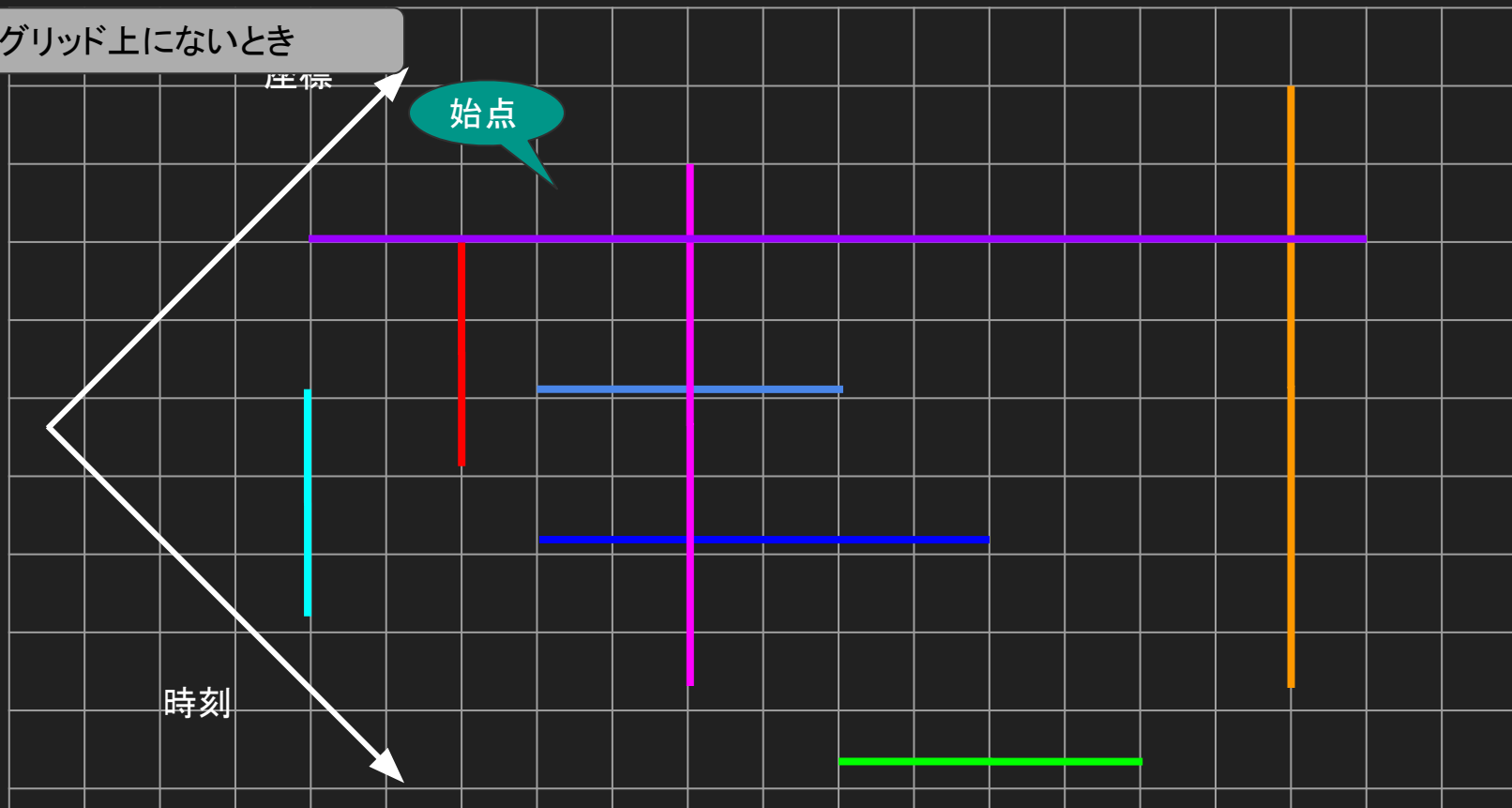
小課題4(20点)

小課題2,3



小課題4(20点)

始点がグリッド上にあるとき



小課題4(20点)

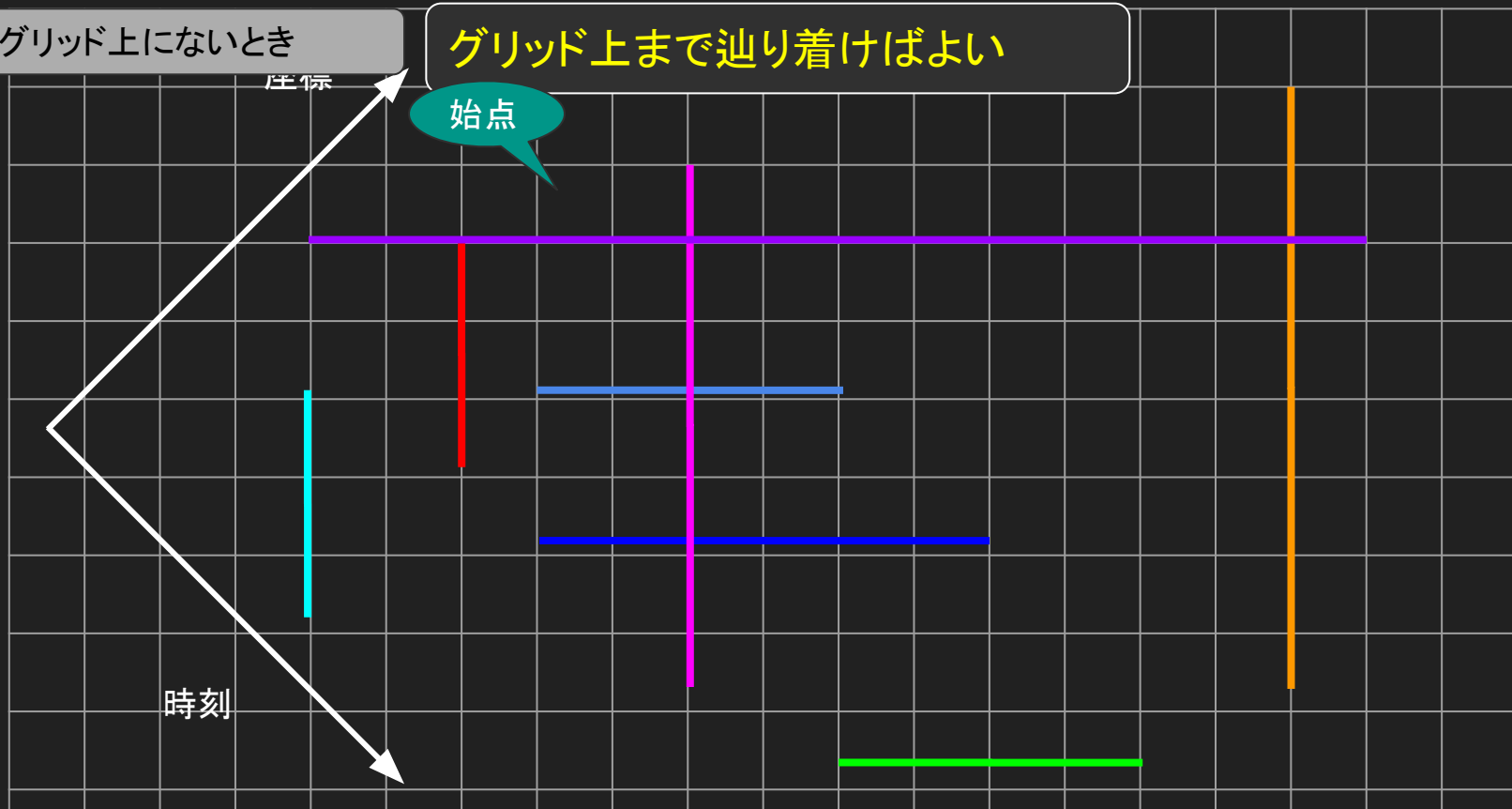
始点がグリッド上にないとき

グリッド上まで辿り着けばよい

始点

座標

時刻



小課題4(20点)

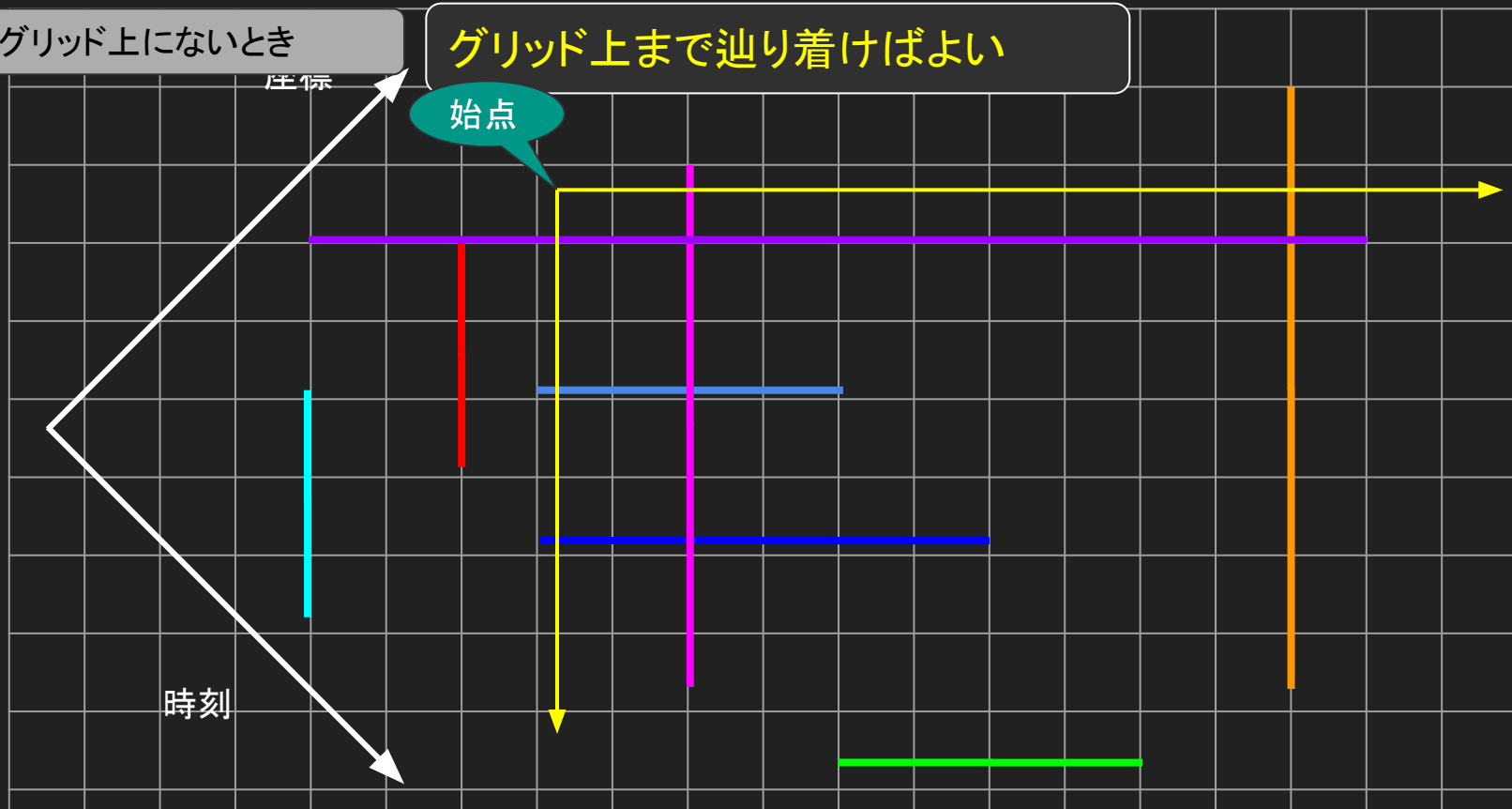
始点がグリッド上にないとき

グリッド上まで辿り着けばよい

始点

座標

時刻



小課題4(20点)

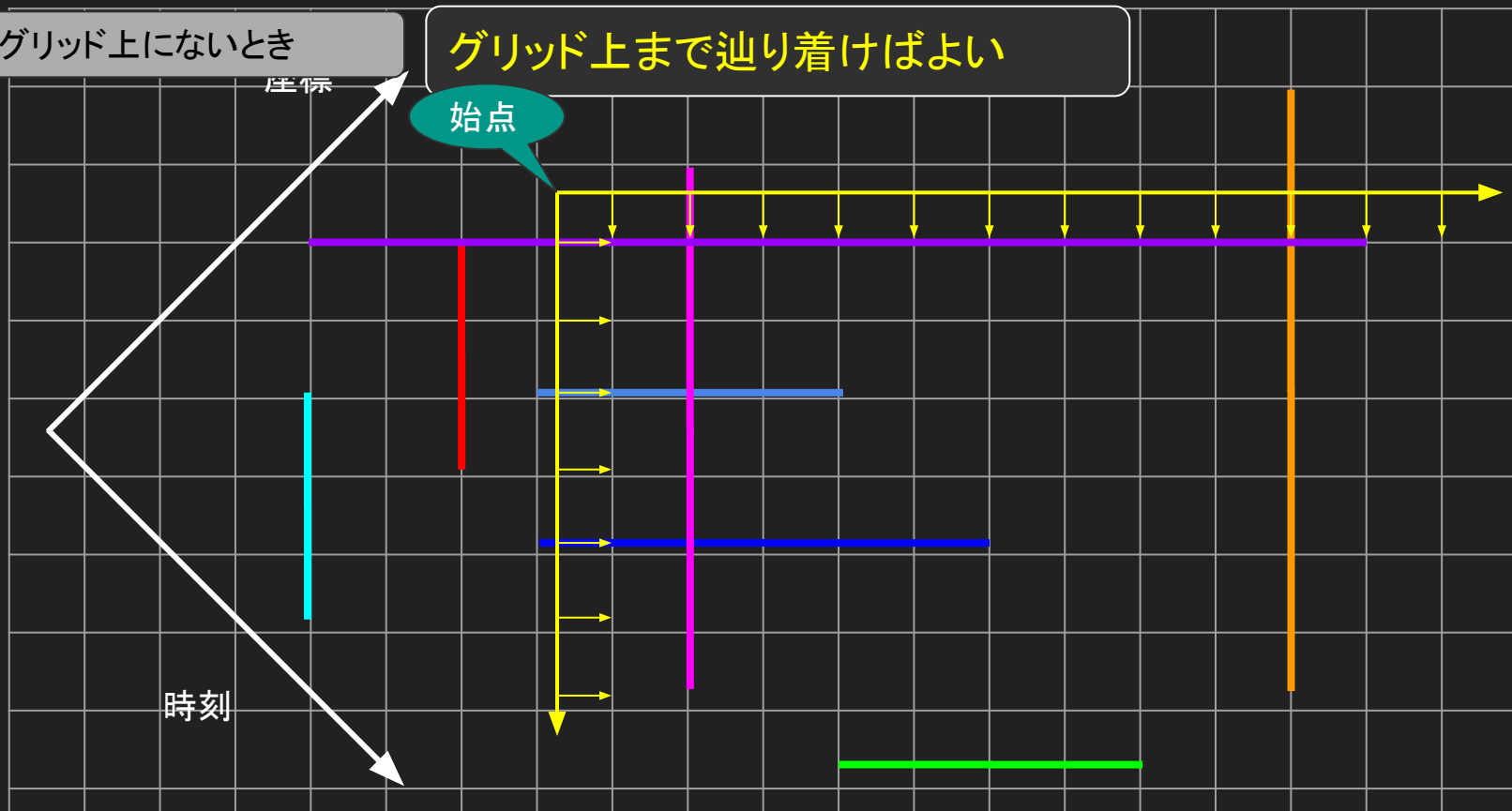
始点がグリッド上にないとき

グリッド上まで辿り着けばよい

始点

座標

時刻



小課題4(20点)

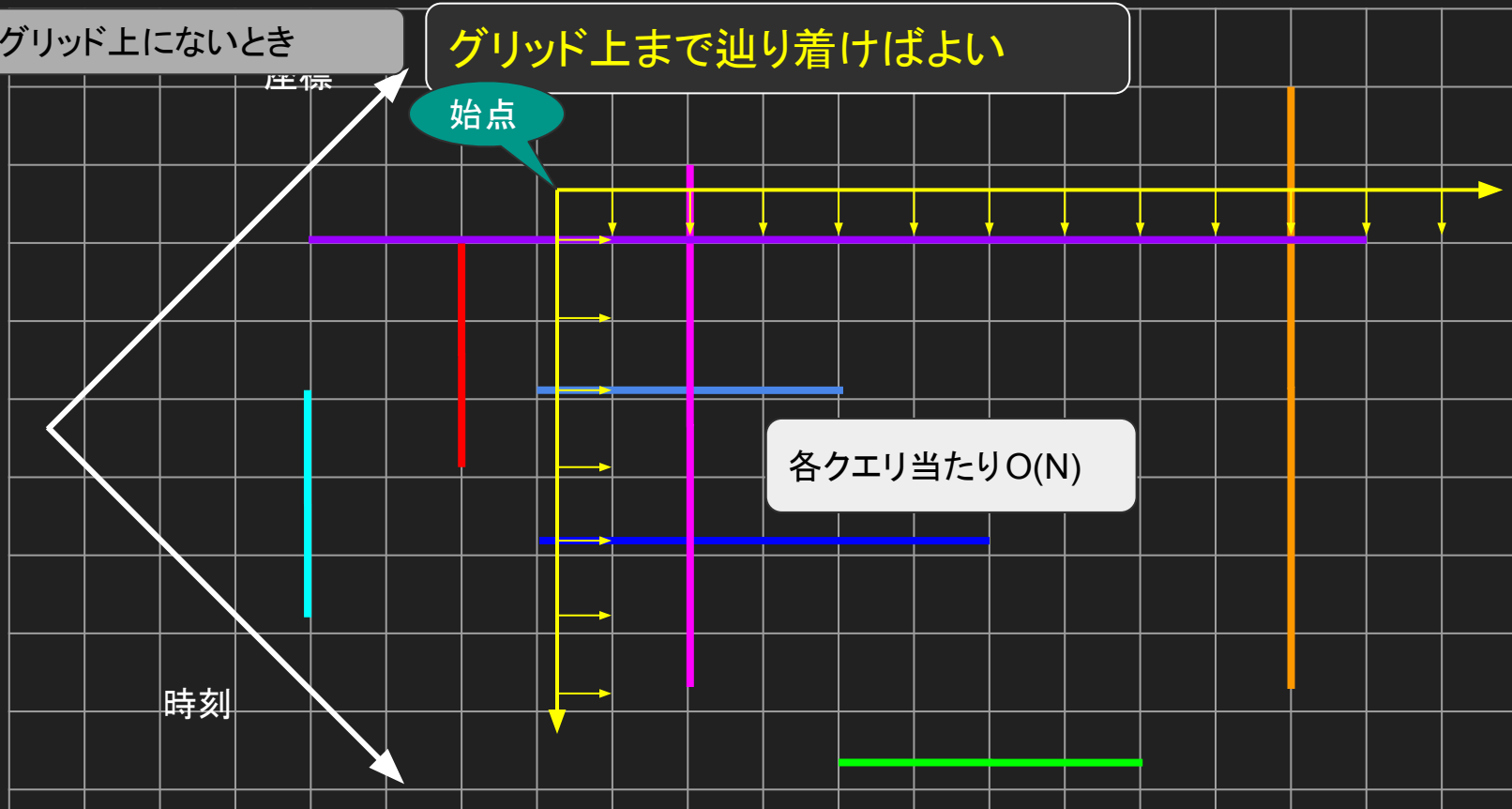
始点がグリッド上にないとき

グリッド上まで辿り着けばよい

始点

各クエリ当たり $O(N)$

時刻



小課題4(20点)

始点がグリッド上にないとき

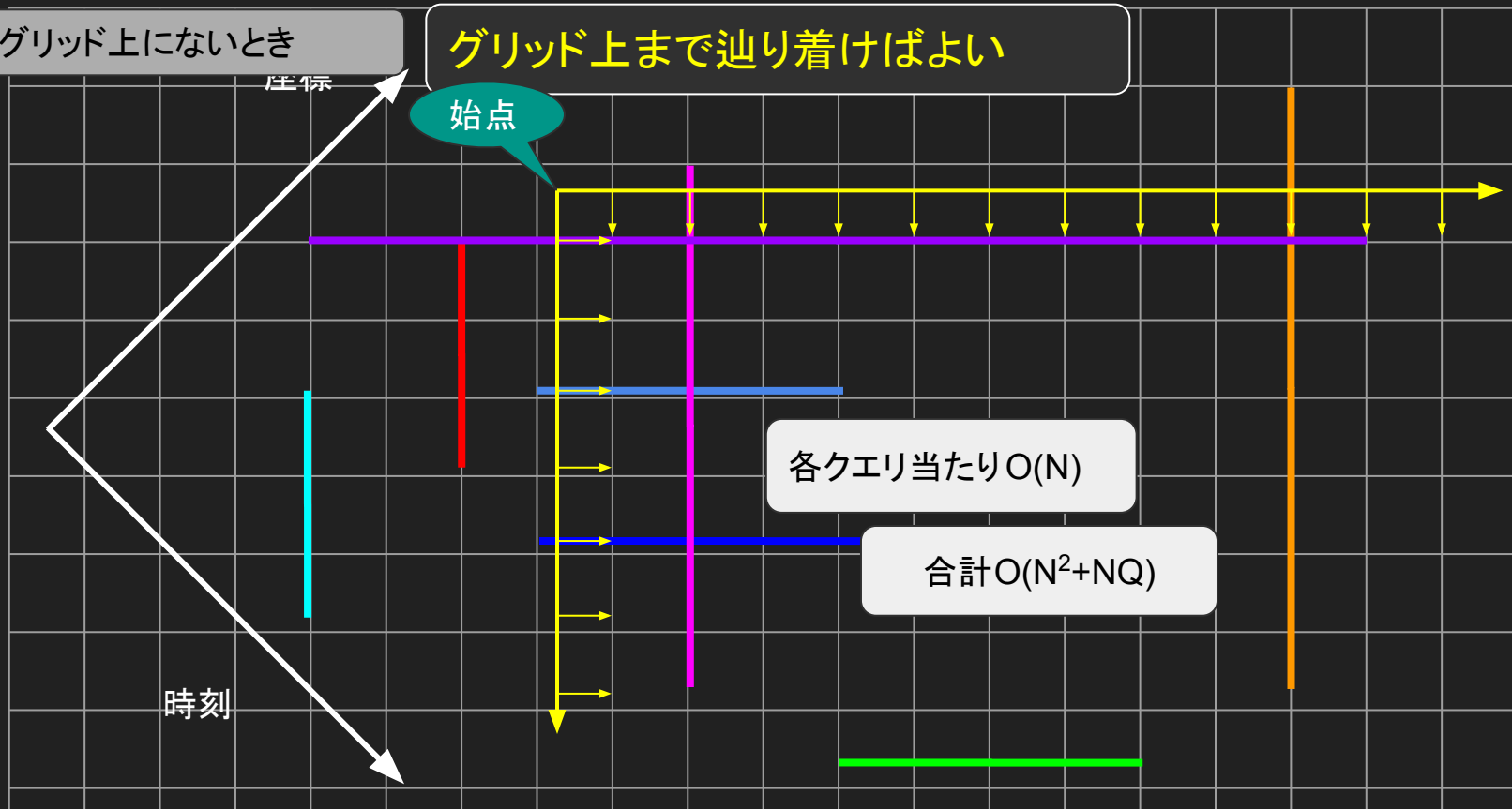
グリッド上まで辿り着けばよい

始点

各クエリ当たり $O(N)$

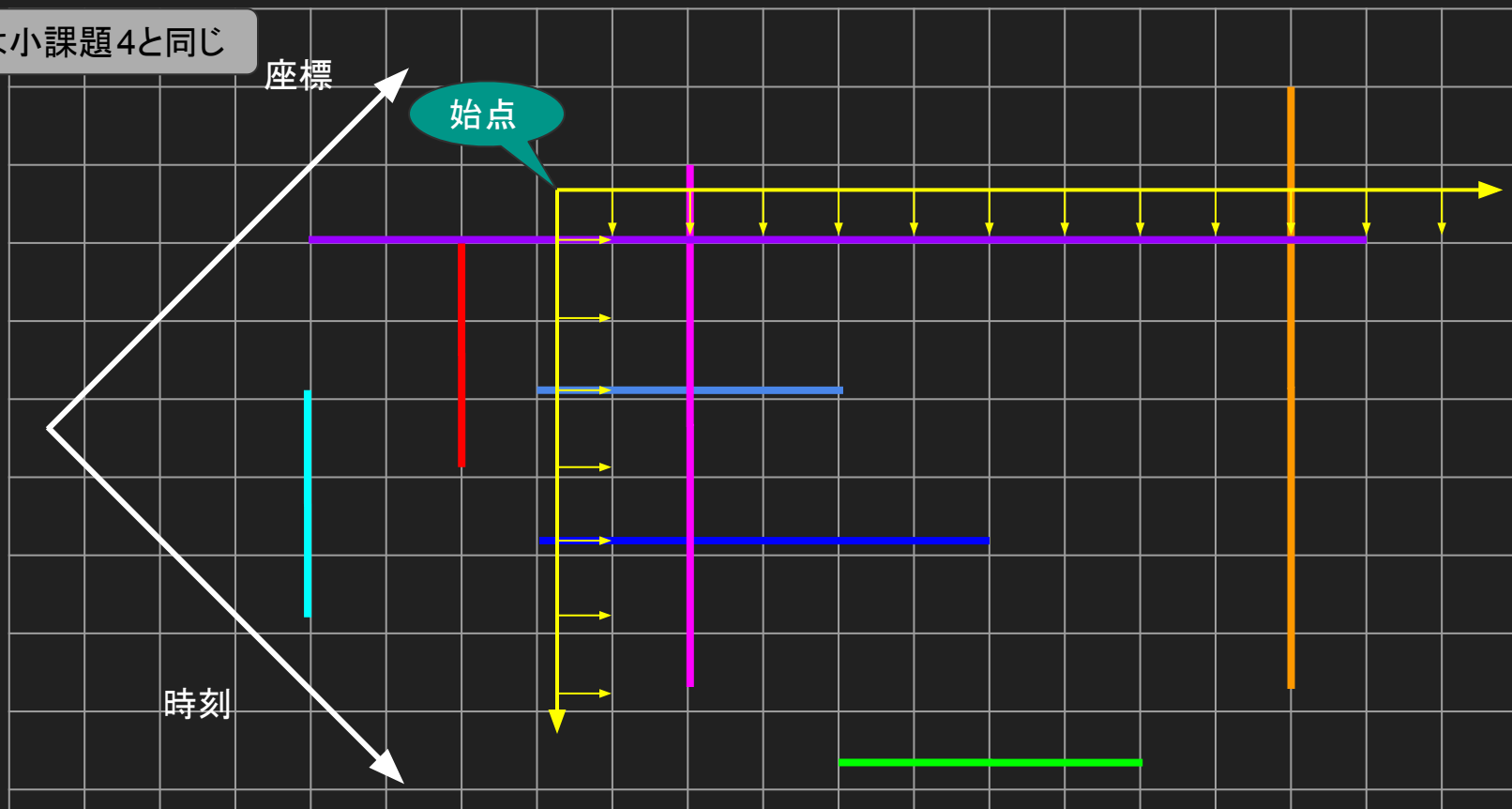
合計 $O(N^2 + NQ)$

時刻



小課題5(52点)

方針は小課題4と同じ



小課題5(52点)

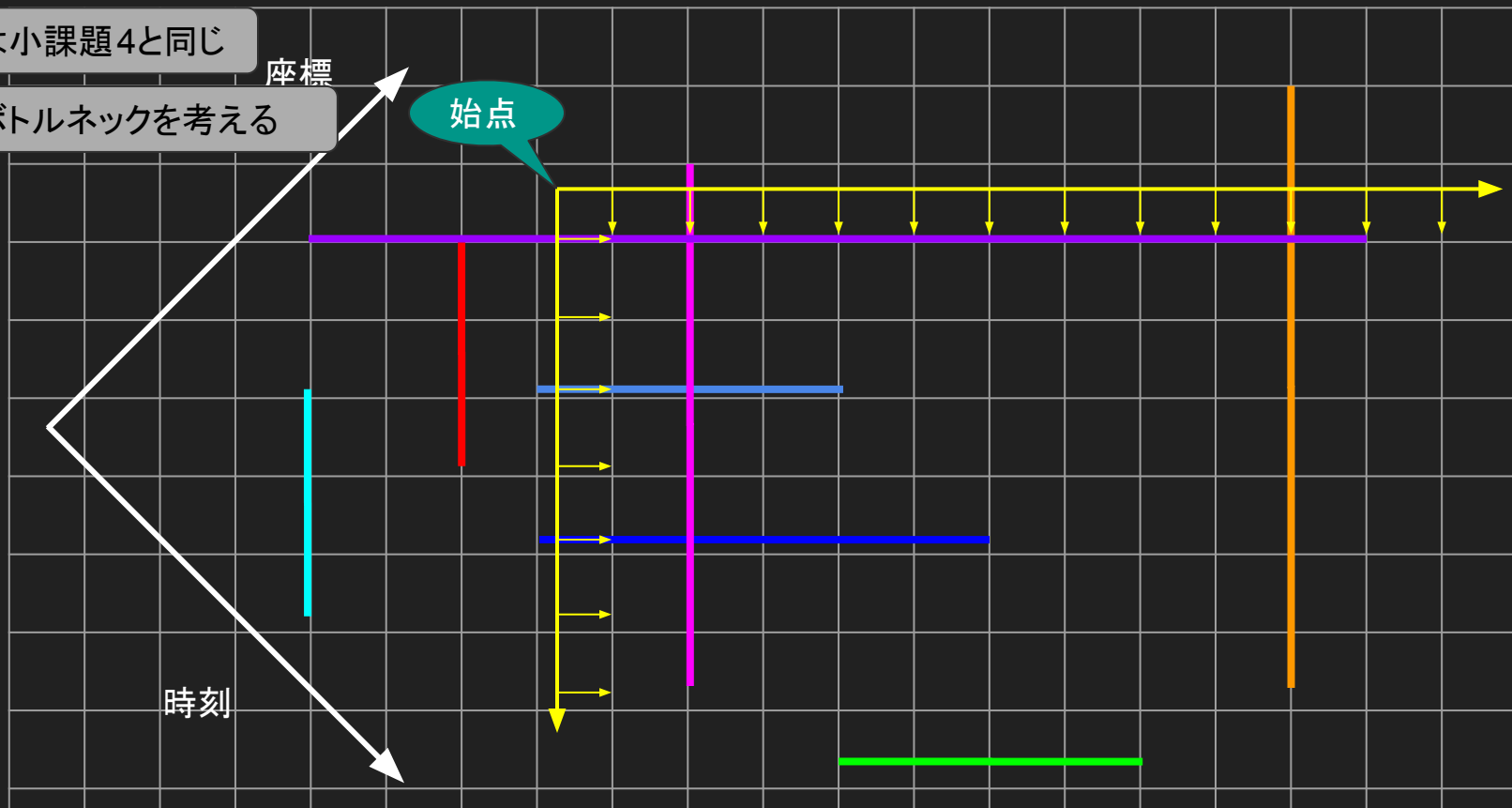
方針は小課題4と同じ

ボトルネックを考える

座標

始点

時刻



小課題5(52点)

方針は小課題4と同じ

ボトルネックを考える

座標

始点

各クエリ当たり $O(N)$

時刻

小課題5(52点)

方針は小課題4と同じ

ボトルネックを考える

座標

始点

各クエリ当たり $O(N)$

どうにかならないか？

時刻

小課題5(52点)

方針は小課題4と同じ

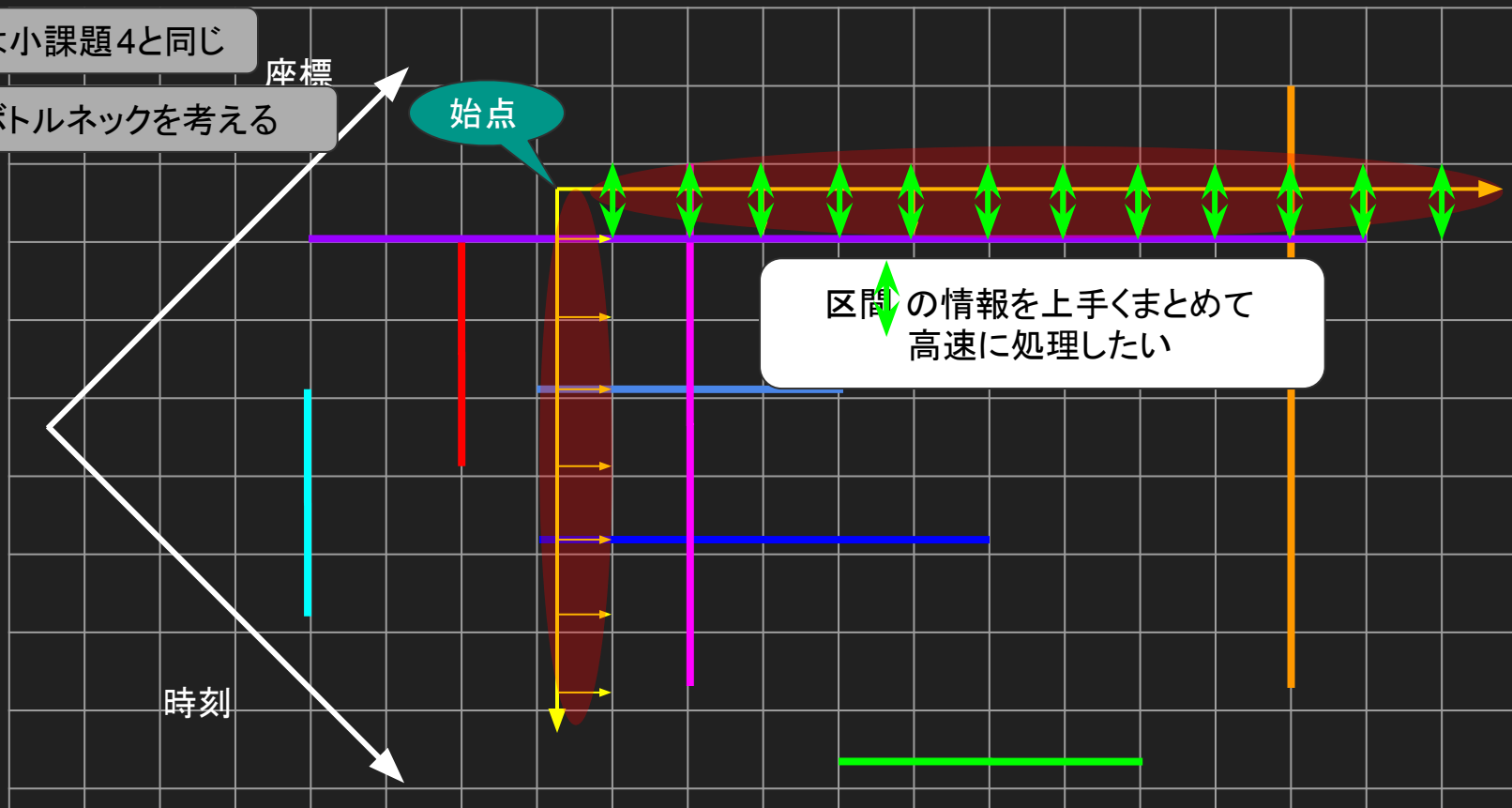
ボトルネックを考える

座標

始点

区間の情報を上手くまとめて
高速に処理したい

時刻



小課題5(52点)

方針は小課題4と同じ

ボトルネックを考える

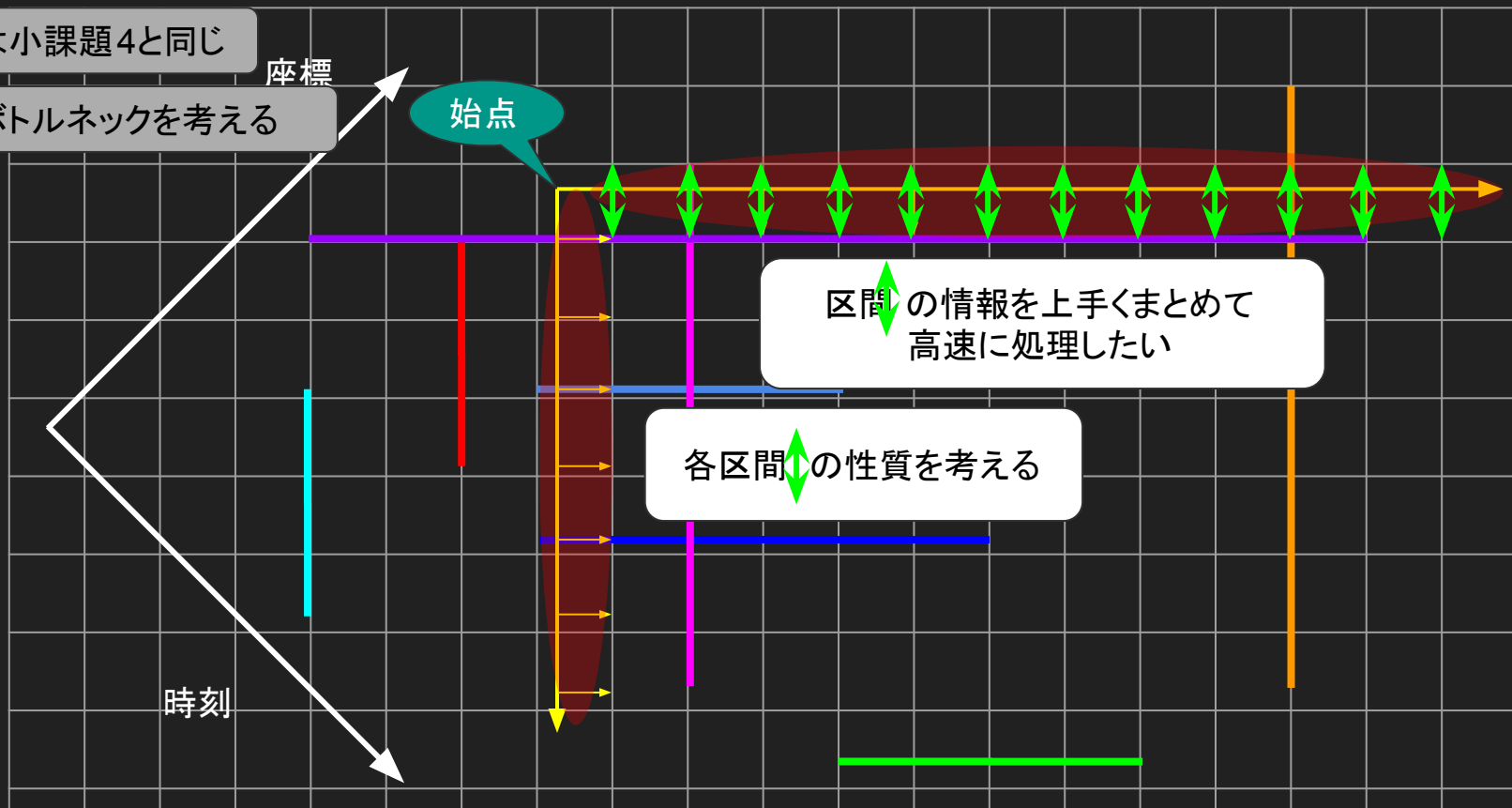
座標

始点

区間の情報を上手くまとめて
高速に処理したい

各区間の性質を考える

時刻



小課題5(52点)

方針は小課題4と同じ

ボトルネックを考える

座標

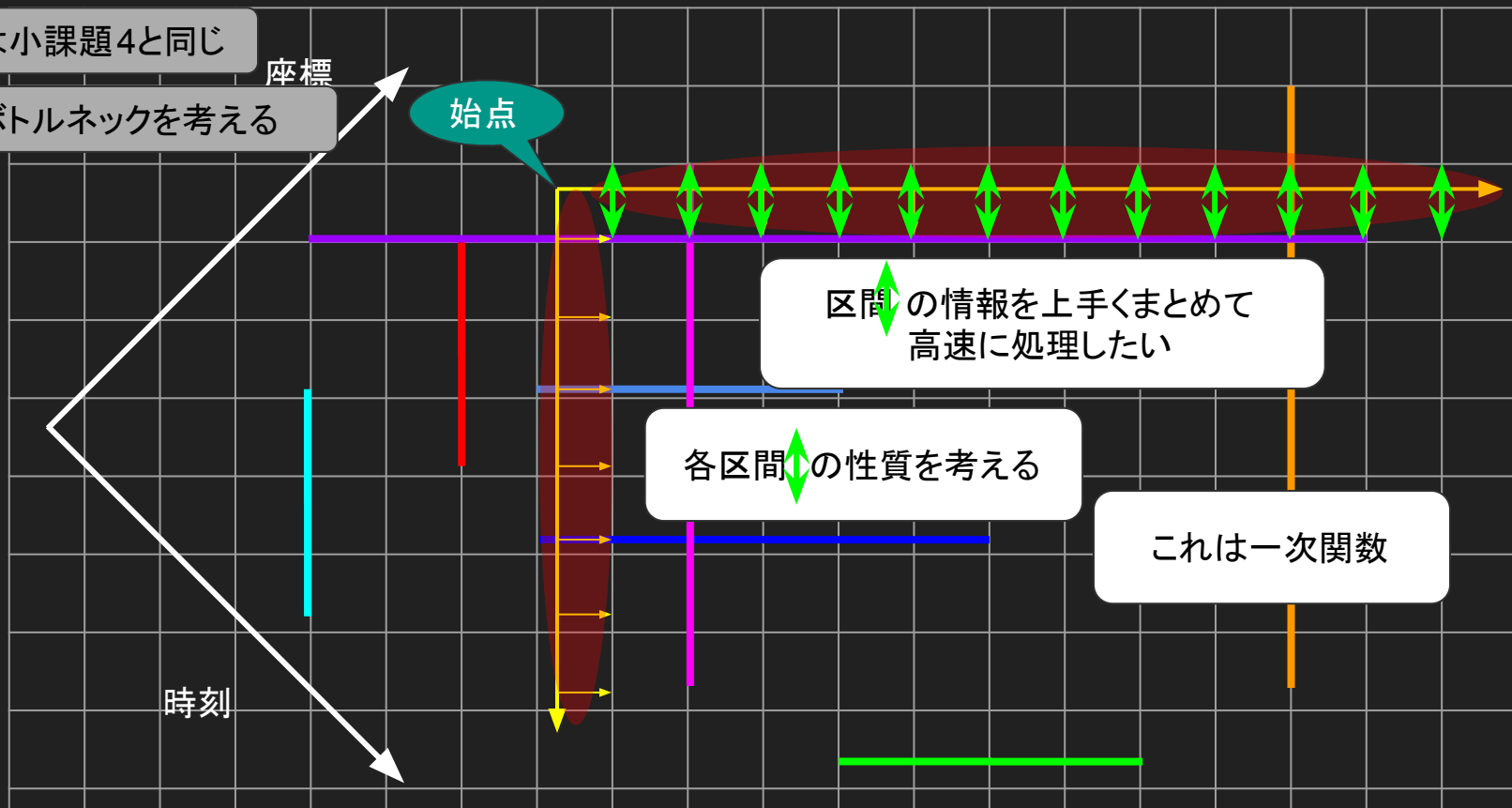
始点

区間の情報を上手くまとめて
高速に処理したい

各区間の性質を考える

これは一次関数

時刻



小課題5(52点)

方針は小課題4と同じ

ボトルネックを考える

座標

始点

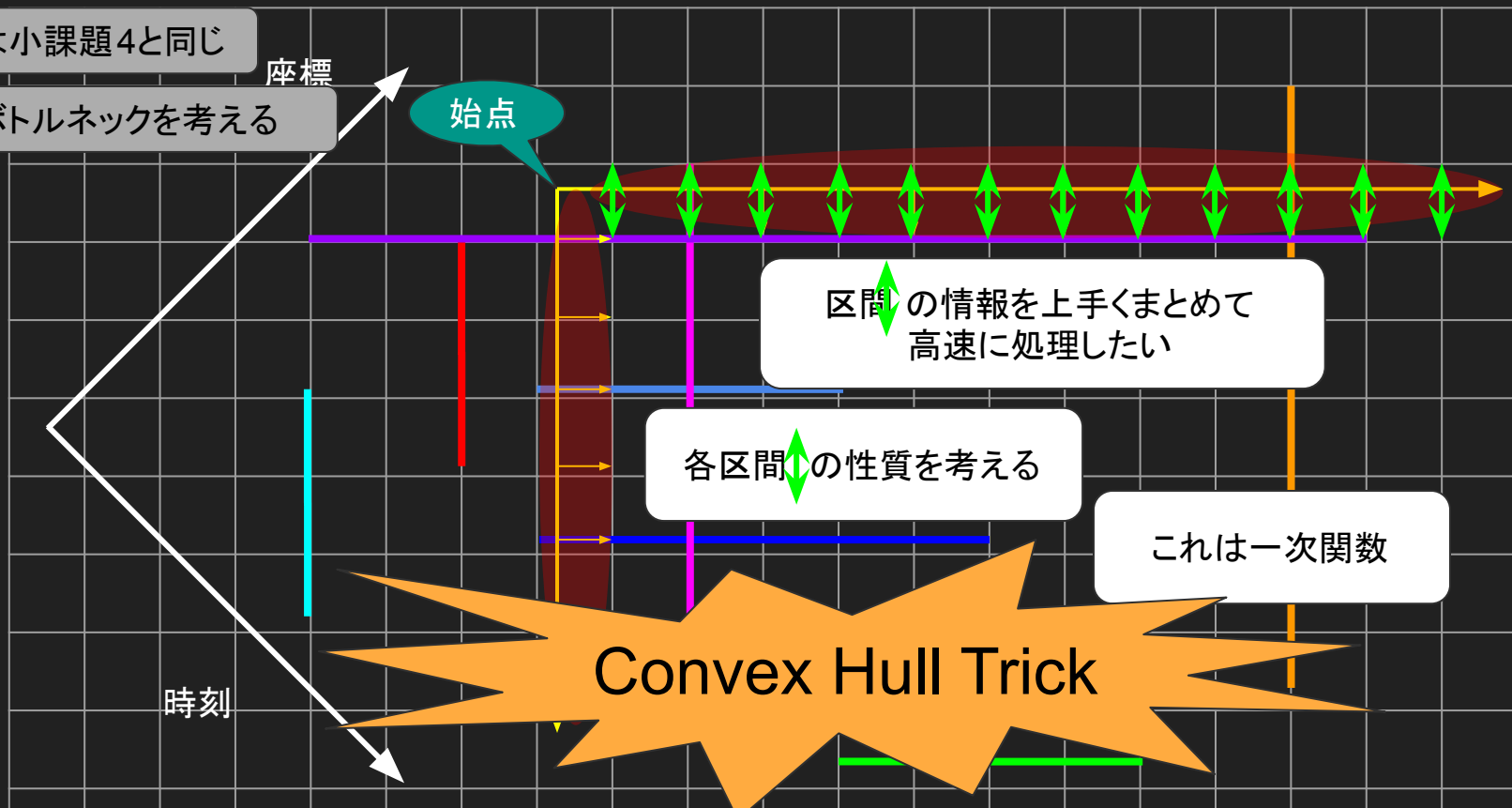
区間の情報を上手くまとめて
高速に処理したい

各区間の性質を考える

これは一次関数

時刻

Convex Hull Trick



小課題5(52点)

平面走査をします

一次関数追加は $O(1)$ 各クエリで最大値を求めるのに $O(\log N)$

$O(N^2 + Q \log N)$ で解けました

小課題5(52点)

平面走査をします

一次関数追加は $O(1)$ 各クエリで最大値を求めるのに $O(\log N)$

$O(N^2 + Q \log N)$ で解けました

終

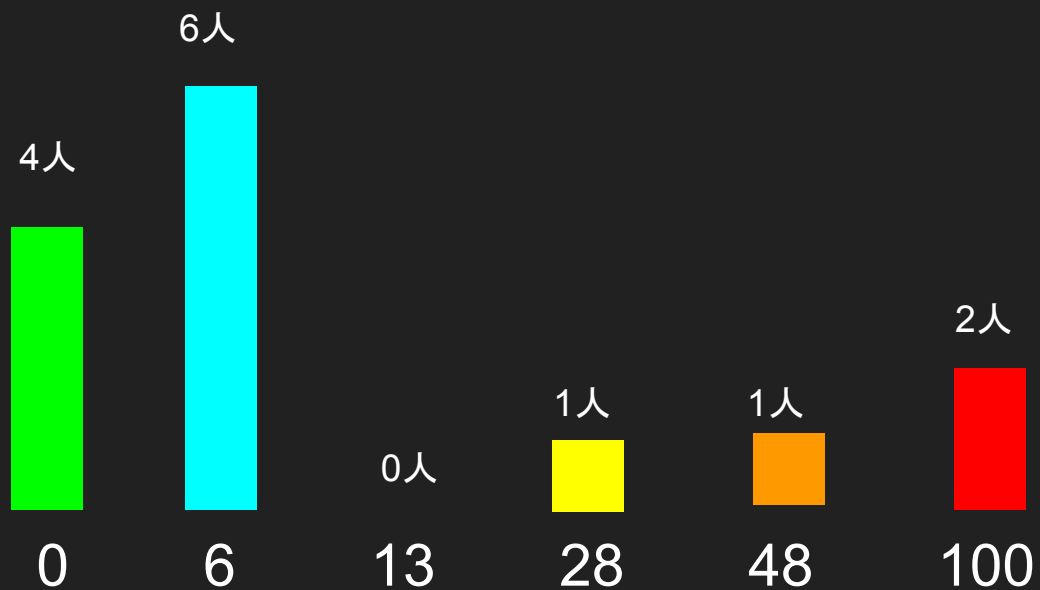
制作・著作

©H©T

得点分布(理想)



得点分布(現実)



Meetings2 解説

担当 : 高谷 悠太

問題概要

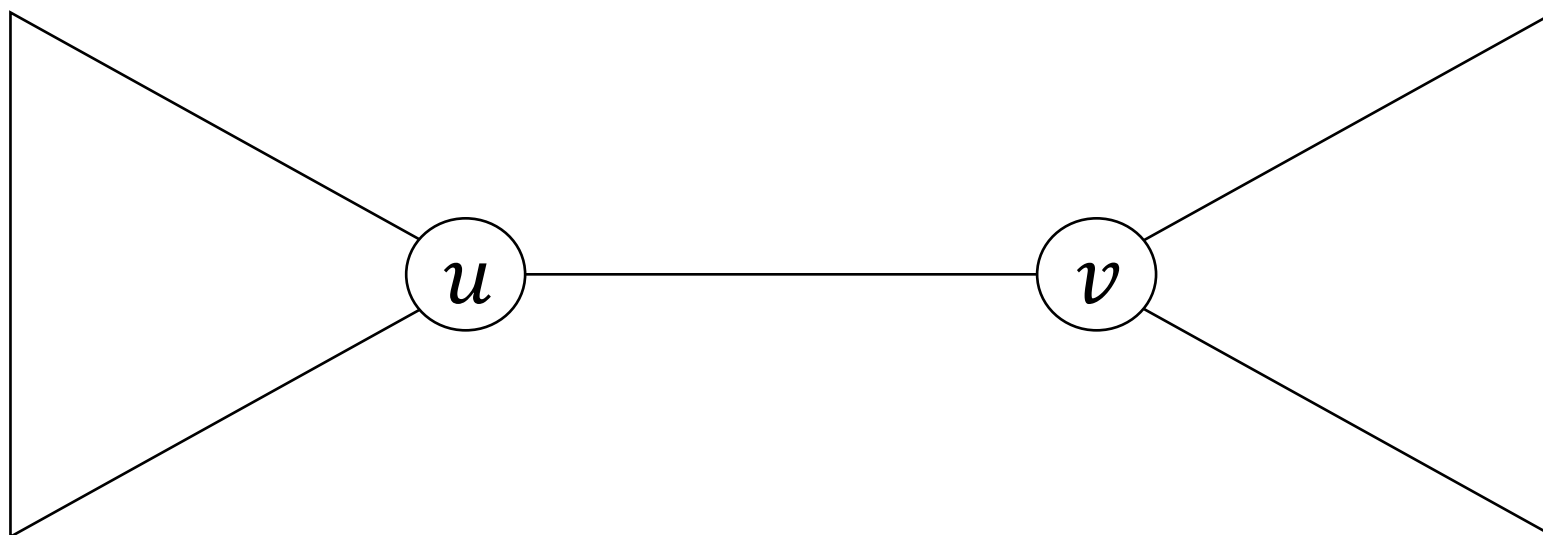
- N 頂点からなる木がある.
- $1 \leq k \leq N$ なる各整数 k について
頂点を k 個選んだとき, それらからの距離の和が最小となる頂点の個数としてありうる最大値
を求めよ.

小課題 1 : $N \leq 16$

- 頂点の選び方を 2^N 通り試す.
- 各頂点につき $O(N)$ で距離の和を計算する.
- 全体で $O(N^2 2^N)$ の計算量で求まる.

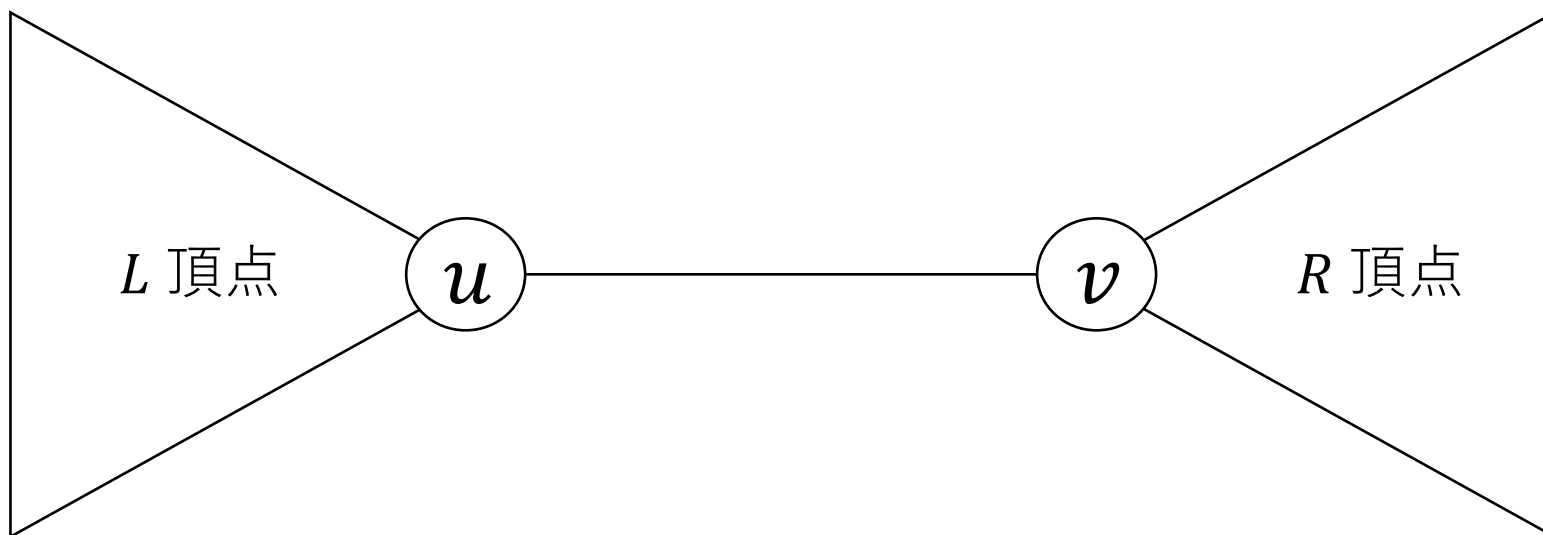
小課題 2 : $N \leq 4,000$

- 頂点集合を決めたとき，距離の和が最小となる頂点が 2 つ以上ある場合を考える．

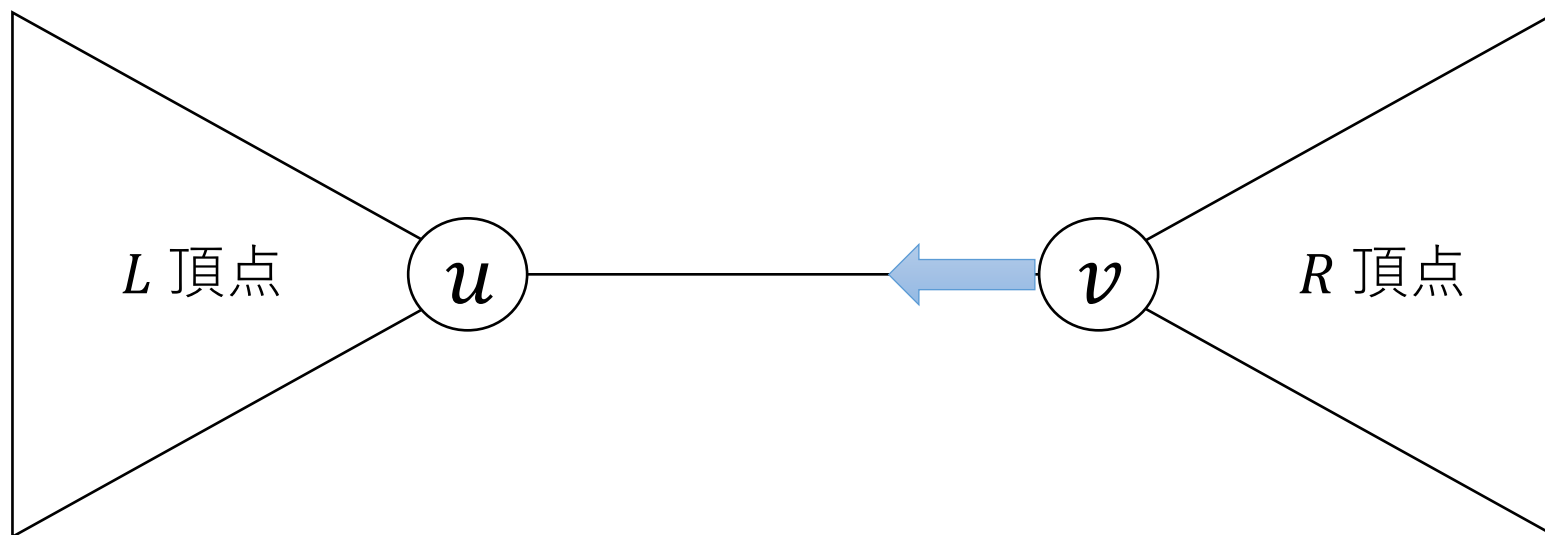


小課題 2 : $N \leq 4,000$

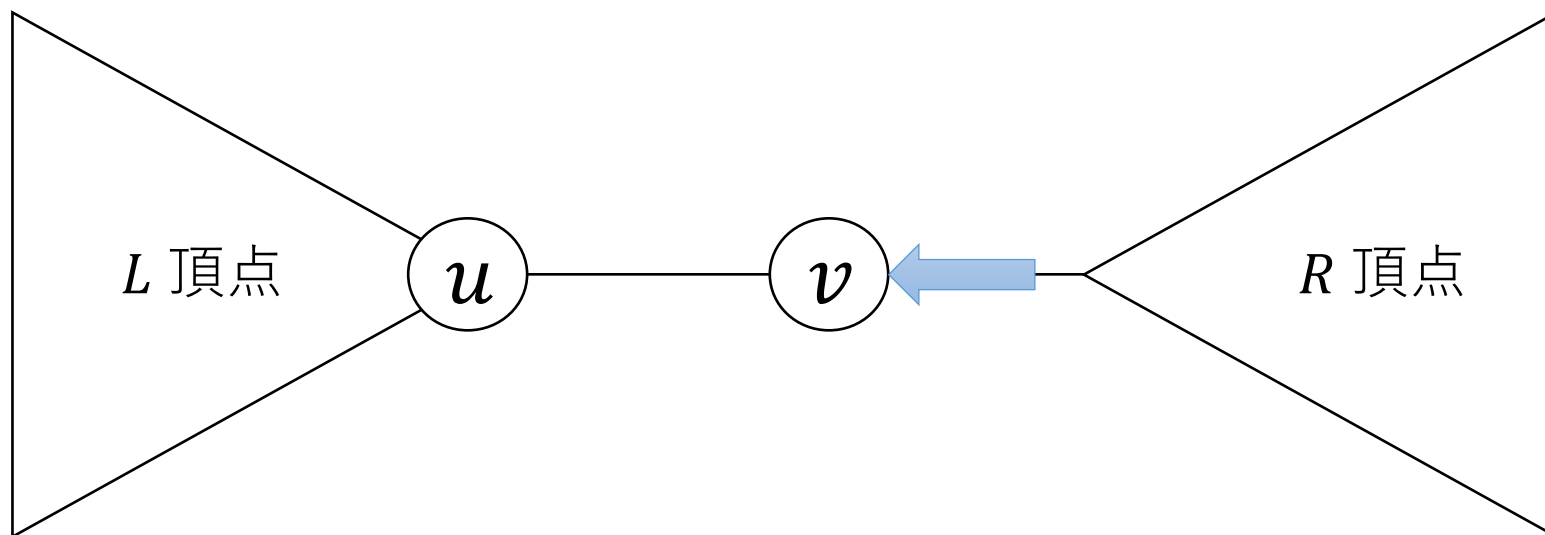
- L : 左の部分木内にある選ばれた頂点の個数
- R も同様.



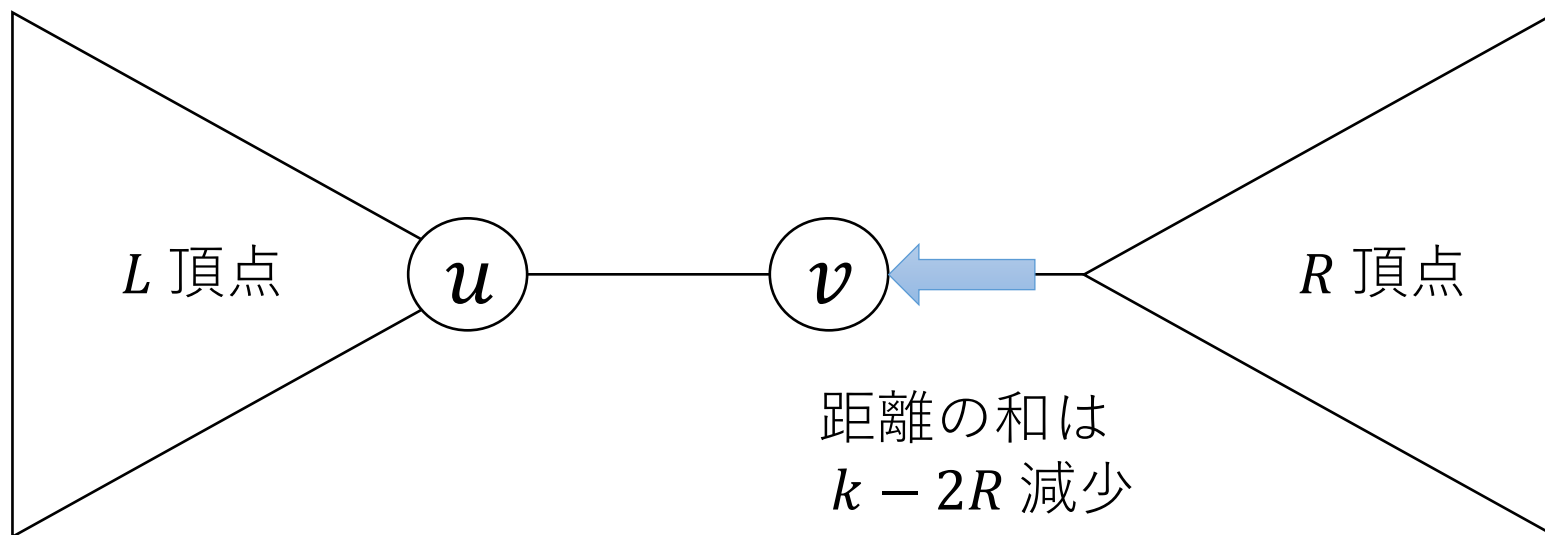
小課題 2 : $N \leq 4,000$



小課題 2 : $N \leq 4,000$

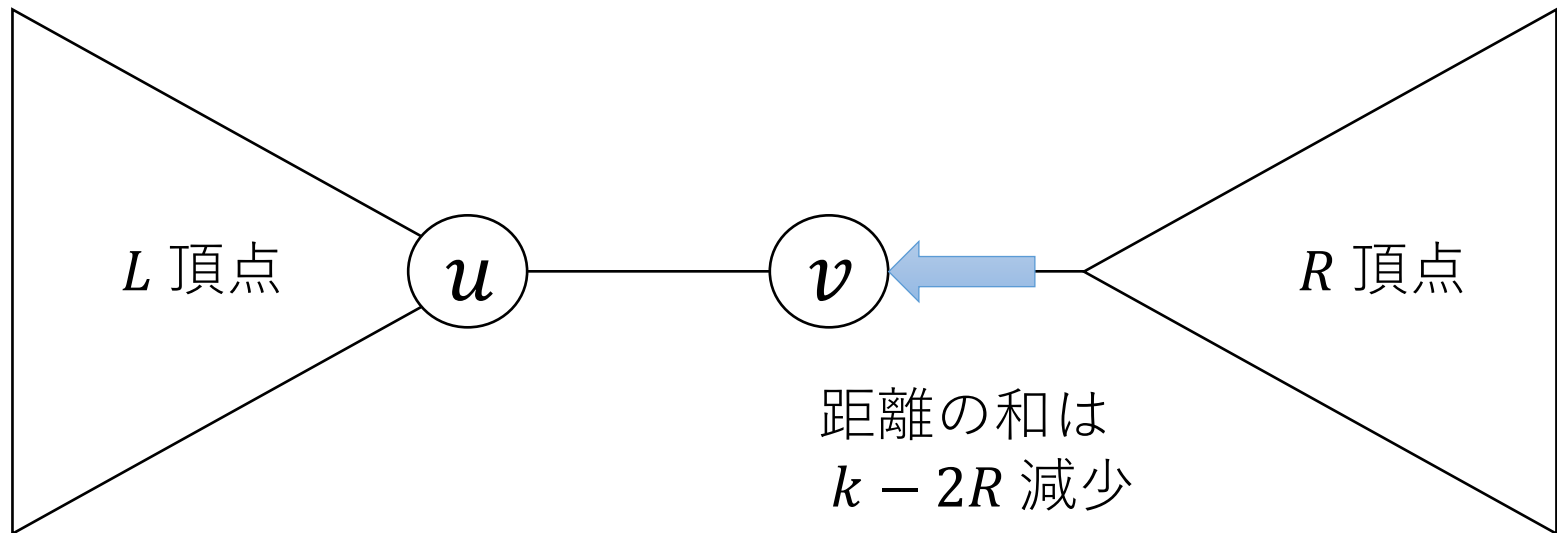


小課題 2 : $N \leq 4,000$



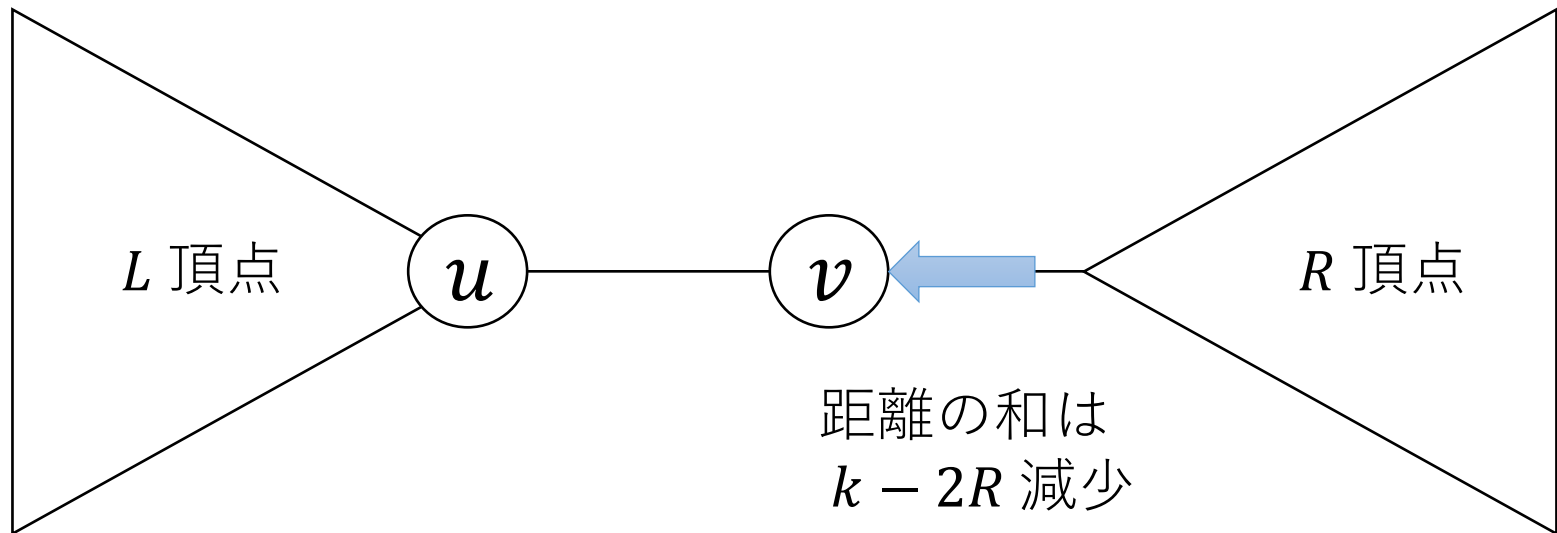
小課題 2 : $N \leq 4,000$

- v の最小性から $k - 2R \leq 0$



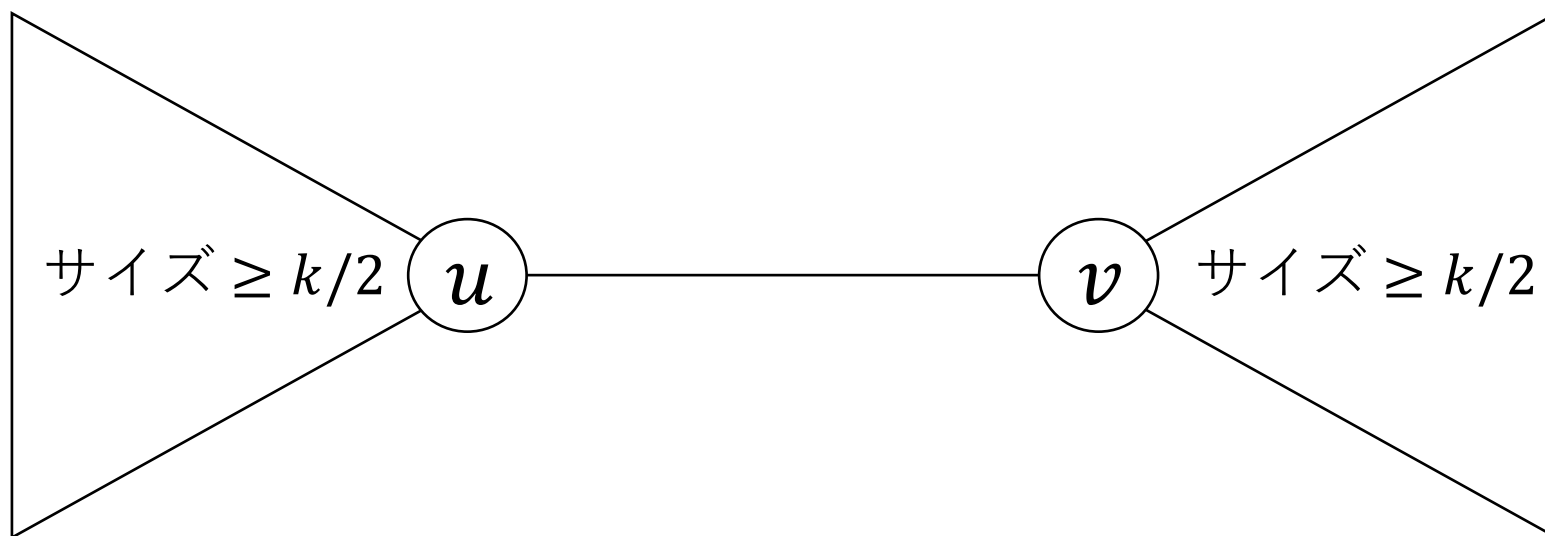
小課題 2 : $N \leq 4,000$

- v の最小性から $k - 2R \leq 0$
- 同様に $k - 2L \leq 0$ なので $L = R = k/2$



小課題 2 : $N \leq 4,000$

- k が奇数の場合は答えは 1
- k が偶数の場合は, 以下を満たす u, v の距離の最大値 + 1 が答え.

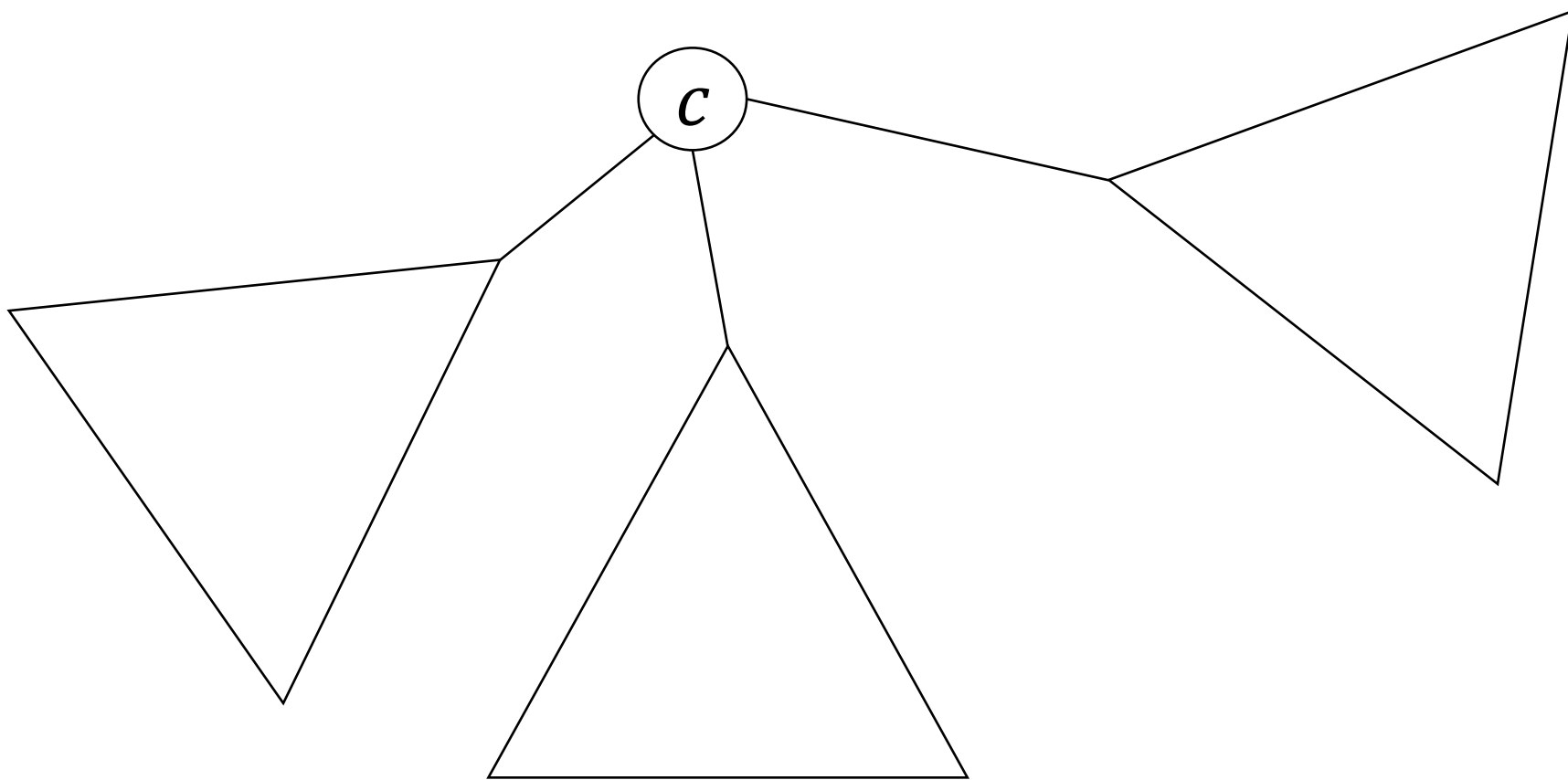


小課題 2 : $N \leq 4,000$

- u を N 通り試す.
- 各 v について, 以下を求める.
 1. u からの距離
 2. v を含まない u に"接続する"部分木のサイズ
 3. u を含まない v に"接続する"部分木のサイズ
- 各 u ごとに $O(N)$ ですべての値が求まる.

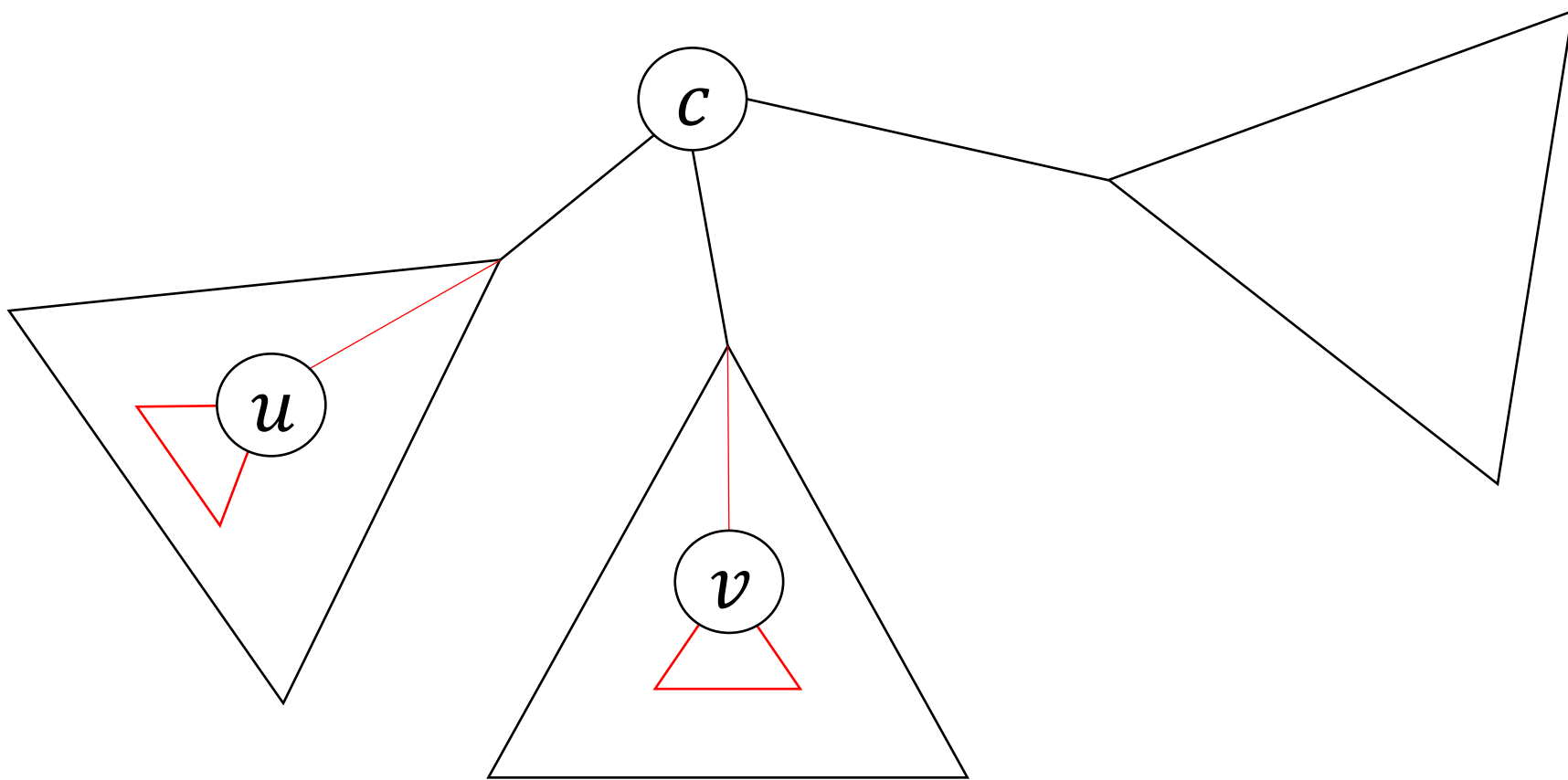
小課題 3 : $N \leq 200,000$

- 木の重心 c をとる.



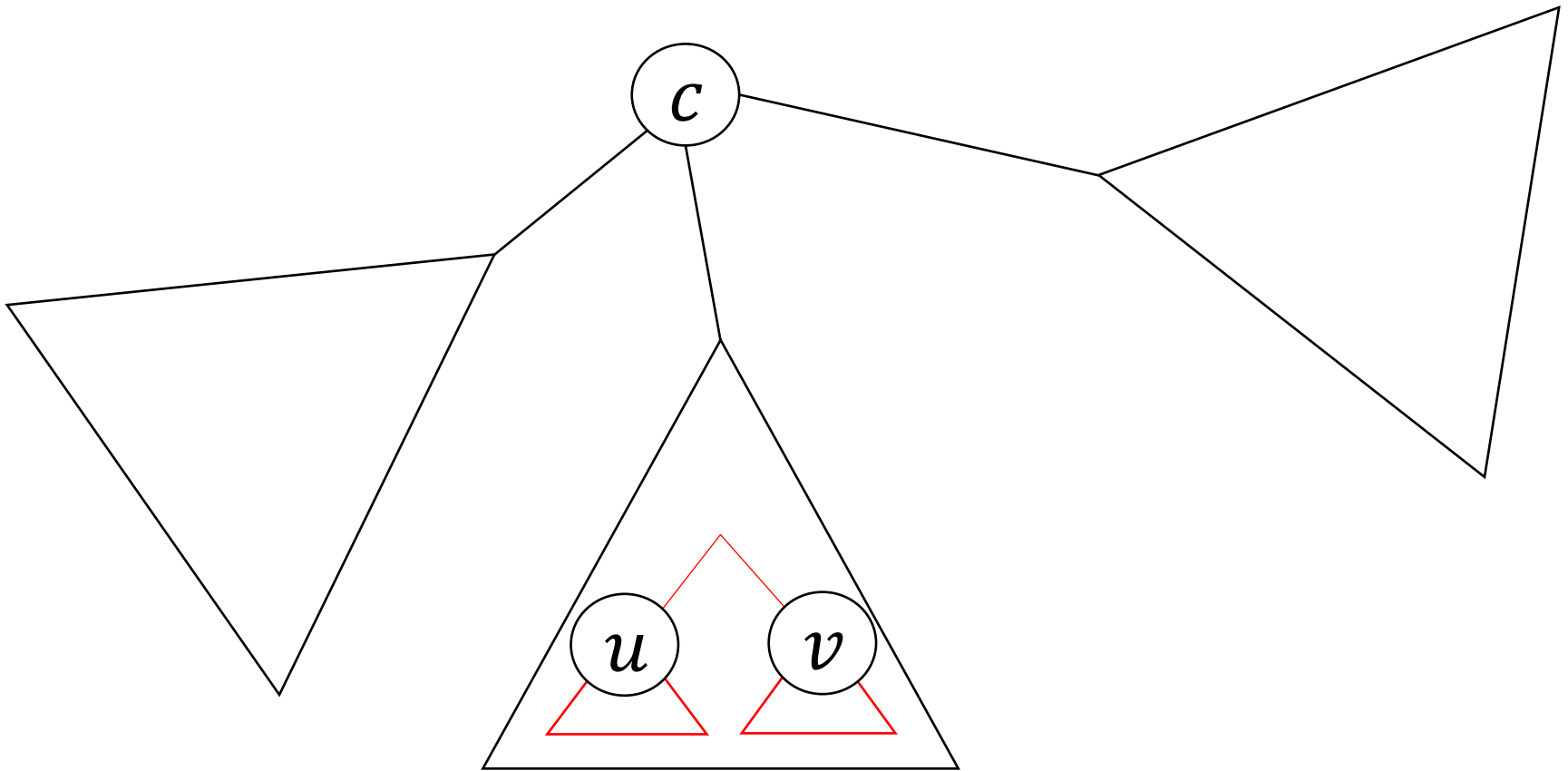
小課題 3 : $N \leq 200,000$

- u, v, c の位置関係を考える.



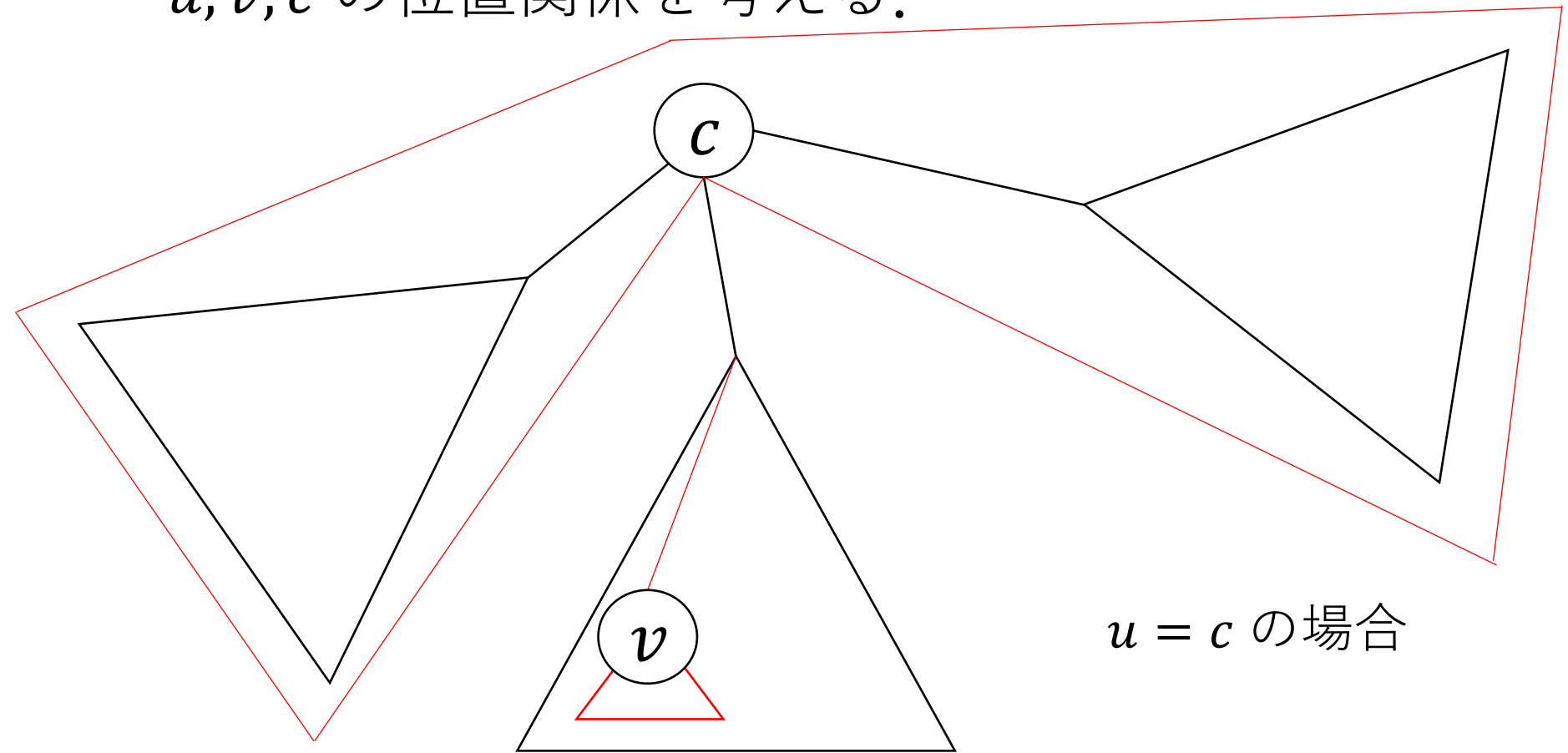
小課題 3 : $N \leq 200,000$

- u, v, c の位置関係を考える.



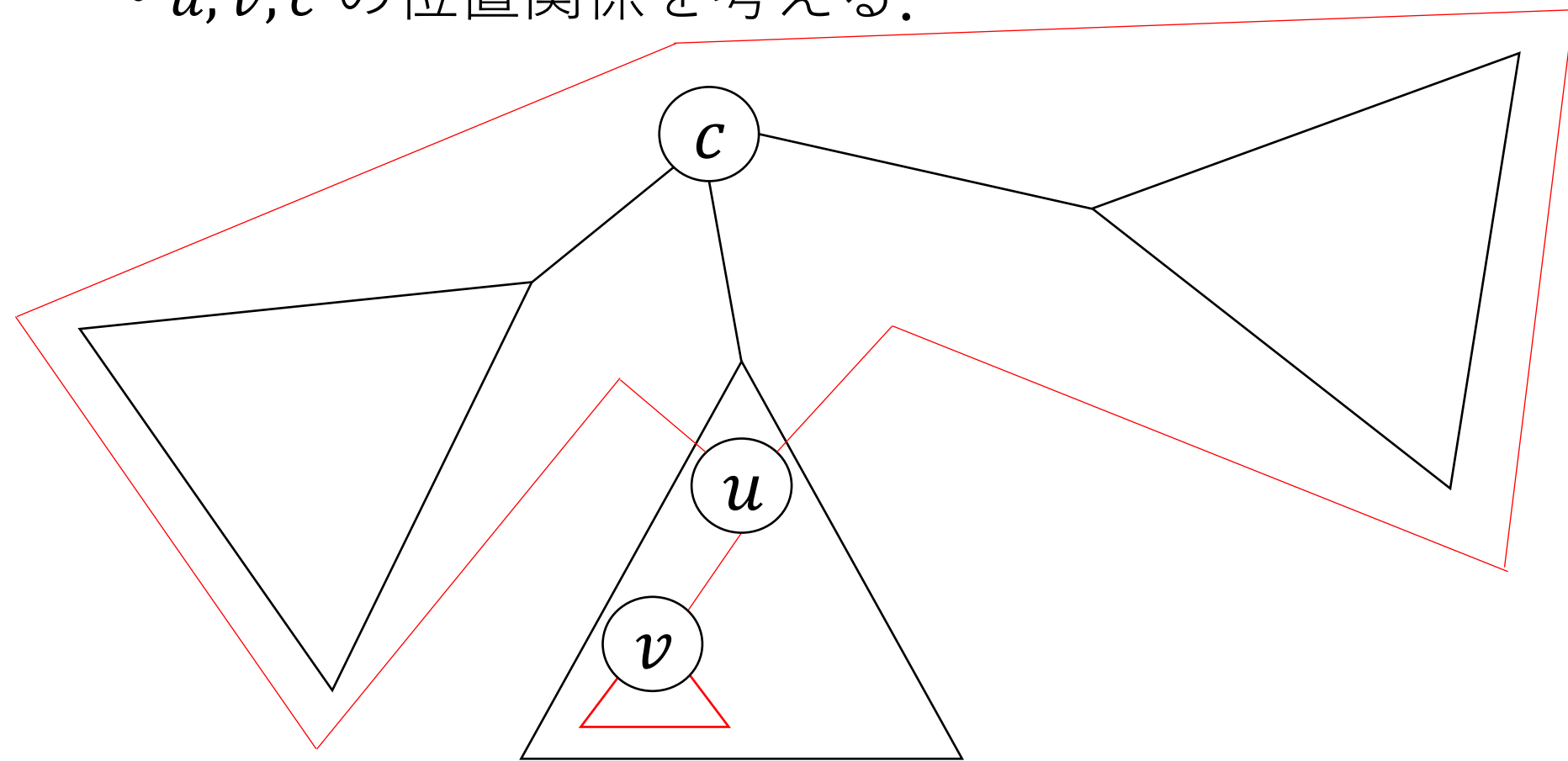
小課題 3 : $N \leq 200,000$

- u, v, c の位置関係を考える.



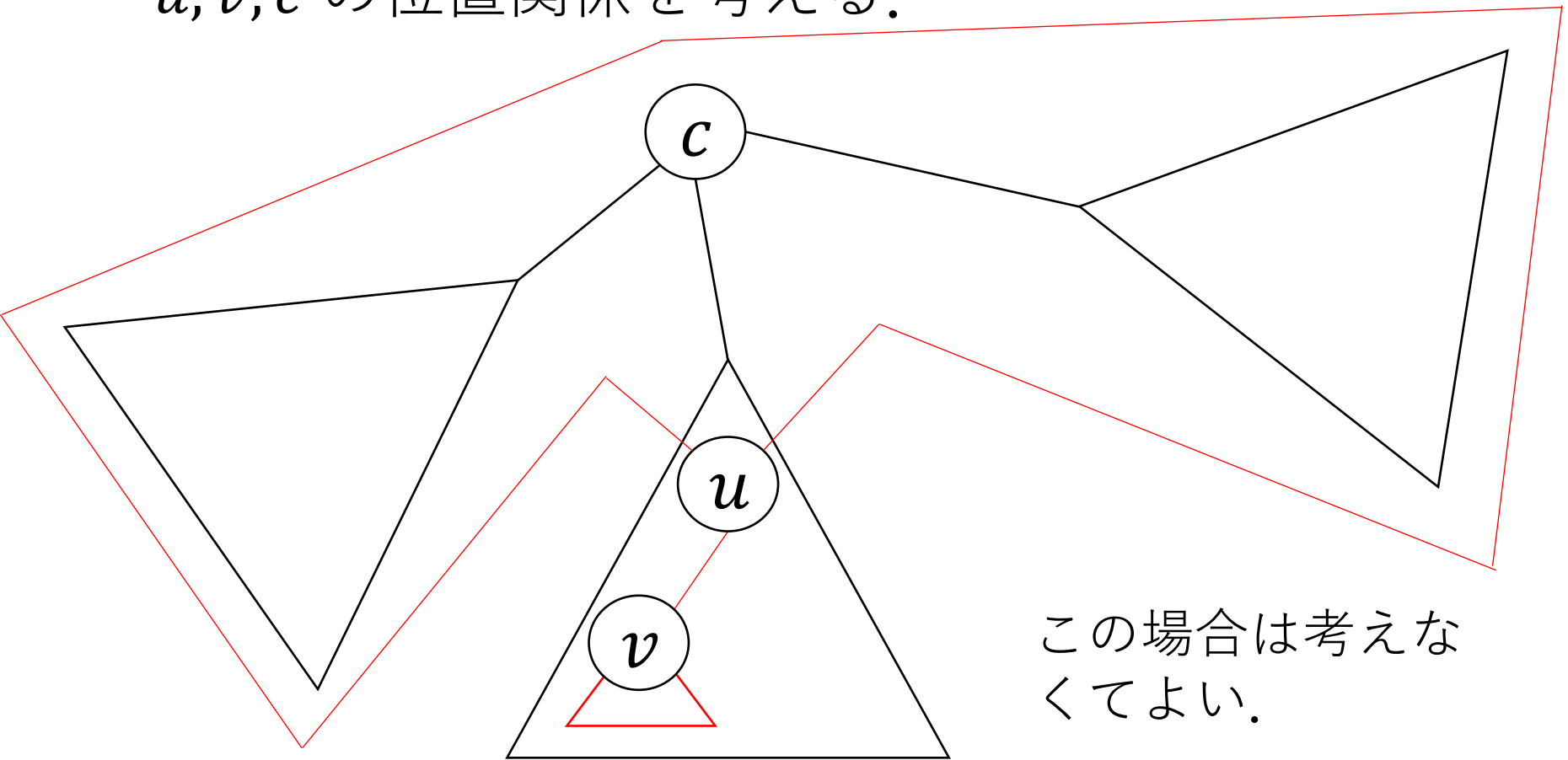
小課題 3 : $N \leq 200,000$

- u, v, c の位置関係を考える.



小課題 3 : $N \leq 200,000$

- u, v, c の位置関係を考える.



この場合は考えなくてよい.

小課題 3 : $N \leq 200,000$

- u, v の一方が c の場合は簡単なので、以下そうでない場合を考える.
- 先の考察より, u に接続する v を含まない部分木は c を含まないとしてよい.
 - 逆も同様.

小課題 3 : $N \leq 200,000$

- c を根として根付き木とする.
- u に接続する v を含まない部分木は, 根付き木としたとき, u を根とする部分木に対応する.
 - そのサイズは v によらない値となる.
- 各 $1 \leq k \leq N/2$ について以下を求めればよい.
 - 対応する部分木のサイズが k 以上である 2 頂点の距離の最大値.

小課題 3 : $N \leq 200,000$

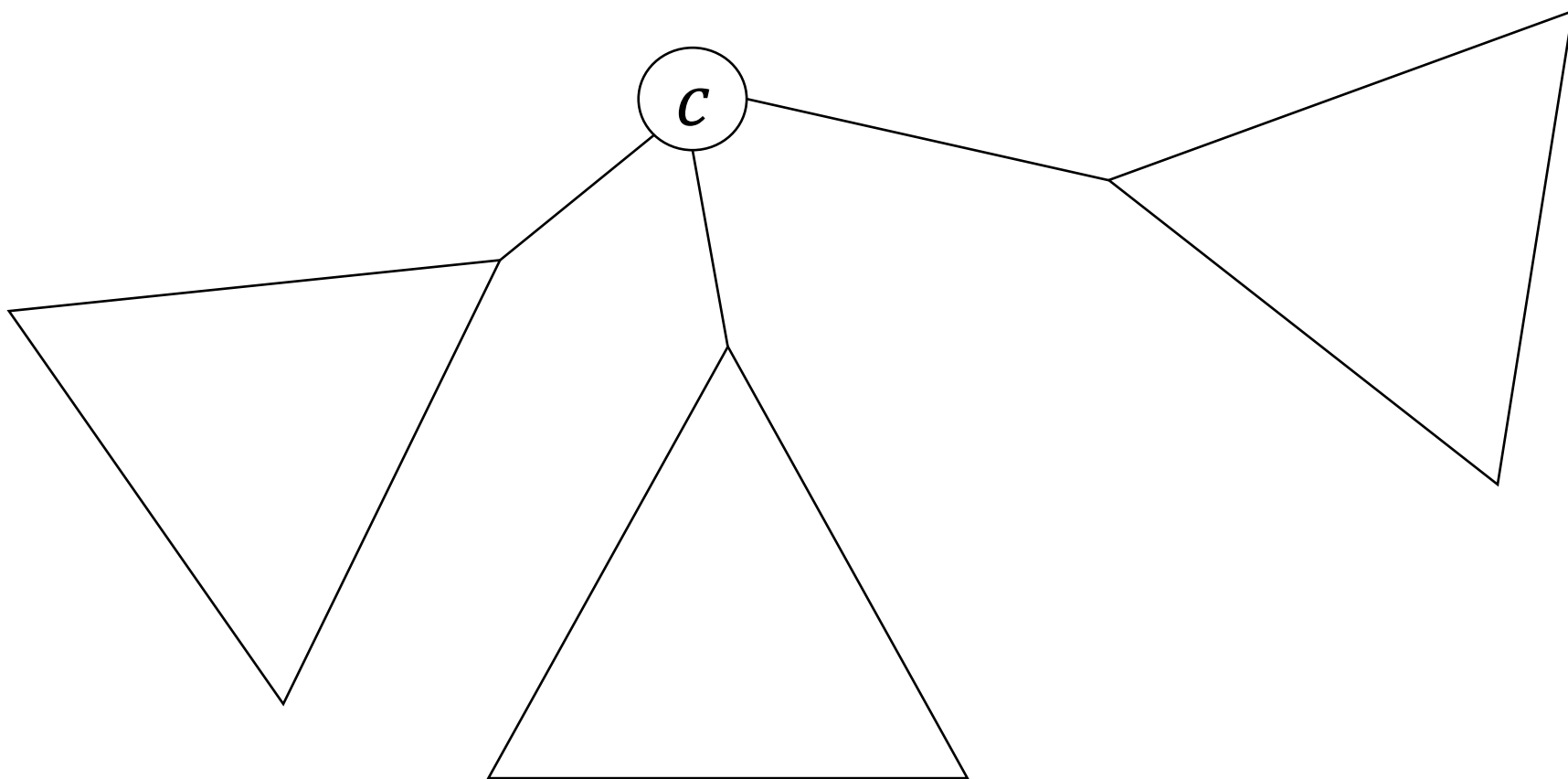
- 頂点を 1 つずつ追加していき，直径を求めることができるように.
- 各時点で，直径の端点を保持しておく.
- 追加する頂点と追加前の端点の距離が分かれば，直径とその端点を更新できる.

小課題 3 : $N \leq 200,000$

- LCA が計算できればよい.
- 計算量は $O(N \log N)$

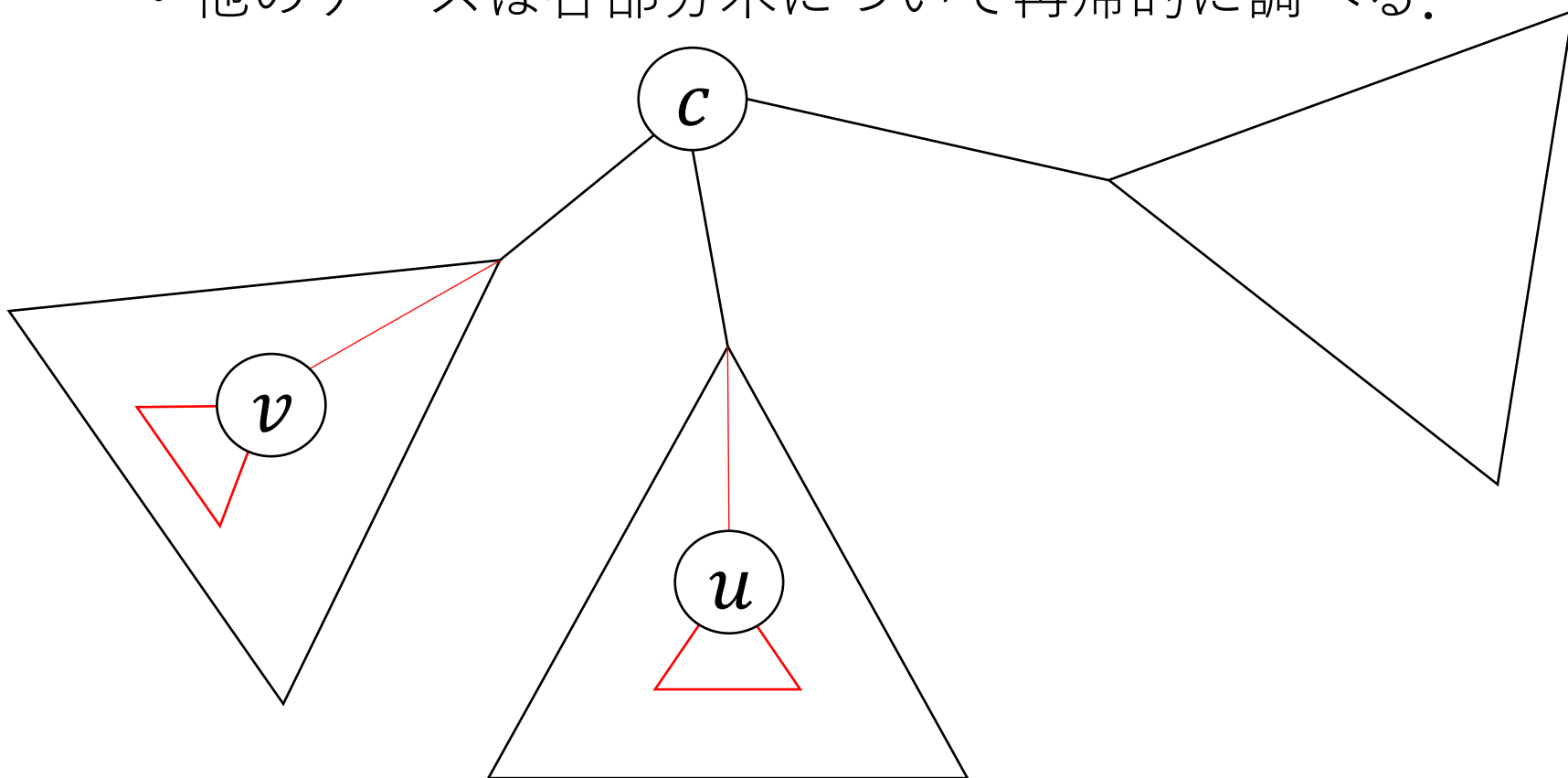
小課題 3 : $N \leq 200,000$ (別解)

- 木の重心 c をとる.



小課題 3 : $N \leq 200,000$ (別解)

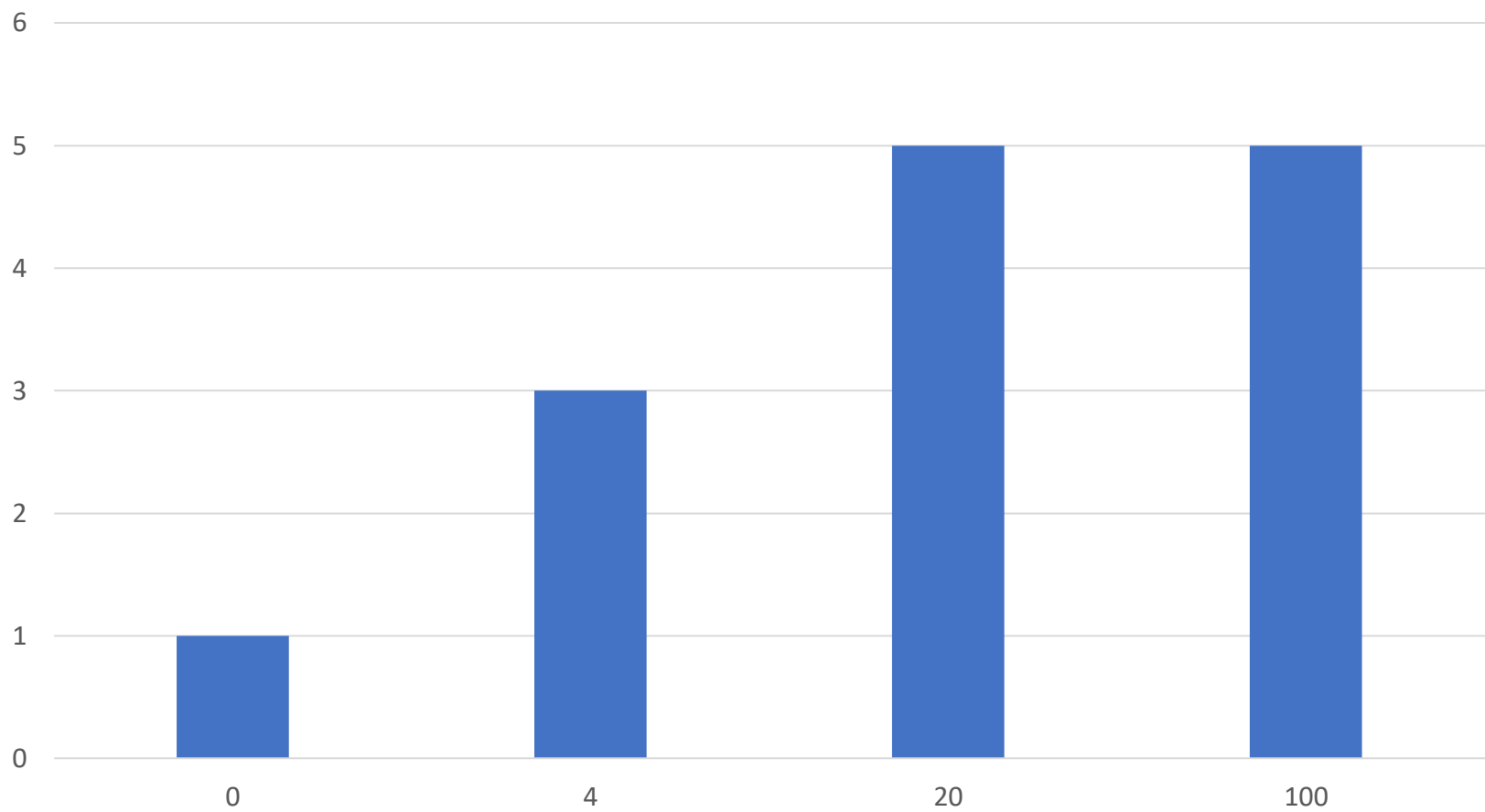
- この場合のみ考えれば十分.
 - 他のケースは各部分木について再帰的に調べる.



小課題 3 : $N \leq 200,000$ (別解)

- 重心まわりの部分木を適当に並べる.
- 左右から部分木を順に見ていき, 各段階で
 - ある一定のサイズ以上の部分木の重心からの距離の
最大値
が分かればよい.
- RMQ を使えば可能. 計算量は $O(N(\log N)^2)$

得点分布





イベント巡り 2

(Event Hopping 2)

解説：北村祐稀

JOI 2020/2021 春合宿 Day 4

問題概要

- N 個の开区間 (L_i, R_i) が与えられる
- 重ならないように K 個の区間 $a_1, \dots, a_K$ を選ぶ
- 辞書順最小となる数列 $(a_1, \dots, a_K)$ を求めよ

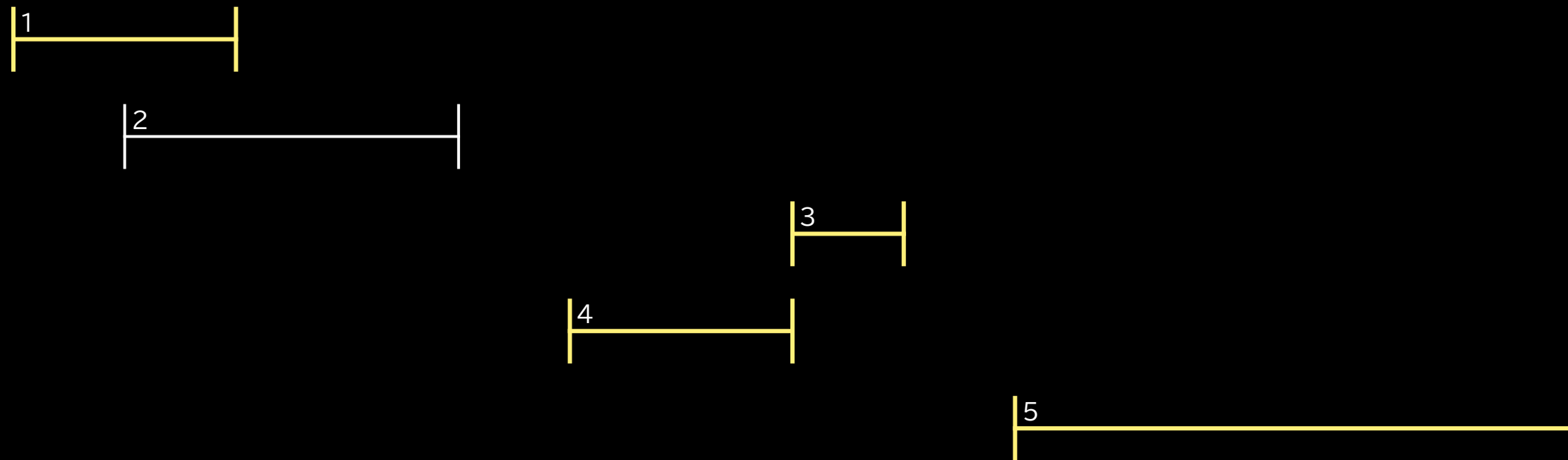
入出力例 1

$$N = 5, K = 4$$



入出力例 1

$$N = 5, K = 4$$



$$(a_1, \dots, a_K) = (1, 3, 4, 5)$$

入出力例 1

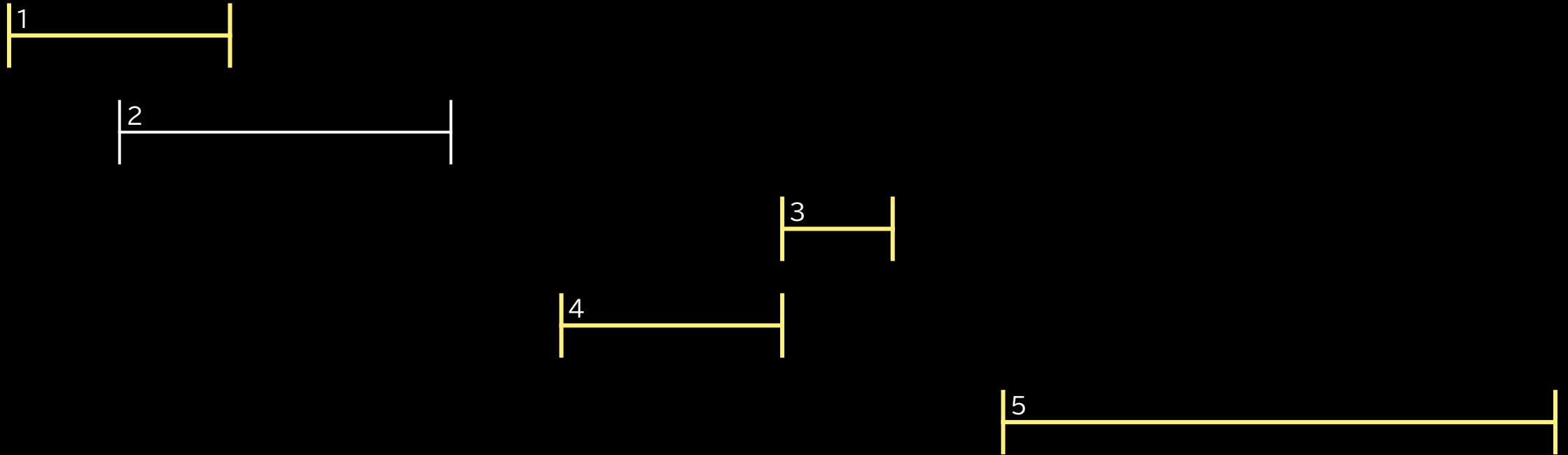
$$N = 5, K = 4$$



$$(a_1, \dots, a_K) = (2, 3, 4, 5)$$

入出力例 1

$N = 5, K = 4$



$(1,3,4,5) < (2,3,4,5)$ より答えは $(1,3,4,5)$

目次

小課題 1 $L_i \leq L_{i+1}$

9 / 14 人正解

小課題 2 $N \leq 20$

12 / 14 人正解

小課題 3 $N \leq 3\,000$

12 / 14 人正解

小課題 4 追加制約なし

5 / 14 人正解

統計情報

目次

小課題 1 $L_i \leq L_{i+1}$

9 / 14 人正解

小課題 2 $N \leq 20$

12 / 14 人正解

小課題 3 $N \leq 3\,000$

12 / 14 人正解

小課題 4 追加制約なし

5 / 14 人正解

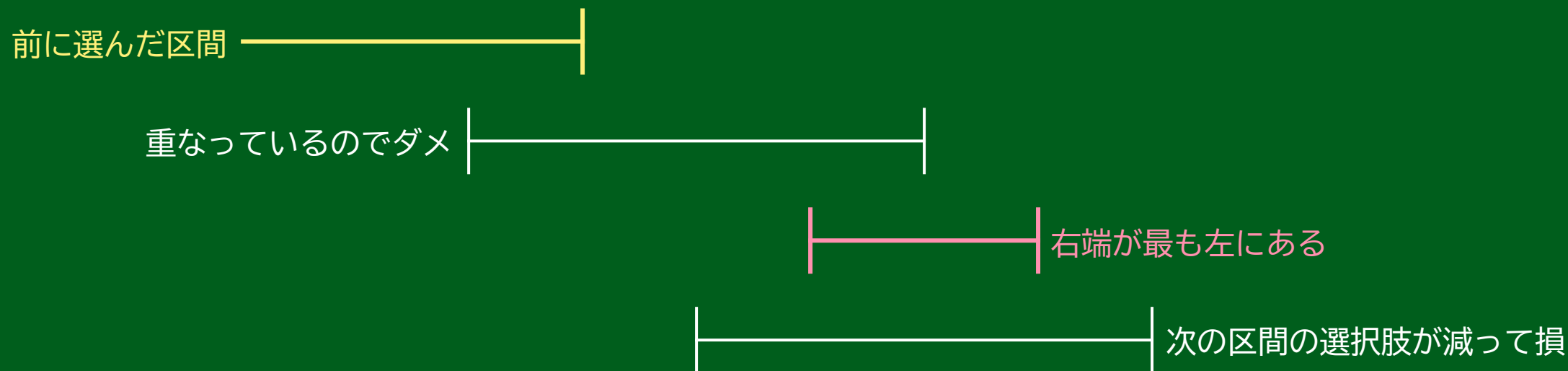
統計情報

小課題 1

$$L_i \leq L_{i+1} \quad (1 \leq i \leq N - 1)$$

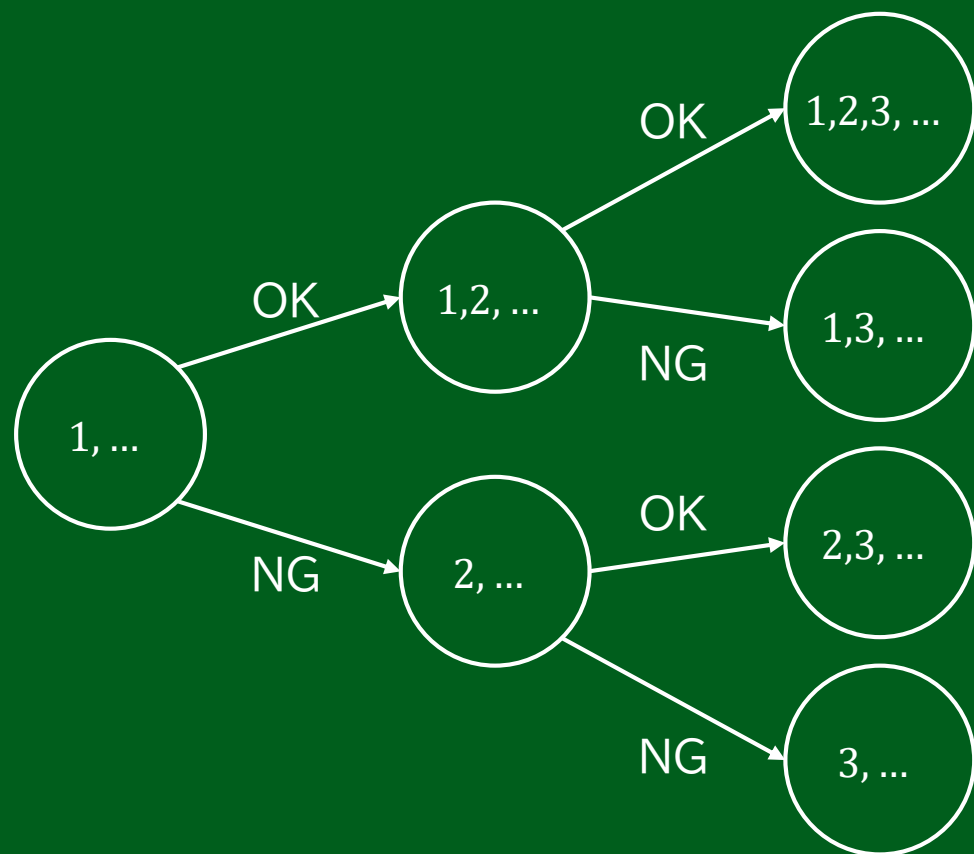
典型 重ならない区間の個数の最大化

- 貪欲法が使える
 - 右端でソートし，左から順に見て行く
 - 次に選べる区間のうち，右端が最も左の区間を選ぶ



典型 辞書順最小化

- 条件を満たす状態を維持しつつ選んでいく



2,1,3 みたいなのも考えないと
いけないと大変だけど今回は不要

1 を使ってみる

2 を使ってみる

3 を使ってみる

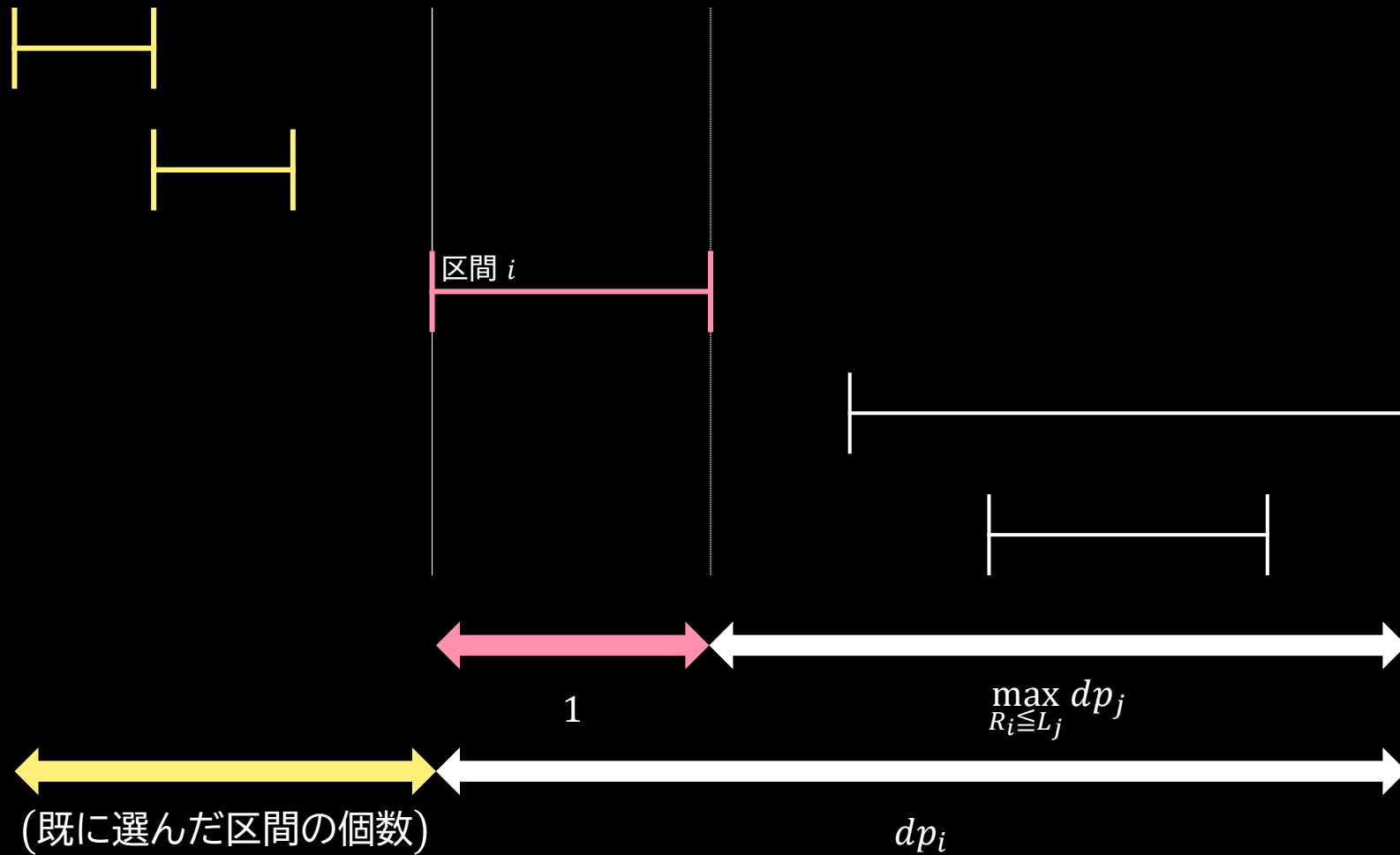
考察：方針

- 2 つの典型を組み合わせる
 - 左から順に選んでいく → 辞書順最小化
 - 条件を満たすか判定 → 重ならない区間の個数の最大化
 - (既に選んだ区間の個数) +
(今選んだ区間とそれより右で選べる個数の最大値) $\geq K$

考察：DP

- (今選んだ区間とそれより右で選べる個数の最大値) を DP で持っておくとよい
 - $dp_i := (\text{区間 } i \text{ とそれより右で選べる個数の最大値})$
 - $dp_i = 1 + \max_{R_i \leq L_j} dp_j$
 - $R_i \leq L_j$ な j は連続しているので、
累積 max + 二分探索などで dp が求められる
- $O(N \log N)$ 時間

図示



実装例

```
for(int i = N - 1; i >= 0; i--){
    int j = lower_bound(L, L + N, R[i]) - L;
    dp[i] = 1 + (j < N ? dpmax[j] : 0);
    dpmax[i] = max(dp[i], i + 1 < N ? dpmax[i + 1] : 0);
}

int now = 0;
for(int i = 0; i < N && now < K; i++){
    if(now + dp[i] >= K){
        cout << i + 1 << endl;
        now++;
        i = lower_bound(L, L+N, R[i]) - L - 1;
    }
}
```

目次

小課題 1 $L_i \leq L_{i+1}$

9 / 14 人正解

小課題 2 $N \leq 20$

12 / 14 人正解

小課題 3 $N \leq 3\,000$

12 / 14 人正解

小課題 4 追加制約なし

5 / 14 人正解

統計情報

目次

小課題 1 $L_i \leq L_{i+1}$

9 / 14 人正解

小課題 2 $N \leq 20$

12 / 14 人正解

小課題 3 $N \leq 3\,000$

12 / 14 人正解

小課題 4 追加制約なし

5 / 14 人正解

統計情報

小課題 2

$N \leq 20$

考察

- 区間の選び方 2^N 通りを bit 全探索で試して,
 - 選んだ区間が被っていない : $O(N^2)$ で全組チェック
 - 選んだ区間の数が K 個
- な選び方のうち辞書順最小のものを出力する
- 区間 i と下から $N - 1 - i$ 番目の bit を対応付けると辞書順最小の判定が楽
- $O(2^N N^2)$ で間に合う
 - 区間をソートしておいて判定パートを軽くすると $O(2^N N)$

実装例

```
for(int i = (1 << N) - 1; i >= 0; i--){
    if(__builtin_popcount(i) != K) continue;
    bool ok = true;
    for(int j = 0; j < N; j++){
        if(~(i >> N - 1 - j) & 1) continue;
        for(int k = 0; k < N; k++){
            if(~(i >> N - 1 - k) & 1 || j == k) continue;
            ok &= (R[j] <= L[k] || R[k] <= L[j]);
        }
    }
    if(ok){
        for(int j = 0; j < N; j++){
            if((i >> N - 1 - j) & 1) cout << j + 1 << endl;
        }
        return 0;
    }
}
cout << -1 << endl;
```

目次

小課題 1 $L_i \leq L_{i+1}$

9 / 14 人正解

小課題 2 $N \leq 20$

12 / 14 人正解

小課題 3 $N \leq 3\,000$

12 / 14 人正解

小課題 4 追加制約なし

5 / 14 人正解

統計情報

目次

小課題 1 $L_i \leq L_{i+1}$

9 / 14 人正解

小課題 2 $N \leq 20$

12 / 14 人正解

小課題 3 $N \leq 3\,000$

12 / 14 人正解

小課題 4 追加制約なし

5 / 14 人正解

統計情報

小課題 3

$N \leq 3\,000$

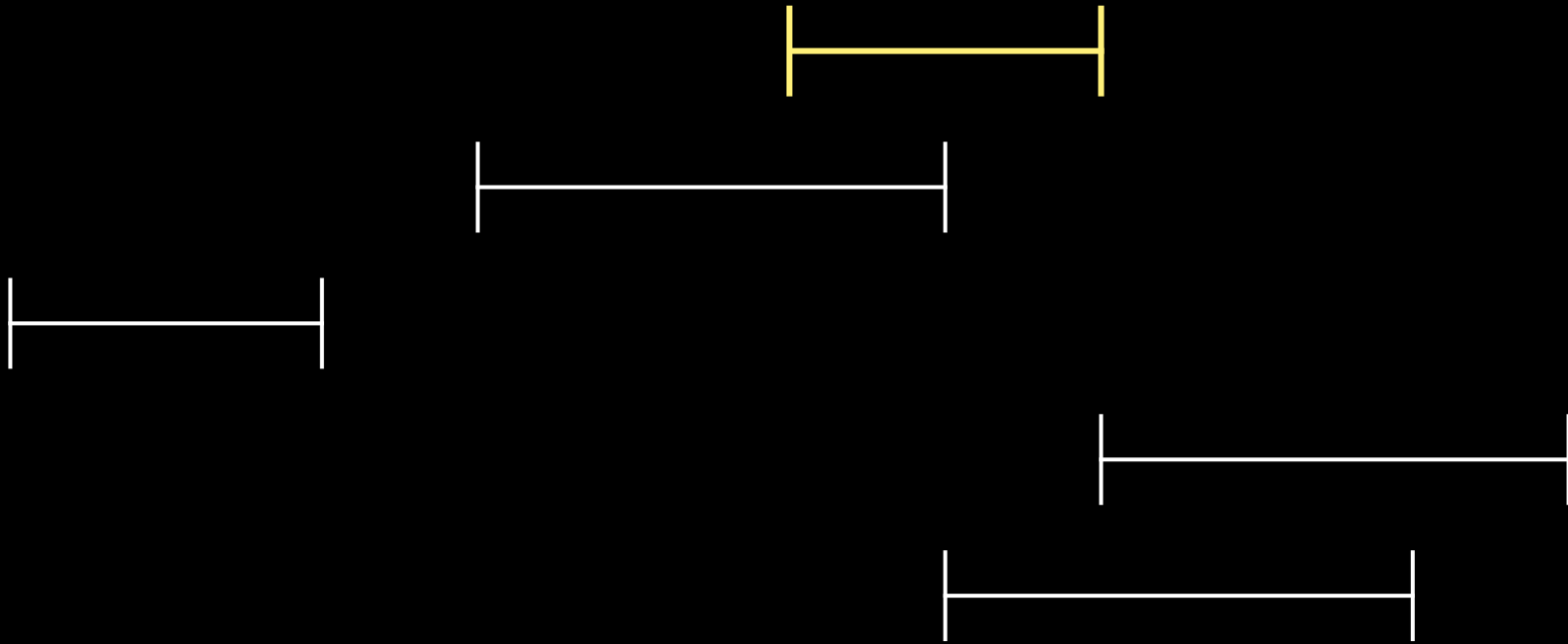
考察：方針

- 2つの典型を組み合わせる
 - インデックスが小さいものから順に選んでいく
→ 辞書順最小化
 - 条件を満たすか判定
→ 重ならない区間の個数の最大化
- どうやって条件を満たすか判定しようか？

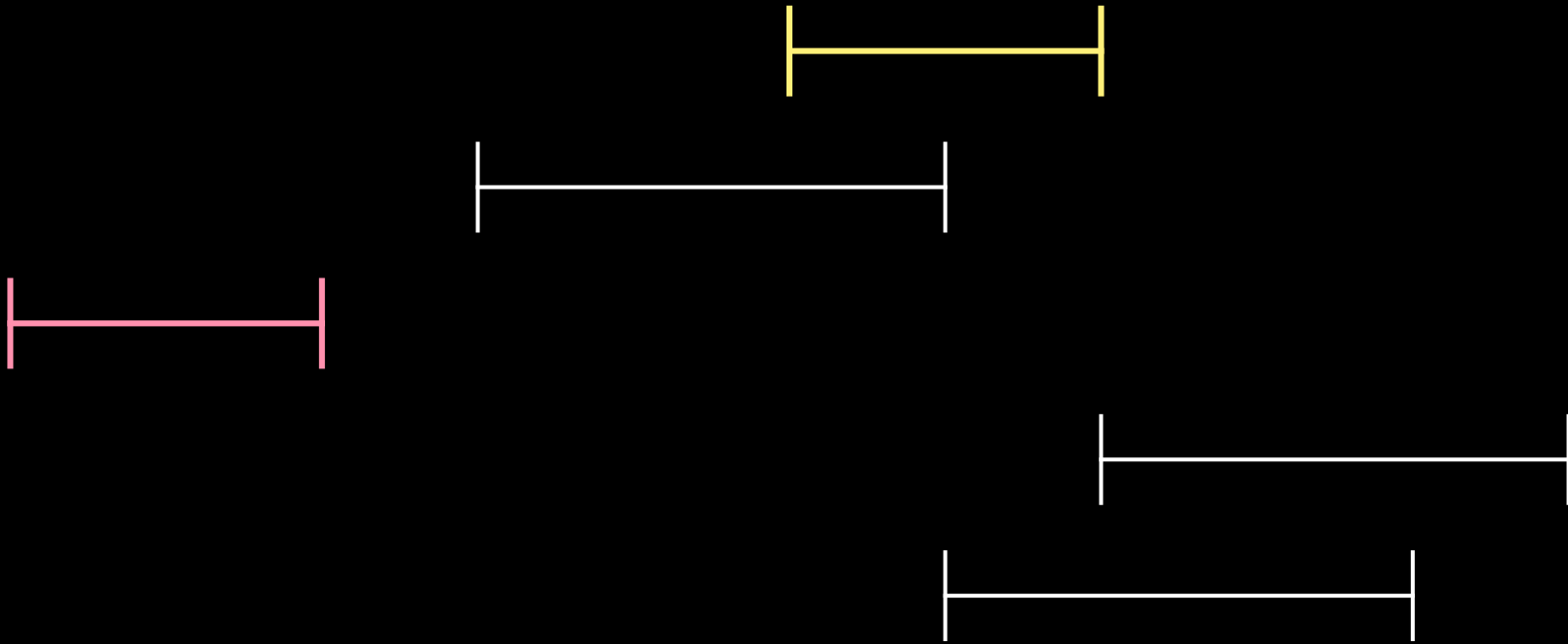
考察：判定方法

- 「重ならない区間の個数の最大化」を普通にやる
- ただし，選ばれている区間は必ず使うようにする

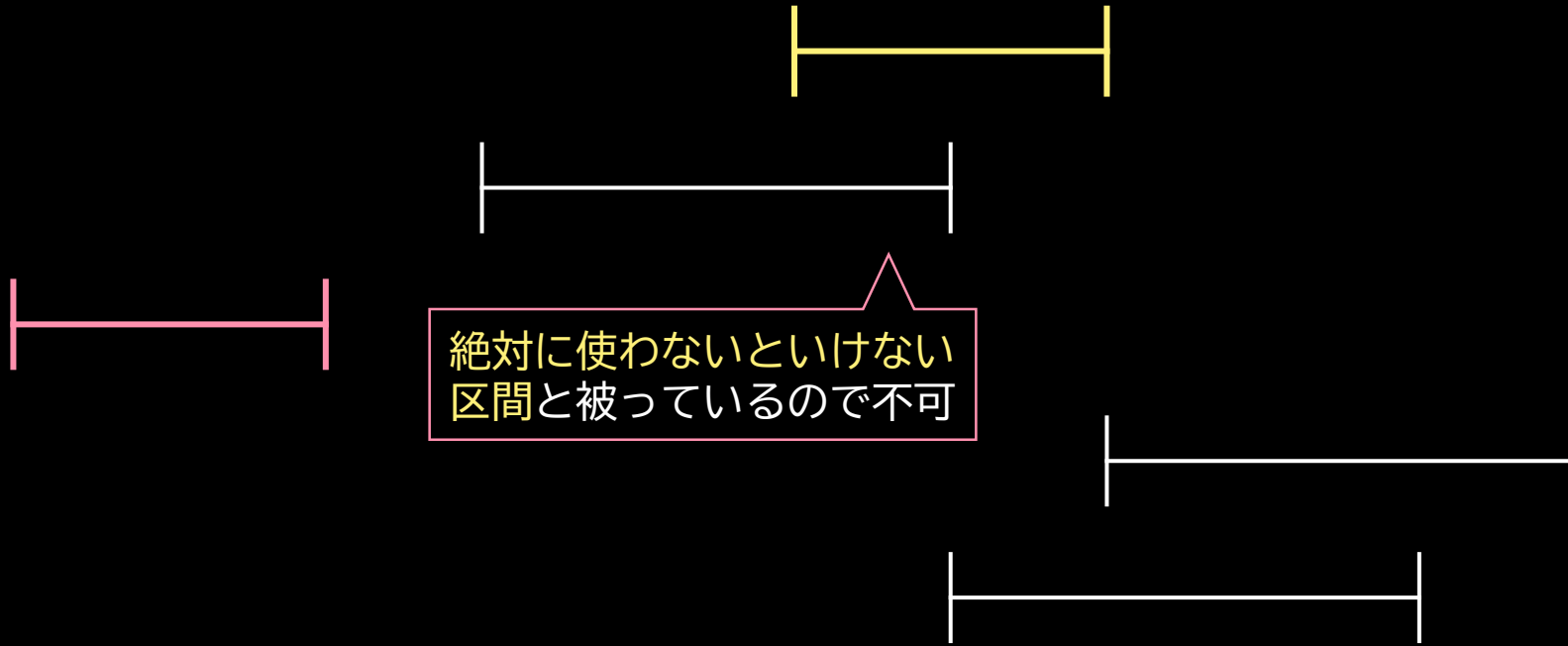
图示



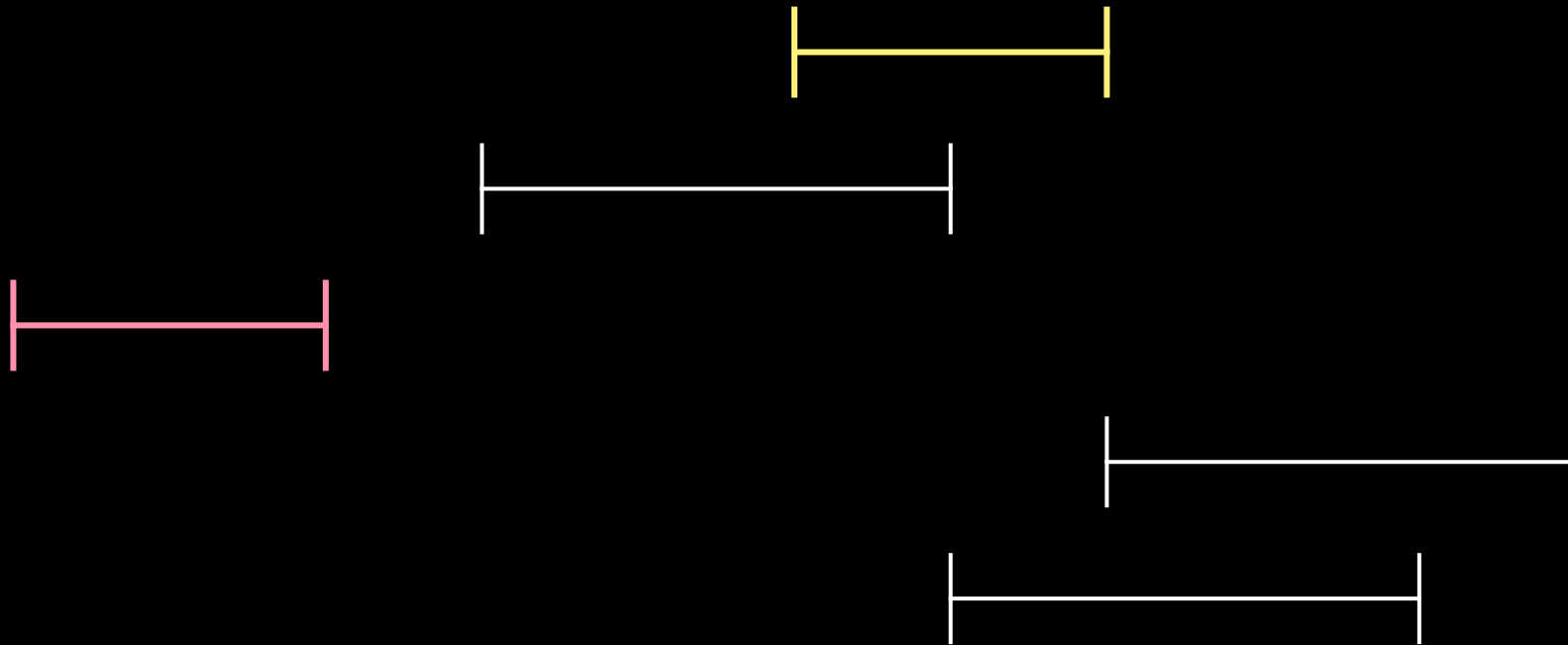
图示



図示

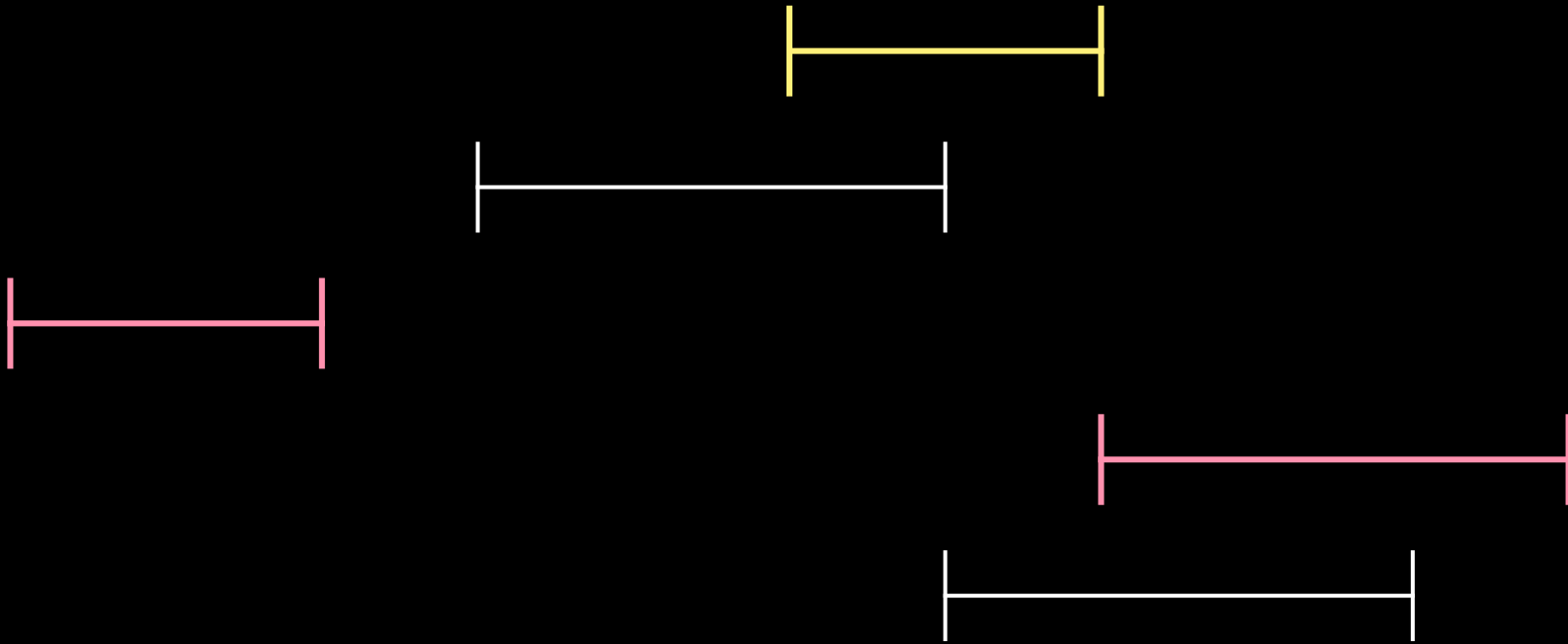


図示

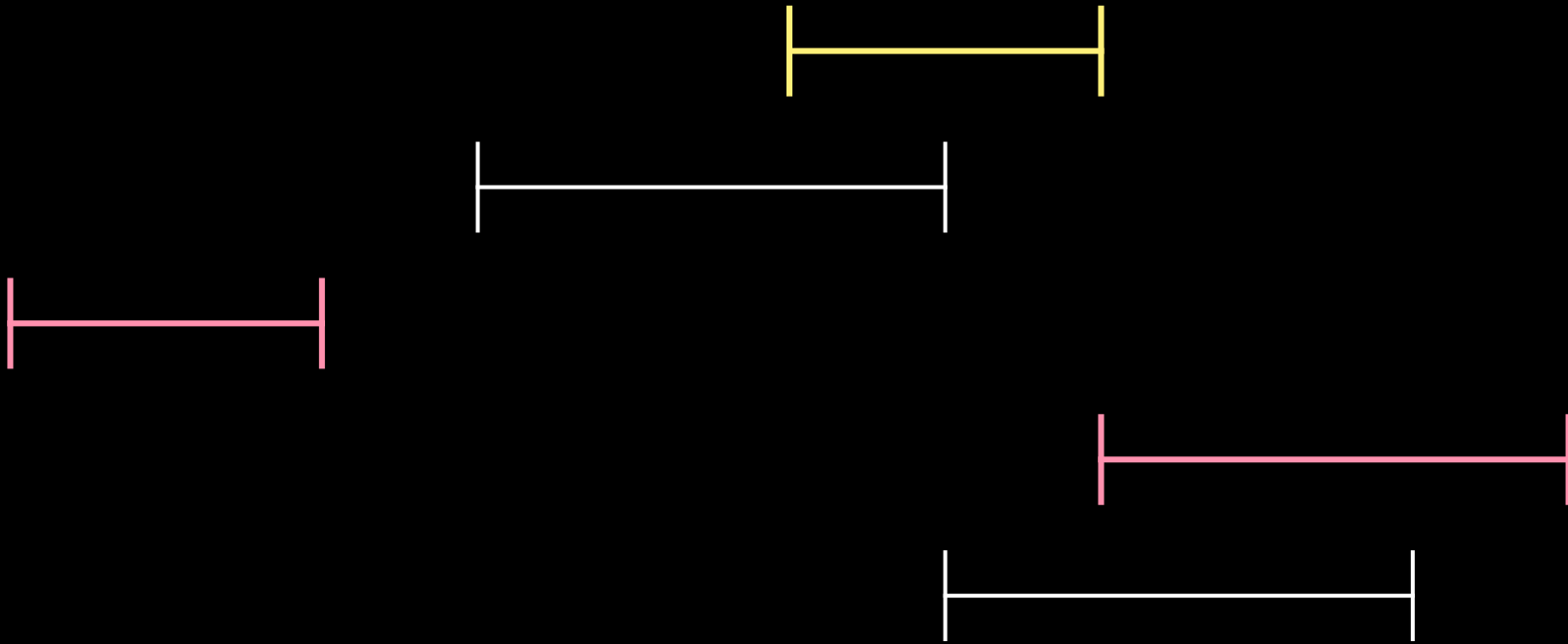


絶対に使わないといけない
区間と被っているので不可

图示



图示



$$3 \geq K ?$$

実装？

- 選択済み区間を `vector` に入れてごにょごにょ . . .

典型 座標圧縮

- 座標圧縮をすると実装が楽になることがある ← 今回
- 座標圧縮をしないと解けないこともある

実装

- 既に選んだ区間で使われてしまっている単位時間を 1 として累積和で管理する
- あとは $sum_r - sum_l > 0$ な区間を選ばないようにして、重ならない区間の個数の最大化をやるだけ
- 全体で $O(N^2)$ 時間
- 他にも実装方針多数
 - たぶんこれが最も細かいことを考えなくてよい

実装例

```
for(int i = 0; i < N; i++) P[i] = {R[i],i};
sort(P, P + N);
if(count() < K){
    cout << -1 << endl;
    return 0;
}
int ans = 0;
for(int i = 0; i < N && ans < K; i++){
    if(sum[R[i]] - sum[L[i]] > 0) continue;
    for(int j = L[i]; j < R[i]; j++) used[j]++;
    for(int j = 0; j < z.size(); j++) sum[j + 1] = sum[j] + used[j];
    if(ans + 1 + count() >= K){
        cout << i + 1 << endl;
        ans++;
    }
    else{
        for(int j = L[i]; j < R[i]; j++) used[j]--;
        for(int j = 0; j < z.size(); j++) sum[j + 1] = sum[j] + used[j];
    }
}
```

実装例

```
pair<int,int> P[MAX_N];
int used[2*MAX_N], sum[2*MAX_N+1];

int count(){
    int bef_r = 0, res = 0;
    for(int i = 0; i < N; i++){
        if(bef_r <= L[P[i].second] && sum[R[P[i].second]] - sum[L[P[i].second]] == 0){
            bef_r = R[P[i].second];
            res++;
        }
    }
    return res;
}
```

目次

小課題 1 $L_i \leq L_{i+1}$

9 / 14 人正解

小課題 2 $N \leq 20$

12 / 14 人正解

小課題 3 $N \leq 3\,000$

12 / 14 人 正解

小課題 4 追加制約なし

5 / 14 人 正解

統計情報

目次

小課題 1 $L_i \leq L_{i+1}$

9 / 14 人正解

小課題 2 $N \leq 20$

12 / 14 人正解

小課題 3 $N \leq 3\,000$

12 / 14 人正解

小課題 4 追加制約なし

5 / 14 人正解

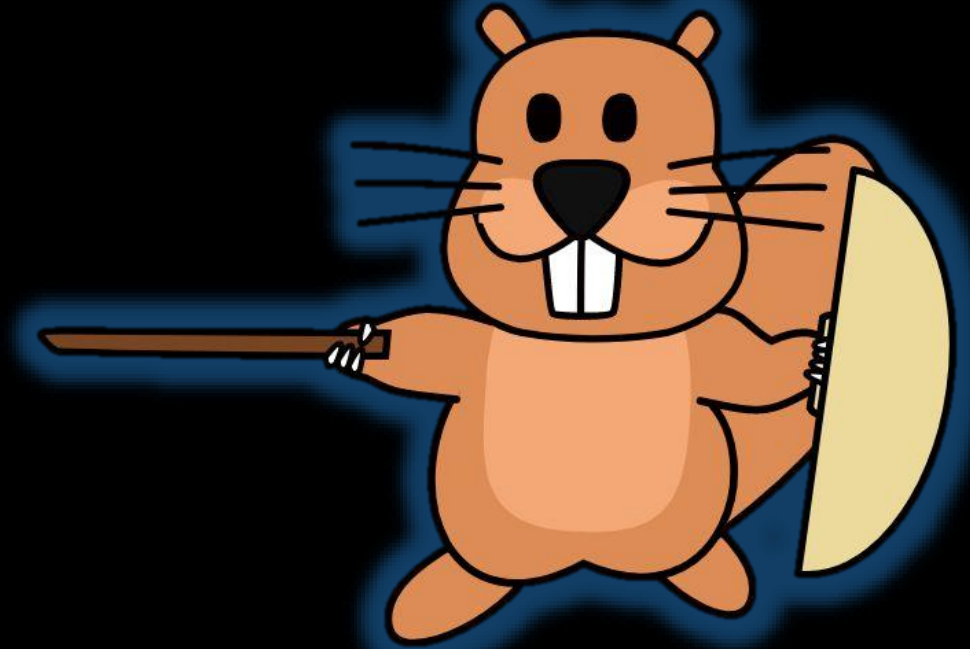
統計情報

小課題 4

追加制約なし

お詫び

- サルでもわかるように解説を書こうとしたら、ものすごく遠回りしてしまった気がします



お気持ち

- 計算回数を少しでも減らしたい . . .
- どこが削れるかな . . .

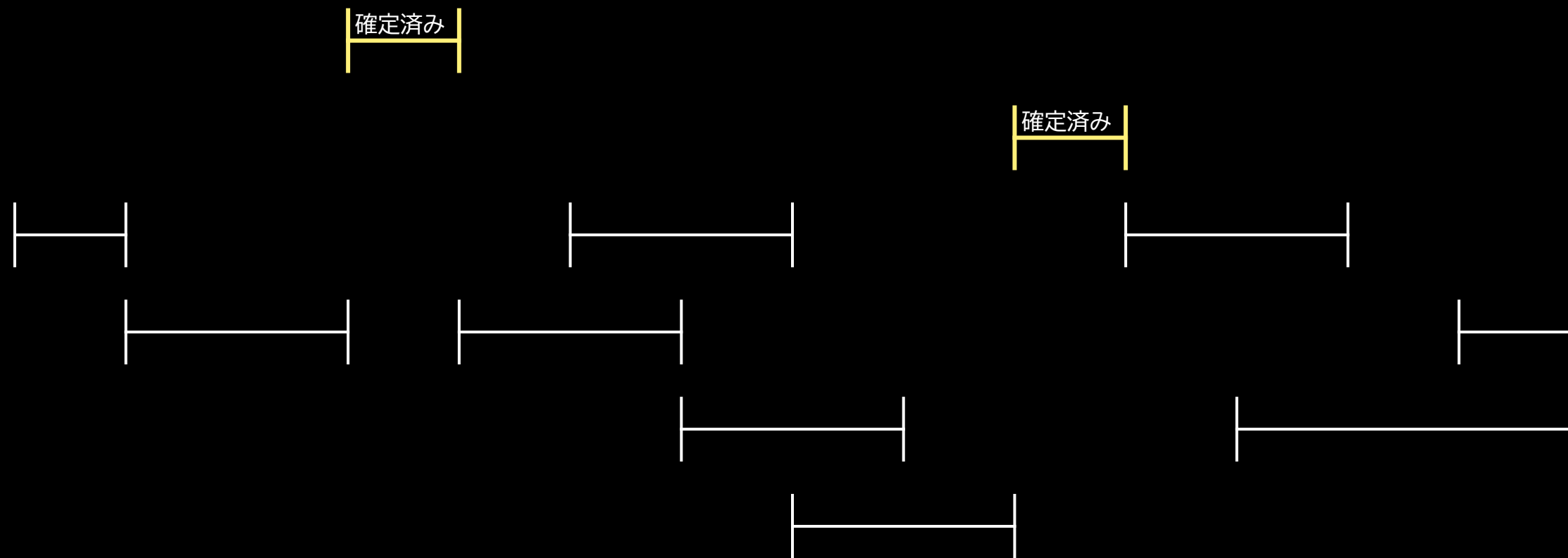
典型 差分

- 毎回全部計算しなおすのは大変
- 差分だけ計算しなおすことで、高速化できることがある

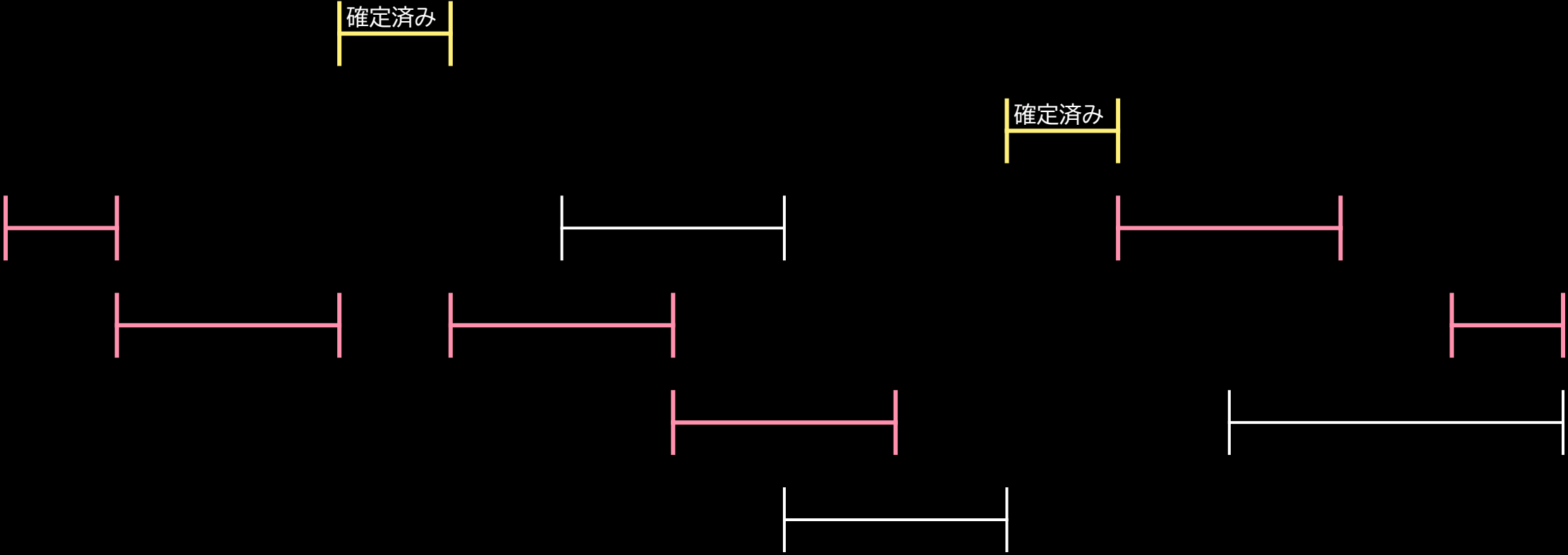
小課題3 考察：判定方法

- 「重ならない区間の個数の最大化」を普通にやる
- ただし，選ばれている区間は必ず使うようにする
- 差分をよくよく観察してみよう

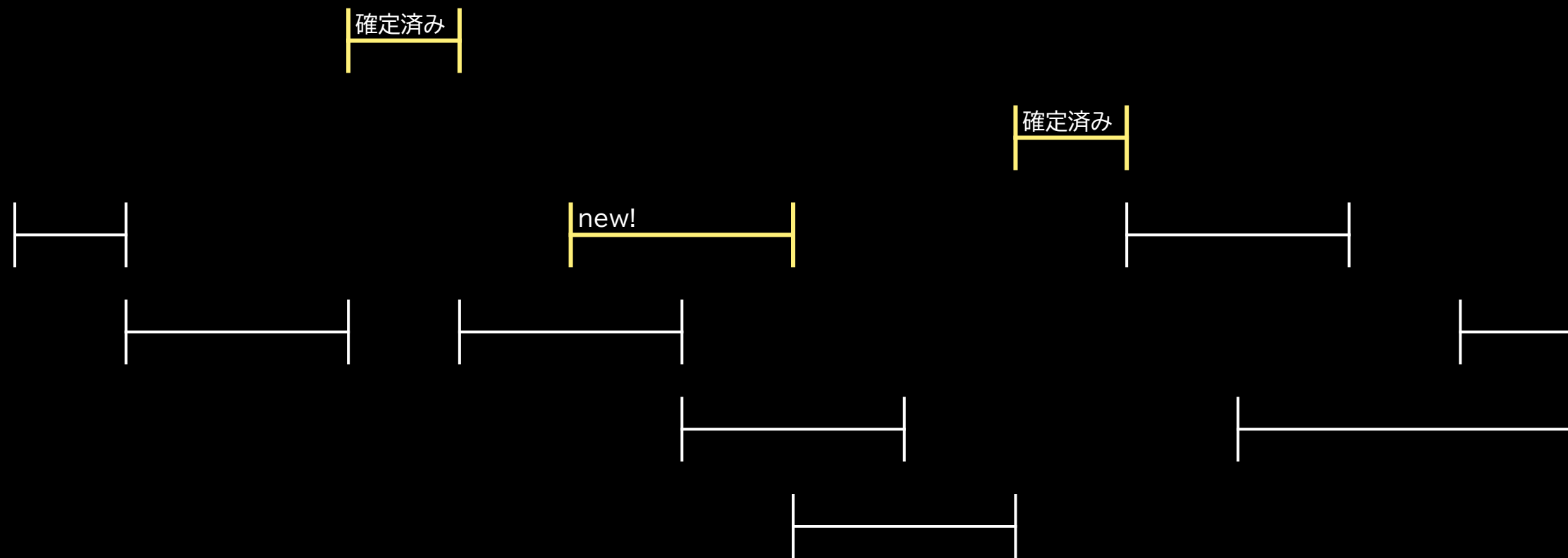
観察：例 1



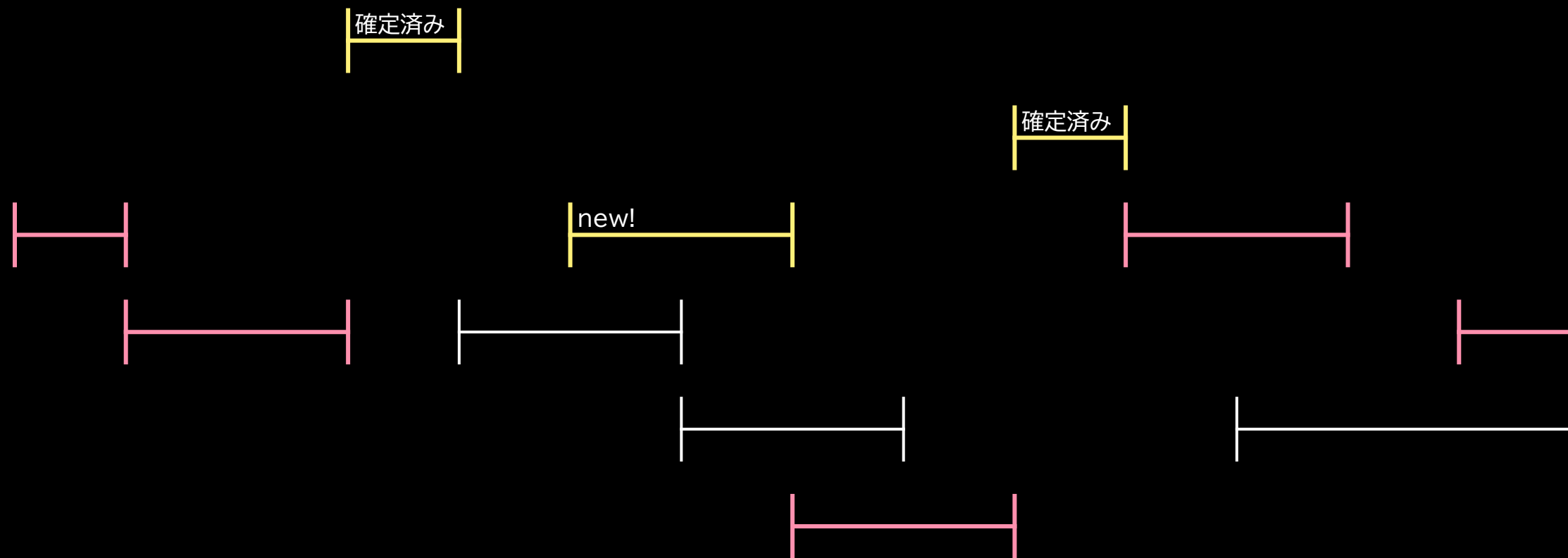
観察：例 1



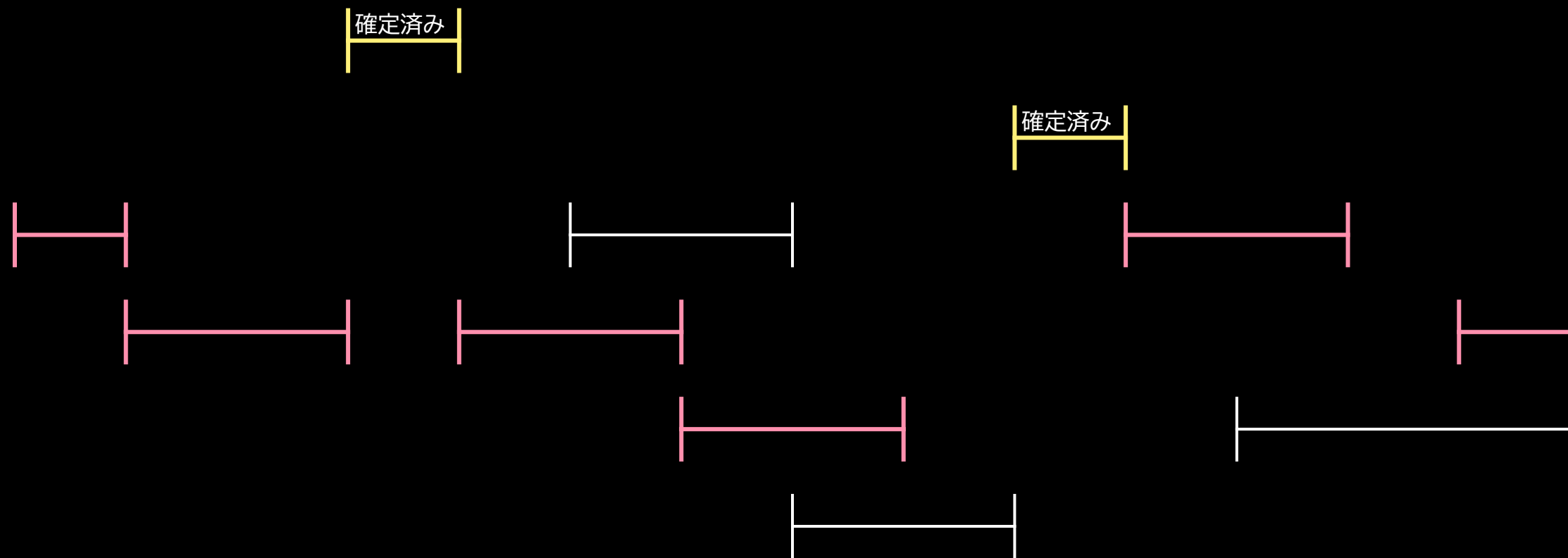
観察：例 2



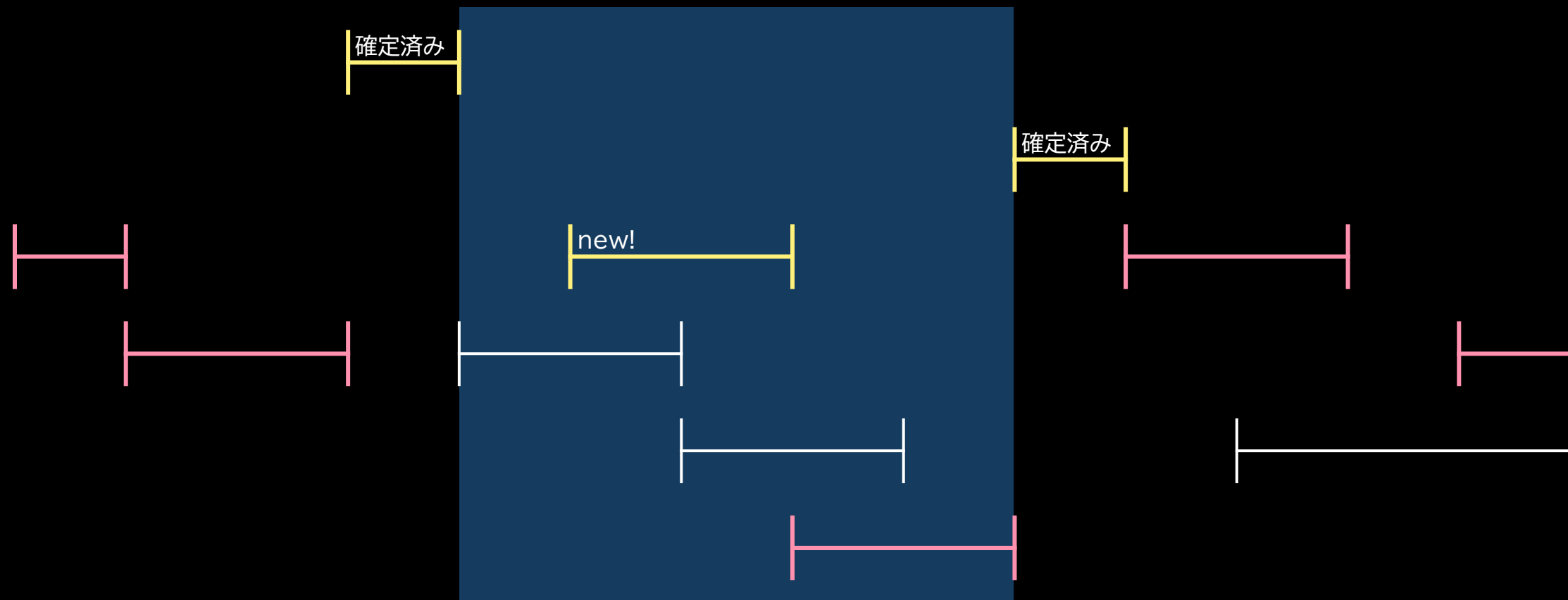
観察：例 2



観察：例 1



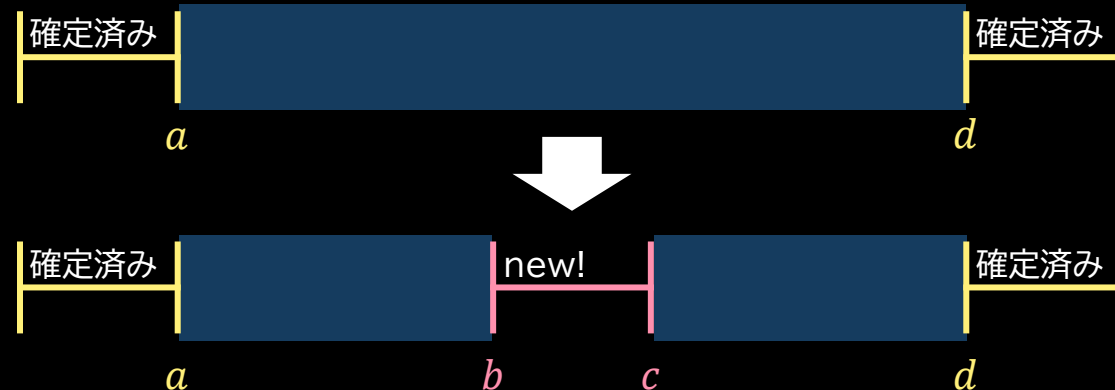
観察：例 2



この範囲しか選び方が変わっていない！

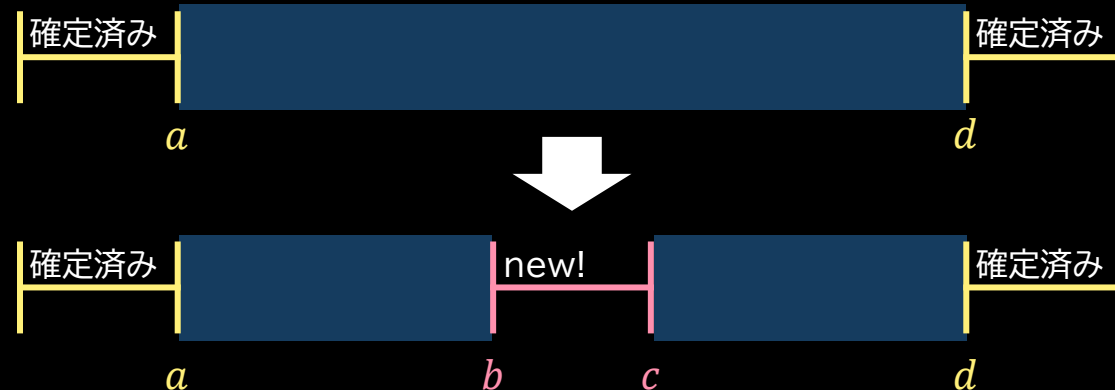
考察：判定方法

- 今まで
 - (既に選んだ個数) + (それを避けたときに選べる最大値) $\geq K$
- 差分に着目した別の表現
 - (前回の個数) - (前回範囲 (a, d) で選んでいた個数)
+ (範囲 (a, b) で選べる最大値) + 1 + (範囲 (c, d) で選べる最大値)
 $\geq K$



考察：判定方法

- 今まで
 - (既に選んだ個数) + (それを避けたときに選べる最大値) $\geq K$
- 差分に着目した別の表現
 - (前回の個数) - (範囲 (a, d) で選べる最大値)
+ (範囲 (a, b) で選べる最大値) + 1 + (範囲 (c, d) で選べる最大値)
 $\geq K$



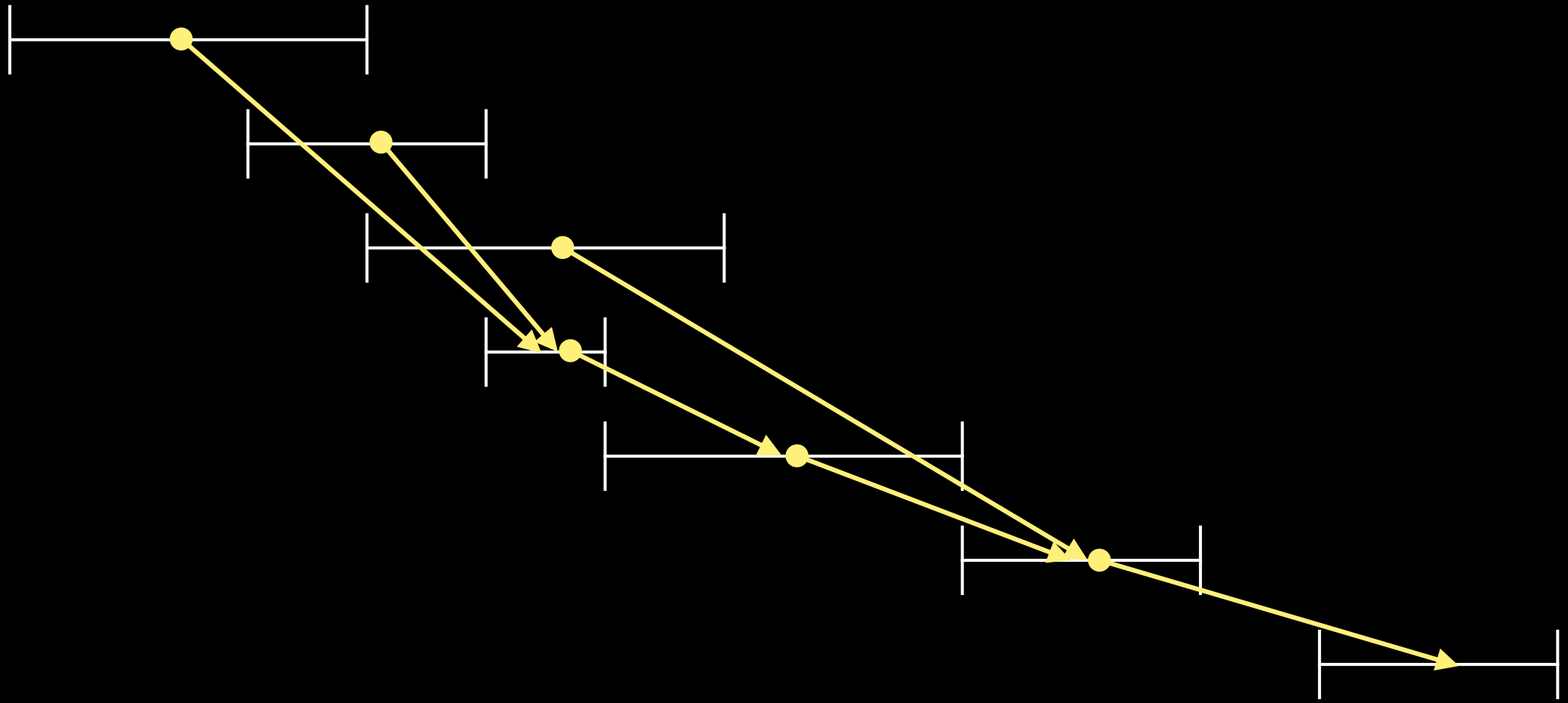
考察：範囲 (l, r) で選べる最大値

- 「範囲 (l, r) で選べる最大値」を高速に求められればよいことがわかった
- 「重ならない区間の個数の最大化」を普通にやっても遅い
- どうしよう・・・

典型 矢印

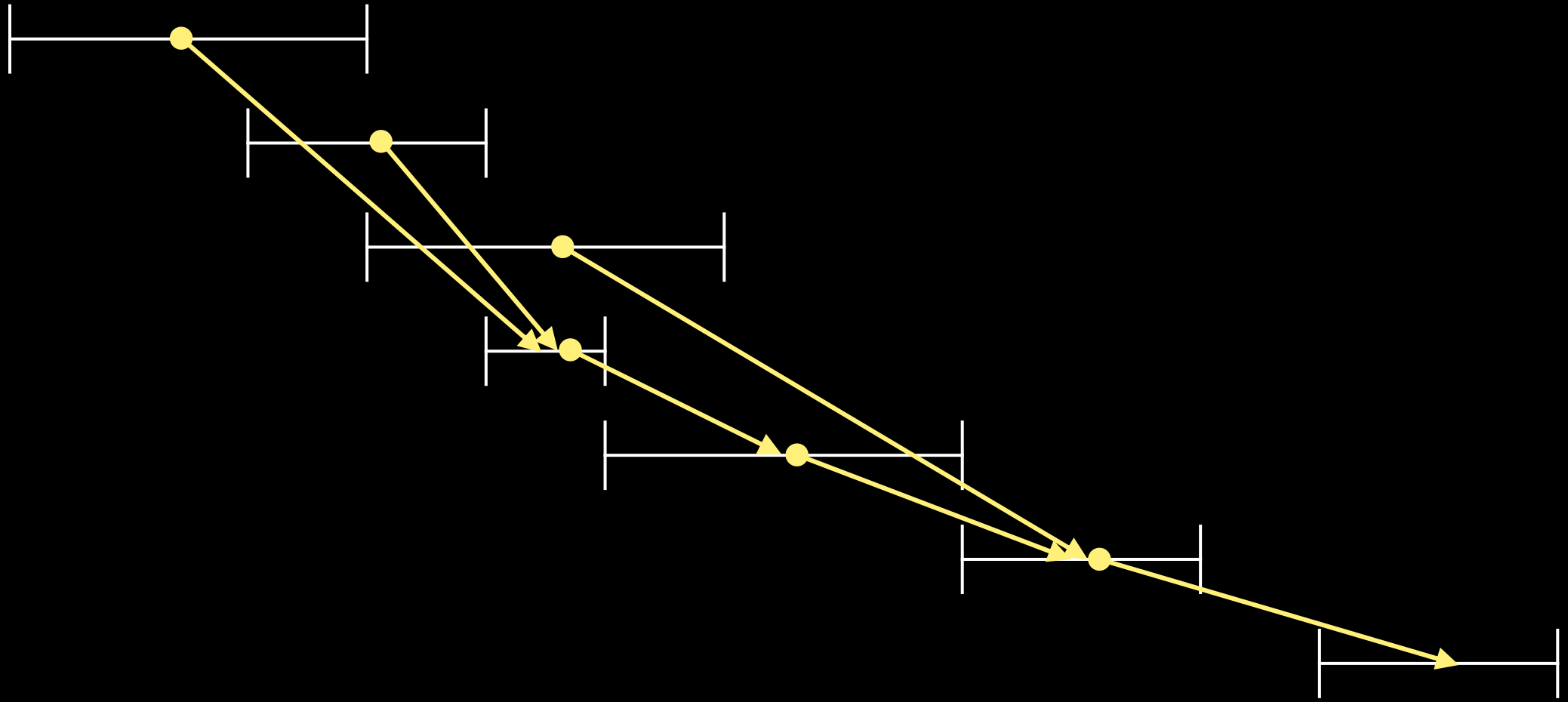
- 「前」「次」みたいな概念があるときは
図に矢印を引いてみることで、考察が捗ることがある

観察



自分より右側にある区間のうち、右端が最も左のものに矢印を引いてみた

観察



根つき木 (森) になっている！

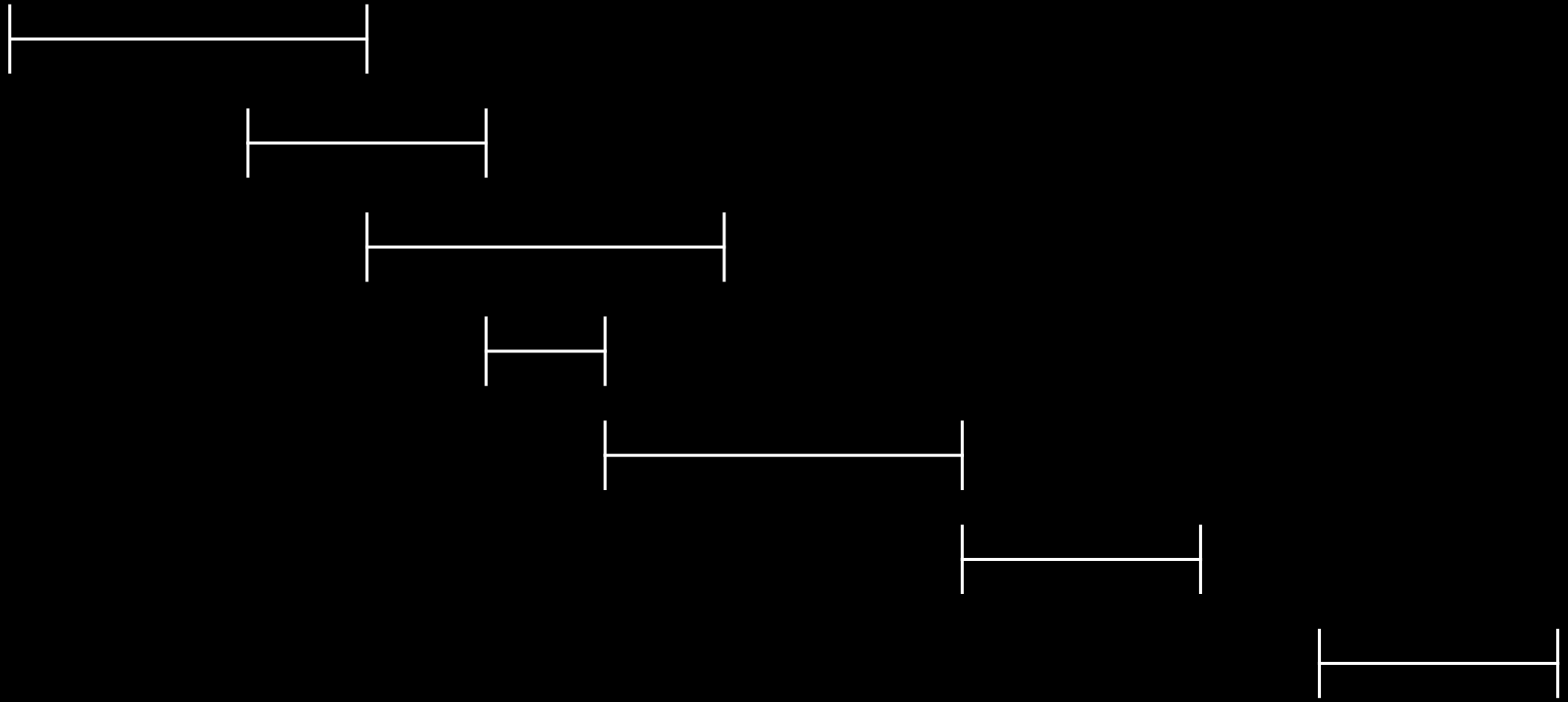
考察：方針

- 「重ならない区間の個数の最大化」はこの木上の最長パスを求めることと対応している
 - R_i が最小の頂点が深さ最大
- 「範囲 (l, r) で選べる最大値」も同じで,
 - $l \leq L_i$ な i のうち R_i が最小の頂点を見つけて
 - $R_j \leq r$ な間, 木を登り続ければよい

考察： $l \leq L_i$ な i のうち R_i が最小の頂点

- 木を作るのと一緒に前計算しておく
 - 座圧しておく
 - 右から平面(?) 走査する
 - 今見ているところより左にある区間のうち、右端が最も小さいものをメモする
 - 区間の左端をくぐったら、右端が最も小さいものを更新する
- 前計算 $O(N \log N)$

図示

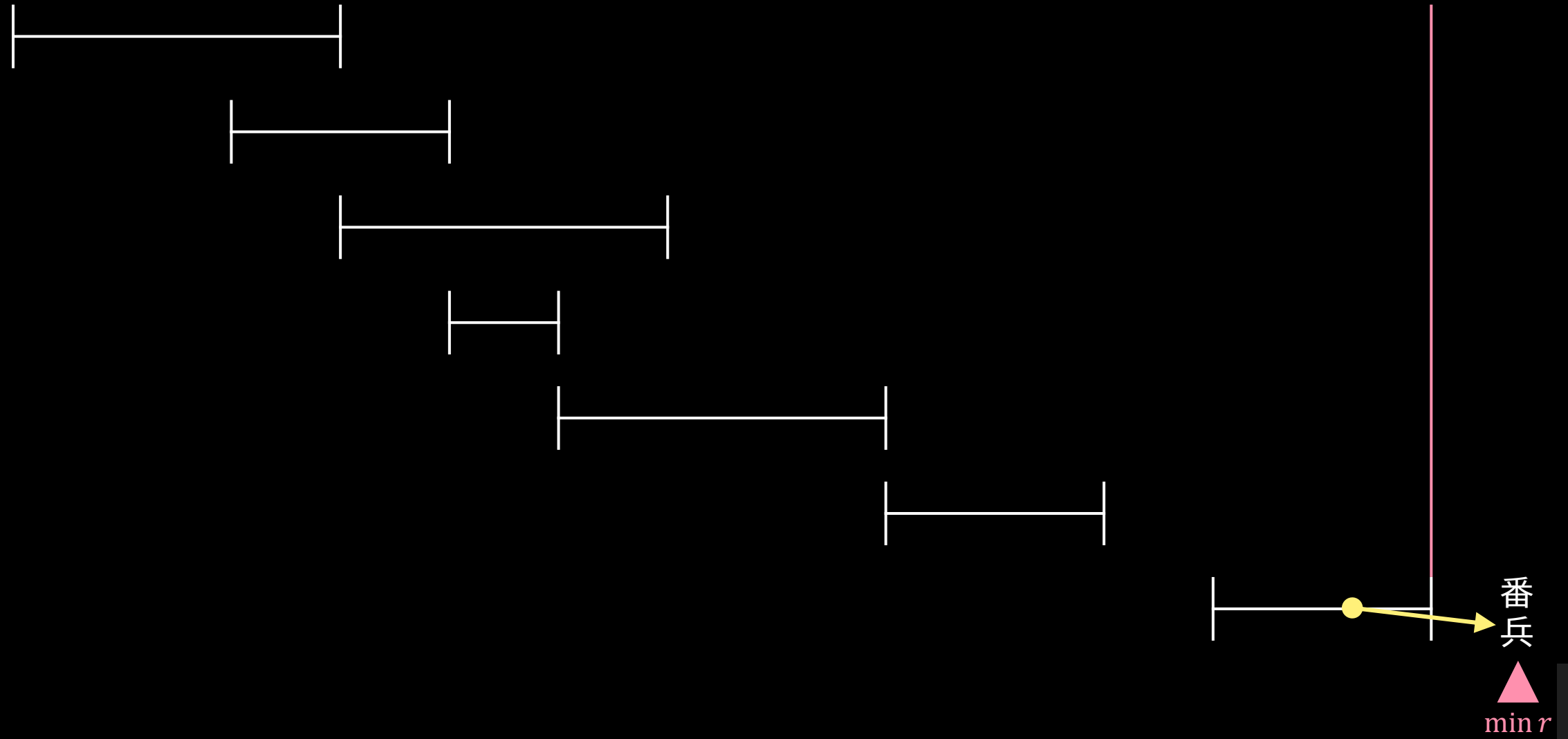


番兵

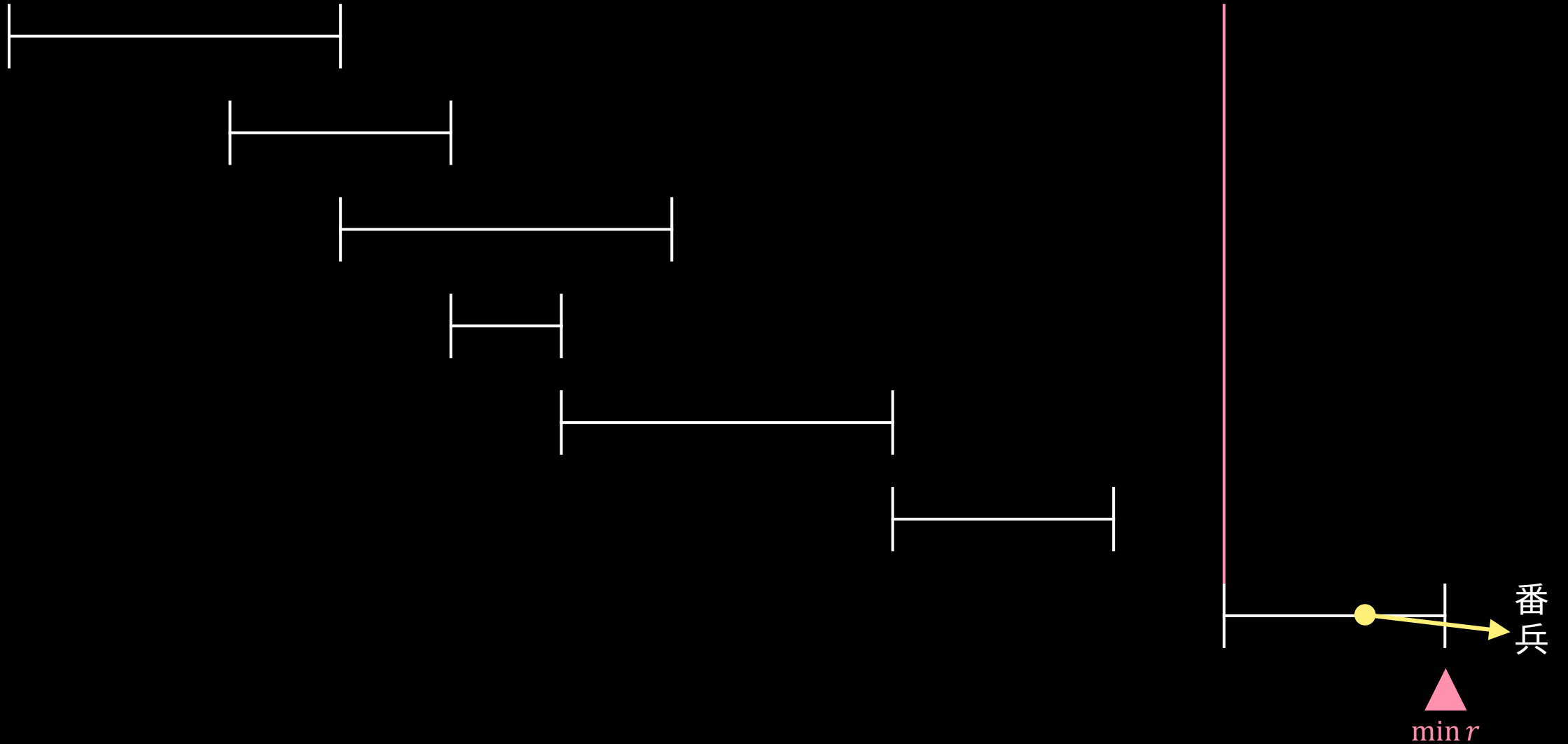


$\min r$

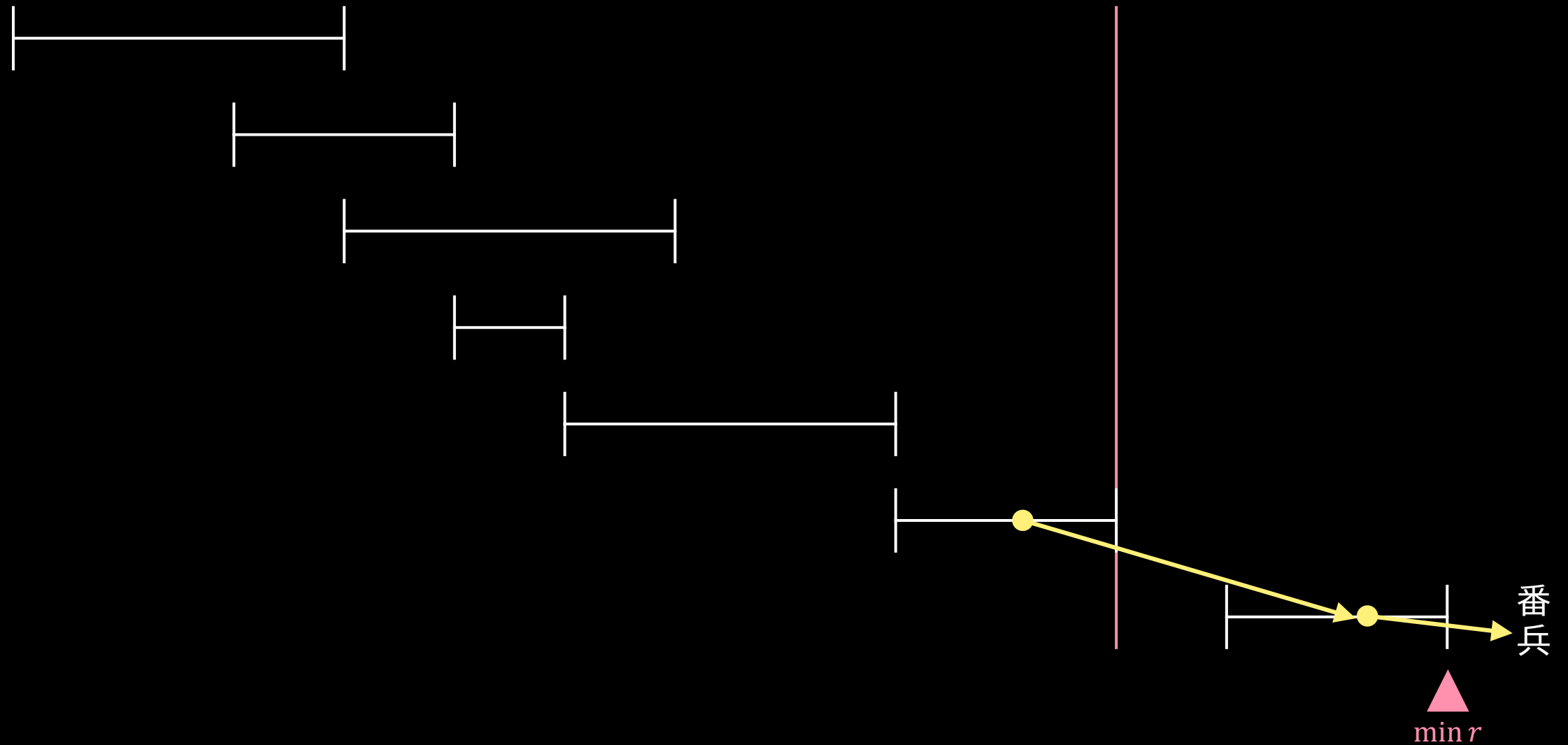
図示



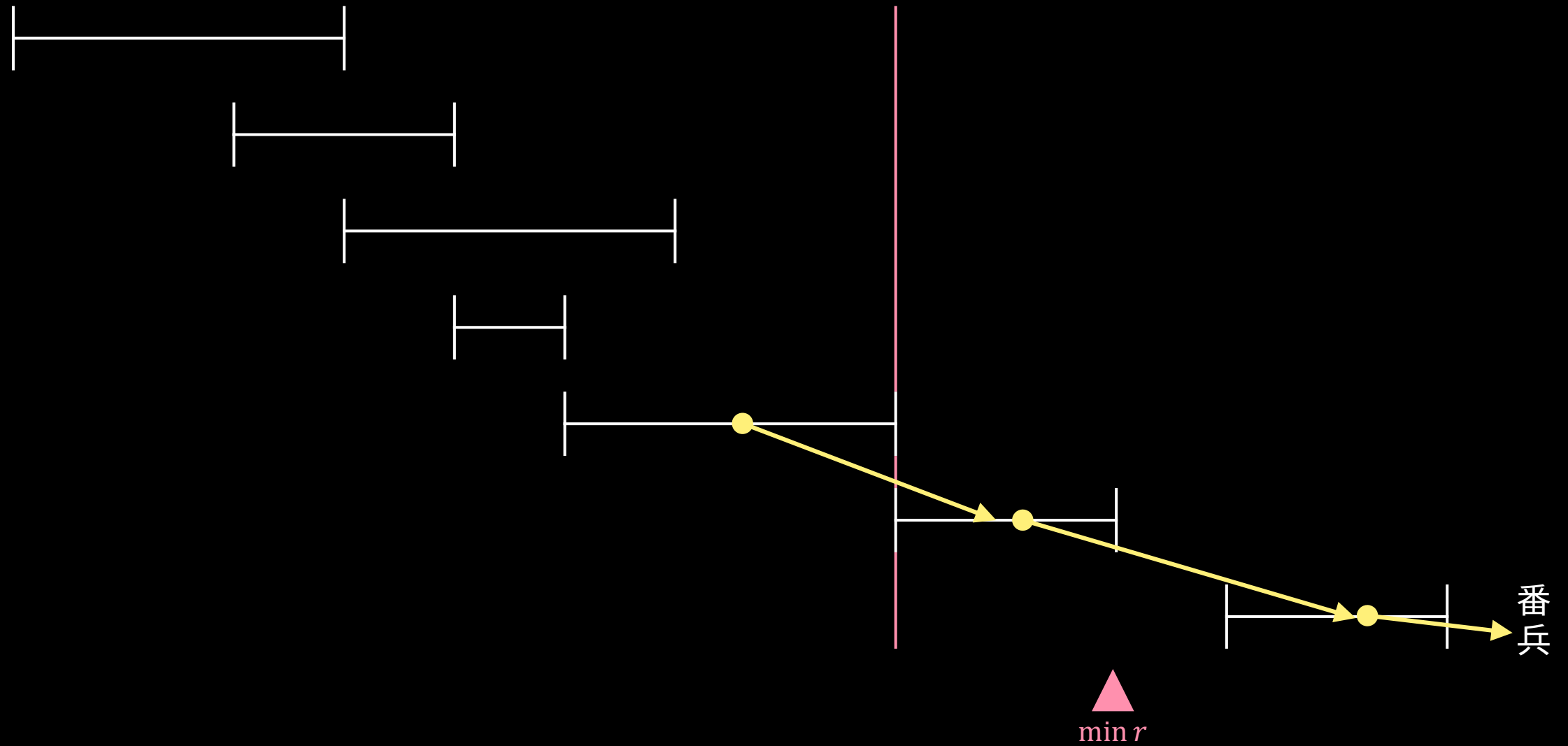
图示



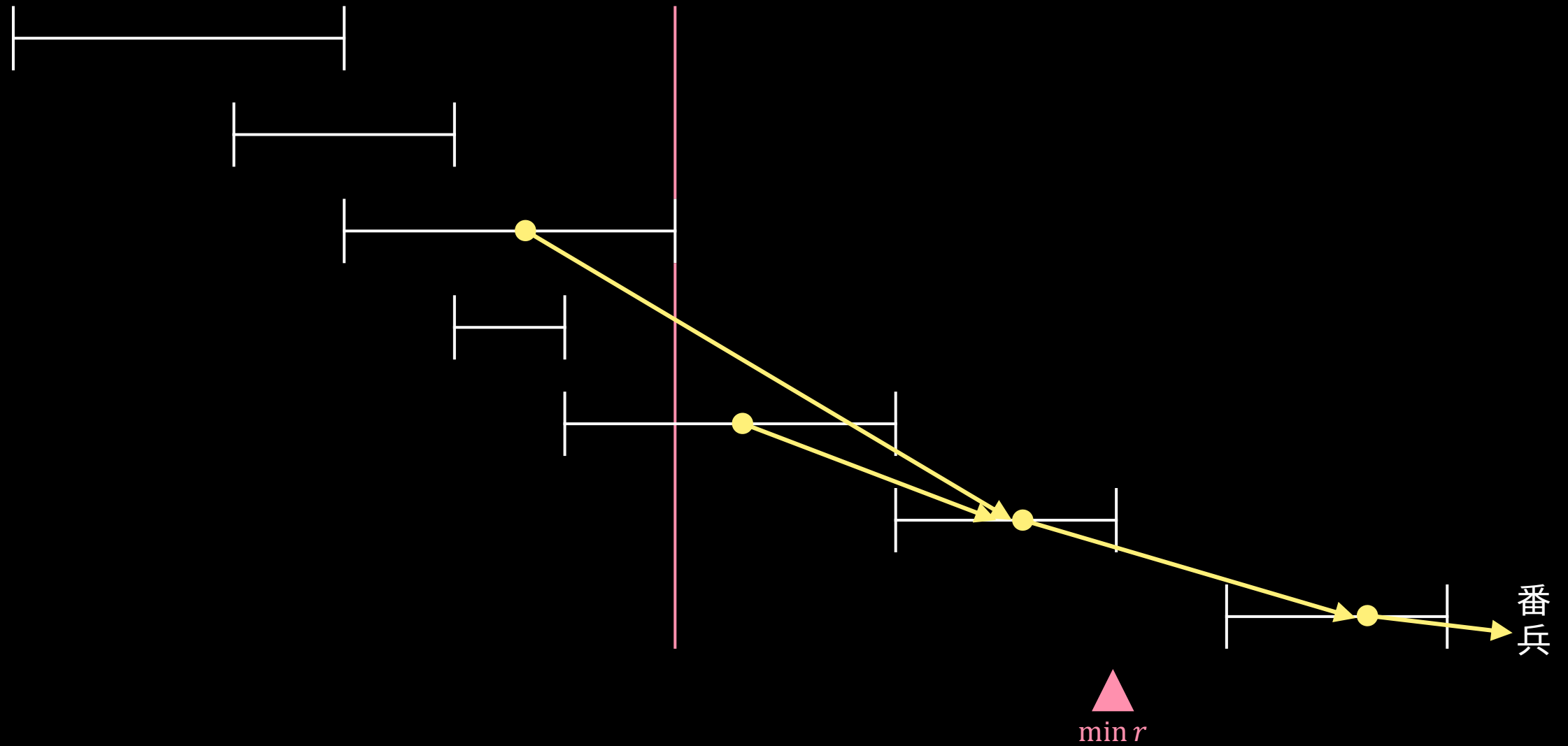
图示



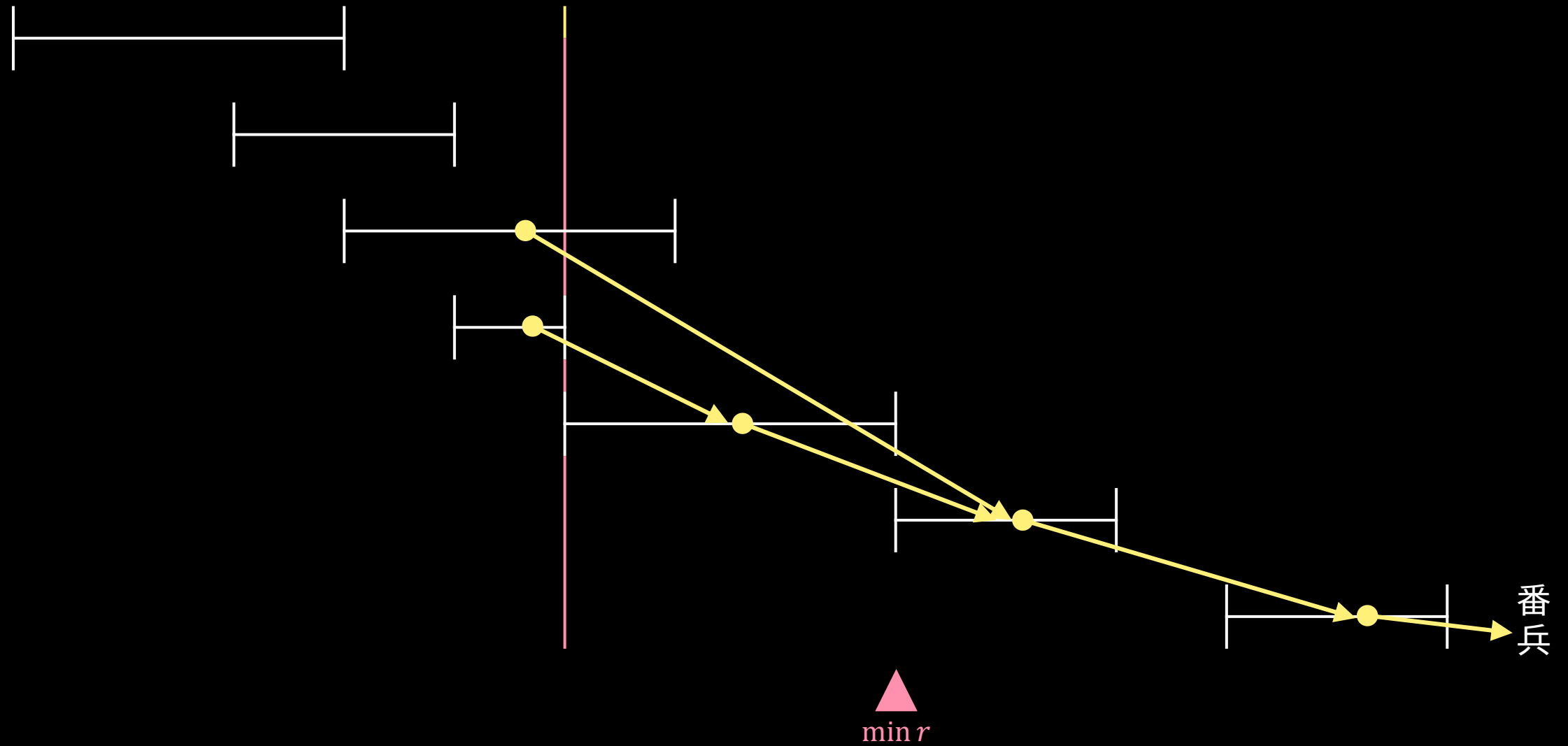
图示



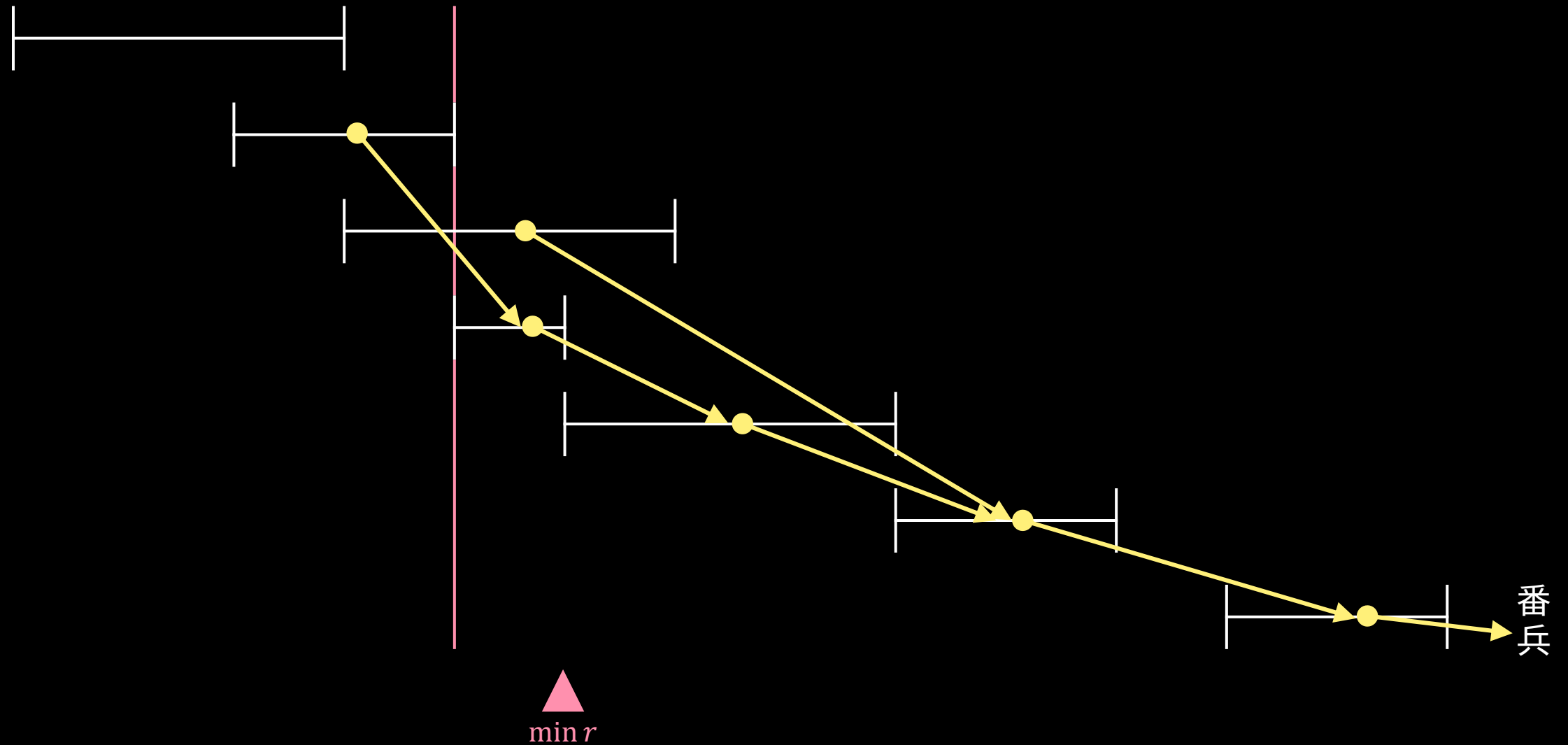
図示



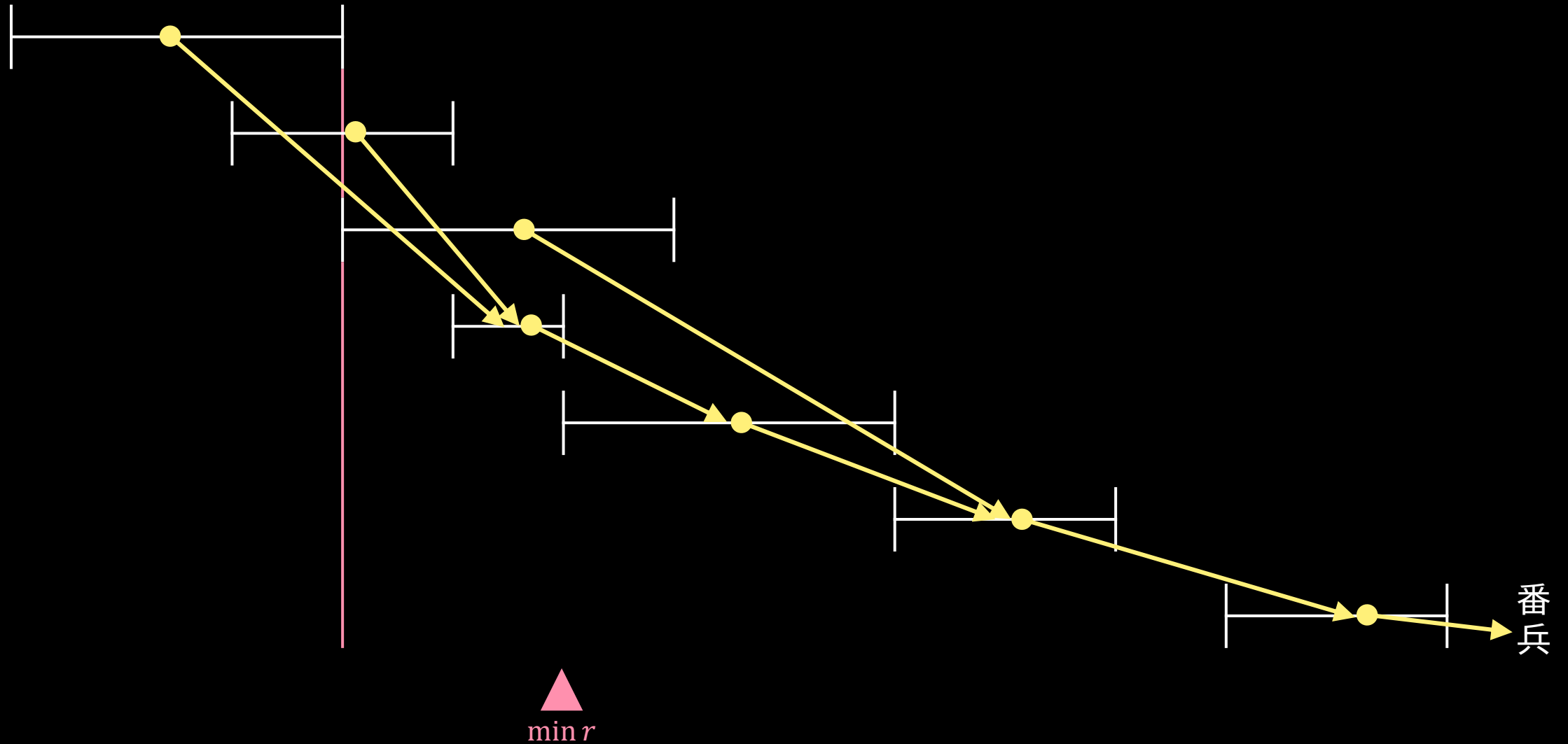
図示



図示



図示



考察： $R_j \leq r$ な間，木を登り続ける

- 1 つずつ登っていったでは毎回 $O(N)$ かかってしまう
- どうしよう . . .

典型 ダブリング

- 特定の条件を満たすギリギリのところまで登るには,
ダブリングが使える
- 発想は二分探索に近い

考察： $R_j \leq r$ な間，木を登り続ける

- 「 2^i 祖先」を前計算しておく
- ダブリングを使って $O(\log N)$ ステップで登る
 - 2^{18} 祖先に登っても大丈夫？
 - 2^{17} 祖先に登っても大丈夫？
 - 2^{16} 祖先に登っても大丈夫？
 - $\vdots$

まとめ

- 座標圧縮
- (明示的でなくてよいので)
木を作ってダブリングを前計算しておく： $O(N \log N)$
- 辞書順最小化の典型通り，条件を満たす状態を維持しながら 1 つずつ決めていく： $O(N \log N)$
 - 条件を満たすかの判定には，差分をダブリングで $O(\log N)$ で求められることを用いる
- 全体で $O(N \log N)$ 時間

実装例（木もどきの構築とダブリング）

```
vector<int> invL[MAX_N * 2 + 1];
int doubling[MAX_N * 2 + 1][MAX_logN];

for (int i = 0; i < N; i++) invL[L[i]].push_back(i);
int r = z.size();
for (int i = z.size(); i >= 0; i--) {
    for (int j = 0; j < invL[i].size(); j++) r = min(r, R[invL[i][j]]);
    doubling[i][0] = r;
}
for (int i = 1; i < MAX_logN; i++) {
    for (int j = 0; j <= z.size(); j++) {
        doubling[j][i] = doubling[doubling[j][i - 1]][i - 1];
    }
}
```

実装例 (範囲 (l, r) で選べる最大値)

```
int count(int l, int r) {
    int res = 0;
    for (int j = MAX_logN - 1; j >= 0; j--) {
        if (doubling[l][j] <= r) {
            l = doubling[l][j];
            res += 1 << j;
        }
    }
    return res;
}
```

実装例 (辞書順最小化)

```
int now = count(0, z.size() - 1);
if (now < K) {
    printf("-1¥n");
    return 0;
}
set<pair<int, int>> used = {{0, 0}, {z.size() - 1, z.size() - 1}};
for (int i = 0; used.size() - 2 < K; i++) {
    int l = prev(used.lower_bound({R[i], 0}))->second;
    int r = used.lower_bound({R[i], 0})->first;
    if (L[i] < l) continue;
    int tmp = now - count(l, r) + count(l, L[i]) + 1 + count(R[i], r);
    if (tmp >= K) {
        printf("%d¥n", i + 1);
        now = tmp;
        used.insert({L[i], R[i]});
    }
}
```

統計情報

得点分布



小課題ごとの正答者数

小課題 1 $L_i \leq L_{i+1}$

9 / 14 人正解

小課題 2 $N \leq 20$

12 / 14 人正解

小課題 3 $N \leq 3\,000$

12 / 14 人正解

小課題 4 追加制約なし

5 / 14 人正解

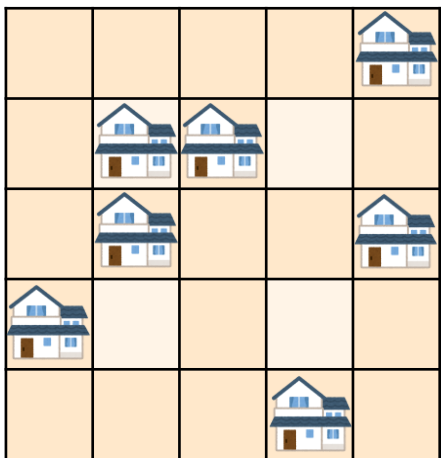


Navigation 2 Editorial

Editorial: E869120 (Masataka Yoneda)

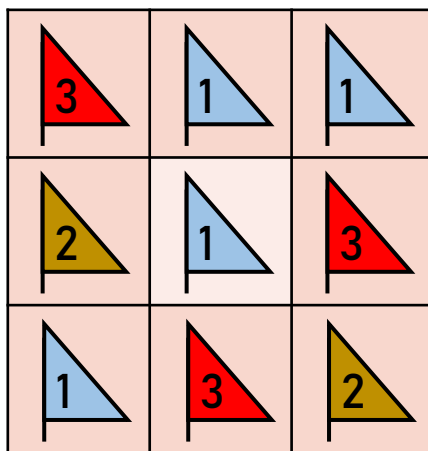
1 問題概要

1



$N \times N$ のマスに
 K 個の家がある

2



Anna は旗に
整数を書き込める

3

マス (a, b) から
Bruno が最短距離で
動くような行動を
教えてください

1

問題概要

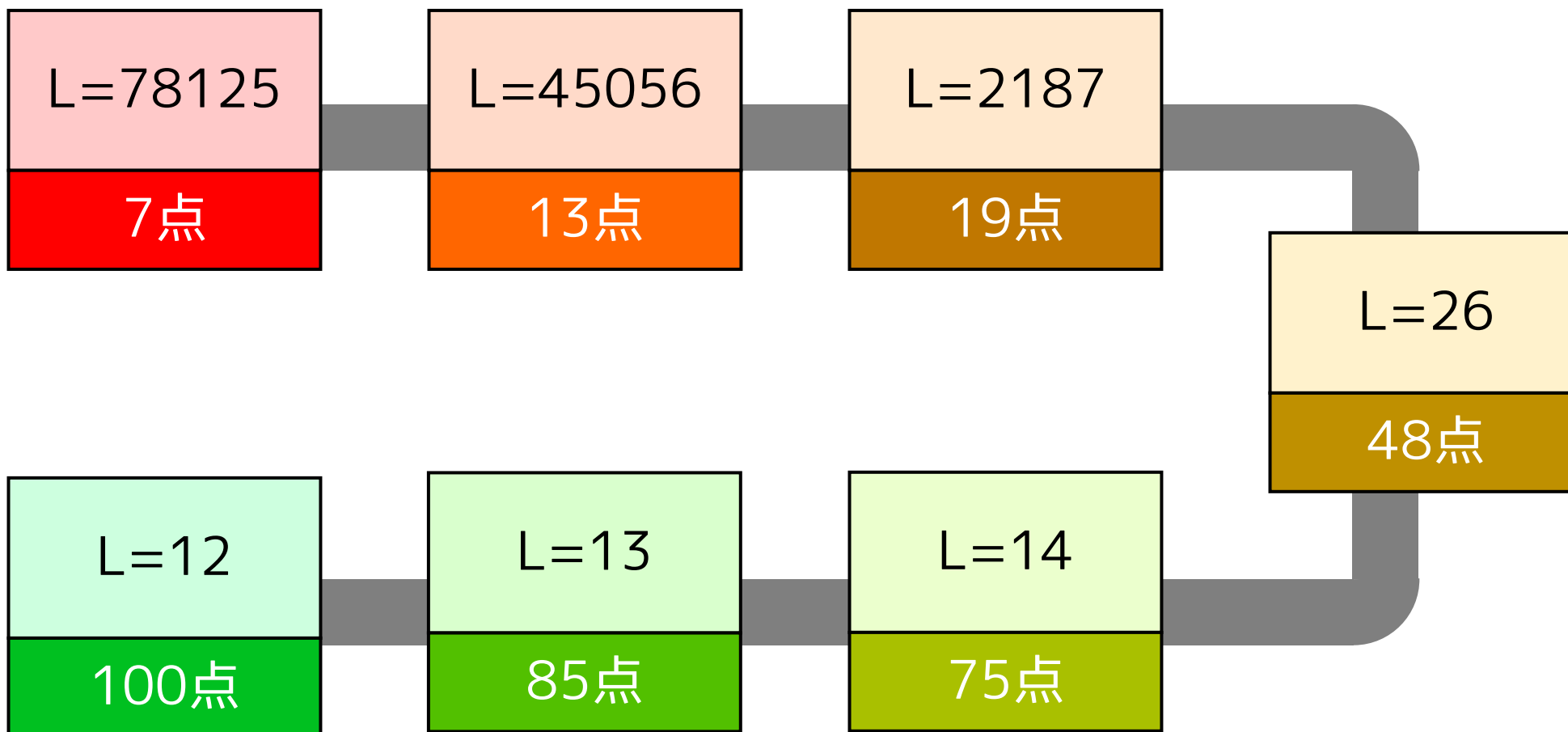
旗に書き込む整数の最大値を
できるだけ小さくしてください

$N \times N$ のマスに
 K 個の家がある

Anna は旗に
整数を書き込める

1

問題概要





$L = 78125$

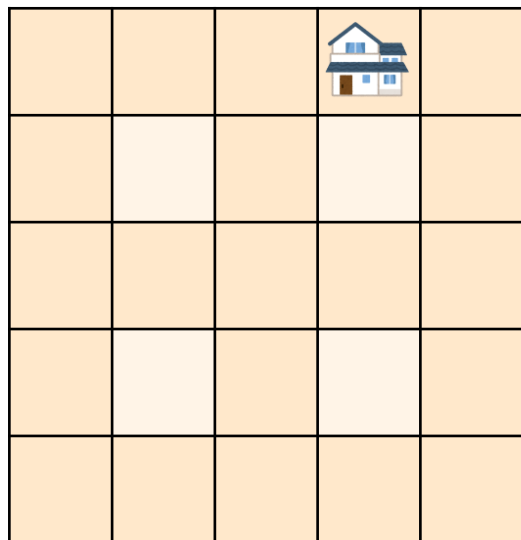
2 $L = 78125$ (7 点)

典型

$K = 1$ など特殊な
ケースを考える

2 $L = 78125$ (7 点)

$K = 1$ のときに解けないか？



2 $L = 78125$ (7 点)

$K = 1$ のときに解けないか？

0	0	0		2
0	0	0	1	2
0	0	0	1	2
0	0	0	1	2
0	0	0	1	2

各マスに対して、

「そのマスから出発した時どの行動を取れば
良いか」

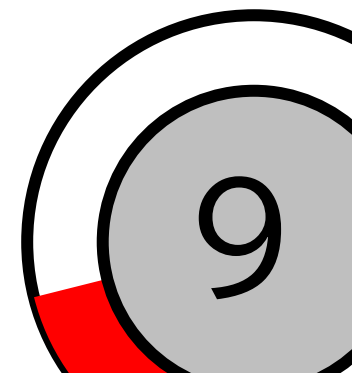
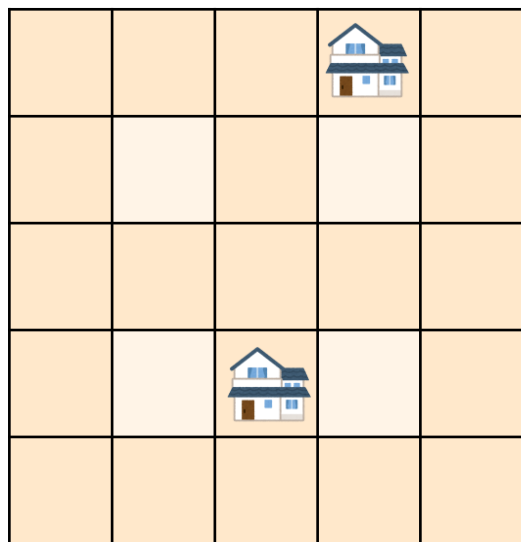
を旗に書きこめばよい ($L = 5$)

※正確には 1-indexed なので 0 を記録してはいけません

※本解説では分かりやすさ重視のため 0-indexed にしています

2 $L = 78125$ (7 点)

K が一般のときに解けないか？



2 $L = 78125$ (7 点)

K が一般のときに解けないか？

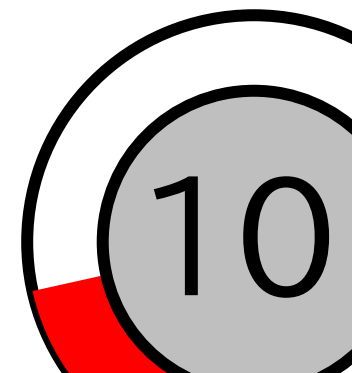
各候補の場合の行動を考えてみよう

0	0	0		2
0	0	0	1	2
0	0	0	1	2
0	0	0	1	2
0	0	0	1	2

候補 0 の場合の行動

0	0	3	2	2
0	0	3	2	2
0	0	3	2	2
0	0		2	2
0	0	1	2	2

候補 1 の場合の行動



2 $L = 78125$ (7 点)

K が一般のときに解けないか？

各候補の場合の行動を考えてみよう

0	0	0		2
0	0	1	2	2
0	0	0	1	2
0	0	0	1	2
0	0	0	1	2

候補 0 の場合の行動

0	0	3	2	2
0	0	3	2	2
0	0	3	2	2
0	0		2	2
0	0	1	2	2

候補 1 の場合の行動

合成して 1 つの整数にできないか？

2 $L = 78125$ (7 点)

K が一般のときに解けないか？

各候補の場合の行動を考えてみよう

0	0	0		2
0	0	0		
0	0	0		
0	0	0	1	2
0	0	0	1	2

候補 0 の場合の行動

典型

0	0	3	2	2
0	0	3	2	2
0	0	3	2	2
0	0		2	2
0	0	1	2	2

候補 1 の場合の行動

A 進数で表す

2 $L = 78125$ (7 点)

K が一般のときに解けないか？

二つの表を 5 進数で合成させてみよう！

0	0	0		2
0	0	0	1	2
0	0	0	1	2
0	0	0	1	2
0	0	0	1	2

候補 0 の場合の行動

0	0	3	2	2
0	0	3	2	2
0	0	3	2	2
0	0		2	2
0	0	1	2	2

候補 1 の場合の行動



0	0	15	14	12
0	0	15	11	12
0	0	15	11	12
0	0	20	11	12
0	0	5	11	12

2 $L = 78125$ (7 点)

K が一般のときに解けないか？

Anna: 二つの表を 5 進数で合成させてみよう！

0	0	0		2
0	0	0	1	2
0	0	0	1	2
0	0	0	1	2
0	0	0	1	2

候補 0 の場合の行動

0	0	3	2	2
0	0	3	2	2
0	0	3	2	2
0	0		2	2
0	0	1	2	2

候補 1 の場合の行動



0	0	15	14	12
0	0	15	11	12
0	0	15	11	12
0	0	20	11	12
0	0	5	11	12

2 $L = 78125$ (7 点)

K が一般のときに解けないか？

Anna: 二つの表を 5 進数で合成させてみよう！

$$2 \times 5 + 1 = 11$$

→11 を記録

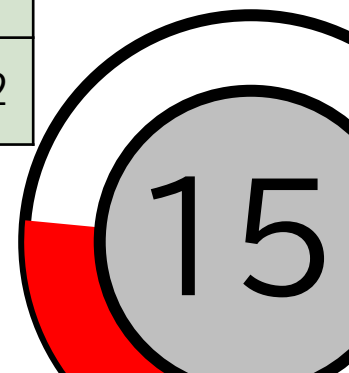
0	0	0		2
0	0	0	1	2
0	0	0	1	2
0	0	0	1	2
0	0	0	1	2

候補 0 の場合の行動

0	0	3	2	2
0	0	3	2	2
0	0	3	2	2
0	0		2	2
0	0	1	2	2

候補 1 の場合の行動

0	0	15	14	12
0	0	15	11	12
0	0	15	11	12
0	0	20	11	12
0	0	5	11	12



2 $L = 78125$ (7 点)

L の値はいくつになる？

7 桁の 5 進整数になるので、最大値は

$$5^7 = 78125$$

ここまでで 7 点が獲得できます





$L = 45056$

2 $L = 78125$ (7 点)

典型

あり得る状態数を見積もる

18

2 $L = 45056$ (13 点)

以下のような場合は存在するのか？

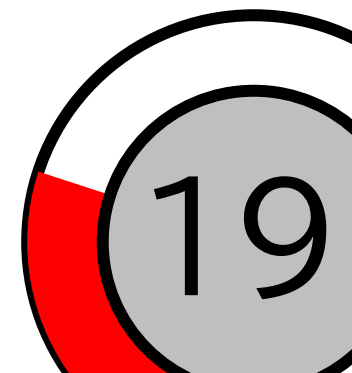
候補 0: マス (a, b) で行動 4

候補 1: マス (a, b) で行動 4

候補 2: マス (a, b) で行動 4

⋮

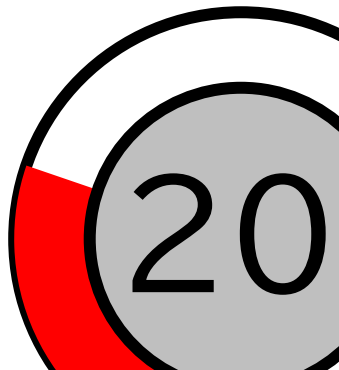
候補 6: マス (a, b) で行動 4



2 $L = 45056$ (13 点)

以下のような場合は存在するのか？

候補 0: マス (a, b) で行動 4	$\rightarrow (X_0, Y_0) = (a, b)$
候補 1: マス (a, b) で行動 4	$\rightarrow (X_1, Y_1) = (a, b)$
候補 2: マス (a, b) で行動 4	$\rightarrow (X_2, Y_2) = (a, b)$
⋮	⋮
候補 6: マス (a, b) で行動 4	$\rightarrow (X_6, Y_6) = (a, b)$



2 $L = 45056$ (13 点)

以下のような場合は存在するのか？

候補 0: マス (a, b) で行動 4 $\rightarrow (X_0, Y_0) = (a, b)$

候補 1: マス (a, b) で行動 4 $\rightarrow (X_1, Y_1) = (a, b)$
 $(X_i, Y_i) \neq (X_j, Y_j)$ より

候補 2: マス (a, b) で行動 4 $\rightarrow (X_2, Y_2) = (a, b)$

こんな場合は存在しない！

候補 6: マス (a, b) で行動 4 $\rightarrow (X_6, Y_6) = (a, b)$

2 $L = 45056$ (13 点)

基本的に、行動 4 が 2 つ以上現れることはない

そこで、 $0 \leq a_i \leq 4$ で $a_i = 4$ となる i が 2 個以上ないような数列 $(a_0, a_1, \dots, a_6)$ の通り数は 45056 通り

→ 各数列に対して ID を予め振っておくと、

$L = 45056$ で解ける (13 点獲得)





$L = 2187$

3

$L = 2187$ (19 点)

典型

$K = 1$ など特殊な
ケースを考える

3 $L = 2187$ (19 点)

$K = 1$ の場合に $L = 3$ で解けないか？



3

$L = 2187$ (19 点)

典型

距離 mod 3 を記録

※ JOI 2020 「Stray Cat」 15点解法

26

3 $L = 2187$ (19 点)

候補となる家からの最短距離 $\text{mod } 3$ を書き込むことを考える

2	1	0	1	2
1	0	2	0	1
0	2	1	2	0
2	1		1	2
0	2	1	2	0

Anna が書く整数



3 $L = 2187$ (19 点)

候補となる家からの最短距離 $\text{mod } 3$ を書き込むことを考える

2	1	0	1	2
1	0	2	0	1
0	2	1	2	0
2	1		1	2
0	2	1	2	0

Anna が書く整数

2	1	0	1	2
1	0	2	0	1
0	2	1	2	0
2	1	0	1	2
0	2	1	2	0

Bruno に見える整数

隣り合う整数は $(1, 0, 0, 0)$ の 4 つである。最短距離 $\text{mod } 3$ が $2 \rightarrow 1$ になっている場合距離が小さくなるので、1 のある方向に移動する。

3 $L = 2187$ (19 点)

最後は 7 点解法と同様に 3 進数にして考えた上で、旗に整数を書き込むことを考える。そのときの L の値は、

$$3^7 = 2187$$

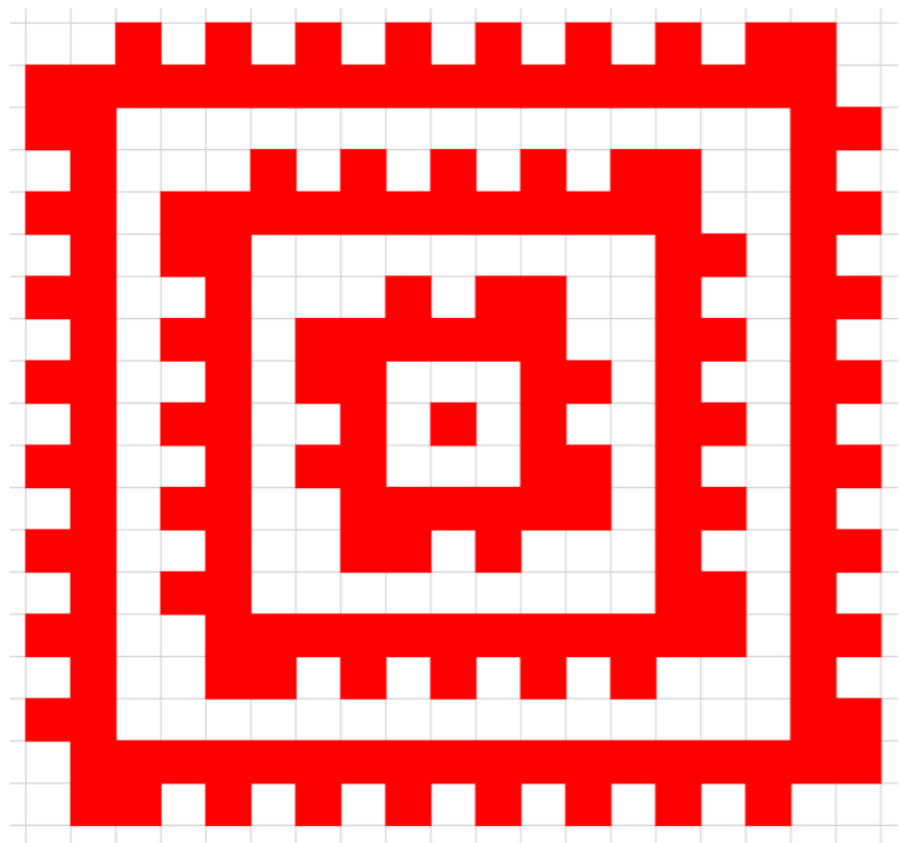
ここまでで 19 点が獲得できます





$L=128$

4 $L = 128$ (39 点)



左図のように旗に番号を記録すると、 $K = 1$ の場合に $L = 2$ で解くことができます

あとは 2 進数表記にします

$$L = 2^7 = 128$$



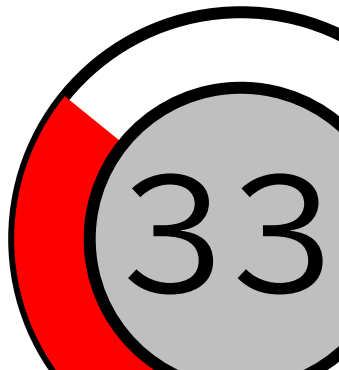


$L=14$

5 $L = 14$ (75 点)

これまでの解法では、各マスについて記録していたが、1 個のマ스에 K 個の情報を記録するのは明らかに無駄そう

→ マスごとに「どの情報を記録するか」の役割を分けられないか？



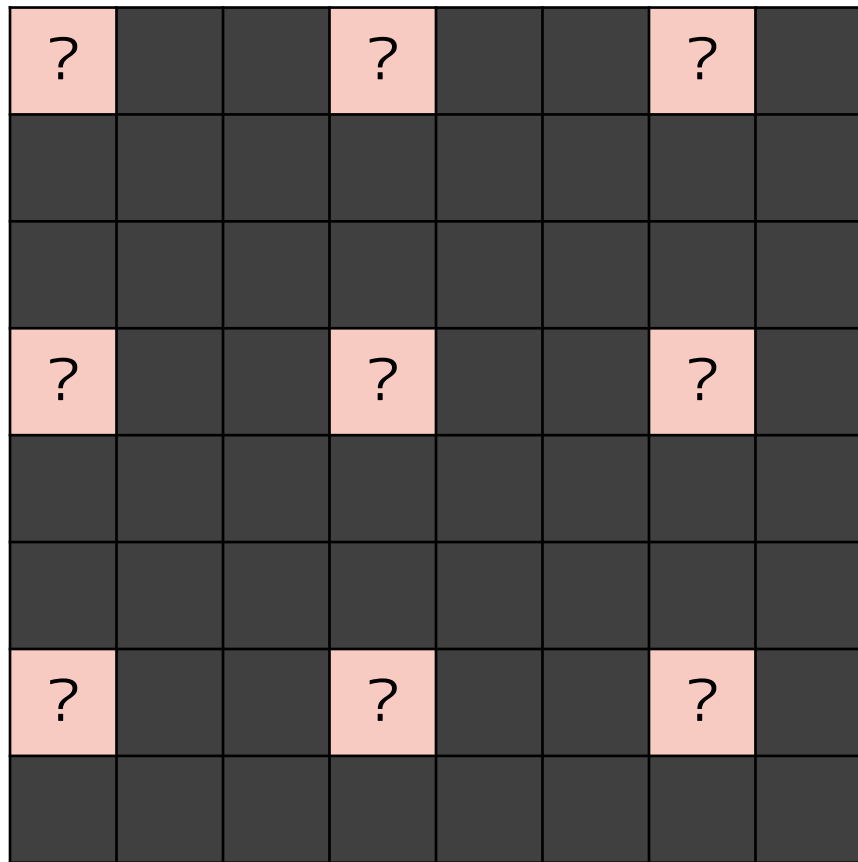
5 $L = 14$ (75 点)

これまでの解法では、各マスについて記録していたが、1 個のマ
スに K 個の情報を記録するのは明らかに無駄そう

典型

$K = 1$ など特殊な
ケースを考える

5 $L = 14$ (75 点)



$K = 1$ で、マスをも (i, j) とすると i, j が
両方 3 の倍数のマスにしか旗に番号
を記録できないような場合を考える

※他のマスの旗に書かれた整数は 0
として考える



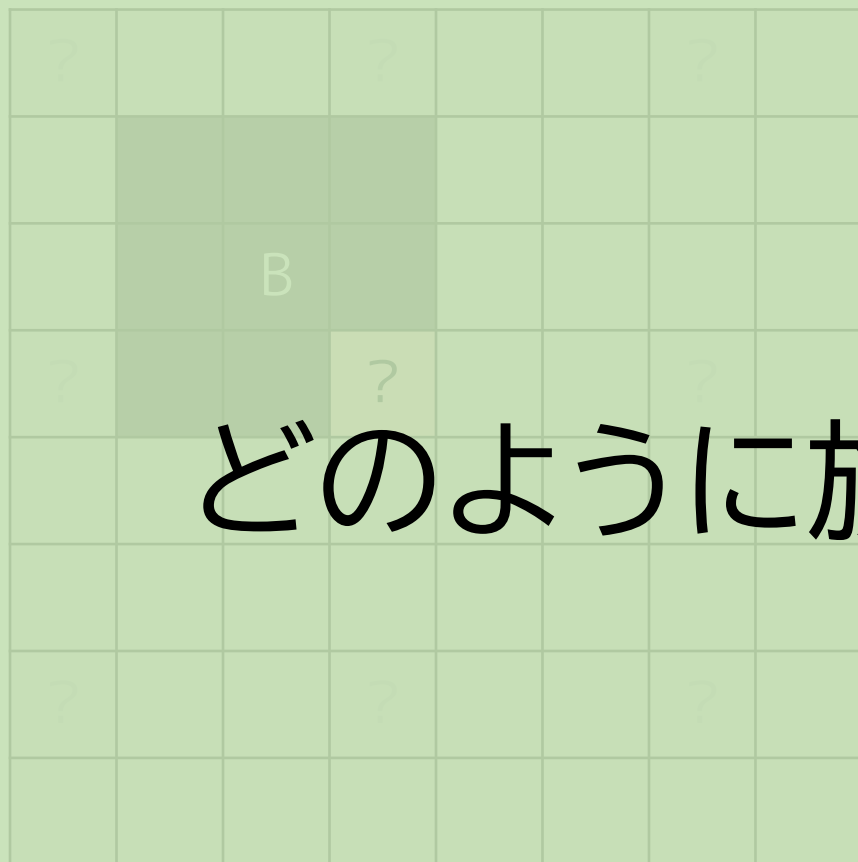
5 $L = 14$ (75 点)

?			?			?	
		B					
?			?			?	
?			?			?	

そのとき、Bruno には必ず 1 つの
「旗に整数が書き込まれているマス」
が存在する

このマスに書かれた整数だけを見て、
答えを判断しなければならない

5 $L = 14$ (75 点)



どのように旗に整数を書き込むか？

そのとき、Bruno には必ず 1 つの
「旗に整数が書き込まれているマス」
が存在する

このマスに書かれた整数だけを見て、
答えを判断しなければならない

5 $L = 14$ (75 点)

お気持ち

この辺り？

候補者の大体の位置が
分かれば良さそう

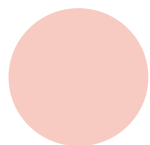
観察

38

5 $L = 14$ (75 点)

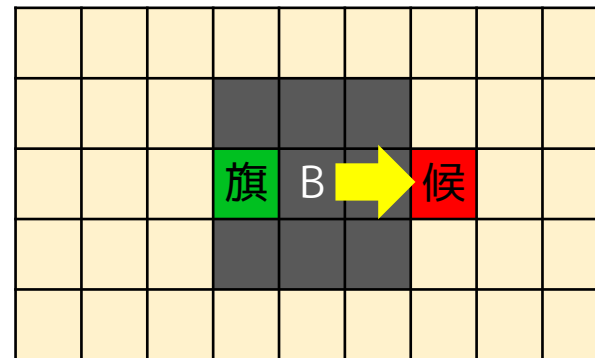
お気持ち

この辺り？



候補者の大体の位置が
分ければ良さそう

観察



x 座標が 2 以上大きければ
右方向に行けばよい

5 $L = 14$ (75 点)

お気持ち

観察

x 座標が 2 以上大きい部分は
すべて同じ整数を記録していいのでは？

候補者の大体の位置が

分ければ良さそう

x 座標が 2 以上大きければ

右方向に行けばよい

40

5 $L = 14$ (75 点)

1



Anna の実装

2



Bruno の実装

3



$K = 7$ に一般化

5 $L = 14$ (75 点) Anna

$K = 1$ で別々の値を記録すると、以下のようなになる

16	15	47	46	44	39	38
17	14	48	11	45	40	37
18	49	1	2	3	41	36
19	13	4	候	6	10	35
20	12	7	8	9	43	34
21	23	25	12	31	32	33
22	24	26	27	28	29	30



5 $L = 14$ (75 点) Anna

x 座標が 2 以上大きい場合、同じ番号にする

16	15	47	46	44	10	10
17	14	48	11	45	10	10
18	49	1	2	3	10	10
19	13	4	候	6	10	10
20	12	7	8	9	10	10
21	23	25	12	31	10	10
22	24	26	27	28	10	10



5 $L = 14$ (75 点) Anna

x 座標が 2 以上小さい場合、同じ番号にする

13	13	47	46	44	10	10
13	13	48	11	45	10	10
13	13	1	2	3	10	10
13	13	4	候	6	10	10
13	13	7	8	9	10	10
13	13	25	12	31	10	10
13	13	26	27	28	10	10



5 $L = 14$ (75 点) Anna

y 座標が 2 以上大きい場合、同じ番号にする

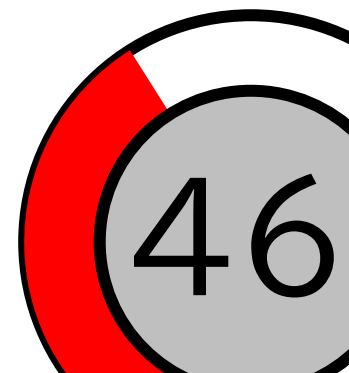
13	13	11	11	11	10	10
13	13	11	11	11	10	10
13	13	1	2	3	10	10
13	13	4	候	6	10	10
13	13	7	8	9	10	10
13	13	25	12	31	10	10
13	13	26	27	28	10	10



5 $L = 14$ (75 点) Anna

y 座標が 2 以上小さい場合、同じ番号にする

13	13	11	11	11	10	10
13	13	11	11	11	10	10
13	13	1	2	3	10	10
13	13	4	候	6	10	10
13	13	7	8	9	10	10
13	13	12	12	12	10	10
13	13	12	12	12	10	10



5 $L = 14$ (75 点) Anna

結局 Anna はそんな感じに整数を書けばよい（候補者が真ん中の場合）

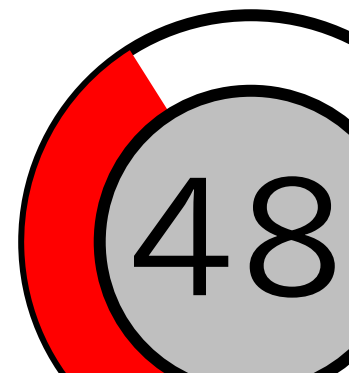
13	13	11	11	11	10	10
13	13	11	11	11	10	10
13	13	1	2	3	10	10
13	13	4	候	6	10	10
13	13	7	8	9	10	10
13	13	12	12	12	10	10
13	13	12	12	12	10	10



5 $L = 14$ (75 点) Anna

y 座標が 2 以上小さい場合、同じ番号にする

13	13	11	11	11	10	10
13	13	11	11	11	10	10
13	13	1	2	3	10	10
13	13	4	候	6	10	10
13	13	7	8	9	10	10
13	13	12	12	12	10	10
13	13	12	12	12	10	10



5 $L = 14$ (75 点)

1



Anna の実装

2



Bruno の実装

3



$K = 7$ に一般化

5 $L = 14$ (75 点) Bruno

13	13	11	11	11	10	10
13	13	11	11	11	10	10
13	13	1	2	3	10	10
13	13	4	候	6	10	10
13	13	7	8	9	10	10
13	13	12	12	12	10	10
13	13	12	12	12	10	10

最後に、Bruno の実装は以下の通り

- 10 必ず右に行く
- 11 必ず左に行く
- 12 必ず下に行く
- 13 必ず上に行く
- 他 旗との相対的な位置関係から計算して判断



5 $L = 14$ (75 点) Bruno

13	13	11	11	11	10	10
13	13	11	11	11	10	10
13	13	1	2	3	10	10
13	13	4	候	6	10	10
13	13	7	8	9	10	10
13	13	12	12	12	10	10
13	13	12	12	12	10	10

例えば Bruno が (1,5) にいた場合



5 $L = 14$ (75 点) Bruno

13	13	11	11	11	10	10
13	13	11	11	11	10	10
13	13	1	2	3	10	10
13	13	4	候	6	10	10
13	13	7	8	9	10	10
13	13	12	12	12	10	10
13	13	12	12	12	10	10

例えば Bruno が (1,5) にいた場合



整数 11 が書かれた旗が見える



5 $L = 14$ (75 点) Bruno

13	13	11	11	11	10	10
13	13	11	11	11	10	10
13	13	1	2	3	10	10
13	13	4	候	6	10	10
13	13	7	8	9	10	10
13	13	12	12	12	10	10
13	13	12	12	12	10	10

例えば Bruno が (1,5) にいた場合



整数 11 が書かれた旗が見える



下に行けばよいことが分かる



5 $L = 14$ (75 点) Bruno

例えば Bruno が (4,3) にいた場合

13	13	11	11	11	10	10
13	13	11	11	11	10	10
13	13	1	2	3	10	10
13	13	4	候	6	10	10
13	13	7	8	9	10	10
13	13	12	12	12	10	10
13	13	12	12	12	10	10



5 $L = 14$ (75 点) Bruno

13	13	11	11	11	10	10
13	13	11	11	11	10	10
13	13	1	2	3	10	10
13	13	4	候	6	10	10
13	13	7	8	9	10	10
13	13	12	12	12	10	10
13	13	12	12	12	10	10

例えば Bruno が (4, 3) にいた場合



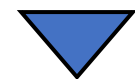
整数 9 が書かれた旗が見える



5 $L = 14$ (75 点) Bruno

13	13	11	11	11	10	10
13	13	11	11	11	10	10
13	13	1	2	3	10	10
13	13	4	候	6	10	10
13	13	7	8	9	10	10
13	13	12	12	12	10	10
13	13	12	12	12	10	10

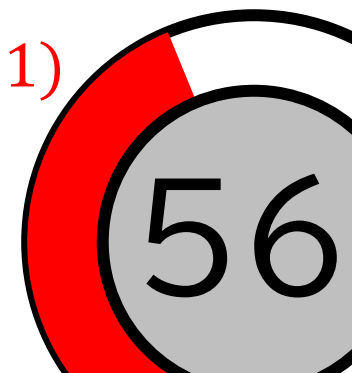
例えば Bruno が (4, 3) にいた場合



整数 9 が書かれた旗が見える



旗は Bruno から見て $(+1, \pm 0)$ の位置にある
ので、候補者は Bruno から見て $(0, -1)$
したがって、上方向に進めばよい



5 $L = 14$ (75 点)

1



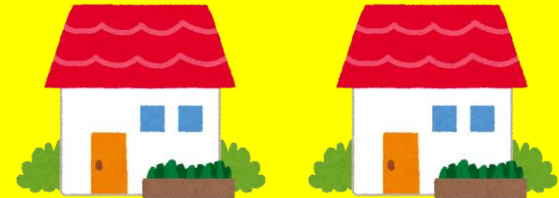
Anna の実装

2



Bruno の実装

3



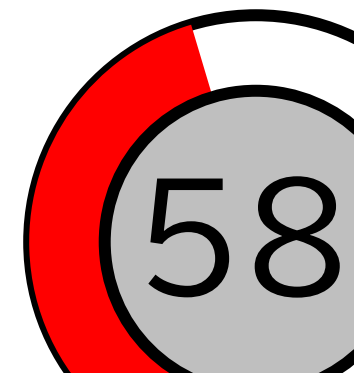
$K = 7$ に一般化

5 $L = 14$ (75 点) $K=7$ のとき

$K = 7$ の場合

以下の表のように候補者の情報を記録することを考える

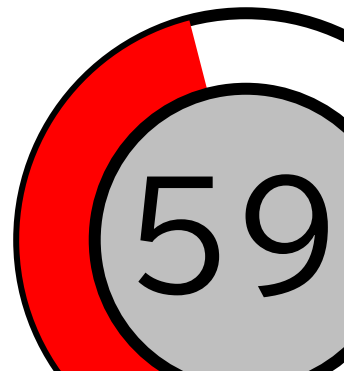
候補 0	候補 1	候補 2	候補 0	候補 1	候補 2	候補 0	候補 1
候補 3	候補 4	候補 5	候補 3	候補 4	候補 5	候補 3	候補 4
候補 6			候補 6			候補 6	
候補 0	候補 1	候補 2	候補 0	候補 1	候補 2	候補 0	候補 1



5 $L = 14$ (75 点) $K=7$ のとき

どの 3×3 のマス目にも、候補 0~6 の情報がすべて含まれているため、Bruno は正しく答えられる

候補 0	候補 1	候補 2	候補 0	候補 1	候補 2	候補 0	候補 1
候補 3	候補 4	候補 5	候補 3	候補 4	候補 5	候補 3	候補 4
候補 6			候補 6			候補 6	
候補 0	候補 1	候補 2	候補 0	候補 1	候補 2	候補 0	候補 1



5 $L = 14$ (75 点) $K=7$ のとき

候補 0 ~ 6 のマス目には 1 以上 13 以下の整数を記録することができるため、**空きマスの片方に 14 を記録すればよい** (空きマスの位置によって、どのマスがどの候補の情報であるかが分かる)

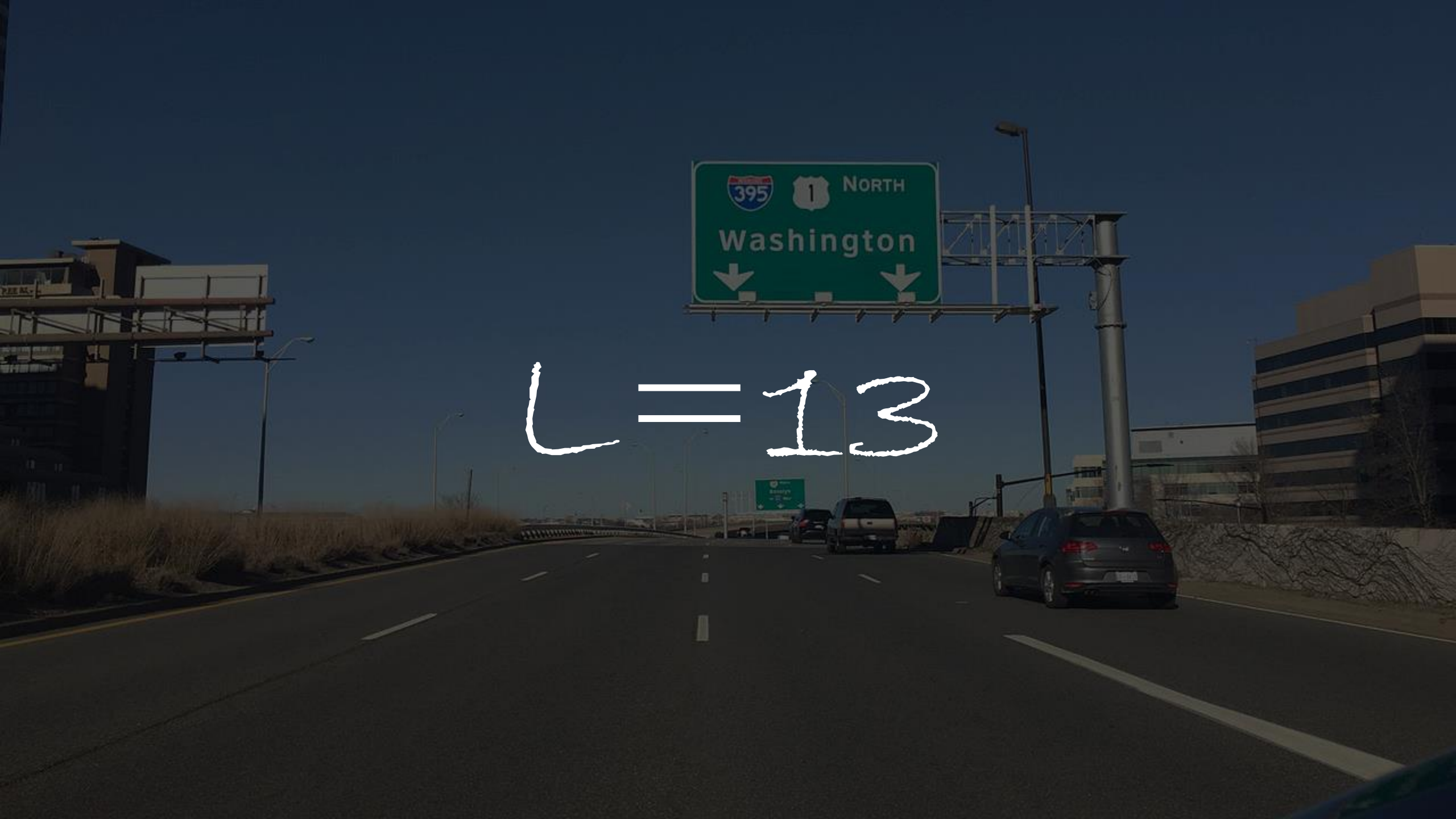
ここまでで **75 点** が獲得できます

候補 0	候補 1	候補 2
候補 3	候補 4	候補 5
候補 6		14





$L=13$



6

 $L = 13$ (85 点)

お気持ち

候補 0	候補 1	候補 2
候補 3	候補 4	候補 5
候補 6	-	(13)

1 つのマスが使われて
いなくて無駄そう

観察

62

6

 $L = 13$ (85 点)

お気持ち

候補 0	候補 1	候補 2
候補 3	候補 4	候補 5
候補 6	-	(13)

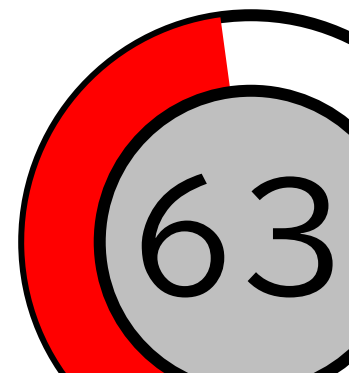
1 つのマスが使われて
いなくて無駄そう

観察

1 3 5 ~~7~~ 9
~~2~~ 4 6 8

整数 1~9 のうち **全体** で使われな
い整数が 2 個以上存在する

※各候補について、9 以下の整数は高々 1 回しか書き込まれないため



6

$L = 13$ (85 点)

お気持ち

0	1	2
3	4	5
6		

観察

空いた 1 マスのうち片方を
「使われない整数」の記録に使えないか？

2 つのマスが使われて
いなくて無駄そう

整数 1~9 のうち使われない整数
が 2 個以上存在する

※各候補について、9 以下の整数は高々 1 回しか書き込まれないため

64

6 $L = 13$ (85 点)

空きマスに「使われない整数」を記録した場合…

- 使われない整数を val とする
- そのとき、 $val + 1$ 以上の整数は全部 1 減少させる



6 $L = 13$ (85 点)

空きマスに「使われない整数」を記録した場合…

- 使われない整数を val とする
- そのとき、 $val + 1$ 以上の整数は全部 1 減少させる

1	9	11
8	5	12
4	-	13

$val = 7$ の場合



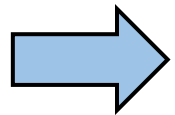
6 $L = 13$ (85 点)

空きマスに「使われない整数」を記録した場合…

- 使われない整数を val とする
- そのとき、 $val + 1$ 以上の整数は全部 1 減少させる

1	9	11
8	5	12
4	-	13

$val = 7$ の場合



1	9	11
8	5	12
4	7	13

空きマスに 7 を記録



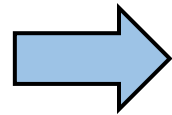
6 $L = 13$ (85 点)

空きマスに「使われない整数」を記録した場合…

- 使われない整数を val とする
- そのとき、 $val + 1$ 以上の整数は全部 1 減少させる

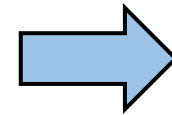
1	9	12
8	5	13
4	-	14

$val = 7$ の場合



1	9	12
8	5	13
4	7	14

空きマスに 7 を記録



1	8	11
7	5	12
4	7	13

減少させる



6 $L = 13$ (85 点)

空きマス位置は、「13」と書き込まれたマス位置を参考にとすると分かる

$val \leq 8$ より最大値 14 は必ず 1 減少して 13 になる
ここまでで 85 点が獲得できます





$L=12$

7

$L = 12$ (100 点)

お気持ち

1 3 5 7 ✓
~~2~~ 4 6 8

使われていない整数の
2 個目を利用できないか？

観察



7

 $L = 12$ (100 点)

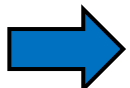
お気持ち

1 3 5 7 9 ✓
~~2~~ 4 6 8

使われていない整数の
2 個目を利用できないか？

観察

1	6	11
8	5	12
4	-	13



1	6	10
8	5	11
4	-	12

9 が使われないことが保証される
特殊な場合だと 1 個減らせる

72

7 $L = 12$ (100 点)

お気持ち

1 3 5 7
✕ 4 6 8

使われていない整数の
2 個目を利用できないか？

観察

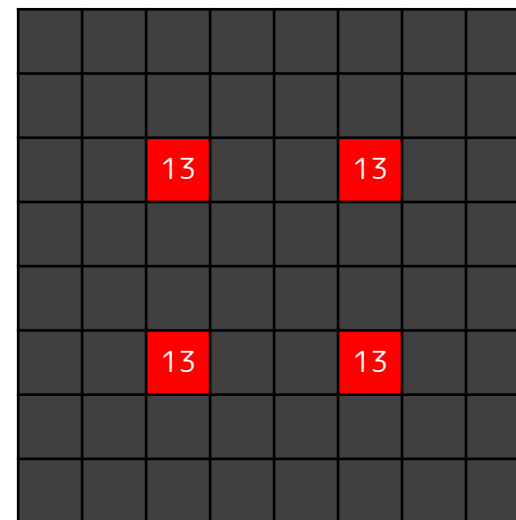
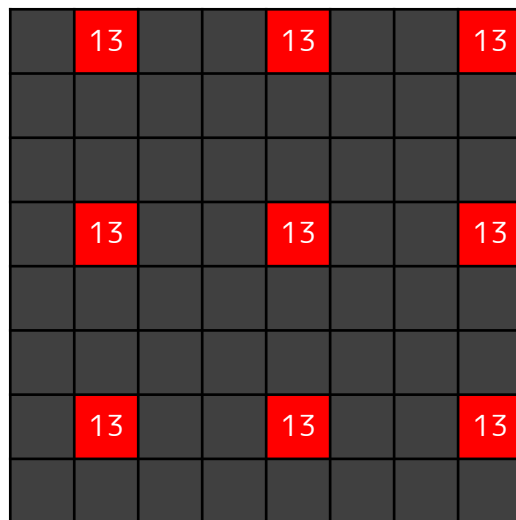
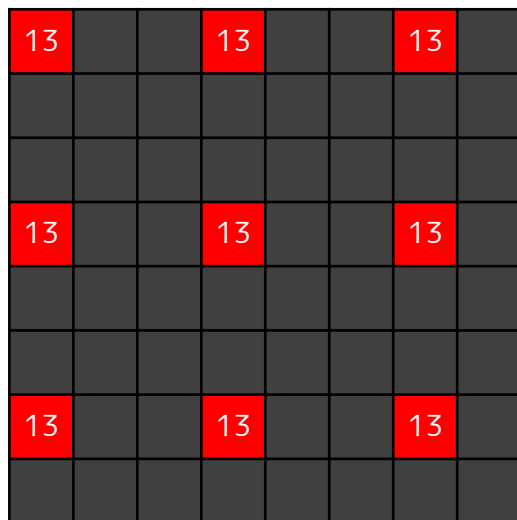
1	6	11
2	3	12
4	-	13

1	6	10
2	3	11
4	-	12

9 を無理矢理使わせない
方法は存在するか？

9 が使われないことが保証される
特殊な場合だと 1 個減らせる

7

 $L = 12$ (100 点)

最大値 (85 点解法では 13) を記録する位置を全探索すると
9 通り中 2 通り以上では 9 が使われない

74

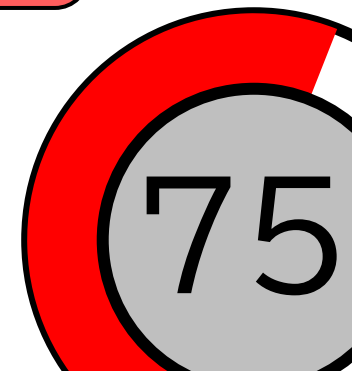
7 $L = 12$ (100 点)

したがって、以下のステップで**最大値 12** が達成できる

9 通り全探索して、「9」が使われない書き方を見つける

そうすると、最大値 13 ($=14-1$) の構成ができる

あとは 85 点解法の通りにさらに 1 減らす

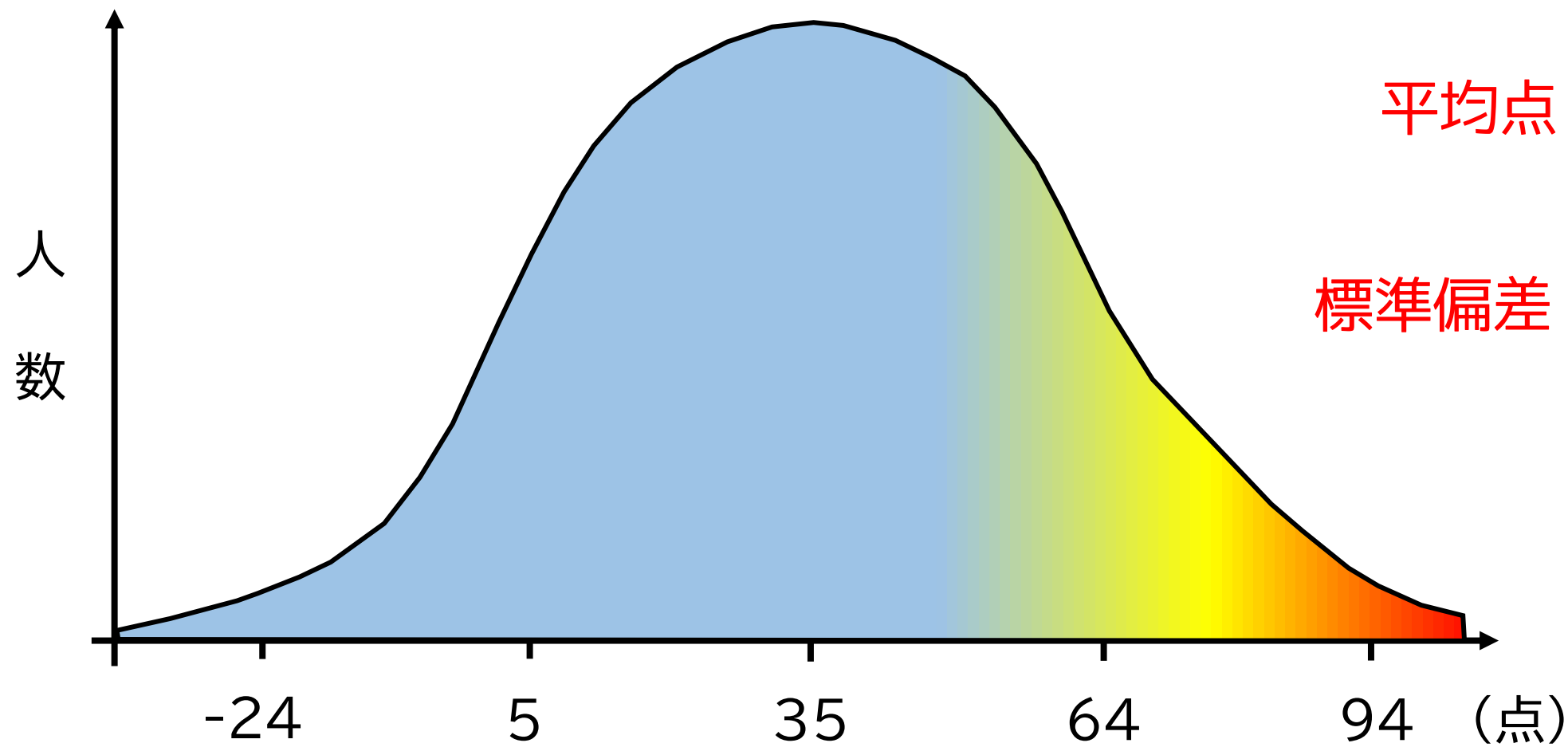




Score Distribution

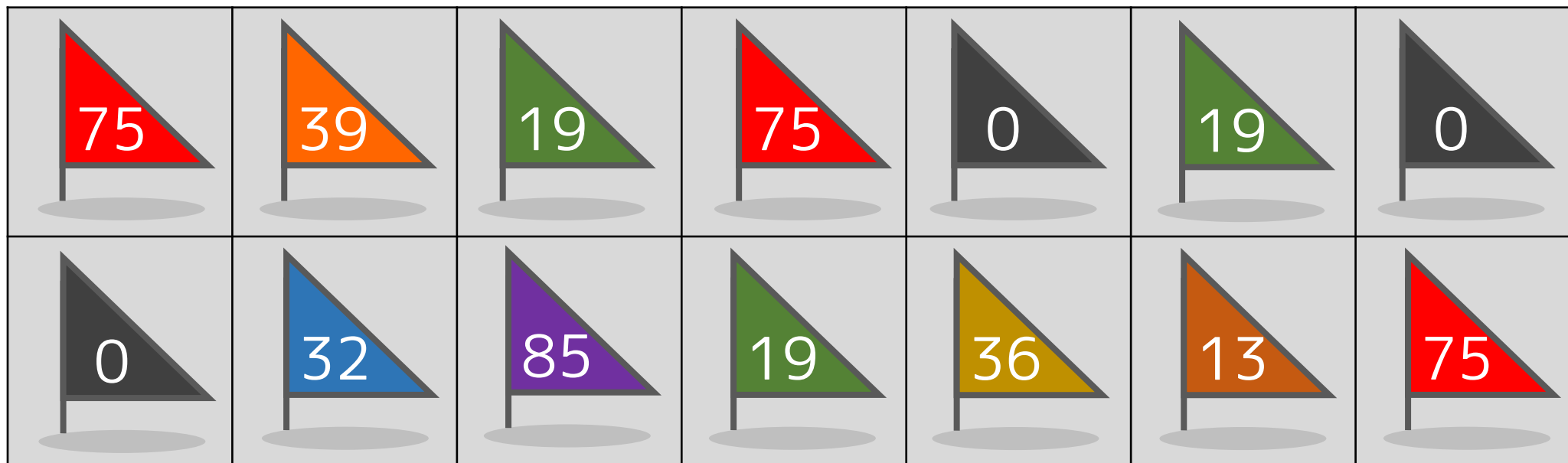
8

得点分布（正規分布）



8

得点分布



ご清聴ありがとうございました

78

Worst Reporter 4

解説：岸田陸玖

問題概要

- 頂点数 N の functional graph（各頂点の出次数が 1 の有向グラフ） G と長さ N の数列 H, C が与えられる

操作：コスト C_i かけて、 H_i を任意の値を変える

条件： $\forall (u, v) \in E(G), H_u \geq H_v$

- コストの総和の最小値は？

考察

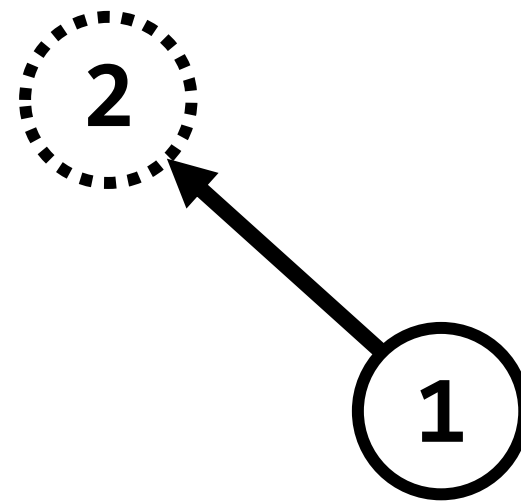
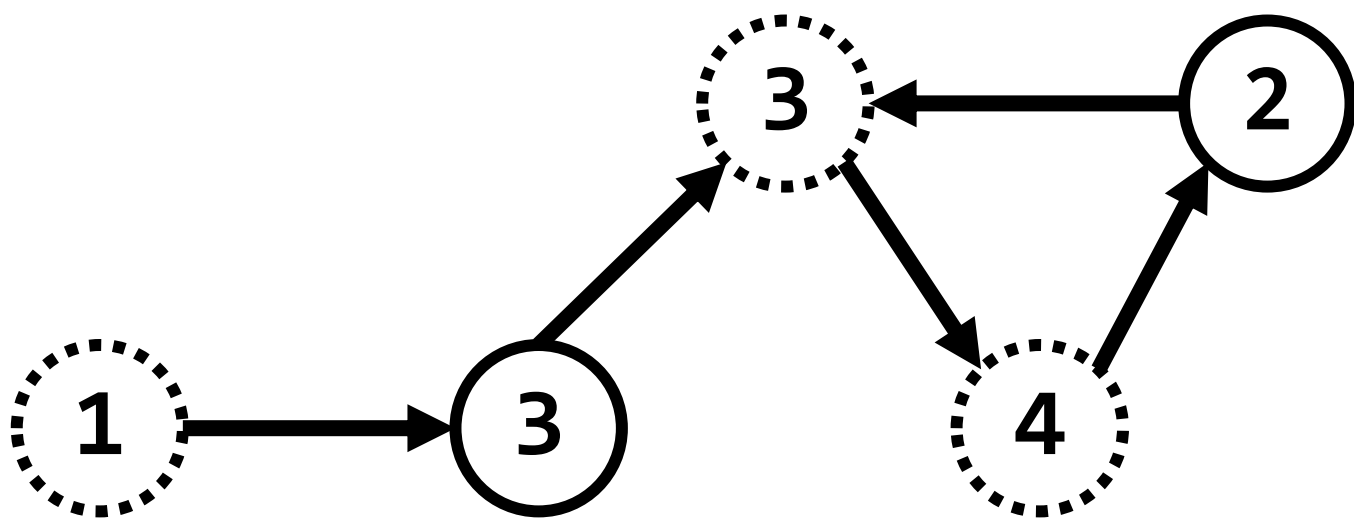
- S を H の値を**変えない**頂点集合とすると
条件を満たすためには (#) を満たす必要がある

$u, v \in S$ かつ $u \rightarrow v$ のパスが存在するならば $H_u \geq H_v \cdots (\#)$

- 逆に (#) を満たすような S は H の値を適切に変えることで条件を満たすことが可能

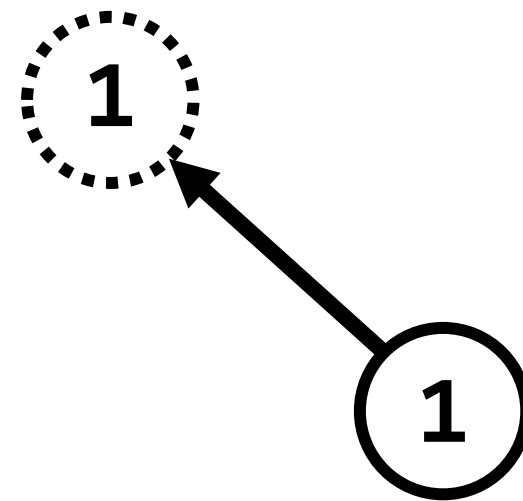
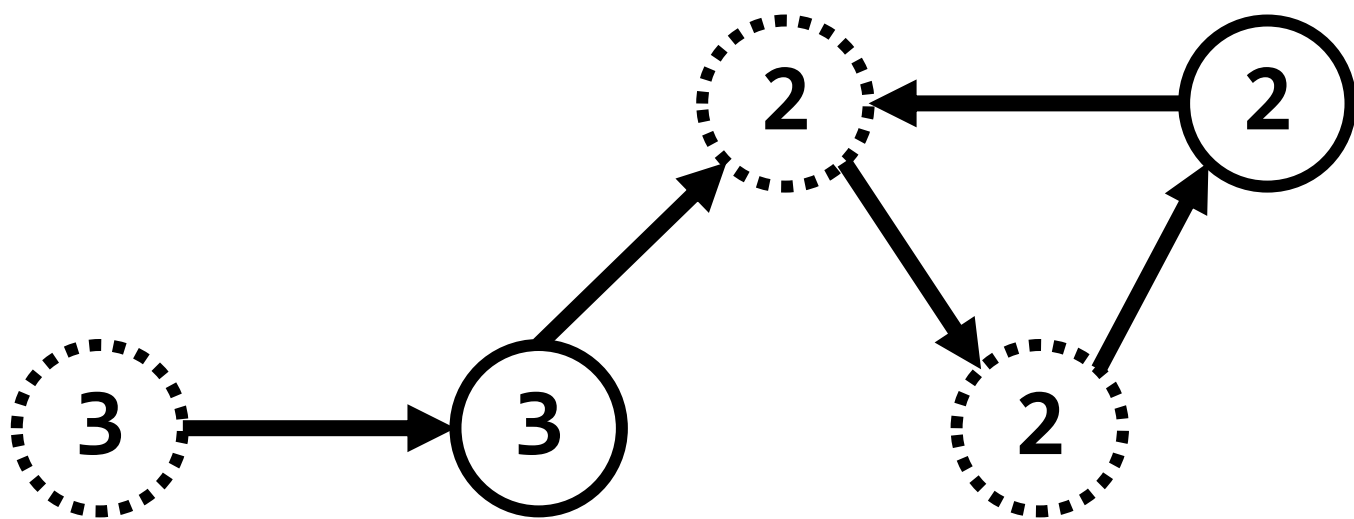
考察

- 辺にそって進んだとき、最初に到達する s の頂点の値に変えればよい
- ただし、到達しない場合は **1** に変える



考察

- 辺にそって進んだとき、最初に到達する s の頂点の値に変えればよい
- ただし、到達しない場合は **1** に変える



考察

$u, v \in S$ かつ $u \rightarrow v$ のパスが存在するならば $H_u \geq H_v \cdots (\#)$

- $(\#)$ を満たす S の集合を $\mathcal{S}$ とすると

$\sum_{v \in \{1, \dots, N\}} C_v - \max\{ \sum_{v \in S} C_v \mid S \in \mathcal{S} \}$ が答え

- 座標圧縮をすることで H は 1 以上 N 以下の値に変換可能

小課題 1

- グラフが木になっている
- $\text{dp}[v][i] := (\#)$ に加え以下を満たす S の中で $\sum_{v \in S} C_v$ の最大値
 $S \subseteq V$ (頂点 v の部分木) and $\forall v \in S, H_v \geq i$
- $\text{dp}[\text{root}][1] = \max\{ \sum_{v \in S} C_v \mid S \in \mathcal{S} \}$ が成り立つ

小課題 1

- $N_v := (v \text{ の子の頂点集合})$ とする
- $H_v < i$ の場合、 $v \notin S$ のみ考慮する

$$\text{dp}[v][i] = \sum_{u \in N_v} \text{dp}[u][i]$$

- $H_v \geq i$ の場合、 $v \notin S$ と $v \in S$ の2通り考慮する

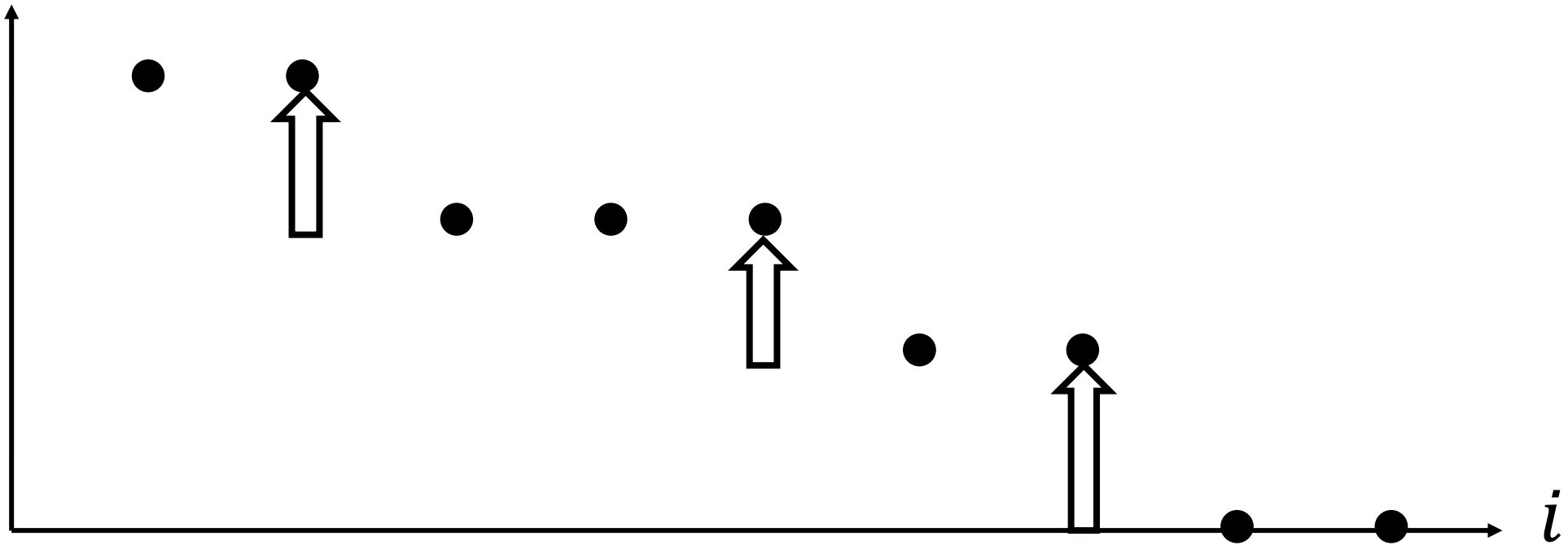
$$\text{dp}[v][i] = \max\left(\sum_{u \in N_v} \text{dp}[u][i], C_v + \sum_{u \in N_v} \text{dp}[u][H_v]\right)$$

- $O(N^2)$ で計算可能

小課題 2

- $N \leq 2 \times 10^5$
- $\text{dp}[v][i] := (\#)$ に加え以下を満たす S の中で $\sum_{v \in S} C_v$ の最大値
 $S \subseteq V$ (頂点 v の部分木) and $\forall v \in S, H_v \geq i$
- 各頂点 v に対し $\text{dp}[v]$ を直接記録するのは避けたい
- $\text{dp}[v][i]$ ($i \in \{1, \dots, N\}$) の値の種類数は
頂点 v の部分木の頂点数以下であり、広義単調減少している

小課題 2



- 増加する箇所のみ $(i, dp[v][i] - dp[v][i + 1])$ を記録すればよい

小課題 2

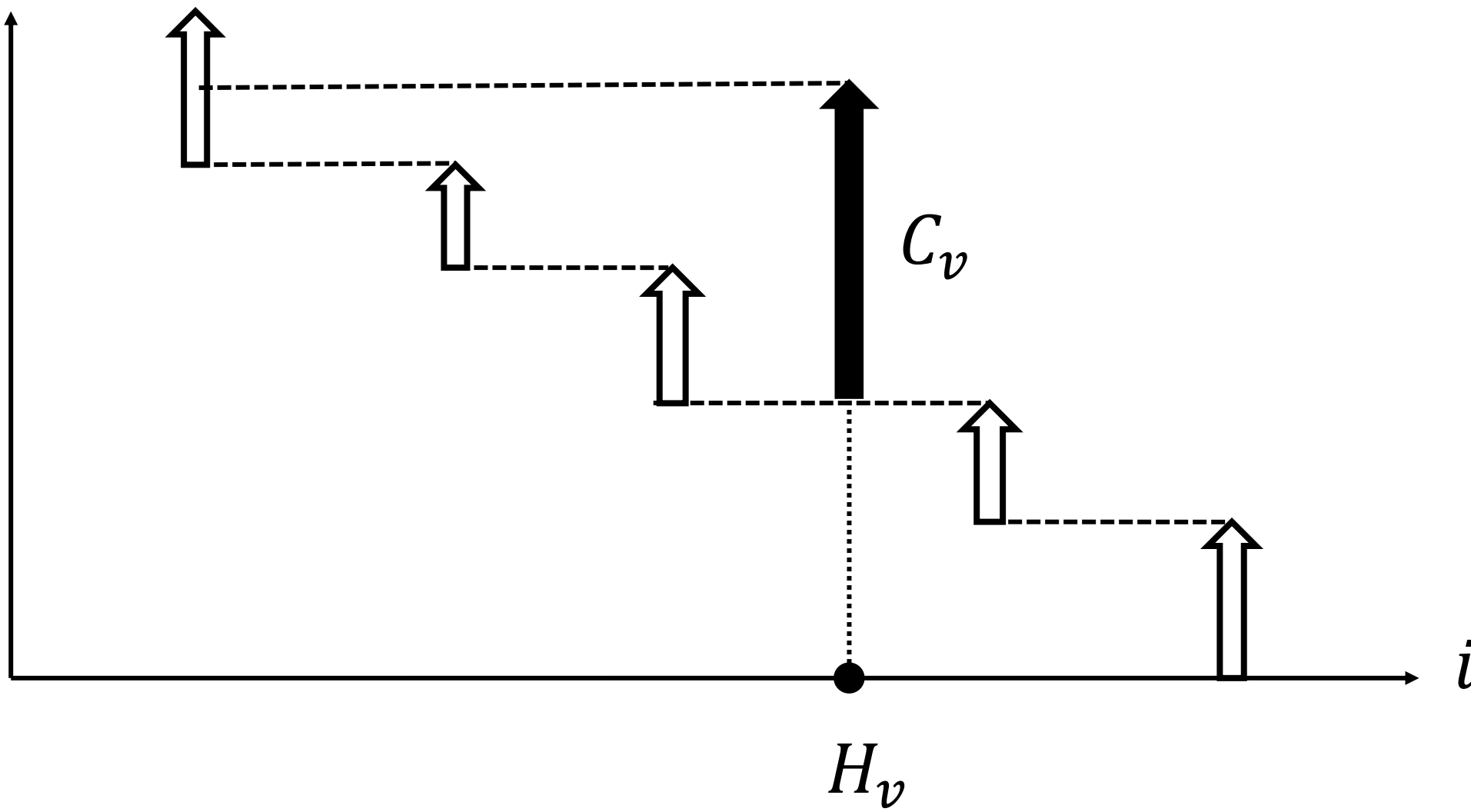
- $\text{dp}[v]$ が $(i, d_v(i))_{i \in X_v}$ (X_v : distinct, $d_i \geq 1$) で表されたとする
- この情報だけで dp を計算したい
- まず dp の遷移における $v \notin S$ の部分 $\sum_{u \in N_v} \text{dp}[u][i]$ 計算したい
- $X_v = \bigcup_{u \in N_v} X_u$ として $(i, \sum_{u \in N_v, i \in X_u} d_u(i))_{i \in X_v}$ で表される

小課題 2

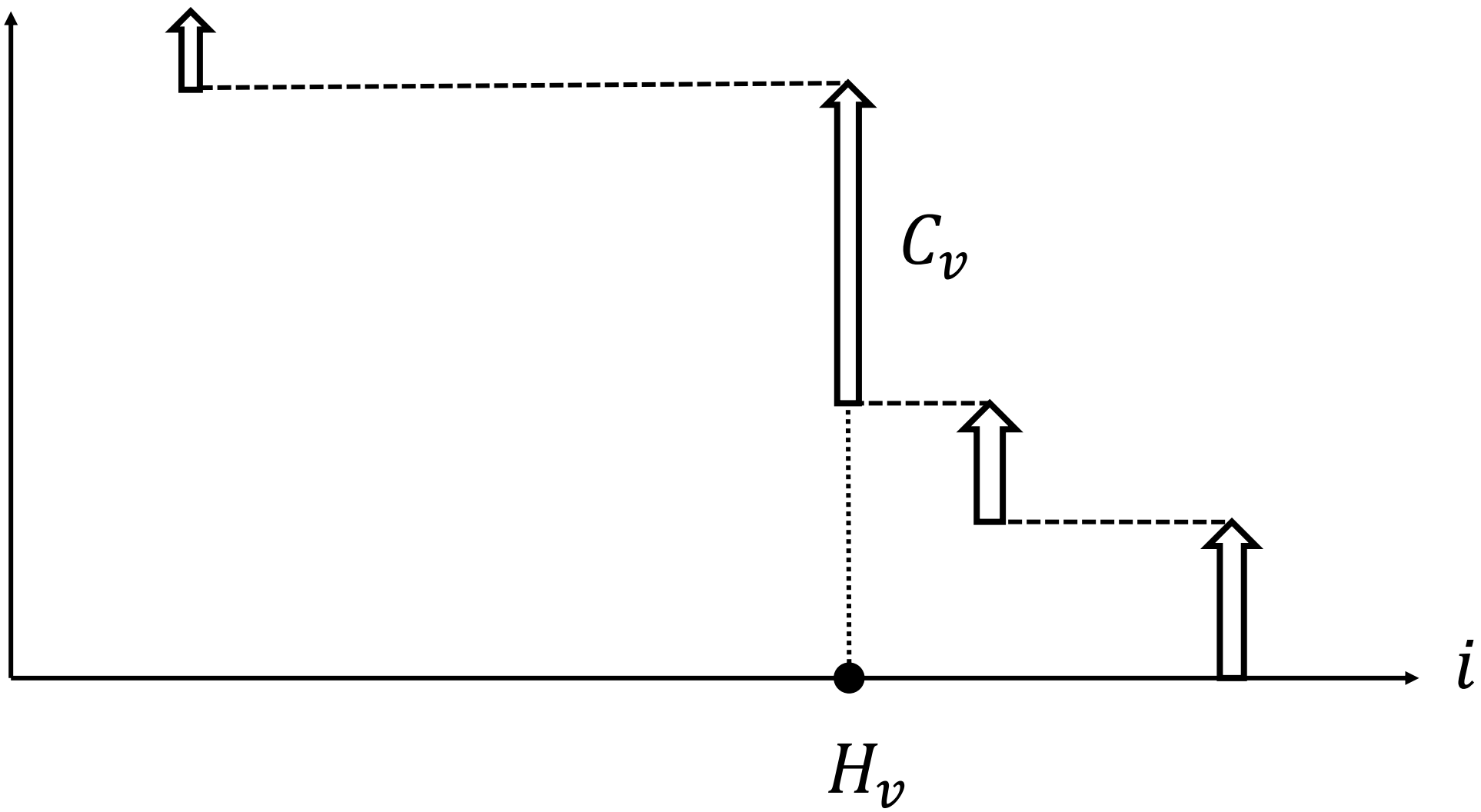
- 次に dp の遷移における $v \in S$ の部分を計算したい

$$\text{dp}[v][i] = \max\left(\sum_{u \in N_v} \text{dp}[u][i], C_v + \sum_{u \in N_v} \text{dp}[u][H_v]\right) \quad (i \leq H_v)$$

小課題 2



小課題 2



小課題 2

- H_v 未満であるような i で大きいほうから順に $(i, d_v(i))$ が消滅または d_i の値が減少する
- 以上の計算は `std::map` で高速に計算可能
`std::map::erase` , `std::map::lower_bound` を利用すれば容易
- しかしこのままでは $O(N^2 \log N)$ になり計算量が悪化する
- $(i, \sum_{u \in N_v, i \in X_u} d_u(i))_{i \in X_v}$ の計算、すなわち `map` の `merge` を高速化したい

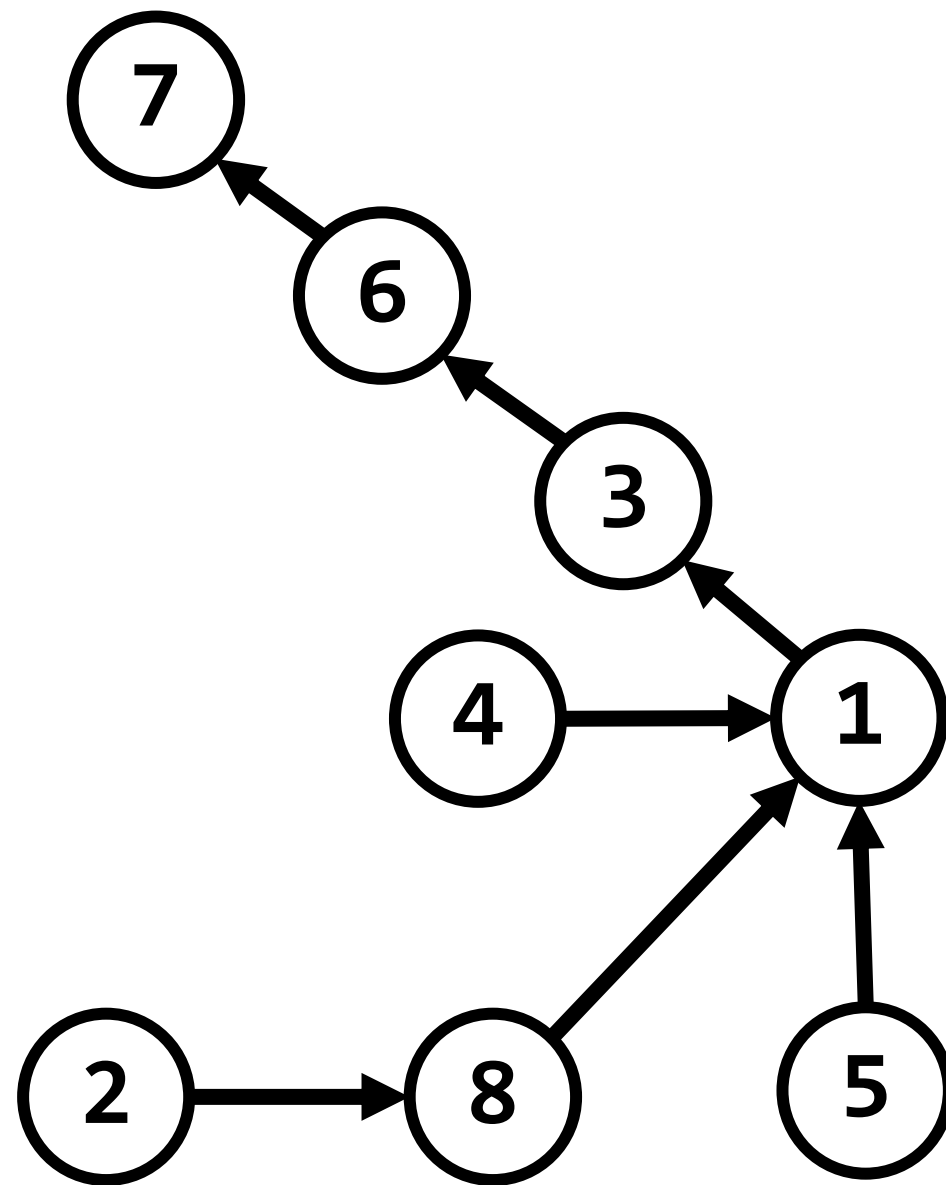
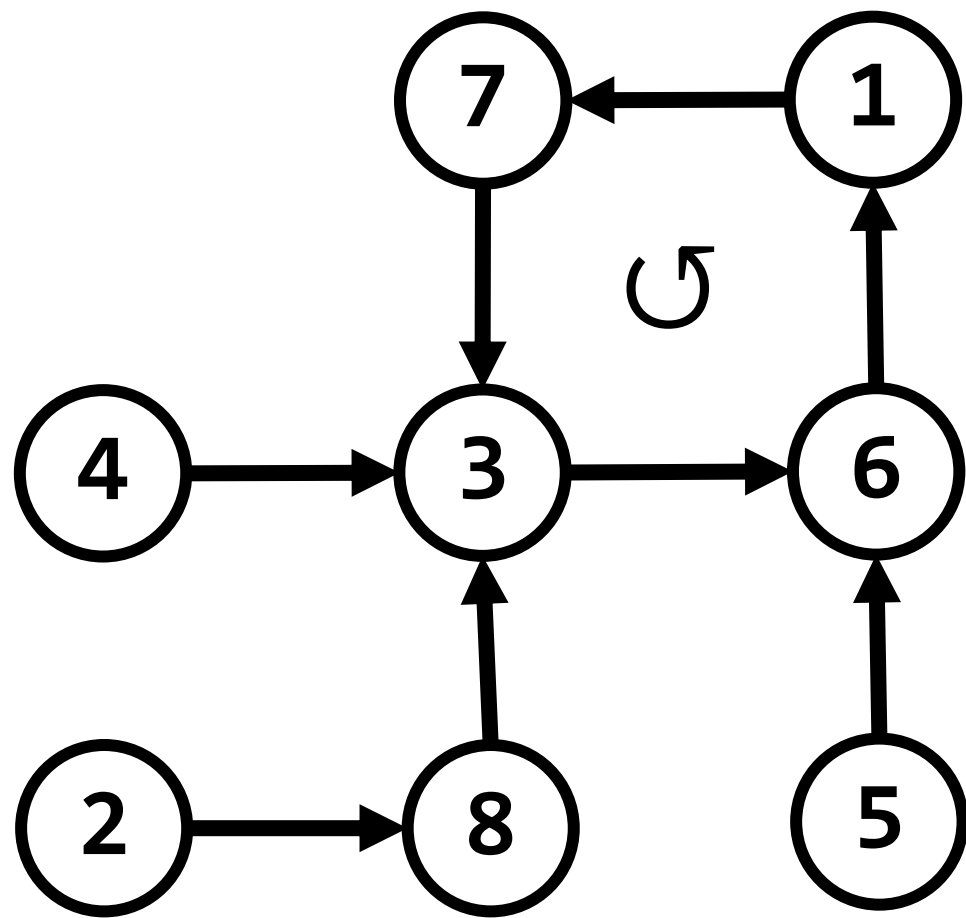
小課題 2

- `map` の `size` が小さい方から大きい方に `merge` を行うようにする
- この処理を用いると計算量は $O(N(\log N)^2)$ に改善される
- 証明は省略
- 「データ構造をマージする一般的なテク」を調べてみましょう

小課題 3

- グラフが木ではなくなった
- 一般的な `functional graph` も木の場合に帰着させたい

小課題 3



小課題 3

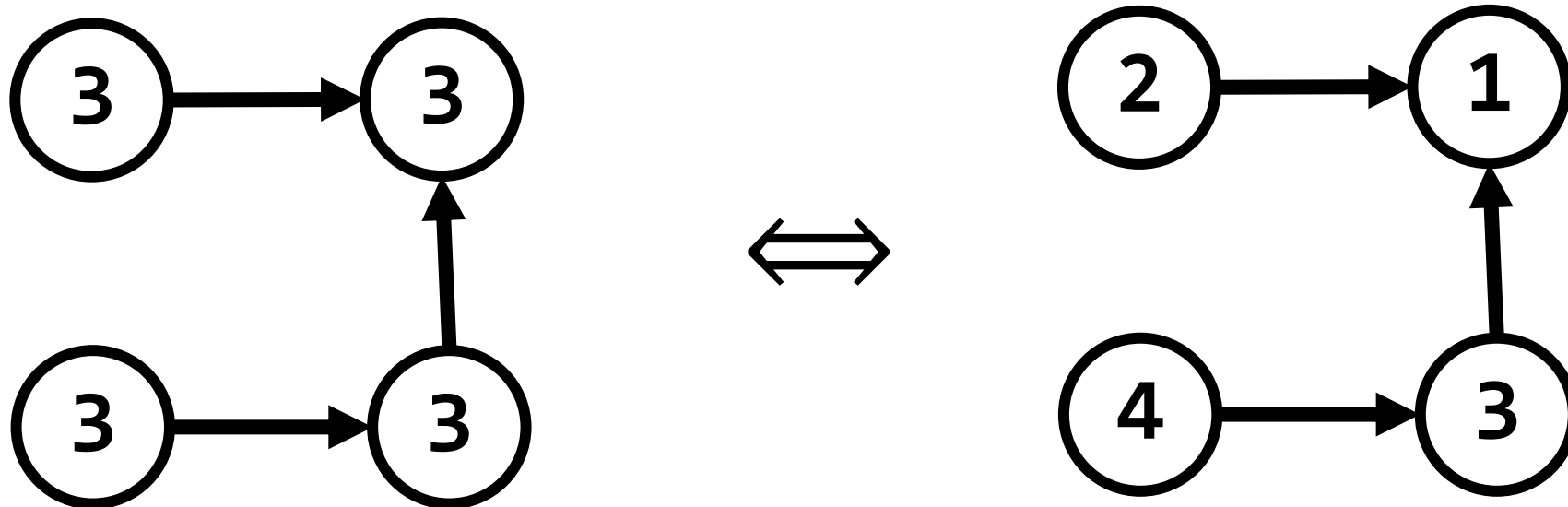
- 一般的な `functional graph` も木の場合に帰着させたい
- サイクル部分を降順に並べ、木の部分の `root` の行先をサイクル部分の H が最も小さい頂点にする
- 線形時間で木の場合に帰着可能
- 100点

別解法

- $O(N \log N)$ で解くことも可能
- $\text{dp}[v][i] := (\#)$ に加え以下を満たす S の中で $\sum_{v \in S} C_v$ の最大値
 $S \subseteq V$ (頂点 v の部分木) and $\forall v \in S, H_v \geq i$
- $i = N, \dots, 1$ として $\text{dp}[v][i]$ の値がどのように変化するかを着目してみる

別解法

- H の値は $(1, \dots, N)$ の permutation の場合に帰着できる
- 同じ値を比較するるとき、深い方を大きいとみなす
深さが一致する場合はどちらでもよい



別解法

- v を $H_v = i$ を満たす頂点とすると

$$\text{dp}[v][i] = \text{dp}[v][i + 1] + C_v$$

$$\text{dp}[u][i] = \text{dp}[u][i + 1] \text{ (} u \text{ は } v \text{ の祖先でない)}$$

$$\text{dp}[u][i] = \max(\text{dp}[u][i + 1], \sum_{w \in N_u} \text{dp}[w][i]) \text{ (} u \text{ は } v \text{ の祖先)}$$

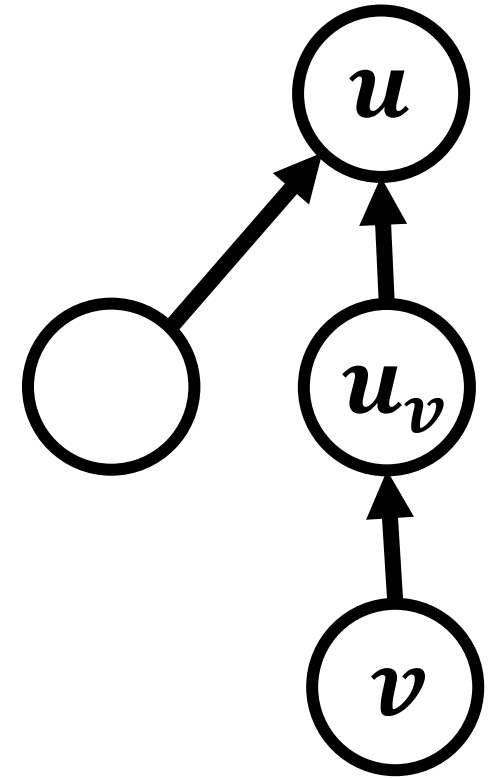
別解法

- v の祖先のみ着目すればよい

$$\text{dp}[u][i] = \max(\text{dp}[u][i + 1], \sum_{w \in N_u} \text{dp}[w][i])$$

- $b[u][i] := \text{dp}[u][i] - \sum_{w \in N_u} \text{dp}[w][i]$ と定義すると

$$\begin{aligned} b[u][i] &= \max(\text{dp}[u][i + 1] - \sum_{w \in N_u} \text{dp}[w][i], 0) \\ &= \max(b[u][i + 1] - \sum_{w \in N_u} (\text{dp}[w][i] - \text{dp}[w][i + 1]), 0) \\ &= \max(b[u][i + 1] - (\text{dp}[u_v][i] - \text{dp}[u_v][i + 1]), 0) \end{aligned}$$



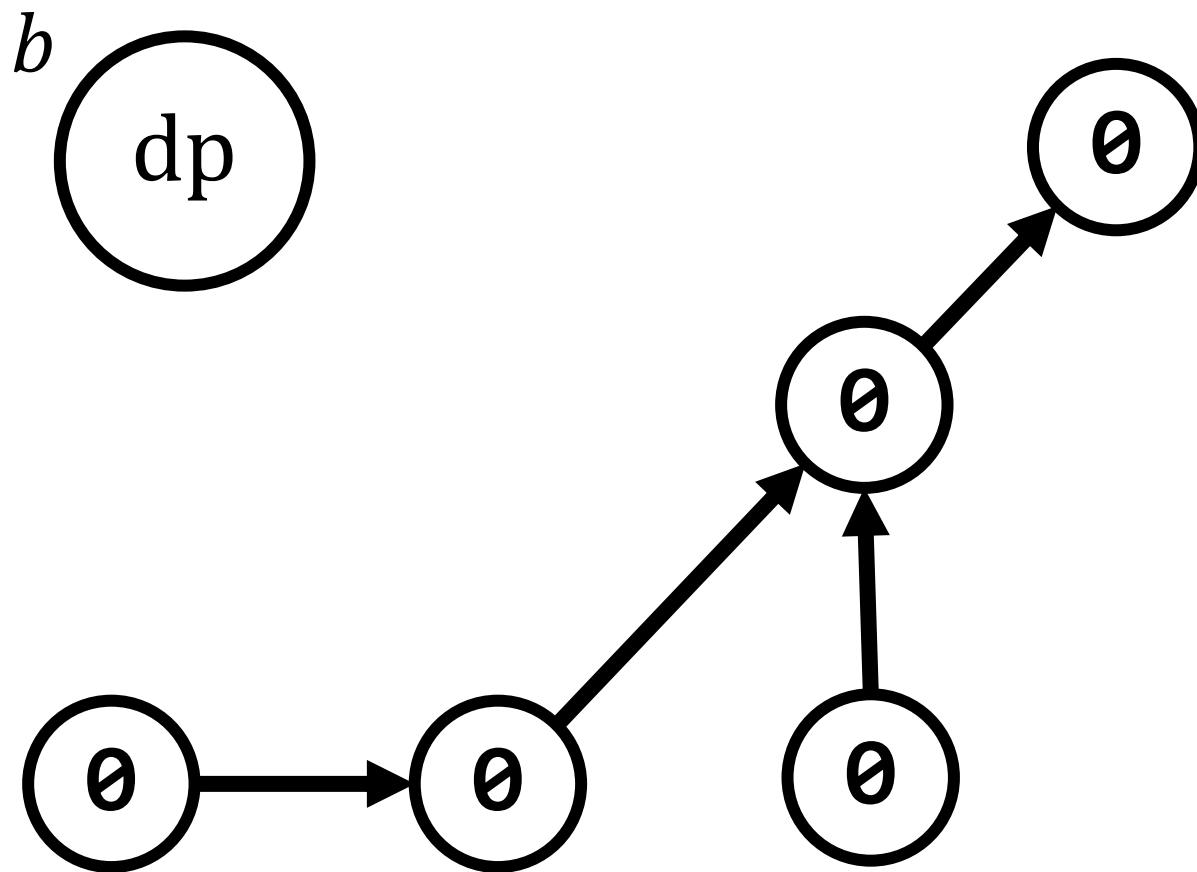
別解法

- $s[u][i] := \text{dp}[u][i] - \text{dp}[u][i + 1]$ と定義すると

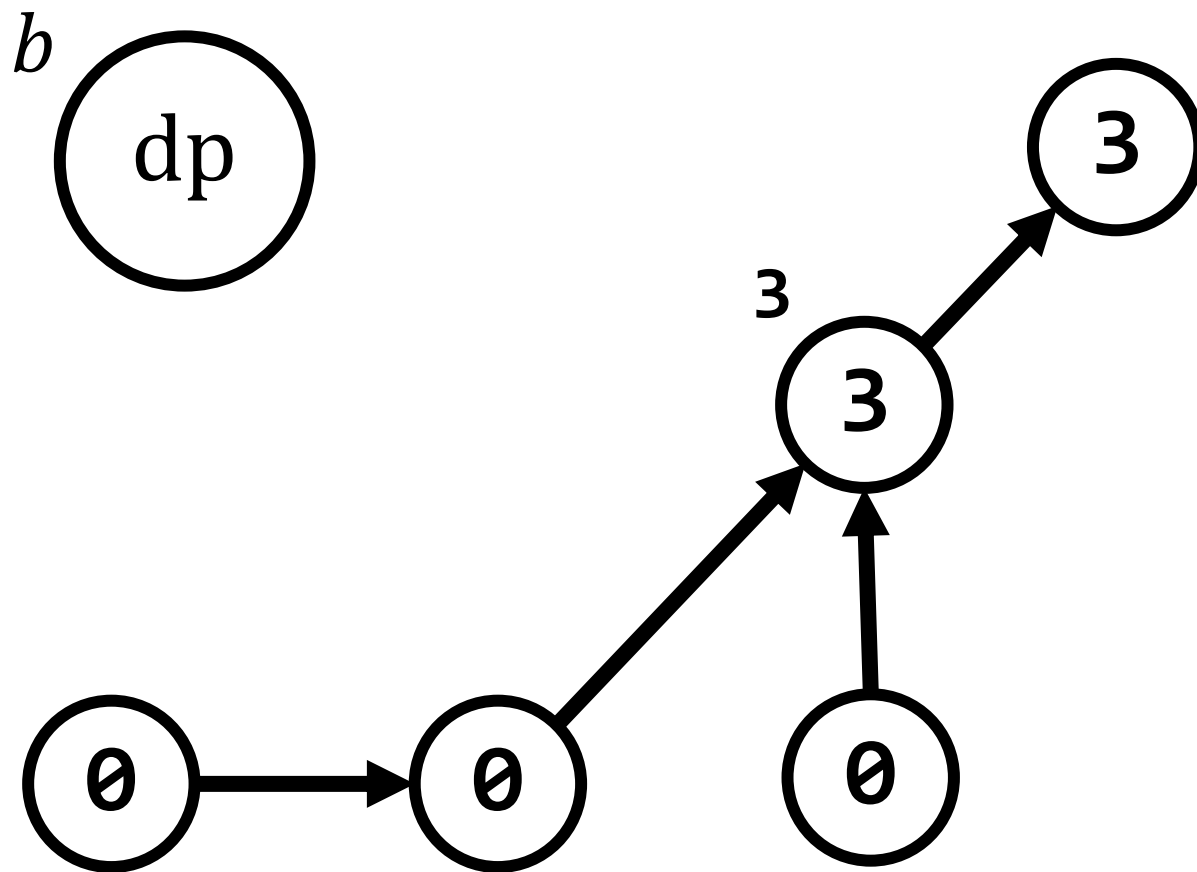
$$b[u][i] = \max(b[u][i + 1] - s[u_v][i], 0)$$

$$\begin{aligned} s[u][i] &= b[u][i] - b[u][i + 1] + \sum_{w \in N_u} (\text{dp}[w][i] - \text{dp}[w][i + 1]) \\ &= b[u][i] - b[u][i + 1] + s[u_v][i] \end{aligned}$$

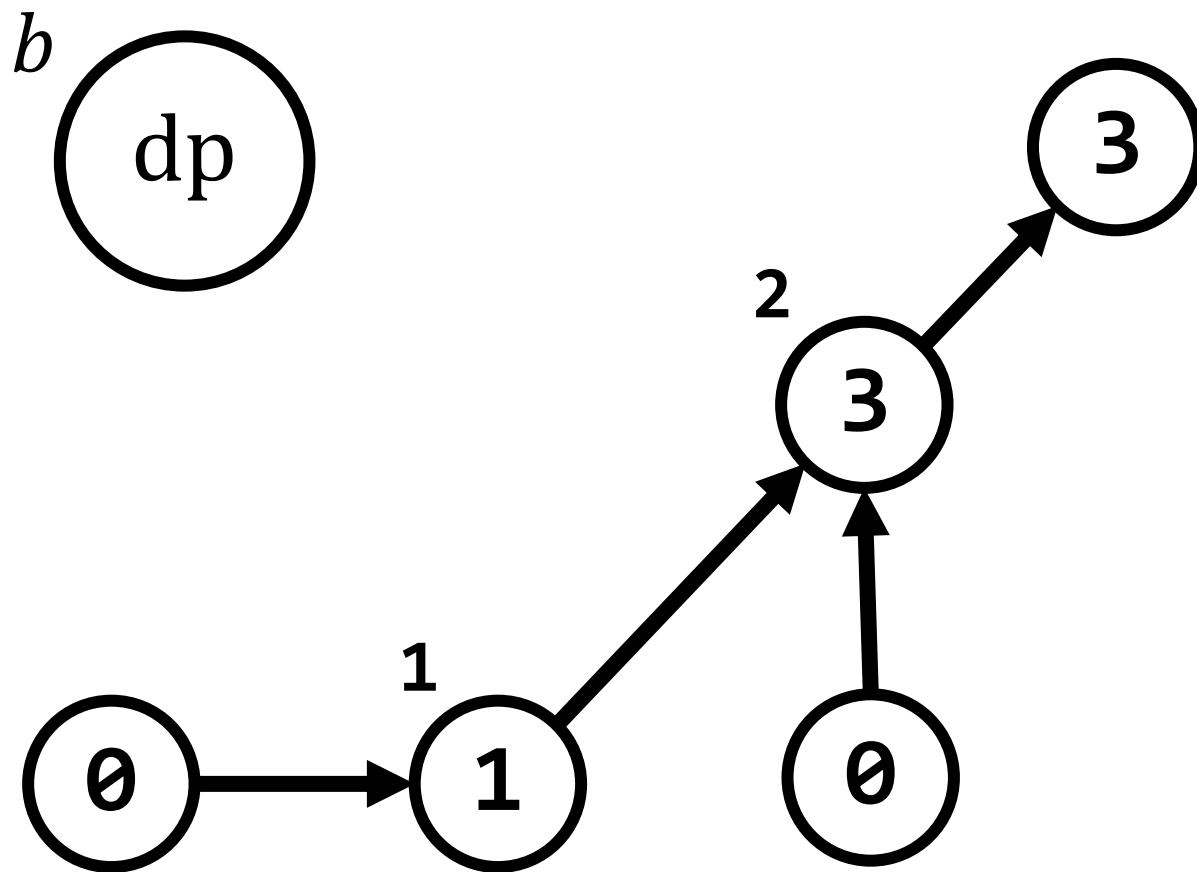
別解法



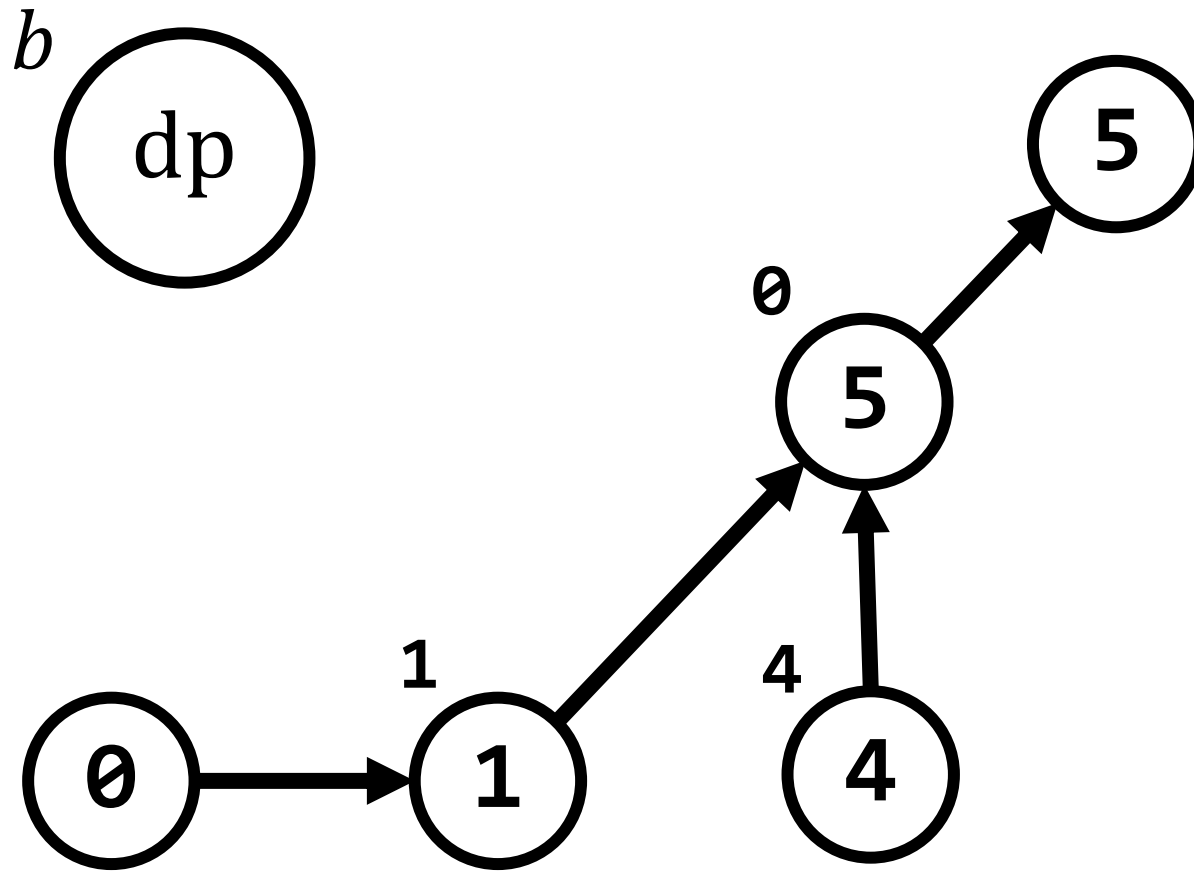
別解法



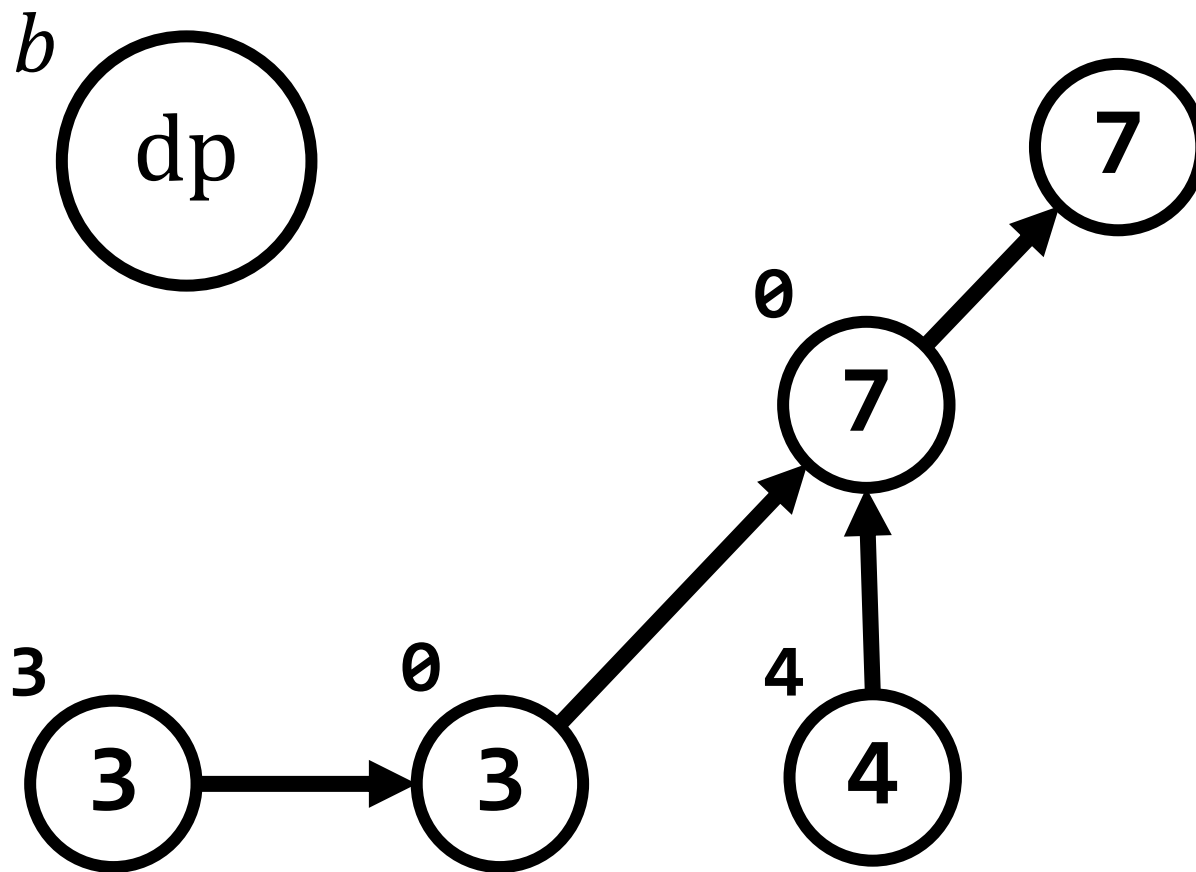
別解法



別解法



別解法



b の値を吸収するように
dp の値が増加する

別解法

- $dp[root][1]$ だけ計算すれば十分
- v の祖先であり $b[u][i]$ の値が正であるような u の中で最も深い頂点を高速に計算できるようにしたい

別解法

- T を頂点集合として、以下のクエリを高速に処理したい

Add : T に v を追加

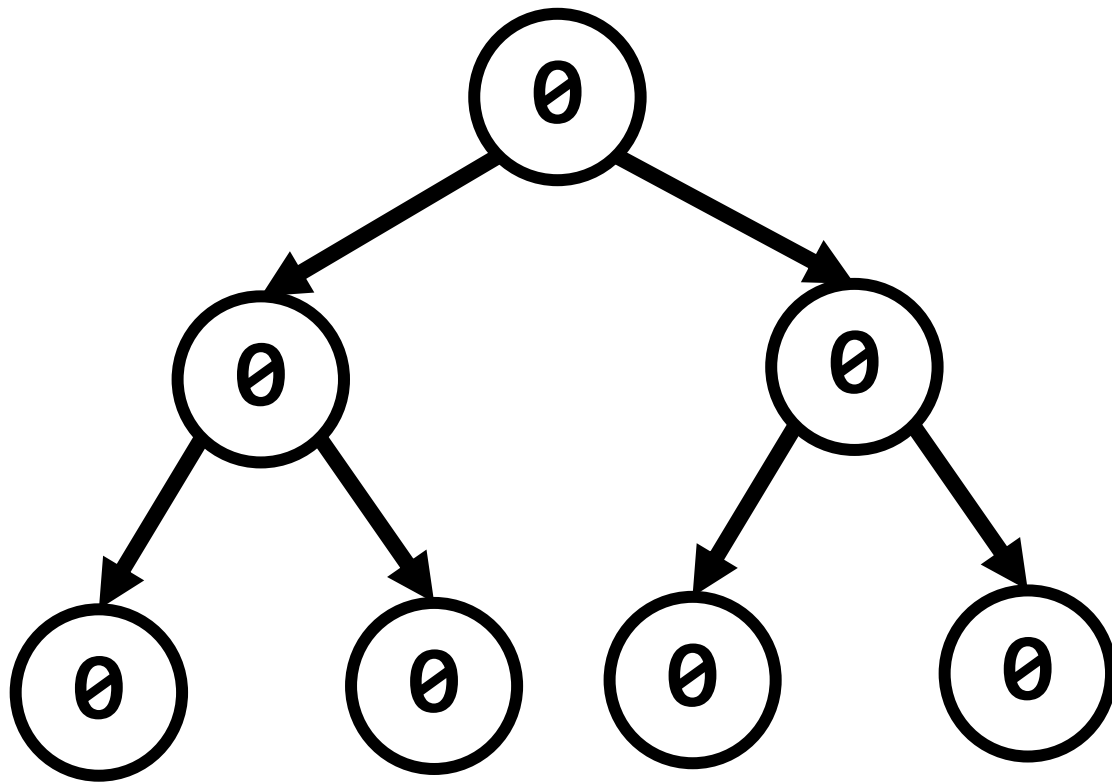
Erase : T から v を削除

Access

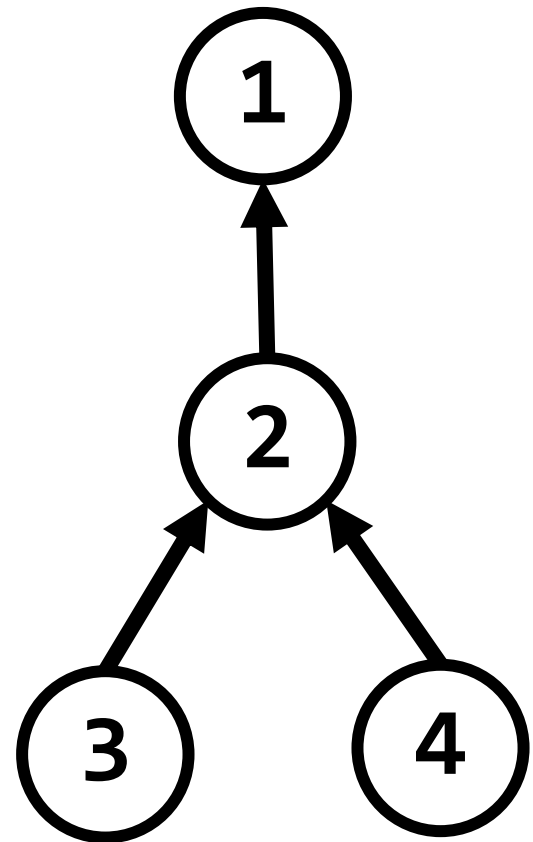
v の祖先(v を含む)であり T に含まれている頂点で最も深い頂点を計算

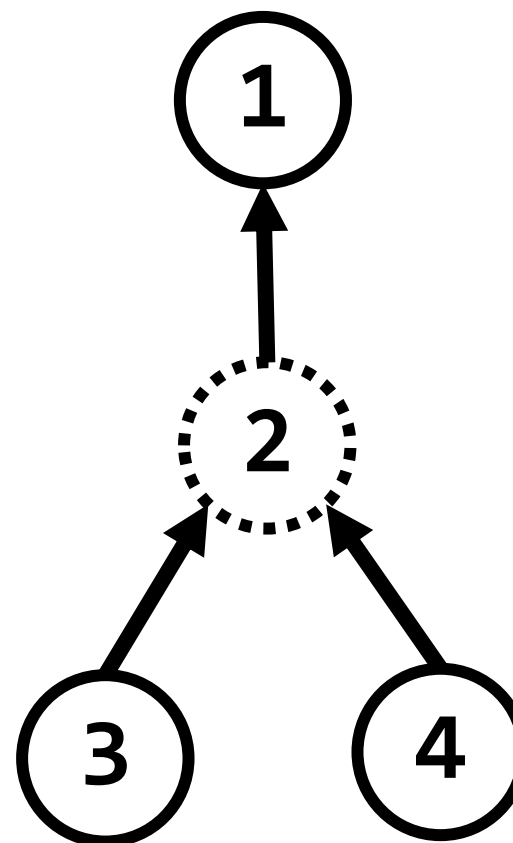
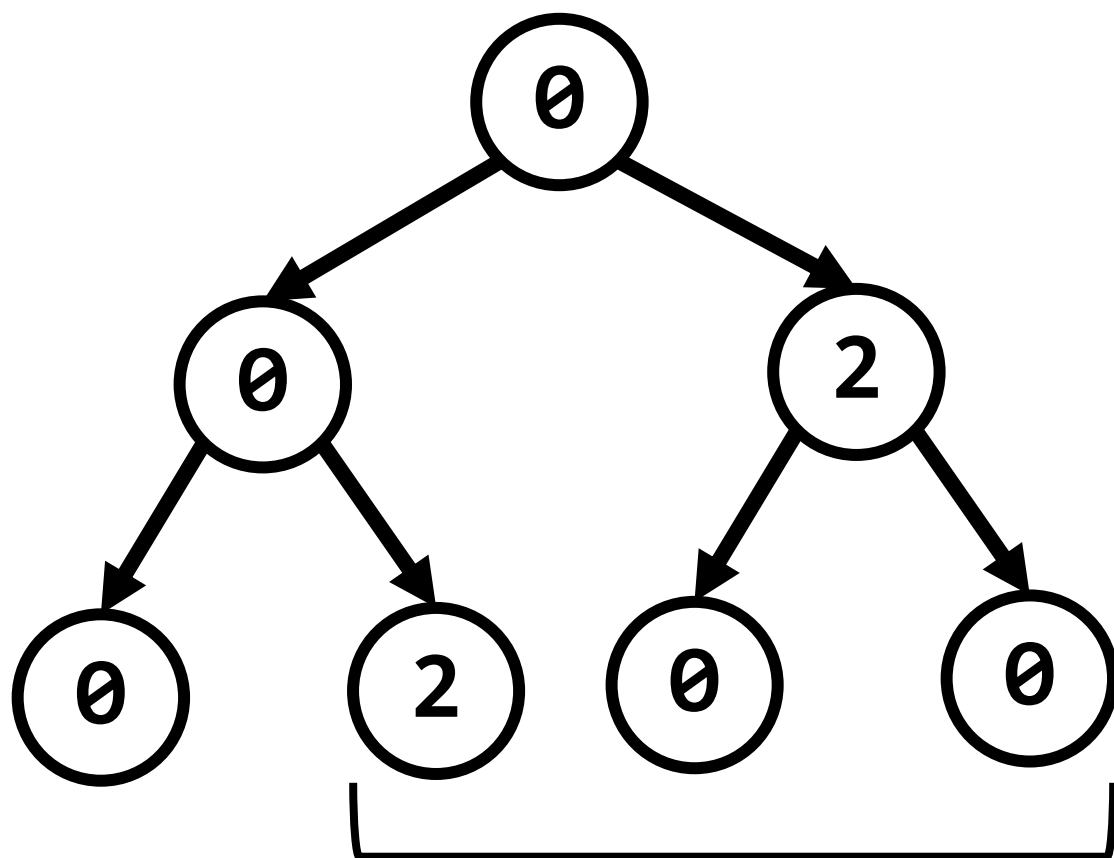
別解法

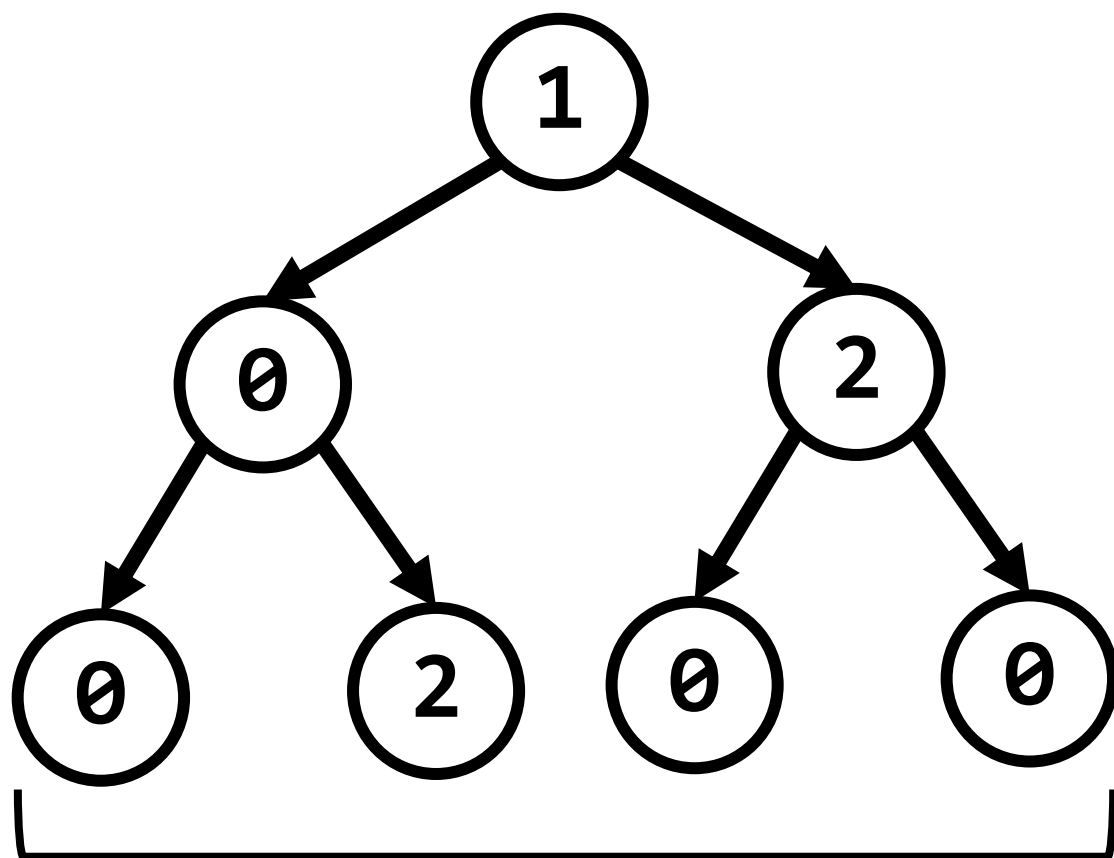
- Add, Erase は部分木内に影響を及ぼす
- 頂点をDFS順序に並べることで部分木を区間に変換することができる
- Add は区間に `chmax` するクエリとみなせる
- Segment tree で効率的に計算できる



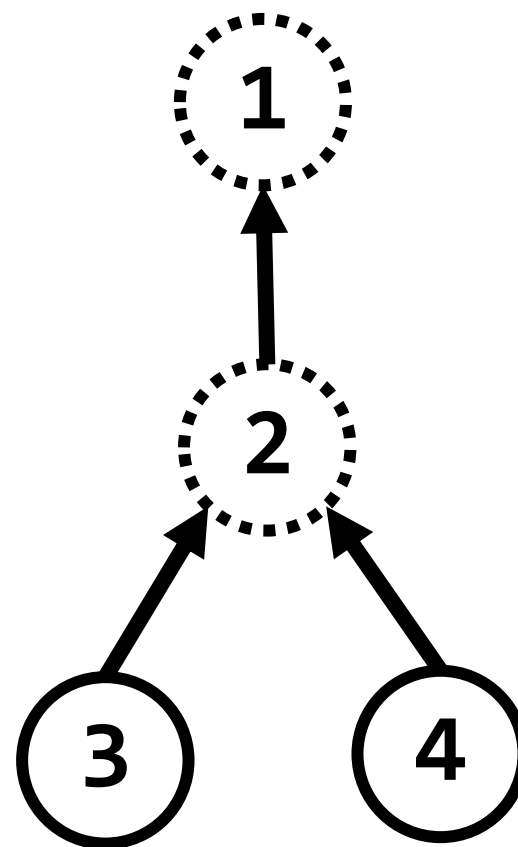
Segment tree

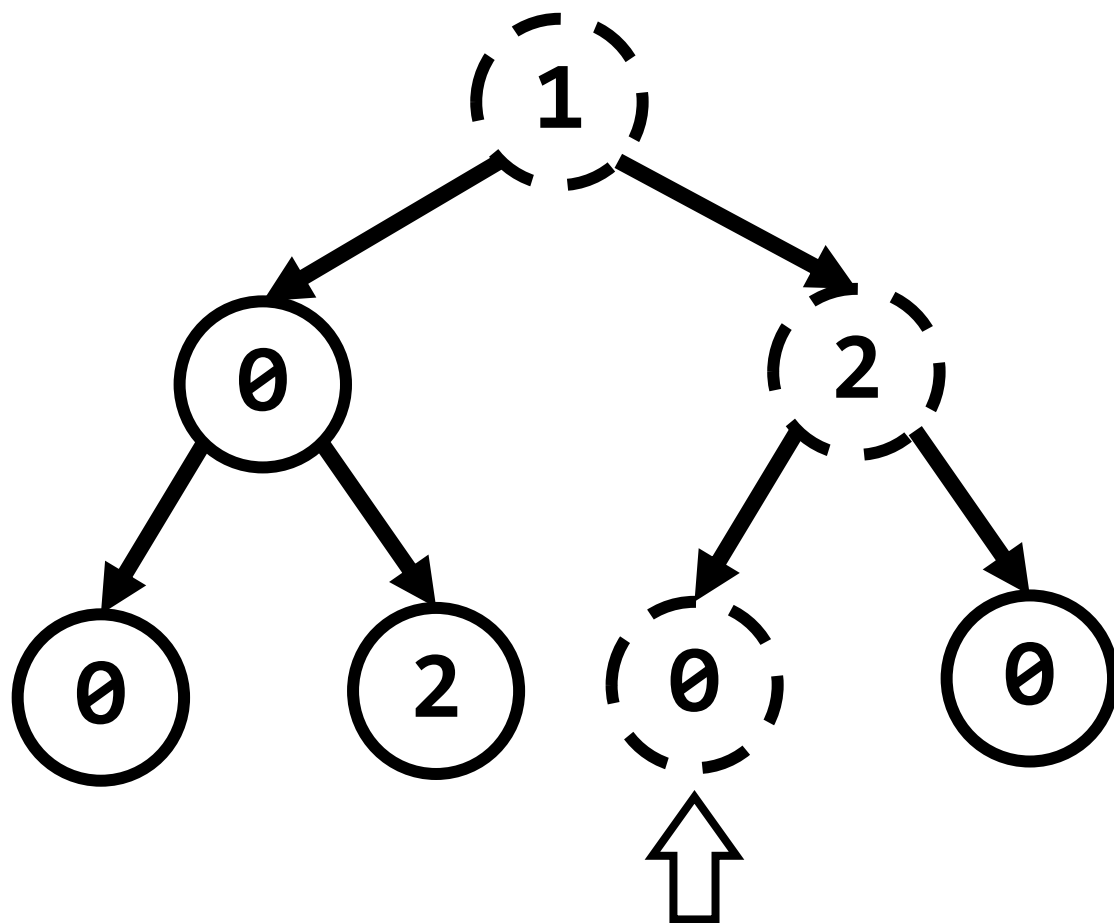




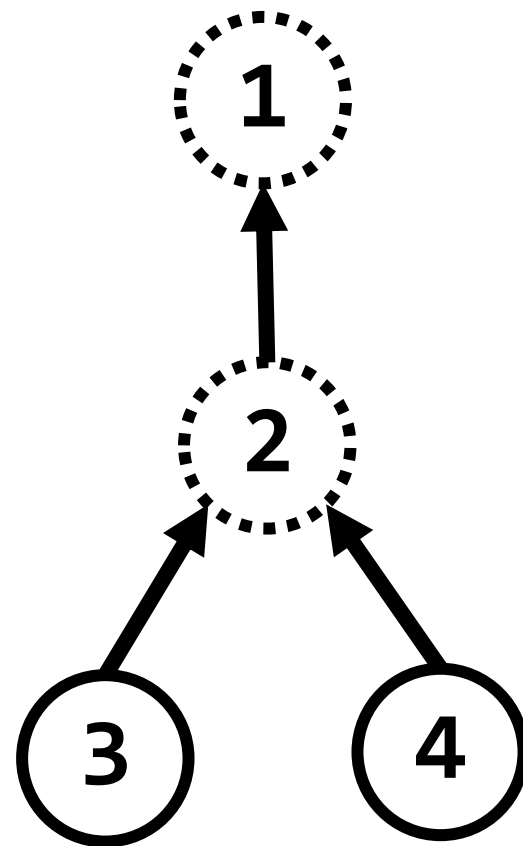


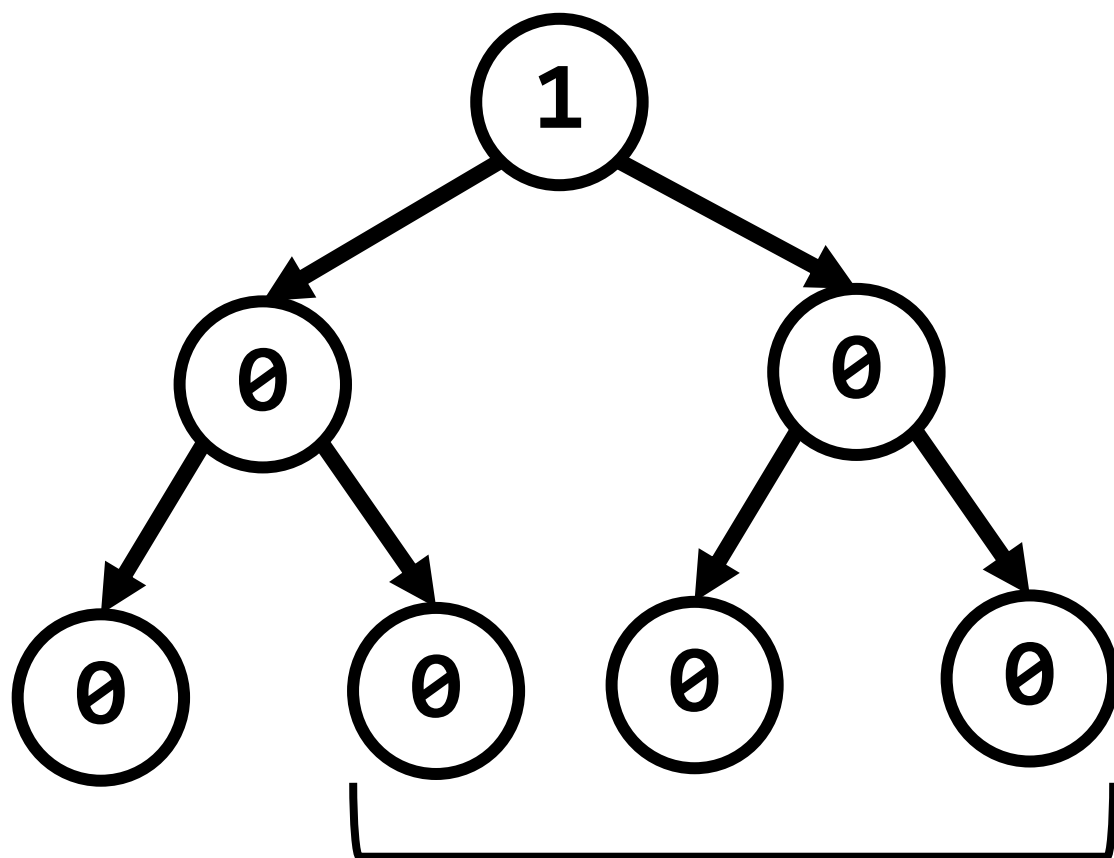
Add 1



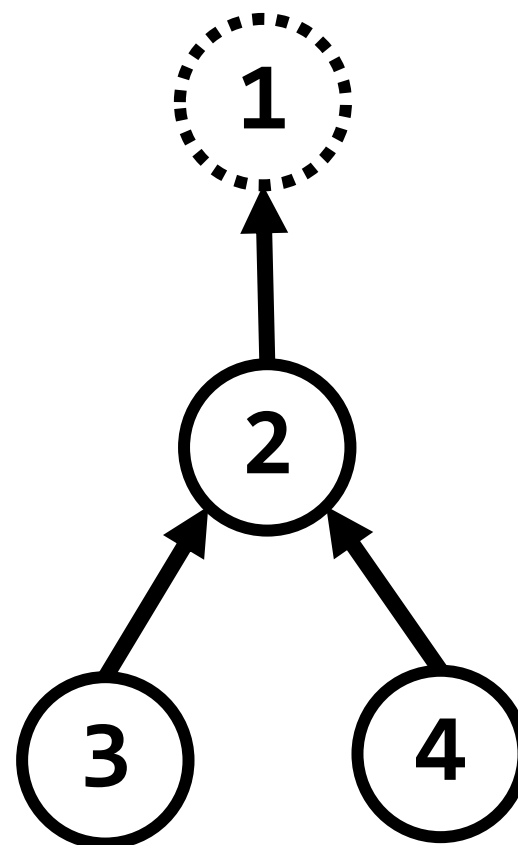


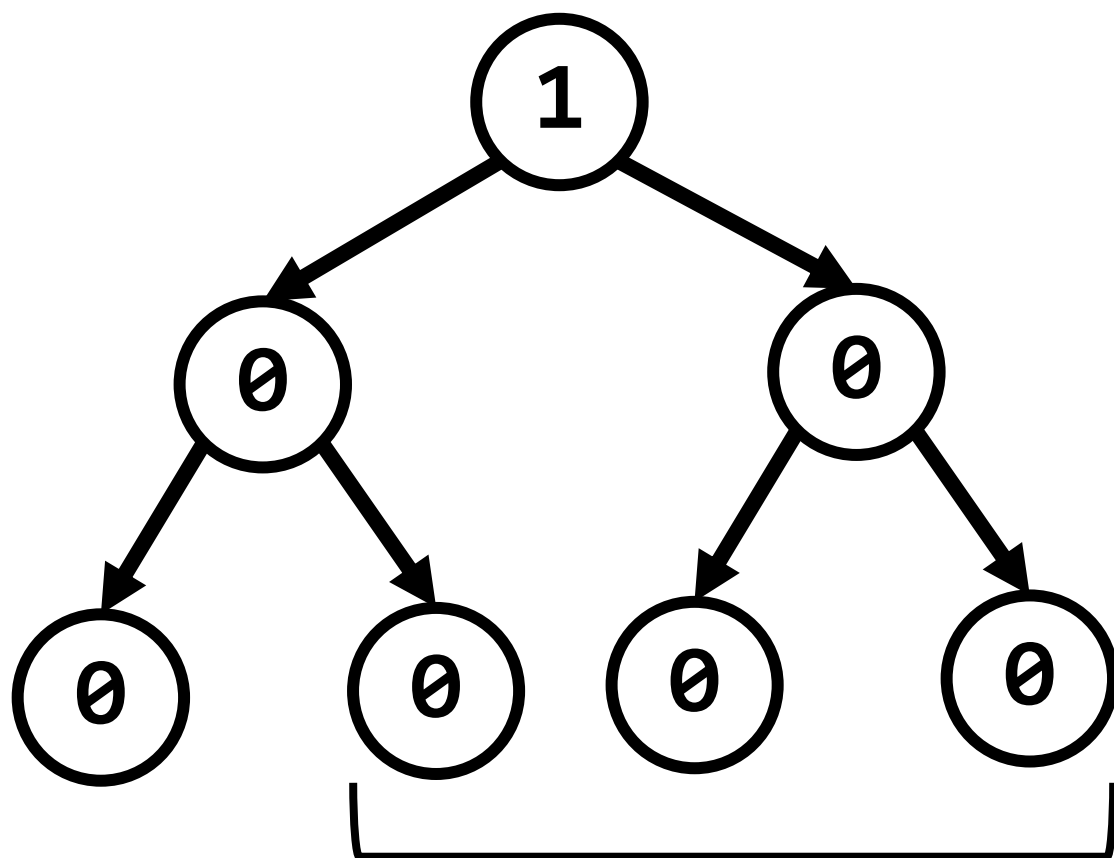
Access 3



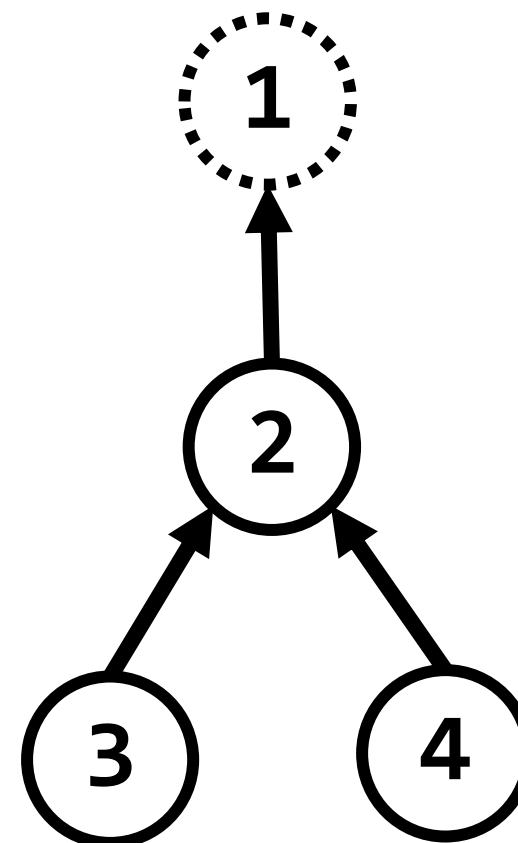
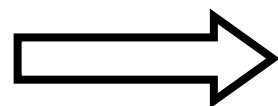


Erase 2





Erase 2



$v = \text{Access parent}(2)$

$2 \rightarrow \emptyset$

Add v

別解法

- 各クエリ $O(\log N)$ 計算可能
- 全体の計算量は $O(N \log N)$

得点分布

